PROJECT HANDBOOK · EXPLANATION · VIVA GUIDE

Production-Grade Microservices Platform with Zero-Downtime Deployment

Order & Inventory Management System

How the project is built, deployed, secured, observed and operated on AWS — every service, every important file, and the evidence behind each claim.

React 18 · Vite · Spring Boot 4.1 · Java 17 · MySQL 8.4 · Flyway · Docker · Jenkins · SonarQube · Trivy · Amazon ECR · Amazon EKS 1.36 · Helm · AWS Load Balancer Controller · Amazon RDS · Terraform · Ansible · IAM · KMS · Secrets Manager · CloudWatch · SNS · SQS · AWS Budgets · Elasticsearch · Filebeat · Kibana

Repository	github.com/SamruddhiDoke/DevOps-Project (branch main)
AWS region	ap-south-1 (Mumbai)
Environment state	suspended (verified read-only on 2026-10-25)
Document generated	2026-10-25

Contains no passwords, keys, tokens or secret values. Project facts are taken from the repository and recorded evidence (docs/TESTING.md, docs/evidence/); general explanations are marked "In general".

Contents

1 Evaluation-day quick start, verification and shutdown 7

Verified current state 7
Before you start: directory, tools and AWS identity 8
Exact resume instructions 9
Getting the current application URL and Jenkins address 10
After the restore: what happens by itself 10
Verifying every layer before the evaluation 11
Live demonstration plan 12
Exact shutdown instructions 13
Common errors and safe recovery 13

2 Project overview 15

In simple English 15
Title, problem, motivation 15
Objectives (and where each is met) 15
Scope 16
Main features 16
Technology stack (actual versions from the repository) 16
Why this is a DevOps project, not only a CRUD application 17
What makes it production-style 17
"Microservices": an accurate description 17
Limitations and trade-offs (honest list) 17
Future improvements 18
Component status matrix 18

3 Architecture and flows 20

High-level system architecture 20
AWS VPC and network layout 21
Application request flow (EKS) 21
Database interaction (create order) 22
GitHub-to-Jenkins CI/CD pipeline 22
Docker image build and ECR publishing 23
Helm deployment to EKS and ALB traffic 24
CloudWatch → SNS / SQS notification flow 24
ELK logging flow 25
Terraform provisioning flow 26
Ansible configuration of Jenkins 27
Suspension and restoration sequence 28

4 The role of every service 29

React, JavaScript and Vite (frontend) 29
nginx (frontend web server and reverse proxy) 30
Java and Spring Boot (backend) 31
REST API 32
MySQL 32
Flyway (database migrations) 33
Docker 33
Docker Compose 34
Git and GitHub 34
Jenkins (CI/CD server)

SonarQube 36
Trivy and secret scanning 37
Amazon ECR 38
Shell and PowerShell automation scripts 39
Amazon EKS and Kubernetes 40
Helm 41
AWS Load Balancer Controller and the ALB 42
VPC, subnets, route tables, Internet Gateway and NAT Gateway 43
Amazon EC2 (Jenkins host) 43
Ansible 44
Terraform and the S3 remote state 45
IAM 46
Security groups 46
AWS KMS 47
AWS Secrets Manager 47
Amazon RDS (backups and recovery) 48
Amazon CloudWatch (alarms and logs) 48
Amazon SNS and SQS 49
AWS Budgets 49
ELK: Elasticsearch, Filebeat and Kibana (no Logstash) 50
Health checks, rolling updates, rollback and zero downtime 51
Supporting tools 51

5 Repository and file inventory 53
Directory tree (tracked files) 54
Files that are intentionally not in Git (from .gitignore) 56
Root files 56
Application — backend (order-inventory-platform/backend) 57
Application — frontend (order-inventory-platform/frontend) 61
Application — local environment and logging 64
Kubernetes and Helm 65
Terraform (terraform/) 66
Ansible (ansible/) 67
CI (local) and scripts 68
Documentation 70
Generated, third-party and evidence files (explained by category) 70
Coverage checklist 71

6 Frontend: React, JavaScript and Vite 72
What the frontend is 72
Build and tooling files 72
Entry point and routing 74
API communication — src/services/api.js (line by line) 74
Hook and components 76
Pages 77
Production build and how the container serves it 78
Frontend tests (21 tests, Vitest + Testing Library) 79
Viva questions (frontend) 79

7 Backend: Java and Spring Boot 81
Architecture of the backend 81
Build: pom.xml (block by block) 81

REST endpoints 84
DTOs and validation 87
Entities (JPA mapping) 88
Repositories 89
Services, transactions and concurrency 89
Error handling: GlobalExceptionHandler and ErrorResponse 91
Tests (29, JUnit 5) 92
Build and run 93
Viva questions (backend) 93

8 MySQL, RDS and Flyway 95
Schema and relationships 95
How Flyway works here 98
Baselining 98
How the application gets its database configuration 98
RDS networking and security (from terraform/main.tf) 99
Connection handling during restarts 99
Stopping RDS 100
Four different kinds of "state" (do not mix them up) 100
Viva questions 100

9 Docker and Docker Compose 102
What Docker does (and does not do) in this project 102
Backend Dockerfile (line by line) 102
Frontend Dockerfile (line by line) 103
nginx configuration (frontend/nginx/default.conf.template) 104
Docker Compose (order-inventory-platform/docker-compose.yml) 105
Image tagging and scanning 106
Local vs production containers 106
Viva questions 106

10 Terraform: every file and resource 108
Files and variables 108
Network 110
Container registry 110
EKS 111
Database (main.tf, explained in detail in MySQL, RDS and Flyway) 112
Jenkins (jenkins.tf) 112
Encryption, storage, monitoring 113
Outputs (outputs.tf) 114
Every AWS resource: suspension, restoration and cost 114
Plan, refresh, apply, state and drift 115
Destruction and dependencies 115
Viva questions 115

11 Ansible 116
Terraform vs Ansible in this project 116
Running it: run-playbook.sh (line by line) 116
Configuration files 117
playbooks/jenkins.yml (line by line) 118
Roles (task by task) 118
Idempotency, error handling, linting and testing 120
Safe commands 120
Viva questions

12 Jenkins and the CI/CD pipeline 122
Where Jenkins runs and how it is configured 122
Pipeline-level settings (Jenkinsfile lines 10–43) 122
Every stage 123
Trace of one commit (real: 18a2bc6, build #8, 2026-10-19 UTC) 126
scripts/deploy-k8s.sh (line by line) 126
scripts/smoke-test.sh (what the checks are) 127
Immutable tags and rollback 127
Local CI (ci/) 127
Build history on AWS (from docs/TESTING.md) 128
Troubleshooting failed builds 128
Viva questions 128

13 EKS, Kubernetes, Helm and the ALB 130
EKS: control plane vs worker nodes 130
Kubernetes objects in the application namespace 130
The Helm chart (helm/order-inventory/) 131
Zero-downtime rolling update — how the pieces fit 133
Ingress → ALB (AWS Load Balancer Controller) 133
Scaling: replicas vs nodes 134
Bootstrap scripts 134
Database outage behaviour on Kubernetes 135
Inspection commands 135
Viva questions 135

14 Security: SonarQube, Trivy, IAM and secrets 137
SonarQube (static analysis + quality gate) 137
Trivy (vulnerability, misconfiguration and secret scanning) 137
gitleaks and .gitleaksignore 138
Secret handling 138
IAM least privilege 139
Network exposure 139
Container and workload security 139
Historical secret removal and Git history 140
Root account and MFA (owner actions) 140
Configured control vs verified outcome 140
Viva questions 141

15 Monitoring and logging 142
CloudWatch alarms 142
SNS → SQS (and email) notification path 143
AWS Budgets 144
ELK on EKS (k8s/logging/elk.yaml) 144
Tracing one request with its request id 145
Commands 145
Viva questions 146

16 Suspend and resume, line by line 147
Suspension is not destruction 147
Shared library scripts/lib/evaluation-common.sh 147
suspend-evaluation.sh 148
resume-evaluation.sh 150

Recovery manifest (docs/recovery/evaluation-manifest.json) 151
Order dependencies 151
Interrupted halfway — what happens 151
What stays billable and why 152
Troubleshooting 152
Test record (2026-10-19) 152

17 Viva and cross-question bank 154
A. The hard cross-questions first 154
B. Project overview and design 156
C. Frontend 156
D. Java and Spring Boot 157
E. REST and HTTP status codes 157
F. SQL, MySQL, transactions, concurrency 157
G. Docker and Compose 158
H. Git and GitHub 158
I. Terraform 158
J. Ansible 159
K. Jenkins and CI/CD 159
L. SonarQube and Trivy 159
M. AWS services 159
N. Monitoring, logging, cost, recovery 160

18 Presentation scripts 161
30-second introduction 161
One-minute introduction 161
Three-minute technical explanation 161
Five-minute architecture explanation (follow diagram 1 and 2 in Architecture and flows) 162
Live demonstration script (5–10 minutes) 162
CI/CD pipeline — detailed explanation 162
Zero-downtime deployment — detailed explanation 163
AWS architecture — detailed explanation 163
Security and cost optimisation explanation 163
Honest limitations and future improvements 163

19 Glossary 164
Networking and AWS 164
Containers and orchestration 165
Delivery, quality and operations 166

20 Quick reference 168
Identity and names 168
Commands 168
Five numbers to remember 169
Never do 169

CHAPTER 1

Evaluation-day quick start, verification and shutdown

This chapter is deliberately first. It tells you exactly how to bring the suspended project back, prove that it works, demonstrate it and suspend it again. Every command was checked against the real scripts in the repository, and the state below was read from AWS (read-only) while this document was produced.

> **Never use `terraform destroy` to "turn the project off".** The database is configured with `skip_final_snapshot = true` (a lab setting in `terraform/main.tf`), so a destroy would delete it *without* a final snapshot, together with the secrets, the encryption key and the ECR images. Suspension (section Exact shutdown instructions) keeps everything.

1.1 Verified current state

Read-only checks on **2026-10-25 at 05:11 UTC** with the `order-inventory` AWS CLI profile. Nothing was changed.

Item	Observed state	How it was checked
AWS identity	account `038215676861`, `arn:aws:iam::038215676861:user/order-inventory-deployer`	`aws sts get-caller-identity`
Region	`ap-south-1` (Mumbai)	profile and the scripts' `AWS_REGION` default
EKS cluster	`order-inventory-eks`, **ACTIVE**, Kubernetes 1.36	`aws eks describe-cluster`
Node group	`order-inventory-nodes`, ACTIVE, **min 0 / desired 0 / max 3**, 0 instances	`aws eks describe-nodegroup`
RDS	`order-inventory-mysql`, **stopped** (MySQL 8.4.11, db.t3.micro); last event "DB instance stopped" 2026-10-18 21:34 UTC	`aws rds describe-db-instances`, `describe-events`
Jenkins EC2	`i-03288b455e674765e`, **stopped**, no public IP while stopped	`aws ec2 describe-instances`
Running EC2 instances	0	filter `running,pending`
Load balancers	**0** (the application ALB is removed while suspended)	`aws elbv2 describe-load-balancers`
NAT gateway	1 available (kept deliberately)	`aws ec2 describe-nat-gateways`
ECR backend tags	`2-06f7acc 4-bf7450d 5-eaf6380 6-ca5c22c 7-ca5c22c 8-18a2bc6`	`aws ecr describe-images`
CloudWatch alarms	9 Terraform alarms present: 8 `INSUFFICIENT_DATA` (nothing emits metrics), the NAT alarm `OK`; the 4 ALB alarms are recreated on resume	`aws cloudwatch describe-alarms`
AWS Budget	limit $350/month; Budgets reported actual **$0.00**, forecast **$7.13** for October (indicative; Budgets data lags and is net of credits)	`aws budgets describe-budgets`
Recovery manifest	`docs/recovery/evaluation-manifest.json`: release **`8-18a2bc6`**, Helm revision at suspend 14, suspended 2026-10-18 21:13 UTC, **`rdsAutoRestartAfter` 2026-10-25 21:13 UTC**	file

	the history rewrite, so compare with `origin/main` explicitly	`git rev-list --left-right --count main...origin/main`

What this means in plain words:

- **Release restored by resume:** `8-18a2bc6` = Jenkins build #8, commit `18a2bc6`, which Jenkins built by itself after detecting a GitHub push. `7-ca5c22c` (build #7) is the previous good release and is still in ECR.
- **RDS auto-start:** AWS starts a stopped RDS instance automatically after 7 days — for this instance around **2026-10-25 21:13 UTC (2026-10-26 02:43 IST)**. If the environment is not needed then, run the suspend command again afterwards; it will only stop RDS.
- **Your public IP has changed** since the last restore test (checked 2026-10-25). Resume detects this and makes a targeted Terraform change to allow the new address (stage 1 below). That branch was dry-run with a saved plan on 2026-10-25: exactly *2 to add, 1 to change, 2 to destroy* (the two Jenkins security-group rules replaced, the EKS API allow-list updated in place), which the script's safety guard accepts. It has not yet been applied for real, so expect a few extra minutes (not measured).
- **Commits not yet built:** commits pushed while Jenkins was stopped (documentation only) will be built and deployed automatically about 5–15 minutes after resume (see After the restore).
- Cost figures quoted later (≈ $126/month suspended, ≈ $311/month running) are **estimates from on-demand list prices computed on 2026-10-18/19**, not invoices.

1.2 Before you start: directory, tools and AWS identity

Open **PowerShell** (or Git Bash) and change to the repository root — every command in this chapter runs from here:

```
cd "C:\Users\Siddesh\Desktop\DevOps Project"
```

Tool	Why it is needed	Check
Git for Windows (Git Bash)	runs the `.sh` scripts; the `.ps1` wrappers call `C:\Program Files\Git\bin\bash.exe`	`git --version`
AWS CLI v2 + profile `order-inventory`	every AWS call	`aws --version`
Terraform ≥ 1.10 (1.15.8 used)	`terraform init` reads IDs from the S3 state; the targeted admin-IP update	`terraform -version`
kubectl	talks to EKS	`kubectl version --client`
Helm 4.x	deploys the release	`helm version --short`
Node.js ≥ 20.19	small JSON helpers inside the scripts	`node -v`
curl	health checks	`curl --version`

Verify the AWS account and region:

```
aws sts get-caller-identity --profile order-inventory     # Account 038215676861, user/order-inventory-deployer
aws configure get region --profile order-inventory        # ap-south-1
```

The scripts enforce this themselves: they stop if the account is not `038215676861` (`EXPECTED_ACCOUNT` in `scripts/lib/evaluation-common.sh`), refuse the root user, and force the region to `ap-south-1`.

1.3 Exact resume instructions

PowerShell:

```
powershell -ExecutionPolicy Bypass -File scripts\resume-evaluation.ps1
```

Git Bash (the form used for the verified end-to-end restore test on 2026-10-19):

```
bash scripts/resume-evaluation.sh
```

The `.ps1` wrapper only locates Git Bash, sets `AWS_PROFILE=order-inventory` if it is not set, changes to the repository root and runs the `.sh` script with the same arguments, returning its exit code. (Wrapper status: the suspend wrapper was executed with `--plan`; the resume wrapper itself has not been executed, the resume script has.)

What the script starts, in order (stage names exactly as printed; durations measured on 2026-10-19):

Stage	What it does	Duration
Preflight	tools present; AWS identity = expected account, not root; `terraform init` on the S3 backend; reads cluster, RDS and Jenkins IDs from Terraform outputs; state bucket reachable; cluster ACTIVE; manifest present; ECR images for the manifest tag exist	11 s
1. Admin network access	compares this laptop's public IP with the EKS API allow-list; if different, rewrites the git-ignored `terraform/admin.auto.tfvars` and applies a **targeted** plan (EKS API CIDRs + two Jenkins SG rules only — anything else is refused)	1 s if unchanged; a few extra minutes if the IP changed
2. Start RDS, Jenkins EC2 and the EKS node group (in parallel)	`start-db-instance`, `start-instances`, node group min 1 / desired 2 / max 3	8 s (requests only)
3. Kubernetes nodes and cluster add-ons	kubeconfig; 2 nodes `Ready`; `coredns`, `aws-load-balancer-controller`, `aws-node`, `kube-proxy` ready; Secret `mysql-secret` present	≈ 1.5 min
4. Database	waits for RDS `available`	≈ 3 min — **longest wait**
5. Application release	`scripts/deploy-k8s.sh <registry> <tag>`: Helm upgrade with chart defaults (2+2 replicas, Ingress enabled), automatic rollback on failure	≈ 1 min
6. Application Load Balancer	waits for the Ingress hostname, ALB `active`, both target groups with ≥ 2 healthy targets, `/api/health` = 200 through the new DNS name	≈ 1 min
7. Database connectivity, migrations and smoke tests	`/api/health`, `/api/health/ready`, `/` = 200; prints Flyway log lines; full read/write `scripts/smoke-test.sh` (20 checks)	≈ 10 s
8. ALB alarms	`scripts/create-alb-alarms.sh` recreates 4 alarms for the new ALB	≈ 7 s
9. Logging (ELK)	scales Elasticsearch and Kibana to 1, waits for them and Filebeat, checks Kibana `/api/status` through a temporary port-forward	≈ 2.5 min
10. Jenkins	waits for the instance, reads its **new** public IP, waits for `/login` = 200, checks its security group has no `0.0.0.0/0` rule	seconds
11. Update recovery manifest	writes `resumedAt`, `application.currentUrl`, `jenkins.publicIpAtResume`	< 1 s

Recognising success. Each successful step prints a line starting with `OK`. Real lines from the 2026-10-19 log:

```
[02:22:24]   OK  2 node(s) Ready (43s)
[02:25:41]   OK  RDS available (165s)
[02:27:45]   OK  both ALB target groups have >=2 healthy targets (4s)
All 20 checks passed against http://k8s-orderinv-orderinv-00335110a2-1122521152.ap-south-
1.elb.amazonaws.com
```

A failure prints `FAILED in stage: <stage>`, a `reason:` line and diagnostic commands, then exits with code 1. **Fix the reason and run the same command again** — every step checks the current state first.

The final block (format from `scripts/resume-evaluation.sh` lines 180–194; the values differ every run):

```
======================================================================
 ENVIRONMENT RESTORED AND VERIFIED in 9m 14s
   Application : http://<new ALB hostname>
   Release     : 8-18a2bc6 (Helm release order-inventory, namespace order-inventory)
   Checks      : nodes Ready, add-ons ready, RDS available, 2/2 backend + 2/2 frontend,
                 ALB targets healthy, health/readiness 200, read/write smoke test passed,
                 ELK ready, Jenkins /login 200
   Jenkins     : http://<new public IP>:8080   (admin /32 only; password: Secrets Manager order-
inventory/jenkins-admin)
   Kibana      : kubectl -n logging port-forward svc/kibana 5601:5601  ->  http://localhost:5601
   Note        : the ALB hostname and Jenkins IP can differ from the previous run.
   Suspend     : bash scripts/suspend-evaluation.sh
======================================================================
```

1.4 Getting the current application URL and Jenkins address

The ALB hostname and the Jenkins IP change on every restore: suspend deletes the Ingress (so AWS deletes the load balancer) and resume creates a new one with a new DNS name; Jenkins has no Elastic IP, so it gets a new public IP on every start. Read the actual values at any time:

```
export AWS_PROFILE=order-inventory AWS_REGION=ap-south-1
kubectl -n order-inventory get ingress order-inventory-ingress -o
jsonpath='{.status.loadBalancer.ingress[0].hostname}'; echo
aws ec2 describe-instances --instance-ids i-03288b455e674765e --query
'Reservations[0].Instances[0].PublicIpAddress' --output text
node -e "const m=require('./docs/recovery/evaluation-manifest.json');console.log(m.application.currentUrl,
m.jenkins.publicIpAtResume)"
```

`terraform output jenkins_public_ip` is **not** reliable after a restart: it shows the last IP stored in the state.

1.5 After the restore: what happens by itself

1. **Jenkins builds the newest commit.** The job polls GitHub every 5 minutes (`H/5 * * * *`). Commits pushed while Jenkins was stopped start a full pipeline (tests → SonarQube → Trivy → images `9-<commit>` → ECR → Helm rolling update → ALB checks). Expect it 5–15 minutes after resume, taking about 8 minutes. It is a zero-downtime rolling update. Start resume **at least 45 minutes before the evaluation**, or use this build as the live CI/CD demo.
2. **Kibana starts empty.** Elasticsearch stores data in an `emptyDir` volume that disappeared with the nodes. New logs are collected again, but the saved data view and dashboard must be re-imported:

```
kubectl -n logging port-forward svc/kibana 5601:5601        # terminal 2, leave running
bash order-inventory-platform/logging/kibana-setup.sh http://localhost:5601
```

(The import script was verified on 2026-10-18/19: "6 objects, success"; re-import after a resume has not yet been executed.)

1.6 Verifying every layer before the evaluation

```
export AWS_PROFILE=order-inventory AWS_REGION=ap-south-1
URL=http://$(kubectl -n order-inventory get ingress order-inventory-ingress -o
jsonpath='{.status.loadBalancer.ingress[0].hostname}')
```

Layer	Command	Expected result
EKS nodes	`kubectl get nodes`	2 nodes `Ready`
Pods	`kubectl -n order-inventory get pods`	2 `backend-*`, 2 `frontend-*`, all `1/1 Running`
Helm release	`helm -n order-inventory history order-inventory --max 3`	last revision `deployed`
Ingress / ALB	`kubectl -n order-inventory get ingress`	`ADDRESS = k8s-orderinv-… elb.amazonaws.com`
ALB targets	`aws elbv2 describe-target-groups --query "TargetGroups[?starts_with(TargetGroupName,'k8s-orderinv')].TargetGroupArn" --output text` then `aws elbv2 describe-target-health --target-group-arn <arn>`	all targets `healthy`
Frontend	`curl -sI $URL/`	`HTTP/1.1 200`
Backend liveness	`curl -s $URL/api/health`	`{"status":"UP"}`
Database (readiness)	`curl -s $URL/api/health/ready`	`{"status":"UP","database":"UP"}`
Migrations	`kubectl -n order-inventory logs deploy/backend | grep -i flyway`	"Successfully validated 2 migrations", "Schema … is up to date"
Read-only functional check	`SMOKE_READ_ONLY=1 bash scripts/smoke-test.sh $URL`	"All 8 read-only checks passed"
Read/write check	`bash scripts/smoke-test.sh $URL`	"All 20 checks passed" (creates uniquely named test records, cancels the order)
Jenkins	browser `http://<jenkins-ip>:8080`	login page; job `order-inventory`
ELK	`kubectl -n logging get pods`	elasticsearch, kibana, filebeat-* Ready
Alarms	`aws cloudwatch describe-alarms --alarm-name-prefix order-inventory --query 'MetricAlarms[].[AlarmName,StateValue]' --output table`	13 alarms; `OK` (new ones `INSUFFICIENT_DATA` for a few minutes)

Jenkins password (copied to the clipboard, never shown on the projector):

```
aws secretsmanager get-secret-value --secret-id order-inventory/jenkins-admin --query SecretString --
output text --profile order-inventory --region ap-south-1 | Set-Clipboard
```

Network note: the EKS API, Jenkins (8080) and SSH accept connections **only from the public IP that ran resume**. Run the demo from the same laptop and network.

Fallback if AWS or the internet fails: run the local stack (`cd order-inventory-platform`, `docker compose up -d --build`, open http://localhost:3000 — Docker Desktop required, build once beforehand while online), the local tests (`mvn -B verify`, `npm test`), and show the recorded evidence in `docs/evidence/` and the results in `docs/TESTING.md`, plus the diagrams in the Architecture chapter.

Exam-pressure checklist

1. `cd "C:\Users\Siddesh\Desktop\DevOps Project"`
2. `aws sts get-caller-identity --profile order-inventory` → deployer user, account 038215676861
3. Run resume ≥ 45 minutes before the slot; wait for `ENVIRONMENT RESTORED AND VERIFIED`
4. Copy the Application URL and Jenkins URL into a text file
5. Open the app; the header shows **API Online**
6. Log in to Jenkins; watch the automatic build finish green
7. Start the Kibana port-forward and run `kibana-setup.sh`
8. Open `docs/evidence/aws-zero-downtime-2026-10-19.txt` and this PDF

1.7 Live demonstration plan

Duration 8–10 minutes. Never break the live system during the evaluation; show failure behaviour with the recorded evidence.

#	Time	Action	What to say
1	0:00	Show the high-level and AWS diagrams (Architecture).	"Browser → ALB → frontend and backend pods on EKS in private subnets → RDS in isolated subnets. Jenkins builds and deploys; Terraform created the infrastructure; Ansible configured Jenkins."
2	0:45	Open the ALB URL → Dashboard → Products.	"React UI served by nginx; it calls `/api` on the same origin; the ALB sends `/api` to Spring Boot."
3	1:30	Orders → **Create Order** (quantity 2) → Products: stock 2 lower → Orders → **Status** → **CANCELLED** → stock restored.	"The order locks the product rows inside one transaction, so two buyers cannot both take the last unit. Cancelling returns stock exactly once."
4	2:30	`curl -s -X POST $URL/api/orders -H "Content-Type: application/json" -d '{"userId":1,"items":[{"productId":1,"quantity":999999}]}'`	Point at `status 400`, the message and the `requestId`.
5	3:15	Jenkins → job → the automatic build (or **Build with Parameters**, DEPLOY ✓, FULL_SMOKE_TEST ✗).	Name the stages: tests, quality gate, Trivy, image build, ECR push, Helm deploy, verification, rollback.
6	4:30	`aws ecr describe-images --repository-name order-inventory-backend --query 'imageDetails[].imageTags[]' --output text`	"Tags are `<build>-<commit>`; the repository is IMMUTABLE."
7	5:00	`kubectl -n order-inventory get deploy,rs,pods,svc,ingress` and `helm -n order-inventory history order-inventory --max 5`	Deployment → ReplicaSet per version → pods; Helm revisions = rollback points.
8	6:00	`docs/evidence/aws-zero-downtime-2026-10-19.txt`	"9,532 requests through the real ALB during releases, failures and a restart: 0 errors."
9	7:00	CloudWatch alarm list; Kibana → Discover → search the `requestId` from step 4.	"One id links the error response to the backend log line, with pod metadata."
10	8:00	Suspend stages and cost table (next section).	"Suspended, not destroyed; one command restores and verifies it in about ten minutes."

1.8 Exact shutdown instructions

After the evaluation, from the repository root:

```
powershell -ExecutionPolicy Bypass -File scripts\suspend-evaluation.ps1 --plan   # read-only preview
powershell -ExecutionPolicy Bypass -File scripts\suspend-evaluation.ps1          # suspend
```

Git Bash: `bash scripts/suspend-evaluation.sh --plan` and `bash scripts/suspend-evaluation.sh`.

Measured on 2026-10-19 (twice): about **23 minutes**, of which about **17 minutes** (1,027–1,028 s) is EKS terminating the two worker nodes. *In general*, a managed node group drains each node and waits for pod evictions up to about 15 minutes; the exact pod that held the drain was not recorded. Success ends with `SUSPENDED. Restore with: bash scripts/resume-evaluation.sh`.

Order	Action	Why in this order
1	Write `docs/recovery/evaluation-manifest.json` (running tag, Helm revision, node-group sizes, IDs; no secrets)	resume restores exactly this release
2	`helm upgrade --reuse-values --set ingress.enabled=false`; wait until the ALB and target groups are gone; delete the 4 ALB alarms	the controller that deletes the ALB runs **on the nodes** — must happen before they go
3	Scale backend/frontend and Elasticsearch/Kibana to 0	clean node drain; no PersistentVolumes exist
4	Node group min 0 / desired 0 (max kept); wait for 0 instances	Terraform ignores `desired_size`, and `node_min_size = 0` allows it
5	Stop Jenkins EC2	Jenkins home, jobs, plugins and SonarQube volumes stay on the encrypted 30 GB root EBS volume
6	Stop RDS	storage, automated backups and point-in-time recovery are kept
7	Verify cluster, images, secrets, state object, PITR window, no ALB	proof that restore is possible

Charges while suspended:

State	Resources
Stopped / removed (no compute charge)	worker nodes (0), application ALB and target groups (deleted, recreated by resume), Jenkins EC2 (stopped), RDS (stopped), app and ELK pods (0 replicas), 4 ALB alarms (deleted, recreated)
Still billed	EKS control plane (≈ $0.10/h, cannot be paused), NAT gateway (≈ $0.056/h) and its Elastic IP, Jenkins EBS volume, RDS storage and backups, Secrets Manager (3), KMS key, 9 CloudWatch alarms, ECR images, S3 (state + CI reports)
Preserved data	RDS database + 7-day PITR, Jenkins configuration and history, SonarQube data, ECR images, Terraform state, secrets, KMS key
Lost on purpose	Elasticsearch logs (demo `emptyDir`)

Estimated suspended cost: ≈ $126/month ≈ $4.15/day (estimate from list prices, 2026-10-19).

1.9 Common errors and safe recovery

`wrong AWS account (...)` / `refusing to run as the root user`	set `$env:AWS_PROFILE='order-inventory'` (PowerShell) or `export AWS_PROFILE=order-inventory`
`terraform init failed (S3 backend reachable?)`	check internet and credentials; `aws s3api head-bucket --bucket order-inventory-tfstate-038215676861`
`cannot reach the Kubernetes API from this machine`	your IP changed: run **resume** (it updates the allowed IP)
`timed out ... RDS available`	`aws rds describe-events --source-identifier order-inventory-mysql --source-type db-instance --duration 120`; re-run
`timed out ... node(s) Ready`	`kubectl get nodes`; `kubectl get events -A --sort-by=.lastTimestamp`; node group health; NAT gateway present; re-run
`ECR image ... not found`	choose an existing tag, set `lastKnownGoodTag` in the manifest, re-run
`Helm deployment of <tag> failed (deploy-k8s.sh rolled back if possible)`	`kubectl -n order-inventory get pods,events`; `kubectl -n order-inventory logs deploy/backend --tail=50`; re-run
waiting on `ALB active` / targets / `api/health returns 200`	new DNS can take minutes; `kubectl -n kube-system logs deploy/aws-load-balancer-controller --tail=50`
`an application ALB exists but the cluster has no nodes ... resume first` (suspend)	run resume, then suspend again
`RDS is '<state>'; it can only be stopped from 'available'` (suspend)	wait until RDS finishes its operation; re-run
Script interrupted (laptop closed, network lost)	run the same script again — idempotent
RDS `available` while everything else is suspended	AWS auto-started it after 7 days: run suspend again

CHAPTER 2

Project overview

2.1 In simple English

A small company sells products and needs to keep track of **customers, products in stock and orders**. When an order is placed, the stock must go down; when an order is cancelled, the stock must come back; it must never be possible to sell more items than exist. That is the application.

The real focus of the project is **how the application is built, tested, secured, deployed, monitored and operated**: every change goes through an automated pipeline that tests it, checks code quality and security, packages it into containers, stores them in a registry and rolls them out to a Kubernetes cluster on AWS **without any downtime**, rolling back by itself if the new version is unhealthy. All cloud infrastructure is written as code (Terraform), the build server is configured as code (Ansible + Jenkins Configuration as Code), and the environment can be suspended and restored with one command to control cost.

2.2 Title, problem, motivation

Item	Content
Title	Production-Grade Microservices Platform with Zero-Downtime Deployment — Order & Inventory Management System
Problem statement	Manual deployments of business applications cause downtime, configuration drift, unreviewed security risk and unpredictable cost. Inventory systems additionally need correctness under concurrent orders.
Motivation	Learn and demonstrate the full path from source code to a monitored, secured cloud deployment, with evidence for each claim (tests, scans, measured zero-downtime runs).

2.3 Objectives (and where each is met)

Objective	Implementation	Status
Correct order/inventory logic under concurrency	pessimistic row locks in `OrderService.createOrder` + DB `CHECK` constraints; `OrderConcurrencyTest` (8 buyers, 3 units)	VERIFIED (H2 test; mutation check documented in `docs/TESTING.md`)
Automated CI/CD from GitHub to EKS	`Jenkinsfile`, Jenkins on EC2 polling GitHub	VERIFIED (builds #7, #8)
Quality and security gates	SonarQube quality gate, Trivy dependency/IaC/secret/image gates	VERIFIED (builds #3 failure = gate working, #4/#7/#8 pass)
Zero-downtime deployment with automatic rollback	Helm `--wait`, `maxUnavailable: 0`, readiness probes, preStop, graceful shutdown, `scripts/deploy-k8s.sh` rollback	VERIFIED (9,532 requests, 0 errors)
Infrastructure as code	Terraform (VPC, EKS, RDS, ECR, IAM, KMS, S3, monitoring, Jenkins EC2)	VERIFIED (apply + no-drift plans)
Configuration management	Ansible roles for the Jenkins host, SonarQube, JCasC	VERIFIED (idempotent re-runs `changed=0`)
Monitoring, alerting, central logs	CloudWatch alarms → SNS → SQS; Budgets; ELK on EKS	VERIFIED except budget/email notifications
Cost control	suspend/resume scripts	VERIFIED (2026-10-19)

2.4 Scope

In scope: one web application (React UI + Spring Boot REST API + MySQL), local Docker Compose and minikube environments, one AWS environment (`dev`) in `ap-south-1`, single Jenkins controller, single NAT gateway, HTTP only. Out of scope: user authentication/authorisation (the login page only navigates), HTTPS/domain, multi-region, autoscaling policies (HPA / Cluster Autoscaler), payment, email delivery of alerts.

2.5 Main features

Application: dashboard KPIs (users, products, orders, revenue excluding cancelled, low-stock list), product CRUD with computed stock status (`IN_STOCK` > 10, `LOW_STOCK` 1–10, `OUT_OF_STOCK` ≤ 0), user creation with case-insensitive unique email, multi-line orders with atomic stock reservation, status changes with stock restoration on cancel, consistent JSON errors with a request id, liveness and readiness endpoints.

Platform: immutable image tags, two quality/security gate types, Helm releases with history, automatic rollback, readiness-gated traffic, PodDisruptionBudgets, KMS-encrypted Kubernetes Secrets, private RDS with TLS-only connections, namespace-scoped Jenkins permissions, CloudWatch alarms with verified delivery, budget guardrail, central logs with request tracing, one-command suspend/resume.

2.6 Technology stack (actual versions from the repository)

Layer	Technology	Where
Frontend	React 18.3, React Router 7, Vite 8, Vitest 5, ESLint 9; served by nginx 1.30 (unprivileged)	`order-inventory-platform/frontend/package.json`, `Dockerfile`
Backend	Java 17, Spring Boot 4.1.1 (Web MVC, Data JPA, Validation, Flyway), Hibernate, HikariCP, JaCoCo	`backend/pom.xml`
Database	MySQL 8.4 (Compose: `mysql:8.4`; AWS: RDS MySQL 8.4.11 db.t3.micro); H2 for tests	`docker-compose.yml`, `terraform/main.tf`
Containers	Docker multi-stage builds, Docker Compose	`*/Dockerfile`, `docker-compose*.yml`
Orchestration	Amazon EKS 1.36, managed node group 2× t3.medium, Helm chart, AWS Load Balancer Controller 3.6.0	`terraform/main.tf`, `helm/order-inventory/`, `scripts/install-lb-controller.sh`
IaC	Terraform 1.15.8, AWS provider ~> 5.0, S3 backend with native locking	`terraform/`
Config management	Ansible core 2.19 (run in a container), roles + JCasC	`ansible/`
CI/CD	Jenkins LTS on EC2 (Ubuntu 24.04), declarative `Jenkinsfile`, SCM polling	`Jenkinsfile`, `ansible/roles/jenkins*`
Quality / security	SonarQube 26.10 Community, Trivy 0.76, gitleaks (manual scans)	`Jenkinsfile`, `.trivyignore.yaml`, `.gitleaksignore`
AWS	VPC, NAT, ECR, EKS, RDS, ALB, EC2, IAM/IRSA, KMS, Secrets Manager, S3, CloudWatch, SNS, SQS, Budgets	`terraform/*.tf`
Logging	Elasticsearch/Kibana/Filebeat 8.19.5, ECS JSON logs	`k8s/logging/elk.yaml`, `order-inventory-platform/logging/`

2.7 Why this is a DevOps project, not only a CRUD application

The application code is deliberately modest: of the 180 tracked files, 71 are application source and 109 are delivery, operations and documentation: pipeline (`Jenkinsfile`), infrastructure (`terraform/`), configuration management (`ansible/`), packaging (`Dockerfile`s, Helm chart), operations (`scripts/`), observability (`k8s/logging`, `terraform/monitoring.tf`) and evidence (`docs/evidence`). The evaluated properties — zero downtime, automatic rollback, gated releases, reproducible infrastructure, least privilege, cost control — are properties of the platform, and each is backed by an executed test.

2.8 What makes it production-style

- Immutable, traceable artefacts (`<build>-<commit>` tags, IMMUTABLE ECR, OCI revision labels).
- Gates that actually stopped bad builds (build #3 failed on 5 HIGH IaC findings and was fixed).
- Health model separating liveness from readiness; graceful shutdown and drain timing.
- Secrets generated by machines, stored in Secrets Manager, KMS-encrypted in Kubernetes, never in Git.
- Private data tier, TLS-enforced DB connections, IMDSv2, non-root read-only containers.
- Remote, encrypted, versioned, locked Terraform state; drift checks after changes.
- Runbooks, alarms with verified delivery, recorded evidence for every claim.

2.9 "Microservices": an accurate description

The **application** is **not** a set of independent microservices. It is a **single Spring Boot service** (a modular monolith with `users`, `products`, `orders` packages that share one MySQL schema and one transaction boundary — order creation and stock deduction happen in the same database transaction) plus a **separate frontend container** (static React files served by nginx). Two independently built, versioned, scaled and health-checked containers are deployed behind a path-routing load balancer.

The **platform** is microservice-ready: each service has its own image repository, Deployment, Service, probes, PDB and ALB target group, and the pipeline/Helm pattern would extend to more services. Say exactly this if asked; splitting orders and inventory into separate services would require distributed transactions or a saga pattern, which would weaken the overselling guarantee this project demonstrates.

2.10 Limitations and trade-offs (honest list)

Limitation	Reason / trade-off
No real authentication; login page only navigates	not in scope; do not expose real data
HTTP only	no domain/ACM certificate; documented as accepted risk
Single NAT gateway, single-AZ Jenkins, single SonarQube	cost; not highly available
RDS single-AZ db.t3.micro, `skip_final_snapshot = true`	cost / clean lab teardown; production would use Multi-AZ, deletion protection, final snapshot
ELK on `emptyDir`, security disabled, single node	demo-sized; logs lost on node replacement; access only via port-forward
Polling instead of webhook (up to 5-minute delay)	Jenkins accepts traffic only from the admin /32
Backend uses the RDS master user	simplicity; a least-privilege DB user is recommended

`ansible/run-playbook.sh` passes the two admin passwords to a local `node` process as arguments before streaming them to the container on stdin	brief local exposure in the workstation's process list (see Security)

2.11 Future improvements

HTTPS with ACM; real authentication (OIDC/Cognito); dedicated DB user with least privilege; Multi-AZ RDS with deletion protection; HPA + Cluster Autoscaler/Karpenter; GitHub webhook through a restricted ingress or GitHub Actions OIDC; External Secrets Operator instead of `create-db-secret.sh`; managed OpenSearch or CloudWatch Container Insights instead of in-cluster ELK; VPC endpoints (ECR, S3, STS) to reduce NAT cost; canary/blue-green with Argo Rollouts; SLO-based alarms and dashboards.

2.12 Component status matrix

Component	Status	Evidence
Backend tests (29) and coverage	VERIFIED	`docs/TESTING.md`; Jenkins builds #4, #7, #8 (`docs/evidence/jenkins-github-*.txt`)
Frontend lint, tests (21), build	VERIFIED	`docs/TESTING.md`, Jenkins builds
Docker images, non-root, Compose acceptance (8/8)	VERIFIED	`docs/TESTING.md`
Helm chart + minikube acceptance (rollout/rollback/outage)	VERIFIED	`docs/evidence/k8s-acceptance-2026-10-18.txt`
Terraform infrastructure	VERIFIED	`docs/AWS_DEPLOYMENT.md`; no-drift plan 2026-10-19
Terraform admin-IP update branch of resume	IMPLEMENTED; targeted plan VERIFIED 2026-10-25 (not applied)	Current state
Ansible configuration of Jenkins	VERIFIED	`docs/TESTING.md`, `docs/evidence/ansible-container-test-2026-10-18.txt`
Jenkins pipeline from GitHub (manual trigger)	VERIFIED	build #7 evidence
Jenkins pipeline triggered by a GitHub push (polling)	VERIFIED	build #8 evidence
GitHub webhook trigger	NOT IMPLEMENTED (by design)	`docs/TESTING.md` "Not yet verified"
SonarQube quality gate	VERIFIED	build evidence: `QUALITY GATE STATUS: PASSED`
Trivy gates	VERIFIED	build #3 failure, later passes
Zero downtime + automatic rollback on AWS	VERIFIED	`docs/evidence/aws-zero-downtime-2026-10-19.txt`
DB interruption behaviour on AWS	VERIFIED	`docs/evidence/aws-rds-interruption-2026-10-19.txt`
CloudWatch → SNS → SQS delivery	VERIFIED	`docs/TESTING.md`
AWS Budget notification	IMPLEMENTED BUT NOT VERIFIED (threshold never reached)	`docs/TESTING.md`

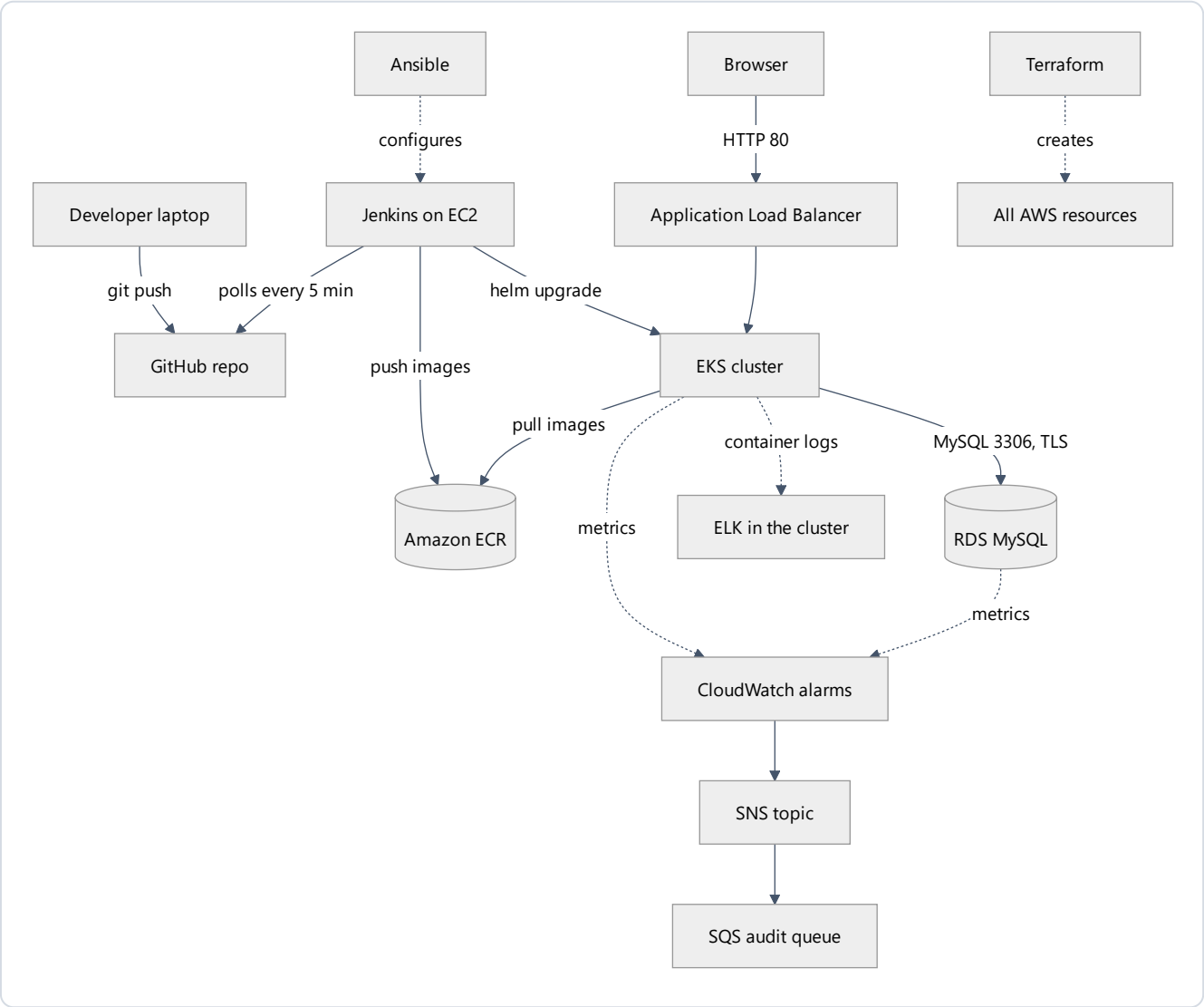
Kibana objects after resume	IMPLEMENTED BUT NOT VERIFIED	After the restore
Suspend / resume / idempotent re-run	VERIFIED	`docs/AWS_DEPLOYMENT.md`
PowerShell wrappers	VERIFIED for `suspend --plan`; resume wrapper IMPLEMENTED BUT NOT VERIFIED (resume ran via Git Bash)	2026-10-19 session
Git history secret removal	VERIFIED on a fresh clone; old SHAs on GitHub need Support purge (**BLOCKED** on GitHub Support)	`docs/SECURITY.md`
Root access key deactivation, MFA on admin user	BLOCKED (owner action)	`docs/SECURITY.md` "Current credential status"
HTTPS, authentication, autoscaling	NOT IMPLEMENTED	Limitations

CHAPTER 3

Architecture and flows

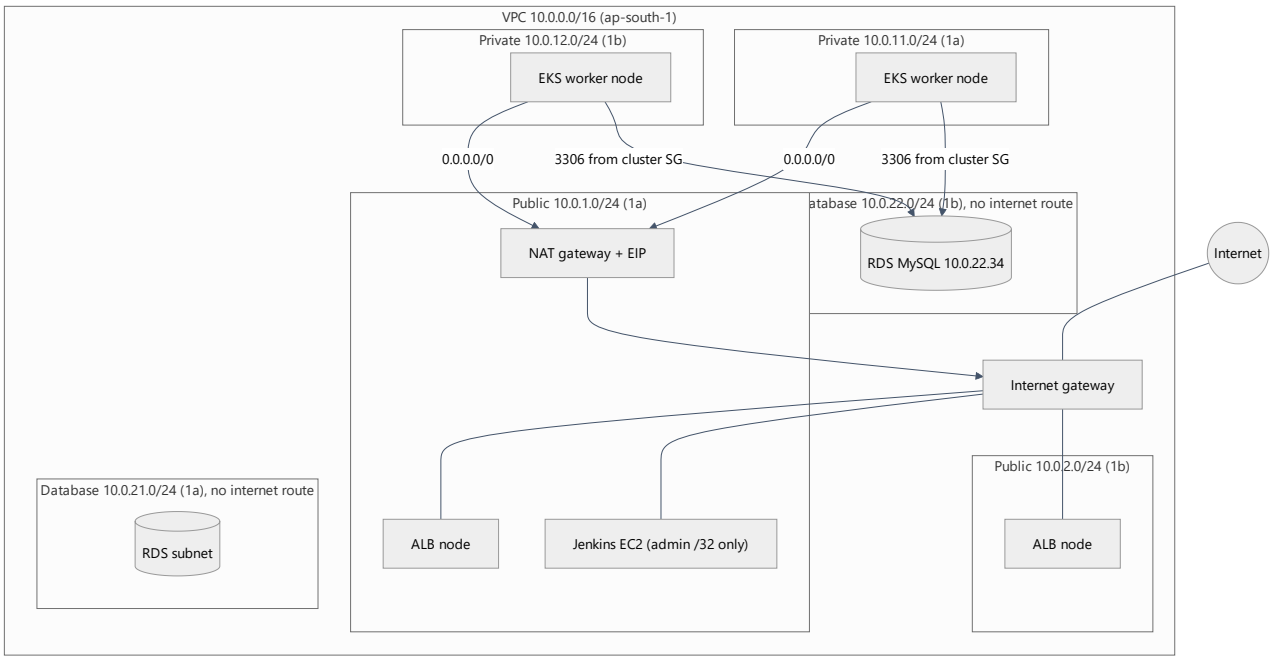
Every box in these diagrams corresponds to a real resource, configuration or file in this repository; nothing is drawn that the project does not implement. (The diagrams were written as Mermaid source and rendered for this document.)

3.1 High-level system architecture



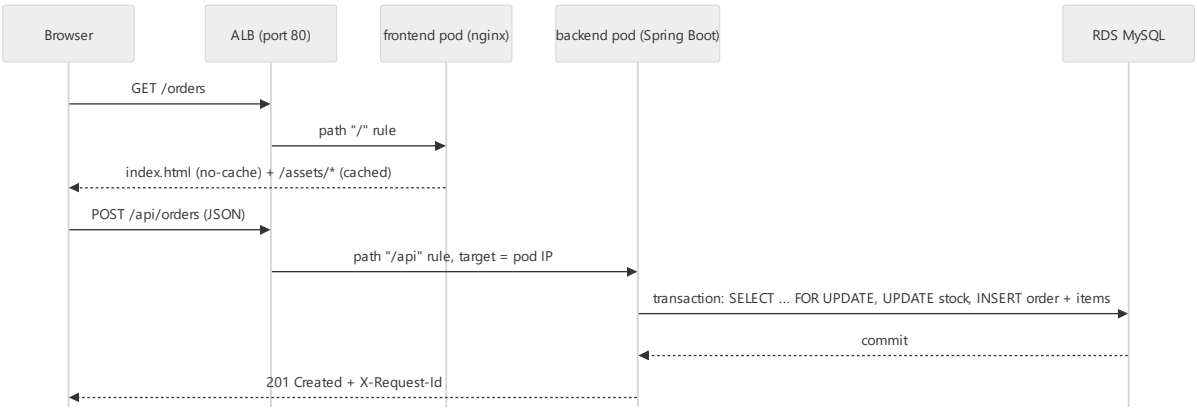
Plain English: developers push to GitHub; Jenkins notices, tests and packages the app, stores images in ECR and deploys them to EKS. Users reach the app through the ALB. The app stores data in RDS. CloudWatch watches the infrastructure and notifies through SNS; ELK keeps searchable logs. Terraform built the AWS side; Ansible configured Jenkins. Files: `Jenkinsfile`, `terraform/*.tf`, `ansible/`, `helm/order-inventory/`.

3.2 AWS VPC and network layout



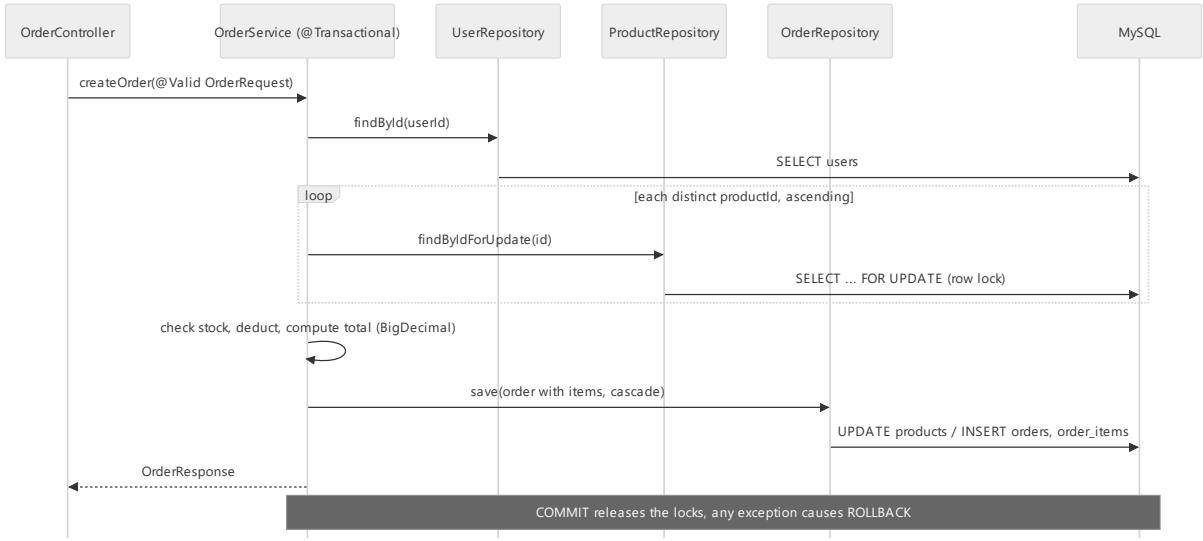
Route tables (`terraform/main.tf`): public → IGW; private → NAT; database → local only. Security groups: ALB SG (controller-managed, port 80), cluster SG (nodes/pods), RDS SG (3306 from cluster SG), Jenkins SG (22 and 8080 from the admin /32), control-plane SG (443 from Jenkins).

3.3 Application request flow (EKS)

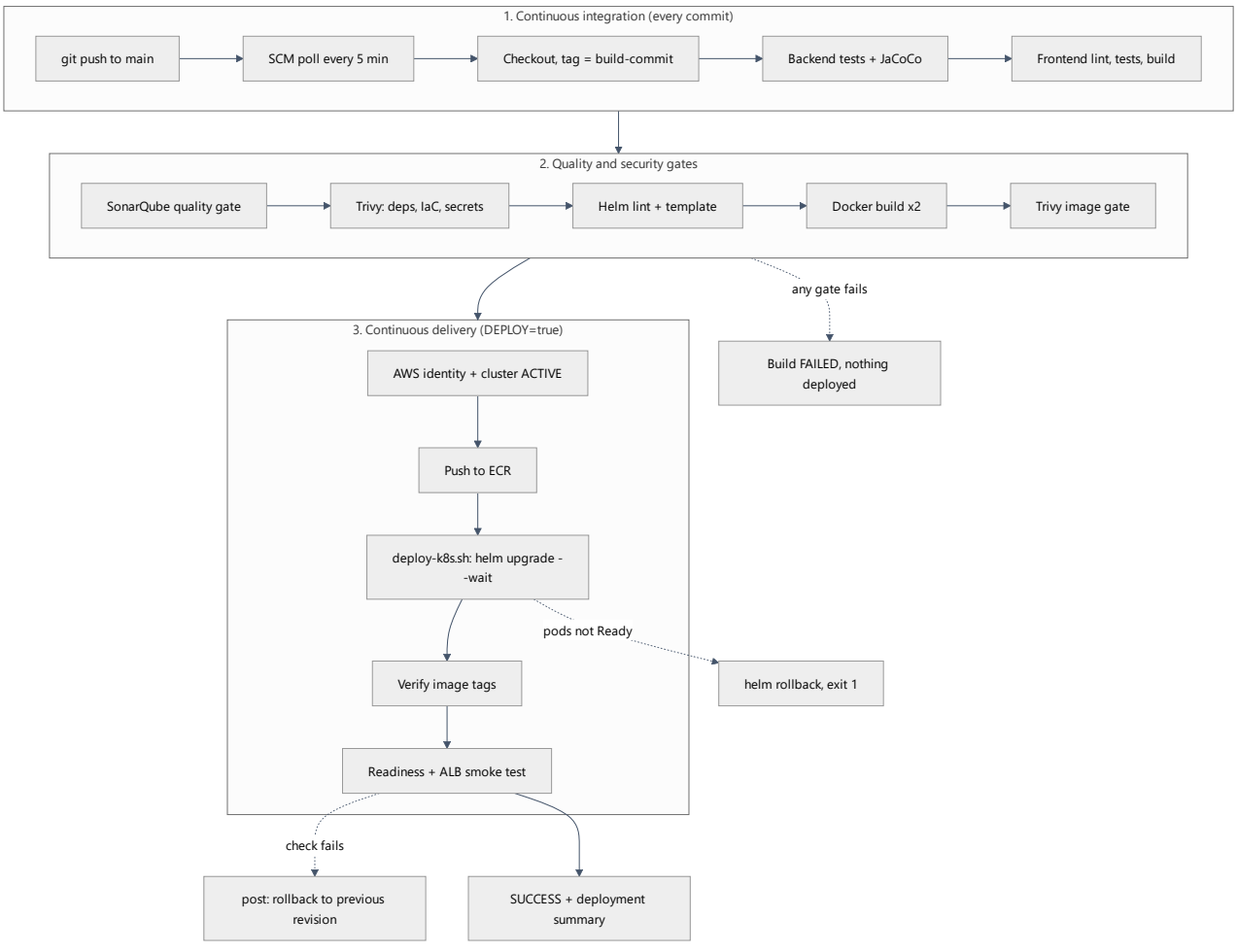


Files: `helm/order-inventory/templates/ingress.yaml`, `frontend/nginx/default.conf.template`, `OrderController.java`, `OrderService.java`.

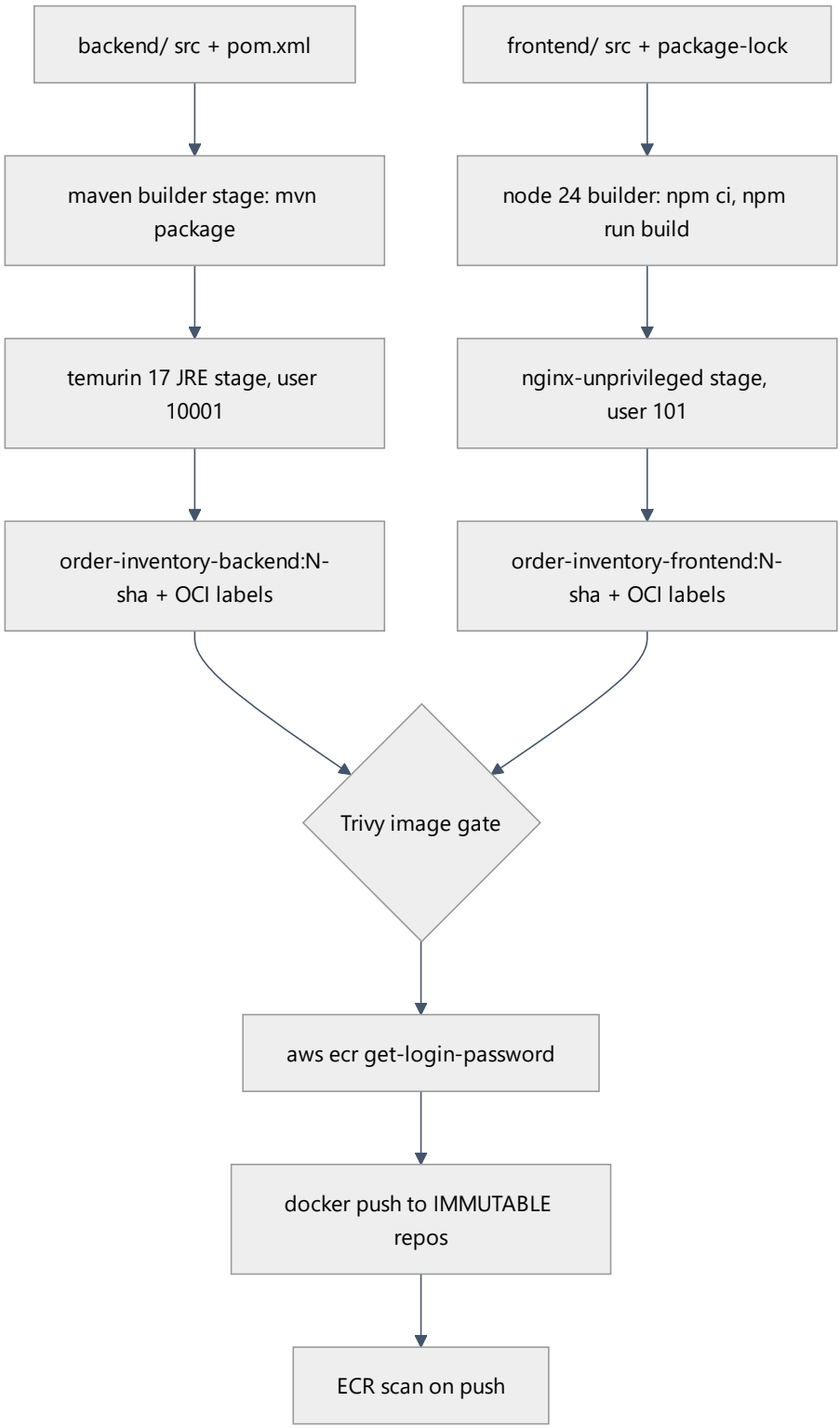
3.4 Database interaction (create order)



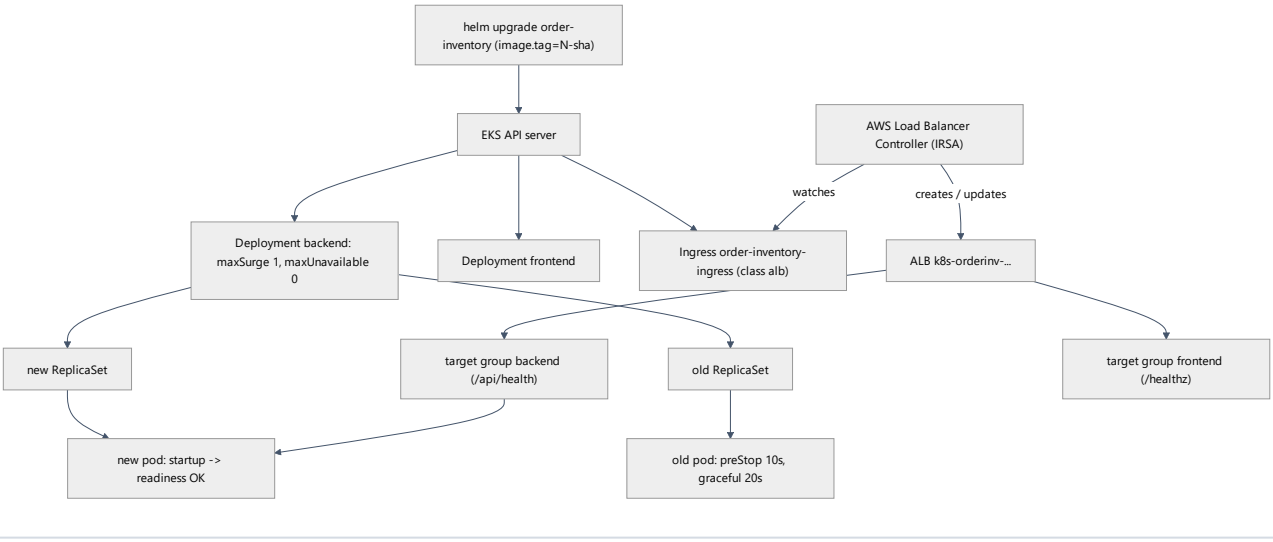
3.5 GitHub-to-Jenkins CI/CD pipeline



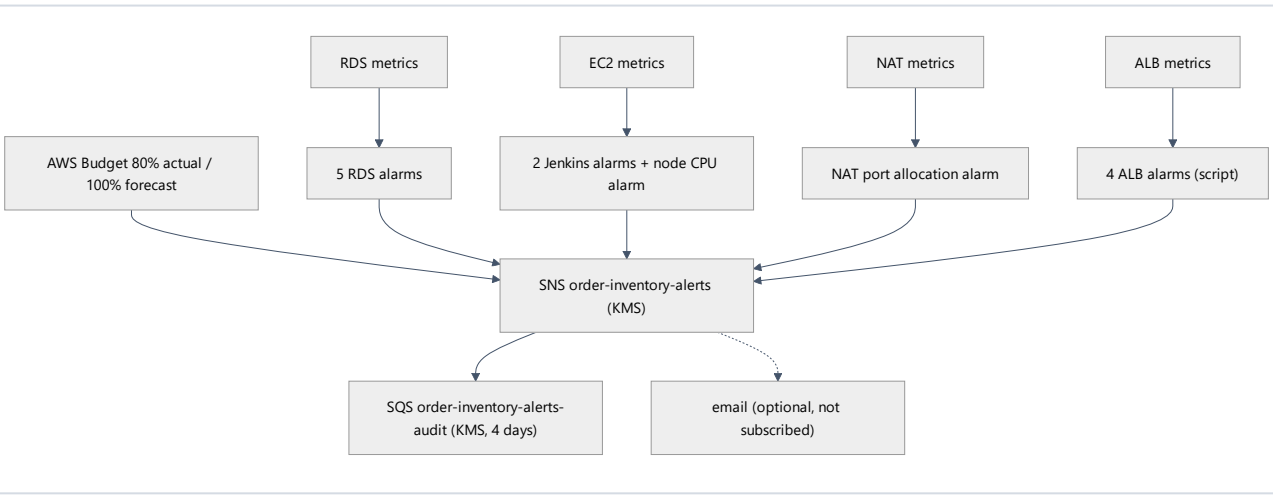
3.6 Docker image build and ECR publishing



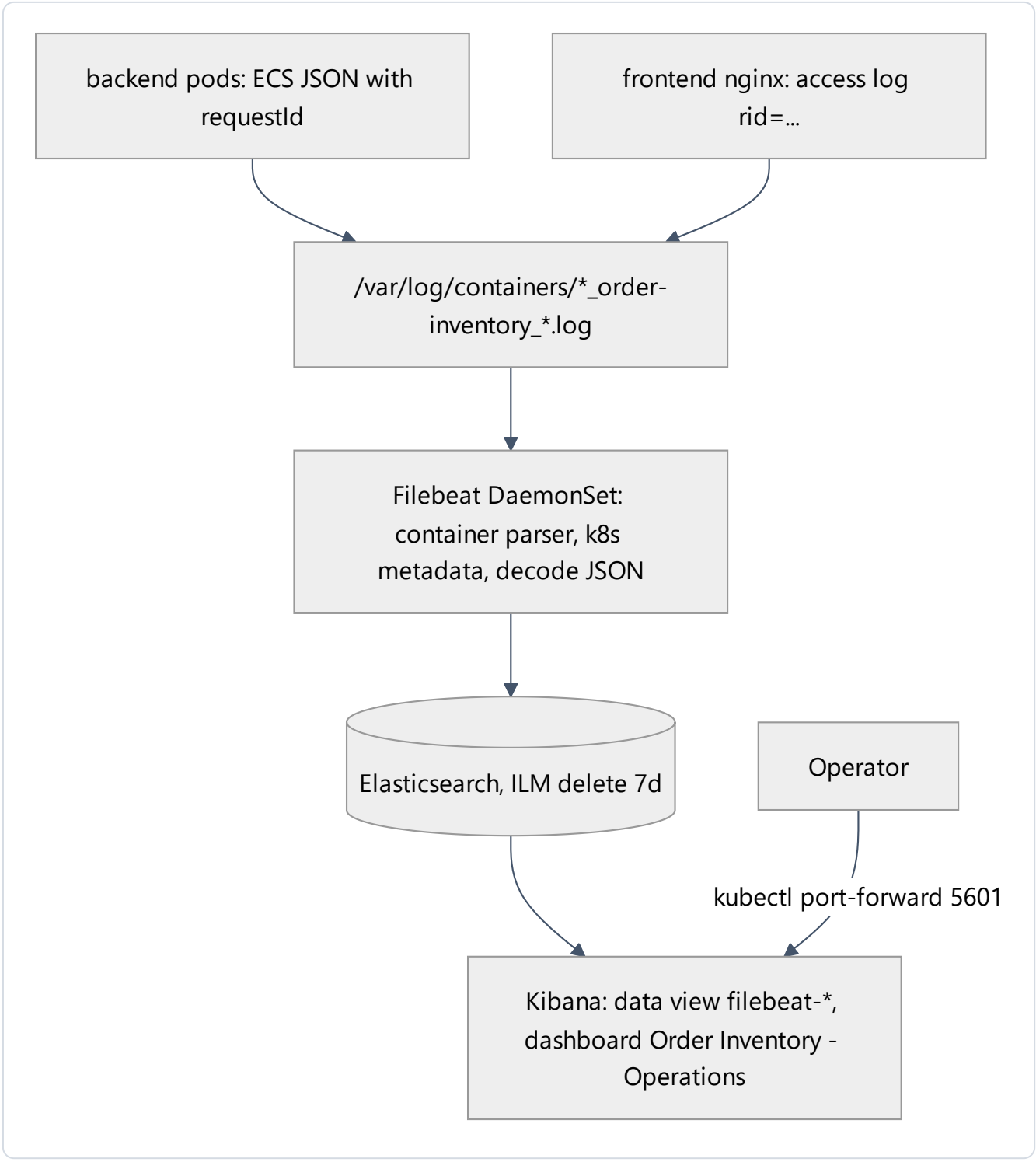
3.7 Helm deployment to EKS and ALB traffic



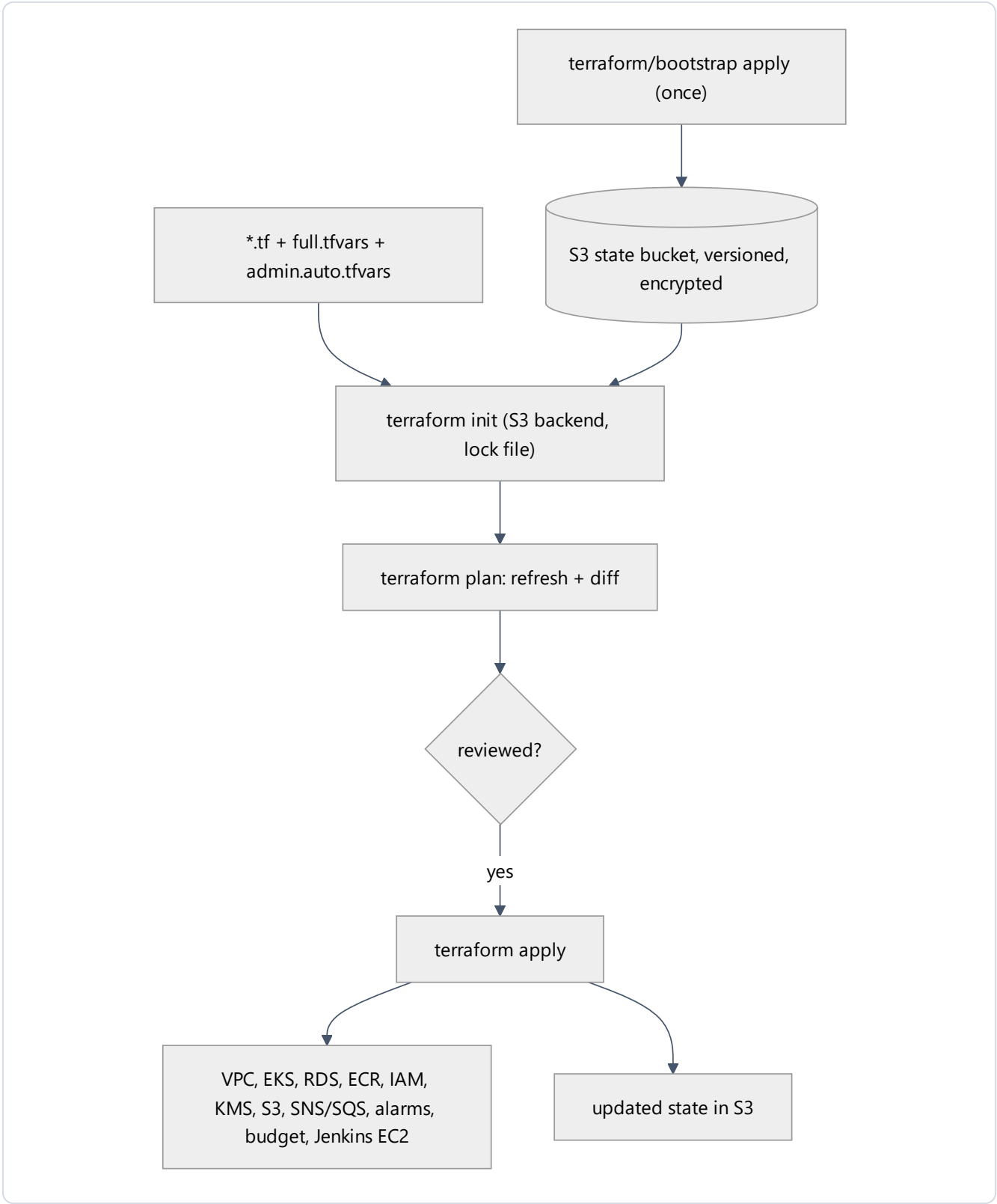
3.8 CloudWatch → SNS / SQS notification flow



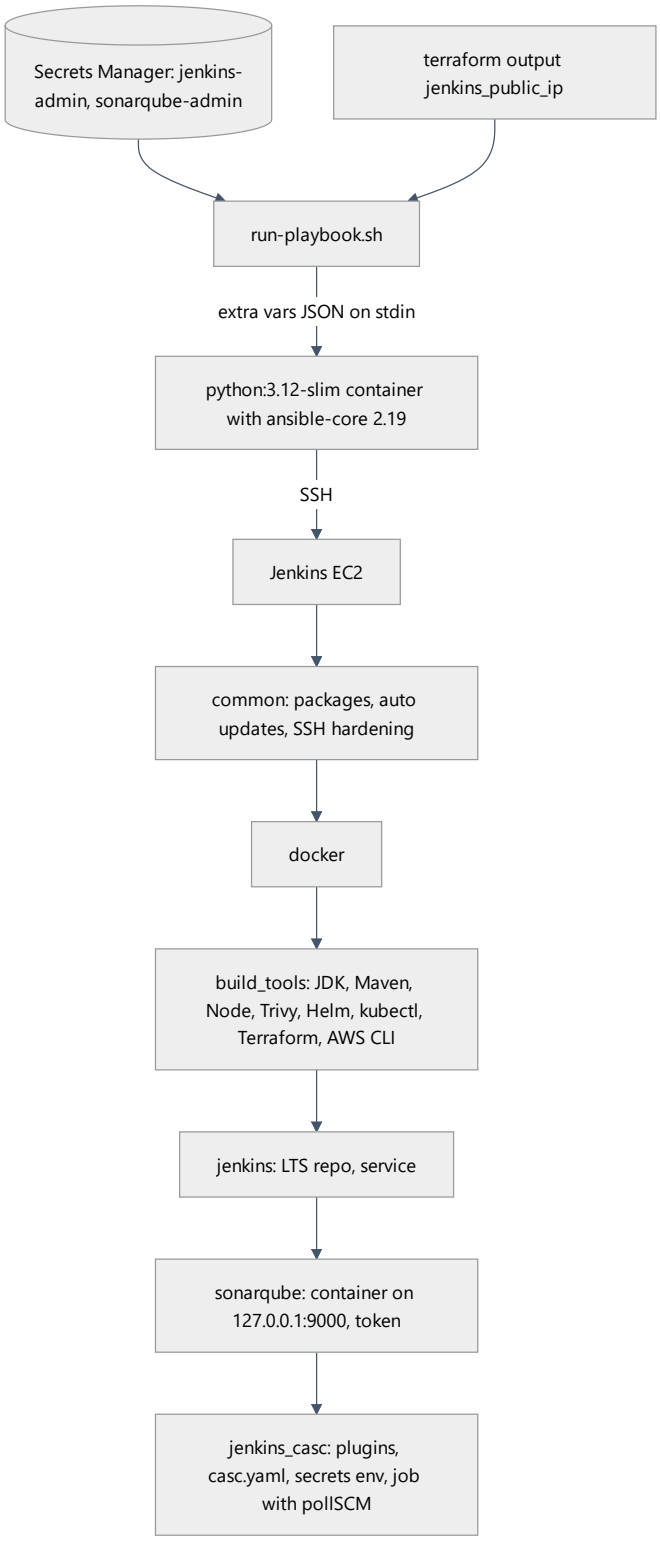
3.9 ELK logging flow



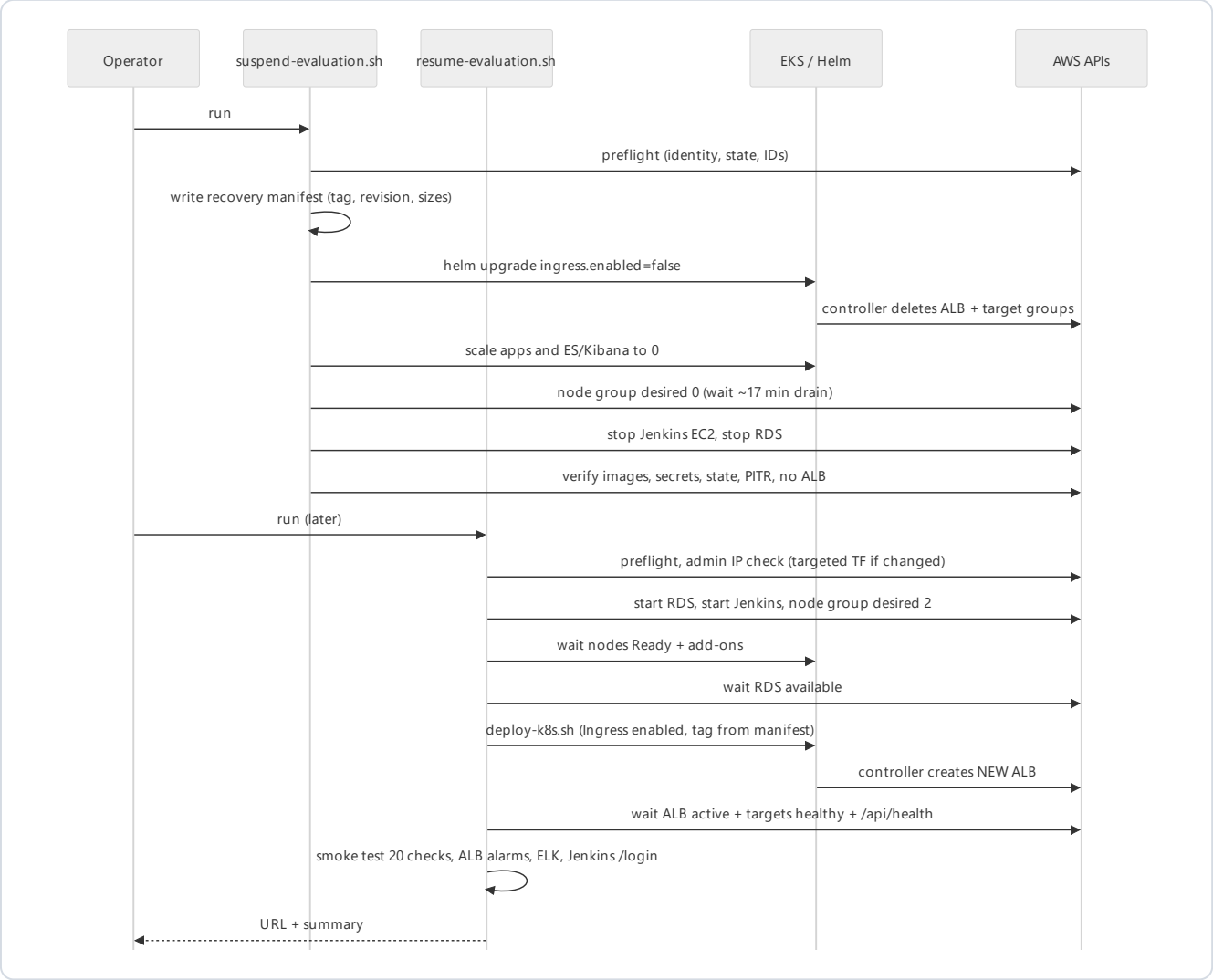
3.10 Terraform provisioning flow



3.11 Ansible configuration of Jenkins



3.12 Suspension and restoration sequence



CHAPTER 4

The role of every service

This chapter answers, for every technology the project really uses, the same twelve questions: what it is, why it is needed, its exact role here, what it talks to, which files implement it, how it behaves in a normal request or deployment, how it behaves in failure and recovery, why it was chosen over an alternative, security, cost, what would happen without it, and a viva question. Facts about this project come from the named files and evidence; sentences starting with *In general* are general knowledge.

Application versus platform. The *application* is two programs: a React single-page app served by nginx and one Spring Boot REST service, backed by one MySQL database. Everything else in this chapter — Docker, Jenkins, SonarQube, Trivy, ECR, EKS, Helm, the ALB, Terraform, Ansible, IAM, KMS, CloudWatch, ELK, the scripts — is the *platform* that builds, secures, deploys, observes and operates those two programs. The application is **not** composed of independent microservices: it is a frontend/backend architecture whose two containers are deployed as independent Kubernetes workloads (see Microservices: an accurate description).

4.1 React, JavaScript and Vite (frontend)

Aspect	In this project
What it is	React 18 is a JavaScript library for building user interfaces from components; Vite 8 is the build tool and dev server that bundles the JSX into static files.
Why needed	Operators need a browser UI to manage users, products and orders.
Role here	Single-page app with Dashboard, Products, Orders, Users and a demo Login page; all data comes from `/api` with `fetch`.
Talks to	the backend REST API (same-origin `/api`); served by nginx in the frontend container.
Files	`order-inventory-platform/frontend/src/**` (pages, components, `services/api.js`, `hooks/useToasts.js`), `vite.config.js`, `package.json`, `eslint.config.js`, `index.html`
Normal operation	Browser loads `index.html` + hashed JS/CSS once; each page calls `api.js` (JSON, 15 s timeout); after every change it reloads data from the server.
Failure / recovery	Backend down → "Unable to connect to backend server..." and header badge "API Offline"; 4xx/5xx → the backend's message in a toast; 503 → "database temporarily unavailable"; no crash, recovers on the next request.
Why chosen	React: component model and ecosystem (React Router, Testing Library). Vite instead of Create React App: CRA is no longer maintained, Vite is much faster and shares config with Vitest.
Security	No secrets in the bundle (everything in it is public); no authentication (documented limitation); runs behind nginx security headers.
Cost	Static files; served by two small pods (100m CPU / 128Mi requests each).
If removed	The API would still work (curl), but there would be no usable UI.
Viva	*How does the browser find the backend?* — "It doesn't need to: it calls `/api` on the same host; the ALB routes `/api` to the backend."

4.2 nginx (frontend web server and reverse proxy)

Aspect	In this project
What it is	A high-performance HTTP server and reverse proxy.
Why needed	Somebody must serve the built static files, handle SPA deep links and (outside EKS) forward `/api`.
Role here	Runs inside the frontend image as non-root UID 101 on port 8080; serves `/usr/share/nginx/html`; `/healthz` probe endpoint; SPA fallback; caching rules; `/api/` proxy with `X-Request-Id`.
Talks to	browsers / the ALB; the backend Service (`BACKEND_URL`) when proxying
Files	`frontend/Dockerfile` (stage 2), `frontend/nginx/default.conf.template`, `frontend/nginx/security-headers.conf`, Helm `templates/frontend.yaml` (`BACKEND_URL`)
Normal operation	ALB sends `/` paths to nginx; nginx returns `index.html` (no-cache) or `/assets/*` (cached 1 year). In Docker Compose it also proxies `/api` to `backend:8080`.
Failure / recovery	`/healthz` does not depend on the backend, so the UI stays up when the backend is down; DNS is resolved per request (`resolver … valid=10s`) so nginx survives backend restarts.
Why chosen	Small, fast, standard; the unprivileged official image avoids root. Alternative: serving files from Spring Boot — would couple UI releases to backend releases.
Security	non-root, `server_tokens off`, `nosniff`, `X-Frame-Options DENY`, `Referrer-Policy`; capabilities dropped in Kubernetes.
Cost	negligible.
If removed	No way to serve the SPA; deep links would 404.
Viva	*Why does refreshing `/orders` work?* — "nginx's `try_files … /index.html` returns the SPA for unknown paths; React Router renders the page."

4.3 Java and Spring Boot (backend)

Aspect	In this project
What it is	Java 17 with Spring Boot 4.1.1: a framework that auto-configures a web server (Tomcat), JSON, JPA/Hibernate, validation and Flyway.
Why needed	Business rules (stock reservation, cancellation, email uniqueness) must run on a trusted server.
Role here	REST API for users, products, orders and health; one transaction per business operation; central error mapping; request-id correlation; ECS JSON logs.
Talks to	MySQL/RDS over JDBC (HikariCP pool); receives HTTP from the ALB or nginx; logs to stdout (Filebeat).
Files	`backend/src/main/java/com/example/orderinventory/**`, `application.properties`, `pom.xml`, `backend/Dockerfile`
Normal operation	`RequestIdFilter` → controller (`@Valid`) → `@Transactional` service → Spring Data repository → MySQL → DTO → JSON.
Failure / recovery	DB unreachable → 503 (fast, 5 s pool timeout) and readiness `DOWN`; unexpected error → generic 500, stack trace only in logs; SIGTERM → graceful shutdown (20 s); OOM → JVM exits and Kubernetes restarts it.
Why chosen	Mature, strongly typed, excellent transaction and validation support. Alternative: Node.js/Express — less built-in support for declarative transactions and row locking.
Security	input validation, no internal details in errors, request ids sanitised against log injection, non-root read-only container, DB credentials only from a Secret.
Cost	Two pods (250m CPU / 512Mi requests, 1Gi limit each).
If removed	No API, no business rules.
Viva	*What does `@Transactional` give you?* — "All-or-nothing database work: an order and its stock deduction commit together or not at all."

4.4 REST API

Aspect	In this project
What it is	HTTP + JSON interface using resource URLs and methods (GET, POST, PUT, DELETE) and status codes.
Why needed	A clear contract between the UI, scripts/tests and the backend.
Role here	`/api/health`, `/api/health/ready`, `/api/users`, `/api/products`, `/api/orders`, `/api/orders/{id}/status`; documented in `docs/API.md`.
Talks to	frontend (`api.js`), `scripts/smoke-test.sh`, Jenkins checks, Kubernetes probes, ALB health checks.
Files	the four controllers, DTOs, `GlobalExceptionHandler.java`, `ErrorResponse.java`
Normal operation	200/201/204 with JSON; every response has `X-Request-Id`.
Failure / recovery	400 validation/business rule, 404 missing, 409 conflict or lock timeout ("retry"), 503 DB unavailable, 500 generic.
Why chosen	Simple, cacheable, tool-friendly; GraphQL/gRPC would add complexity without benefit here.
Security	no stack traces or SQL in responses (tested); CORS configurable; no authentication (limitation).
Cost	—
If removed	Nothing could use the backend.
Viva	*Why 409 for deleting a product that has orders?* — "The request conflicts with existing data: a foreign key forbids it."

4.5 MySQL

Aspect	In this project
What it is	Relational database (MySQL 8.4 LTS, InnoDB engine with transactions and row locks).
Why needed	Durable, consistent storage with constraints and transactions for inventory.
Role here	Database `order_inventory_db` with tables `users`, `products`, `orders`, `order_items`; locally in a container, on AWS as RDS.
Talks to	the backend only.
Files	Flyway migrations `db/migration/V1__…`, `V2__…`; `docker-compose.yml` (local); `terraform/main.tf` (RDS)
Normal operation	Backend opens a transaction, locks product rows `SELECT … FOR UPDATE`, updates stock, inserts the order.
Failure / recovery	Lock waits serialise concurrent orders; constraint violations become 409; outages become 503 and automatic recovery when it returns.
Why chosen	Relational integrity (FKs, CHECK, UNIQUE) fits orders and stock; MySQL 8.4 is LTS (8.0 would incur RDS extended-support charges). Alternative: DynamoDB — no multi-row ACID transactions with simple SQL constraints.
Security	app account (local) / master user (RDS — documented limitation); TLS required on RDS; private network.
Cost	local: free; RDS: see the RDS section.
If removed	No persistence.
Viva	*How is overselling prevented in the database?* — "Row locks plus `CHECK (stock_quantity >= 0)`."

4.6 Flyway (database migrations)

Aspect	In this project
What it is	A schema migration tool that applies versioned SQL scripts in order and records them in `flyway_schema_history`.
Why needed	The schema must change in a controlled, repeatable way across tests, local, minikube and RDS.
Role here	Runs at backend start-up before Hibernate; V1 creates tables, V2 adds indexes and CHECK constraints; `R__demo_data.sql` seeds local demo data only when `users` is empty.
Talks to	the database; Spring Boot starts it.
Files	`src/main/resources/db/migration/*.sql`, `db/seed/R__demo_data.sql`, `application.properties` (`spring.flyway.*`), `docker-compose.yml` (`SPRING_FLYWAY_LOCATIONS`)
Normal operation	On RDS after resume: "Successfully validated 2 migrations … Schema is up to date" (resume log 2026-10-19).
Failure / recovery	A modified applied migration fails validation → the app does not start (protects history); two pods starting together are coordinated by Flyway's lock (observed on RDS).
Why chosen	Plain SQL, simple; alternative Liquibase (XML/YAML changelogs) is heavier; Hibernate `ddl-auto=update` is unsafe for production (it is set to `validate`).
Security	migrations are code-reviewed in Git.
Cost	—
If removed	Schema changes would be manual and drift between environments.
Viva	*How do you add a column?* — "A new `V3__…sql` file, backward compatible for one release; never edit an applied migration."

4.7 Docker

Aspect	In this project
What it is	Container build and runtime: packages an app with its runtime into an image and runs it as an isolated process.
Why needed	Identical artefacts across laptop, CI and Kubernetes.
Role here	Multi-stage builds of the backend (Maven → Temurin 17 JRE, UID 10001) and frontend (Node 24 → nginx-unprivileged, UID 101); runs the local stack, local CI and the Ansible controller container.
Talks to	ECR (push), Trivy (scan), Kubernetes (pulls by tag).
Files	`backend/Dockerfile`, `frontend/Dockerfile`, both `.dockerignore`, `ci/jenkins/Dockerfile`
Normal operation	Jenkins `docker build --pull` with OCI labels and tag `<build>-<commit>`; Trivy scans; push to ECR.
Failure / recovery	Build failure stops the pipeline; health checks restart unhealthy containers in Compose.
Why chosen	Industry standard; Kubernetes runs OCI images. Alternative: Buildpacks/Jib — less transparent for teaching.
Security	non-root numeric users, `apk upgrade`, minimal runtime images, secrets excluded from build context.
Cost	ECR storage (~150 MB, lifecycle keeps 20 images).
If removed	No portable artefact for EKS.
Viva	*Why `exec` in the Java entrypoint?* — "So the JVM is PID 1 and gets SIGTERM for graceful shutdown."

4.8 Docker Compose

Aspect	In this project
What it is	A tool to run multi-container applications from one YAML file.
Why needed	Fast local development and acceptance testing without AWS.
Role here	`mysql` (named volume, healthcheck) → `backend` (waits for healthy MySQL, readiness healthcheck) → `frontend` (waits for healthy backend, host port 3000); optional ELK overlay; local CI (Jenkins + SonarQube).
Talks to	Docker engine; containers talk via service names on a bridge network.
Files	`order-inventory-platform/docker-compose.yml`, `docker-compose.logging.yml`, `.env.example`, `ci/docker-compose.ci.yml`
Normal operation	`docker compose up -d --build` → http://localhost:3000.
Failure / recovery	`restart: unless-stopped` restarts crashed containers (verified: restart count 0 → 1); DB outage → 503 then automatic recovery (verified, 8/8 acceptance checks).
Why chosen	Simplest local orchestration; minikube is used separately for Kubernetes-specific tests.
Security	local-only default passwords overridable via `.env` (git-ignored); ELK ports bound to 127.0.0.1.
Cost	free (local).
If removed	Local testing would need Kubernetes or AWS.
Viva	*What survives `docker compose down`?* — "The named MySQL volume; only `down -v` deletes it."

4.9 Git and GitHub

Aspect	In this project
What it is	Git: distributed version control. GitHub: hosted repository `SamruddhiDoke/DevOps-Project`.
Why needed	Source of truth for code, infrastructure, pipeline and docs; trigger for CI.
Role here	Jenkins polls `main` every 5 minutes and builds the new commit; the commit hash becomes part of the image tag.
Talks to	Jenkins (outbound polling from Jenkins).
Files	`.gitignore`, `.gitattributes` (LF for scripts), `.gitleaksignore`, `CONTRIBUTING.md`
Normal operation	push → poll → build (verified: build #8 "Started by an SCM change").
Failure / recovery	GitHub unreachable → checkout fails, nothing deployed; any commit can be redeployed by tag.
Why chosen	Standard; GitHub hosting. Polling instead of a webhook because Jenkins accepts traffic only from the admin IP.
Security	history was rewritten with `git filter-repo` to remove a leaked (already destroyed) password and pushed with `--force-with-lease` on owner approval; old SHAs still need a GitHub Support purge (BLOCKED on GitHub).
Cost	free.
If removed	No versioning, no CI trigger.
Viva	*Why is removing a secret in a new commit not enough?* — "Old commits still contain it; rewrite history and rotate the credential."

4.10 Jenkins (CI/CD server)

Aspect	In this project
What it is	An automation server that runs pipelines defined in a `Jenkinsfile`.
Why needed	Every change must be built, tested, checked and deployed the same way, automatically.
Role here	Runs on EC2 `i-03288b455e674765e` (t3.large, Ubuntu 24.04), job `order-inventory`: 16 stages from checkout to deployment summary with automatic rollback; configured entirely as code (Ansible + JCasC).
Talks to	GitHub (poll), SonarQube (127.0.0.1:9000), Docker, Trivy, ECR, EKS private API, S3 (reports).
Files	`Jenkinsfile`, `scripts/deploy-k8s.sh`, `scripts/smoke-test.sh`, `ansible/roles/jenkins*`, `templates/casc.yaml.j2`, `terraform/jenkins.tf`
Normal operation	See the Jenkins chapter: one commit → tested, scanned images `N-sha` → Helm rolling update → verified through the ALB.
Failure / recovery	Any gate fails → build fails, nothing deployed; deploy not Ready → `helm rollback` (verified); restart mid-build (#6) → rollback by the `post` block; stop/start keeps all data on EBS.
Why chosen	Self-hosted, plugin ecosystem, pipeline as code, no per-minute cost. Alternative: GitHub Actions — easier but needs either OIDC to AWS or stored keys, and runs outside the VPC (no private EKS endpoint).
Security	no stored AWS keys (instance role, IMDSv2); admin-only network access; admin password generated into Secrets Manager; Kubernetes rights limited to one namespace; `disableConcurrentBuilds`.
Cost	t3.large ≈ $65/month running (estimate); stopped while suspended (EBS only).
If removed	Manual, error-prone releases; no enforced gates.
Viva	*How does Jenkins deploy a specific commit?* — "The checkout gives the commit; the tag `<build>-<short commit>` is built, scanned, pushed and passed to `helm upgrade --set image.tag=…`; the verification stage checks that exactly that tag runs."

4.11 SonarQube

Aspect	In this project
What it is	A static code analysis server with quality gates.
Why needed	Detect bugs, vulnerabilities, code smells, duplication and missing test coverage before release.
Role here	Community 26.10 container on the Jenkins host bound to `127.0.0.1:9000`; two projects (backend with JaCoCo coverage, frontend with LCOV); the pipeline waits for and obeys the quality gate.
Talks to	Jenkins (scanner uploads, gate result).
Files	`ansible/roles/sonarqube/*`, `pom.xml` (sonar properties), `frontend/sonar-project.properties`, Jenkinsfile "Code Quality"
Normal operation	`QUALITY GATE STATUS: PASSED` for both projects (builds #4, #7, #8).
Failure / recovery	Gate failed → build fails; server down → stage fails (or is skipped if `SONAR_HOST_URL` is unset). Data persists in Docker volumes on EBS.
Why chosen	Free community edition, Java and JavaScript support, quality gates. Alternative: SonarCloud — SaaS, would need the code to leave the environment and a token.
Security	not exposed (localhost; SSH tunnel for humans); factory password replaced by a generated one; analysis token stored root:jenkins 0640.
Cost	shares the Jenkins instance (reason for t3.large).
If removed	No automated code-quality gate.
Viva	*What is a quality gate?* — "Pass/fail conditions on the analysis, mainly on new code; with `qualitygate.wait=true` a failed gate fails the build."

4.12 Trivy and secret scanning

Aspect	In this project
What it is	Trivy 0.76: open-source scanner for vulnerabilities, misconfigurations and secrets. gitleaks: Git history secret scanner (run manually).
Why needed	Stop vulnerable dependencies/images, unsafe infrastructure configuration and committed secrets from reaching production.
Role here	Four gates: CRITICAL dependency CVEs; HIGH/CRITICAL IaC misconfigurations (Terraform with `full.tfvars`, Helm, Dockerfiles, YAML) with 5 reviewed exceptions; secrets in the working tree; fixable HIGH/CRITICAL image CVEs.
Talks to	Jenkins; its vulnerability DB is cached under `JENKINS_HOME`.
Files	Jenkinsfile stages "Dependency & IaC Scan" and "Image Scan", `.trivyignore.yaml`, `.gitleaksignore`
Normal operation	Reports archived (`reports/trivy-*.txt`); latest: 0 findings in both images and dependencies (`docs/TESTING.md`).
Failure / recovery	Build #3 failed on 5 HIGH findings in the new ELK manifest → 4 fixed by hardening, 1 accepted with a path scope.
Why chosen	One free tool for four scan types, fast, CI-friendly. Alternatives: Snyk (commercial), Checkov (IaC only).
Security	Gates on fixable issues (`--ignore-unfixed`); every exception justified and narrow. "0 findings" means no *known, fixable* issues at scan time — not "vulnerability-free".
Cost	free.
If removed	Vulnerable or misconfigured artefacts could be deployed unnoticed.
Viva	*Why `--ignore-unfixed`?* — "A CVE with no fix cannot be solved by rebuilding; gating on it would block every release."

4.13 Amazon ECR

Aspect	In this project
What it is	AWS's private container image registry.
Why needed	EKS must pull images from a trusted, private registry close to the cluster.
Role here	Repositories `order-inventory-backend` and `order-inventory-frontend`, **IMMUTABLE** tags, scan on push, lifecycle keeps 20 images. (An unrelated repository named `frontend` exists in the account and is not touched.)
Talks to	Jenkins (push, instance role), EKS nodes (pull, node role read-only).
Files	`terraform/main.tf` (`aws_ecr_repository.*`, `aws_ecr_lifecycle_policy.app`), Jenkinsfile "Push Images to ECR"
Normal operation	`aws ecr get-login-password | docker login`, `docker push …:8-18a2bc6`; nodes pull through the NAT gateway.
Failure / recovery	Pushing an existing tag fails (immutability); suspend/resume verify the manifest tag exists before acting.
Why chosen	Native IAM integration, no extra credentials. Alternative: Docker Hub — public by default, rate limits, separate credentials.
Security	private; IAM-scoped push (Jenkins only, two repos); immutable tags; scan on push.
Cost	storage only (≈ 150 MB, cents per month).
If removed	EKS could not pull release images.
Viva	*Why immutable tags?* — "A tag always means one exact tested image; rollback is deploying an older tag."

4.14 Shell and PowerShell automation scripts

Aspect	In this project
What it is	Bash scripts in `scripts/` (and two PowerShell wrappers) automating deployment, verification, testing and operations.
Why needed	Repeatable operations with checks, used identically by people, Jenkins and tests.
Role here	`deploy-k8s.sh` (Helm deploy + rollback), `smoke-test.sh` (8/20 checks), `create-db-secret.sh`, `install-lb-controller.sh`, `create-alb-alarms.sh`, `aws-zero-downtime-test.sh`, `k8s-acceptance-test.sh`, `compose-acceptance-test.sh`, `suspend-evaluation.sh`, `resume-evaluation.sh` (+ `.ps1`), `lib/evaluation-common.sh`.
Talks to	AWS CLI, Terraform outputs, kubectl, Helm, curl, Node.
Files	`scripts/**`, `ansible/run-playbook.sh`, `ci/start-local-ci.sh`, `order-inventory-platform/logging/kibana-setup.sh`
Normal operation	e.g. resume: 11 ordered stages with timeouts and verification.
Failure / recovery	`die` prints the failed stage, reason and diagnostics; scripts are idempotent — re-run after fixing.
Why chosen	Bash runs everywhere (Git Bash, Linux agents); a heavier framework was unnecessary.
Security	never print secrets; refuse root/wrong account; read IDs from Terraform outputs instead of hard-coding.
Cost	—
If removed	Manual, error-prone operations.
Viva	*What if resume stops halfway?* — "Run it again; each step checks the state first (measured: re-run on a running system took 55 s and changed nothing)."

4.15 Amazon EKS and Kubernetes

Aspect	In this project
What it is	Kubernetes schedules and heals containers; EKS is AWS's managed Kubernetes control plane.
Why needed	Rolling updates, health-based traffic, self-healing and declarative deployments across two AZs.
Role here	Cluster `order-inventory-eks` (1.36), managed node group `order-inventory-nodes` (2 × t3.medium, private subnets); namespaces `order-inventory` (app) and `logging` (ELK); add-ons vpc-cni, coredns, kube-proxy, aws-ebs-csi-driver.
Talks to	ECR (image pulls), RDS (from pods), ALB (via the controller), Jenkins (private API endpoint), the laptop (public endpoint restricted to the admin /32), KMS (Secret encryption), CloudWatch (node CPU alarm).
Files	`terraform/main.tf` (cluster, node group, OIDC, add-ons, IAM), `terraform/jenkins.tf` (access entry), `helm/order-inventory/**`, `k8s/**`
Normal operation	Deployments keep 2 backend + 2 frontend pods; Services give stable names; readiness controls traffic.
Failure / recovery	Crashed container → restarted; lost pod/node → rescheduled; DB outage → pods unready, not restarted; nodes scaled to 0 (suspend) → objects stay in the control plane, pods return when nodes return.
Why chosen	Portable Kubernetes skills and tooling (the same Helm chart was tested on minikube first), rich rollout primitives, DaemonSets for log shipping. Alternative: ECS/Fargate — simpler, no control-plane fee, but AWS-specific.
Security	Secrets envelope-encrypted with KMS; API public endpoint limited to the admin /32; Jenkins limited to one namespace; pods non-root with seccomp; nodes in private subnets.
Cost	Control plane ≈ $0.10/h **always** (even with 0 nodes); 2 × t3.medium ≈ $65/month when running (estimates).
If removed	No runtime for the containers on AWS.
Viva	*Why does EKS cost money with zero nodes?* — "The managed control plane runs 24/7 and is billed per cluster-hour."

4.16 Helm

Aspect	In this project
What it is	Kubernetes package manager: templates + values → manifests, installed as a versioned *release*.
Why needed	One parameterised chart for EKS and minikube; release history and one-command rollback.
Role here	Chart `helm/order-inventory` (Deployments, Services, ConfigMap, PDBs, Ingress, helpers that forbid a missing or `latest` tag); release `order-inventory` (revision 14 at the last suspend).
Talks to	the Kubernetes API (Jenkins, resume script, tests).
Files	`helm/order-inventory/Chart.yaml`, `values.yaml`, `values-minikube.yaml`, `templates/*`, `scripts/deploy-k8s.sh`
Normal operation	`helm upgrade --install --set image.registry=… --set image.tag=… --wait`.
Failure / recovery	`--wait` fails if pods are not Ready → `helm rollback <last good revision>` (automatic in `deploy-k8s.sh`).
Why chosen	Standard, templating + history; alternative Kustomize has no release history/rollback.
Security	no secrets in values (Secret referenced by name); immutable tag enforced in templates.
Cost	—
If removed	Raw manifests with manual tag edits and no release history.
Viva	*How do you roll back?* — "`helm -n order-inventory rollback order-inventory <revision>`; the deploy script does it automatically."

4.17 AWS Load Balancer Controller and the ALB

Aspect	In this project
What it is	A Kubernetes controller (Helm chart 3.6.0 in `kube-system`) that creates and manages AWS Application Load Balancers from Ingress objects.
Why needed	Expose the app on the internet with path routing and health-checked targets, without managing the ALB by hand.
Role here	Ingress `order-inventory-ingress` (class `alb`, internet-facing, HTTP 80, `target-type: ip`): `/api` → backend Service, `/` → frontend Service; health checks `/api/health` and `/healthz`; deregistration delay 20 s.
Talks to	Kubernetes API (watch), ELBv2/EC2 APIs through its IRSA role `order-inventory-aws-lbc-role`.
Files	`scripts/install-lb-controller.sh`, `k8s/iam_policy.json`, `terraform/main.tf` (`aws_iam_role.lbc`), `helm/order-inventory/templates/ingress.yaml`, `values.yaml` annotations, `scripts/create-alb-alarms.sh`
Normal operation	Browser → ALB → pod IPs of ready pods.
Failure / recovery	Unready pods are deregistered; with no healthy target the ALB returns 503; Ingress deleted (suspend) → controller deletes ALB + target groups; Ingress recreated (resume) → **new ALB with a new DNS name**.
Why chosen	Native ALB integration with pod-IP targets; alternative NGINX Ingress Controller behind an NLB — an extra hop and component.
Security	only port 80 open (HTTP only — documented limitation, no certificate); controller permissions via IRSA bound to one service account.
Cost	ALB ≈ $20/month + public IPs while running; **deleted while suspended**.
If removed	No public entry point.
Viva	*Why does the ALB hostname change after suspension?* — "Suspend deletes the Ingress so the controller deletes the ALB; resume creates a new ALB, which AWS gives a new DNS name."

4.18 VPC, subnets, route tables, Internet Gateway and NAT Gateway

Aspect	In this project
What it is	The private AWS network and its routing.
Why needed	Isolation and tiered exposure of public entry points, compute and data.
Role here	VPC `10.0.0.0/16` in ap-south-1a/1b; public subnets 10.0.1.0/24, 10.0.2.0/24 (ALB, NAT, Jenkins; tagged `kubernetes.io/role/elb`); private 10.0.11.0/24, 10.0.12.0/24 (EKS nodes); database 10.0.21.0/24, 10.0.22.0/24 (RDS, **no routes** beyond local); IGW for public; one NAT gateway with Elastic IP for private egress.
Talks to	everything inside it.
Files	`terraform/main.tf` sections 1–8 and the database subnets/route table; `terraform.tfvars`
Normal operation	Users → IGW → ALB; nodes → NAT → ECR/AWS APIs; pods → RDS inside the VPC.
Failure / recovery	NAT down → nodes cannot pull images or join (alarm `nat-port-allocation-errors`); single NAT is not AZ-redundant (cost trade-off).
Why chosen	Three tiers is the standard secure layout. NAT kept while suspended because nodes need it to start; VPC endpoints would be the alternative (more resources).
Security	no public IPs on nodes/RDS; DB tier has no internet path at all; `map_public_ip_on_launch = false`.
Cost	VPC/subnets/routes free; NAT ≈ $0.056/h + data, EIP ≈ $0.005/h (billed while suspended).
If removed	Nothing could be deployed.
Viva	*Why are workloads and RDS in different tiers?* — "Different exposure needs: nodes need outbound internet via NAT; the database needs none, so its subnets have no internet route and its SG only trusts the cluster."

4.20 Ansible

Aspect	In this project
What it is	Agentless configuration management over SSH, with idempotent tasks grouped into roles.
Why needed	Install and configure software on the Jenkins server reproducibly.
Role here	Playbook `playbooks/jenkins.yml` with roles `common`, `docker`, `build_tools`, `jenkins`, `git_mirror` (historical), `sonarqube`, `jenkins_casc`; run through `run-playbook.sh` in a disposable container.
Talks to	the Jenkins host (SSH), Secrets Manager and Terraform outputs (through the wrapper).
Files	`ansible/**`
Normal operation	Re-runs report `changed=0` (verified).
Failure / recovery	`assert`, `validate` (sshd), retries; a configuration change restarts Jenkins — never run it during a build (that caused build #6's interruption). Must pass `JENKINS_REPO_URL=https://github.com/…` or the job is switched back to the old on-host mirror.
Why chosen	Agentless, readable YAML. Alternatives: a baked AMI (Packer) or user-data scripts — less maintainable.
Security	secrets streamed on stdin into the controller container, `no_log`, files root:jenkins 0640; limitation: passed briefly as arguments to a local `node` helper on the laptop.
Cost	—
If removed	Jenkins would be configured by hand (not reproducible).
Viva	*Why both Terraform and Ansible?* — "Terraform creates the server through the AWS API; Ansible configures what runs inside it over SSH."

4.21 Terraform and the S3 remote state

Aspect	In this project
What it is	Infrastructure as code: HCL files → plan → apply against provider APIs, with a state file.
Why needed	Reproducible, reviewable infrastructure; no console click-ops.
Role here	102 resources in the main stack (network, EKS, RDS, ECR, IAM, KMS, S3, SNS/SQS, alarms, budget, Jenkins); state in `s3://order-inventory-tfstate-038215676861/order-inventory/terraform.tfstate` with encryption and native lock file; bootstrap stack creates that bucket.
Talks to	AWS APIs; read by the scripts (`terraform output`).
Files	`terraform/*.tf`, `*.tfvars`, `bootstrap/main.tf`, `.terraform.lock.hcl`
Normal operation	`terraform plan -var-file=environments/full.tfvars -var-file=admin.auto.tfvars` → review → apply.
Failure / recovery	Lock prevents concurrent applies; versioned bucket allows restoring an earlier state; drift is detected by plan (e.g. stopped Jenkins).
Why chosen	Declarative, multi-provider, large ecosystem. Alternative: CloudFormation (AWS-only).
Security	state contains generated passwords → private, encrypted, versioned, TLS-only bucket, never in Git; variable validations reject `0.0.0.0/0`.
Cost	S3 negligible.
If removed	Infrastructure could not be reproduced or reviewed.
Viva	*Why is Terraform state sensitive?* — "It stores every resource attribute, including the generated database password."

4.22 IAM

Aspect	In this project
What it is	AWS identities and permissions.
Why needed	Every human and machine must have only the permissions it needs.
Role here	Deployer user (PowerUser + IAMReadOnly + IAM write scoped to `order-inventory-*`); EKS cluster role; node role; Jenkins role (ECR push to 2 repos, describe 1 cluster, put to 1 bucket) + EKS access entry (edit, 1 namespace); IRSA roles for the Load Balancer Controller and EBS CSI driver.
Talks to	every AWS API call.
Files	`terraform/main.tf`, `terraform/jenkins.tf`, `terraform/kms.tf`, `aws/iam/order-inventory-deployer-iam.json`, `k8s/iam_policy.json`
Normal operation	Jenkins gets temporary credentials from the instance role; pods via IRSA tokens.
Failure / recovery	Missing permission → explicit AccessDenied; the pipeline checks `kubectl auth can-i` before deploying.
Why chosen	Roles instead of long-lived keys; least privilege.
Security	never root (scripts refuse it); open owner items: root access key still active, no MFA on the admin user (**BLOCKED** on the owner).
Cost	free.
If removed	— (nothing works without IAM).
Viva	*What is IRSA?* — "Pods assume an IAM role through the cluster's OIDC provider, bound to one service account."

4.23 Security groups

Aspect	In this project
What it is	Stateful virtual firewalls attached to network interfaces.
Why needed	Allow only the required flows.
Role here	RDS SG: 3306 only from the EKS cluster SG; Jenkins SG: 22 and 8080 only from the admin /32, all egress; EKS control-plane SG: 443 from the Jenkins SG; ALB SG managed by the controller (port 80).
Talks to	—
Files	`terraform/main.tf`, `terraform/jenkins.tf`
Normal operation	SG references (not IP ranges) for internal trust.
Failure / recovery	When your IP changes, resume replaces the two Jenkins rules through a targeted Terraform plan (rules are keyed by CIDR with `for_each`).
Why chosen	SG-to-SG references survive IP changes of pods/nodes.
Security	no `0.0.0.0/0` ingress except the public ALB on 80; resume verifies the Jenkins SG has none.
Cost	free.
If removed	Default-deny would block everything, or wide rules would expose the database.
Viva	*Why can't the internet reach RDS?* — "No route, no public address, and its SG trusts only the cluster SG."

4.24 AWS KMS

Aspect	In this project
What it is	Managed encryption keys.
Why needed	Encrypt Kubernetes Secrets and the alert topic/queue with a key whose policy we control.
Role here	Key `alias/order-inventory-app` (rotation on, 7-day deletion window); EKS envelope encryption of `secrets`; SNS topic and SQS queue encryption; key policy allows CloudWatch, Budgets and SNS to use it.
Talks to	EKS, SNS, SQS, CloudWatch, Budgets.
Files	`terraform/kms.tf`, `main.tf` (`encryption_config`), `monitoring.tf`
Normal operation	transparent.
Failure / recovery	Without the CloudWatch permission, publishing to the encrypted topic would fail silently — the reason a customer-managed key is used.
Why chosen	AWS-managed `aws/sns` key cannot grant CloudWatch use.
Security	rotation, restricted policy; never schedule deletion (data would become unreadable).
Cost	≈ $1/month.
If removed	Secrets stored only with default etcd encryption; alarm delivery would break.
Viva	*Why a customer-managed key for SNS?* — "So the key policy can allow CloudWatch and Budgets to publish encrypted messages."

4.25 AWS Secrets Manager

Aspect	In this project
What it is	Managed storage for secrets.
Why needed	Passwords must be generated, stored and delivered without humans or Git seeing them.
Role here	`order-inventory/rds-master` (JSON username/password/host/port/dbname), `order-inventory/jenkins-admin`, `order-inventory/sonarqube-admin` — all generated by Terraform `random_password`.
Talks to	`scripts/create-db-secret.sh` → Kubernetes Secret `mysql-secret`; `ansible/run-playbook.sh` → Jenkins/SonarQube; the operator (Jenkins login).
Files	`terraform/main.tf`, `terraform/jenkins.tf`, `scripts/create-db-secret.sh`, `ansible/run-playbook.sh`
Normal operation	read at bootstrap/configuration time, never printed.
Failure / recovery	7-day recovery window on deletion; rotation procedure in `docs/SECURITY.md`.
Why chosen	Native, audited, IAM-controlled. Alternative: SSM Parameter Store SecureString (cheaper, fewer features).
Security	values never in Git, logs or images; this document contains none.
Cost	$0.40/secret/month (3 secrets).
If removed	Passwords would end up in files or variables.
Viva	*Where is the DB password?* — "Generated by Terraform into Secrets Manager, copied into a KMS-encrypted Kubernetes Secret, injected as an environment variable."

4.26 Amazon RDS (backups and recovery)

Aspect	In this project
What it is	Managed relational database service.
Why needed	Patching, backups, encryption and monitoring without running MySQL ourselves.
Role here	`order-inventory-mysql`, MySQL 8.4, db.t3.micro, gp3 20→50 GB encrypted, private DB subnets, parameter group (TLS required, slow log, tuned memory), 7-day automated backups with point-in-time recovery, error/slow logs to CloudWatch.
Talks to	backend pods (3306, TLS); CloudWatch.
Files	`terraform/main.tf` (RDS section), `monitoring.tf` (5 RDS alarms, log groups)
Normal operation	Hikari pool of 10 per pod.
Failure / recovery	Reboot test: ~1 s of 503s, liveness 100 %, 0 restarts (evidence file); stopped while suspended (storage + backups kept), AWS auto-starts after 7 days; PITR restore creates a **new** instance (procedure documented, not executed).
Why chosen	Managed backups/encryption. Alternative: MySQL in Kubernetes with a PV — more operational risk.
Security	private, TLS-only, encrypted, SG from the cluster only; the app uses the master user (limitation). Lab settings `skip_final_snapshot = true`, `deletion_protection = false` — never destroy.
Cost	≈ $21/month running incl. storage (estimate); storage only while stopped.
If removed	No persistent data on AWS.
Viva	*What happens if RDS is unavailable?* — "Requests needing the DB get a fast 503; readiness removes pods from traffic without restarting them; everything recovers automatically when RDS returns."

4.27 Amazon CloudWatch (alarms and logs)

Aspect	In this project
What it is	AWS metrics, alarms and log storage.
Why needed	Notice infrastructure problems without watching dashboards.
Role here	9 Terraform alarms (RDS CPU, storage, memory, swap, connections; Jenkins status, CPU; node CPU; NAT port allocation) + 4 ALB alarms from a script; RDS error/slow-query log groups (14 days).
Talks to	SNS (actions).
Files	`terraform/monitoring.tf`, `scripts/create-alb-alarms.sh`
Normal operation	alarms `OK`; while suspended `INSUFFICIENT_DATA` (verified 2026-10-25).
Failure / recovery	A real alarm fired: RDS memory/swap on db.t3.micro → parameter tuning and recalibrated threshold.
Why chosen	Native AWS metrics without extra agents. Alternative: Prometheus/Grafana — more to run.
Security	SNS topic encrypted.
Cost	≈ $0.10/alarm/month.
If removed	Silent failures.
Viva	*What does INSUFFICIENT_DATA mean?* — "No data points, e.g. because the resource is stopped."

4.28 Amazon SNS and SQS

Aspect	In this project
What it is	SNS: publish/subscribe notifications. SQS: message queue.
Why needed	Fan out alerts and keep a verifiable record of them.
Role here	Topic `order-inventory-alerts` (KMS) receives alarms and budget notifications; subscribed queue `order-inventory-alerts-audit` (KMS, 4-day retention); optional email subscription (currently **not** subscribed).
Talks to	CloudWatch, Budgets → SNS → SQS.
Files	`terraform/monitoring.tf`, `terraform/kms.tf`
Normal operation	each alarm transition produces one queue message.
Failure / recovery	Delivery verified by forcing an alarm state and reading the message from SQS (`docs/TESTING.md`).
Why chosen	Decoupled, auditable delivery without a mailbox.
Security	topic policy limits publishers to CloudWatch/Budgets of this account; queue policy limits senders to this topic.
Cost	negligible.
If removed	Alarms would have no destination.
Viva	*How do you know alerts are delivered?* — "I forced an alarm and the message arrived in the SQS audit queue."

4.29 AWS Budgets

Aspect	In this project
What it is	Cost thresholds with notifications.
Why needed	Early warning of unexpected spend.
Role here	`order-inventory-monthly`: $350/month; ACTUAL > 80 % and FORECASTED > 100 % → SNS.
Talks to	SNS.
Files	`terraform/monitoring.tf` (`aws_budgets_budget.monthly`), `kms.tf` (Budgets may use the key)
Normal operation	evaluated by AWS a few times per day.
Failure / recovery	Never triggered (spend below threshold) → **IMPLEMENTED BUT NOT VERIFIED**.
Why chosen	The first two action-free budgets are free.
Security	—
Cost	free.
If removed	No cost alert.
Viva	*Does a budget stop spending?* — "No, it only notifies."

4.30 ELK: Elasticsearch, Filebeat and Kibana (no Logstash)

Aspect	In this project
What it is	Elasticsearch stores and searches documents; Filebeat ships log files; Kibana is the UI. **There is no Logstash** — Filebeat parses JSON itself.
Why needed	Searchable application logs with request-id tracing.
Role here	Namespace `logging`: single-node Elasticsearch 8.19.5 (emptyDir, non-root, read-only root fs), Kibana (ClusterIP, port-forward only), Filebeat DaemonSet reading only `*_order-inventory_*` container logs, adding Kubernetes metadata and decoding the backend's ECS JSON; ILM deletes after 7 days; Kibana data view, 4 saved searches and a dashboard imported by script. Local Compose overlay as well.
Talks to	node log files → Filebeat → Elasticsearch → Kibana.
Files	`k8s/logging/elk.yaml`, `order-inventory-platform/logging/*`, `docker-compose.logging.yml`
Normal operation	Search `requestId : "<id>"` → the exact backend log line with pod and namespace (evidence `elk-eks-2026-10-19.txt`).
Failure / recovery	emptyDir → logs and Kibana objects lost when nodes go (suspend); re-import with `kibana-setup.sh` after resume.
Why chosen	Runs on existing nodes (no extra AWS cost); alternative: CloudWatch Logs/Container Insights or managed OpenSearch (production choice).
Security	never published; security disabled inside the cluster (demo); Filebeat root but read-only and capability-restricted (accepted Trivy exception); password found in 0 indexed documents.
Cost	uses node capacity only.
If removed	Only `kubectl logs` per pod; no cross-pod search.
Viva	*Why no Logstash?* — "Filebeat already parses the JSON and adds metadata; one less component."

4.31 Health checks, rolling updates, rollback and zero downtime

Aspect	In this project
What it is	The combination of Kubernetes and application mechanisms that keep the service available during changes.
Why needed	Releases and failures must not cause user-visible errors.
Role here	Startup/liveness `/api/health` (no DB), readiness `/api/health/ready` (DB); `maxUnavailable: 0`, `maxSurge: 1`; preStop sleep 10 s; Spring graceful shutdown 20 s; ALB deregistration delay 20 s; PDB `minAvailable: 1`; `deploy-k8s.sh` records the last good revision and rolls back; Jenkins `post` rollback.
Talks to	Kubernetes, the ALB, Helm, Jenkins.
Files	`helm/order-inventory/templates/backend.yaml`, `frontend.yaml`, `pdb.yaml`, `values.yaml`, `application.properties`, `HealthController.java`, `scripts/deploy-k8s.sh`, `Jenkinsfile`
Normal operation	New pod Ready → one old pod drains and stops → repeat.
Failure / recovery	Bad image or never-Ready pods: old pods keep serving; Helm times out; automatic rollback; exit code 1.
Why chosen	Rolling updates need no duplicate environment (cheaper than blue/green). Note: no ALB pod-readiness gate is configured; preStop + deregistration delay cover the hand-over.
Security	—
Cost	`maxSurge: 1` needs room for one extra pod per Deployment.
If removed	Releases would drop requests.
Viva	*How do you verify zero downtime?* — "Continuous requests through the public ALB during two releases, a bad image, a never-Ready release and a restart: 9,532 requests, 0 non-200 (`docs/evidence/aws-zero-downtime-2026-10-19.txt`)."

4.32 Supporting tools

Tool	Role here	Files
Maven	builds and tests the backend (`mvn -B clean verify`), dependency management through the Spring Boot parent	`backend/pom.xml`
npm	installs exact frontend dependencies (`npm ci`), runs scripts	`package.json`, `package-lock.json`
JUnit 5, MockMvc, Mockito, H2	29 backend tests (integration, failure paths, concurrency, correlation, unit)	`backend/src/test/**`
JaCoCo	backend coverage report for Jenkins and SonarQube (lines 70.97 %)	`pom.xml`
Vitest, Testing Library, jsdom, ESLint	21 frontend tests, lint with zero warnings	`vite.config.js`, `eslint.config.js`, `src/**/*.test.*`
HikariCP	JDBC connection pool (10 per pod, 5 s timeout → fast 503)	`application.properties`
Hibernate / Spring Data JPA	ORM; `ddl-auto=validate`; pessimistic row locks	entities, repositories
Jenkins Configuration as Code + Job DSL	Jenkins users, credentials, job and poll trigger from YAML	`templates/casc.yaml.j2`, `ci/jenkins/casc.yaml`
SonarScanner	frontend analysis upload	Ansible `build_tools`

minikube	local Kubernetes for the acceptance test before AWS	`scripts/k8s-acceptance-test.sh`, `values-minikube.yaml`, `k8s/testing/`
EKS add-ons	VPC CNI (pod IPs), CoreDNS, kube-proxy, EBS CSI driver (installed; no PersistentVolumes are used on EKS)	`terraform/main.tf`
gitleaks	manual Git-history secret scans	`.gitleaksignore`

CHAPTER 5

Repository and file inventory

The repository has **181 tracked files** (`git ls-files` after this document was added: 180 project files plus this PDF). This chapter shows the real tree, then explains every project-owned file in tables. The last column of each table says which chapter of this document explains the file further and at what depth:

- **L** = line by line / small block by block in that chapter
- **B** = by logical blocks (file is long but repetitive, e.g. UI markup)
- **C** = by category only (generated, third-party or data/evidence files, explained in Generated, third-party and evidence files)

5.1 Directory tree (tracked files)

```
DevOps Project/
├── .gitattributes  .gitignore  .gitleaksignore  .trivyignore.yaml
├── CONTRIBUTING.md  README.md  Jenkinsfile
├── order-inventory-platform/                ← the application
│   ├── .env.example  README.md
│   ├── docker-compose.yml                   ← local MySQL + backend + frontend
│   ├── docker-compose.logging.yml           ← optional local ELK overlay
│   ├── database/README.md
│   ├── logging/  filebeat.yml  ilm-policy.json  kibana-saved-objects.ndjson  kibana-setup.sh
│   ├── backend/                             ← Spring Boot REST API (Java 17)
│   │   ├── pom.xml  Dockerfile  .dockerignore
│   │   └── src/
│   │       ├── main/java/com/example/orderinventory/
│   │       │   ├── OrderInventoryApplication.java
│   │       │   ├── config/      CorsConfig  RequestIdFilter
│   │       │   ├── controller/  HealthController  OrderController  ProductController  UserController
│   │       │   ├── dto/         HealthResponse  OrderItemRequest  OrderItemResponse  OrderRequest
│   │       │   │                OrderResponse  OrderStatusUpdateRequest  ProductRequest  ProductResponse
│   │       │   │                UserRequest  UserResponse
│   │       │   ├── entity/      Order  OrderItem  OrderStatus  Product  User
│   │       │   ├── exception/   BadRequestException  ErrorResponse  GlobalExceptionHandler
│   │       │   │                ResourceNotFoundException
│   │       │   ├── repository/  OrderItemRepository  OrderRepository  ProductRepository  UserRepository
│   │       │   └── service/     OrderService  ProductService  UserService
│   │       ├── main/resources/  application.properties
│   │       │   └── db/ migration/V1__initial_schema.sql  V2__indexes_and_integrity_checks.sql
│   │       │       seed/R__demo_data.sql
│   │       └── test/  java/.../ ApiIntegrationTest  FailurePathWebTest  OrderConcurrencyTest
│   │                            RequestCorrelationTest  dto/ProductResponseTest
│   │                   resources/application-test.properties
│   └── frontend/                            ← React + Vite SPA served by nginx
│       ├── package.json  package-lock.json  vite.config.js  eslint.config.js  index.html
│       ├── Dockerfile  .dockerignore  sonar-project.properties
│       ├── nginx/  default.conf.template  security-headers.conf
│       └── src/  main.jsx  App.jsx  App.test.jsx  index.css
│           ├── components/  ConfirmDialog Header Icons Modal Sidebar StatCard StatusBadge Toast (.jsx)
│           ├── pages/       Dashboard Login Orders Products Users (.jsx)  workflows.test.jsx
│           ├── hooks/       useToasts.js  useToasts.test.js
│           ├── services/    api.js  api.test.js
│           └── test/        setup.js  helpers.js
├── helm/order-inventory/                    ← Helm chart deployed to EKS / minikube
│   ├── Chart.yaml  values.yaml  values-minikube.yaml
│   └── templates/  _helpers.tpl backend.yaml frontend.yaml configmap.yaml ingress.yaml pdb.yaml NOTES.txt
├── k8s/  namespace.yaml  iam_policy.json  logging/elk.yaml  testing/mysql-test-fixture.yaml
├── terraform/                               ← AWS infrastructure
│   ├── backend.tf main.tf jenkins.tf kms.tf storage.tf monitoring.tf variables.tf outputs.tf
│   ├── terraform.tfvars  environments/full.tfvars  environments/learning.tfvars
│   ├── .terraform.lock.hcl
│   └── bootstrap/  main.tf  .terraform.lock.hcl     ← creates the S3 state bucket
├── ansible/                                 ← configures the Jenkins EC2 host
│   ├── ansible.cfg  requirements.yml  inventory.example.ini  run-playbook.sh
│   ├── group_vars/all.yml  playbooks/jenkins.yml  test/test-in-container.sh
│   └── roles/ common docker build_tools jenkins git_mirror sonarqube jenkins_casc
├── ci/                                      ← local Jenkins + SonarQube (Docker)
│   ├── docker-compose.ci.yml  start-local-ci.sh
│   └── jenkins/  Dockerfile  casc.yaml  plugins.txt
├── scripts/                                 ← deploy, test and operations scripts
│   ├── deploy-k8s.sh smoke-test.sh create-db-secret.sh install-lb-controller.sh create-alb-alarms.sh
│   ├── aws-zero-downtime-test.sh k8s-acceptance-test.sh compose-acceptance-test.sh
│   ├── suspend-evaluation.sh resume-evaluation.sh (+ .ps1 wrappers)
│   └── lib/evaluation-common.sh
├── aws/iam/order-inventory-deployer-iam.json
└── docs/  API ARCHITECTURE AWS_DEPLOYMENT OPERATIONS_CHECKLIST RUNBOOKS SECURITY TESTING (.md)
```

5.2 Files that are intentionally not in Git (from `.gitignore`)

Pattern	What it is	Why excluded
`**/node_modules/`, `**/frontend/dist/`, `**/frontend/coverage/`, `**/backend/target/`, `*.class`	dependencies and build output	reproducible from `package-lock.json` / `pom.xml`; large; history was cleaned of them
`**/.terraform/`, `*.tfstate*`, `*.tfplan`, `tfplan`	provider binaries, state, saved plans	state and plans can contain secrets (e.g. the generated RDS password); state lives in S3
`*.tfvars` (except `terraform/terraform.tfvars`, `terraform/environments/*.tfvars`, `*.tfvars.example`)	local variable files, e.g. `terraform/admin.auto.tfvars` (your IP)	personal/sensitive values
`.env`, `.env.*` (except `.env.example`), `ci/.env`	local passwords	secrets
`*.pem`, `*.key`, `*.crt`, `*.p12`, `*.jks`	keys/certificates	secrets
`ansible/inventory.ini`	real Jenkins host address and key path	environment-specific
`.deploy/`	`previous-revision`, evaluation logs, zero-downtime test output	run-time state
`reports/`	CI reports	build output (archived by Jenkins and in S3)
`.idea/`, `.vscode/`, `*.iml`, `.DS_Store`, `Thumbs.db`, `*.log`, `*.tmp`	editor/OS/log files	noise

5.3 Root files

File	Purpose and important contents	Used by / when	Depends on	Design rationale	If wrong or missing	Likely viva question	Chapter
`Jenkinsfile`	Declarative pipeline: 15 stages from Checkout to Deployment Summary, params `DEPLOY`, `FULL_SMOKE_TEST`, `post` rollback	Jenkins job `order-inventory` on every build	`scripts/deploy-k8s.sh`, `scripts/smoke-test.sh`, Helm chart, tools on the agent, instance role	pipeline as code, versioned with the app	no CI/CD; a syntax error fails every build	"Walk me through your pipeline."	Ch. 12 (L)
`README.md`	project overview, status banner, quick start, cost table, limitations	humans, GitHub front page	—	single entry point	onboarding harder	—	—
`CONTRIBUTING.md`	branch/commit workflow, local quality gates, rules (no secrets, Flyway only, immutable images, pinned tools)	contributors	—	codifies conventions	conventions drift	"How would a new developer contribute?"	Ch. 5
`.gitattributes`	`* text=auto`						

	`eol=lf` ; `*.ps1 eol=crlf` ; images binary	Git on checkout/commit	—	shell scripts must have LF to run in Linux containers; PowerShell prefers CRLF	`bash\r: command not found` errors on Linux agents	"Why does a Windows repo force LF?"	Ch. 5
`.gitignore`	ignore rules (see Files not in Git)	Git	—	keep secrets and build output out of history	secrets/state could be committed (this happened before the remediation)	"How do you keep state files out of Git?"	Ch. 5
`.gitleaksignore`	two reviewed false-positive fingerprints (SonarQube factory-default `admin:admin` in an old version of `ci/start-local-ci.sh`)	gitleaks scans	the commit `939b41a…`	documented exceptions instead of disabling the scanner	the scan reports the same false positive forever	"Is ignoring a finding safe?"	Ch. 14
`.trivyignore.yaml`	5 accepted Trivy misconfiguration IDs with statements (AVD-AWS-0040/0041/0104/0132, KSV-0118 path-scoped to `k8s/logging/elk.yaml`)	`trivy config --ignorefile .trivyignore.yaml` in the Jenkinsfile	—	every exception is justified and narrow	build fails on accepted findings, or (if too broad) hides real ones	"Why did you ignore some Trivy findings?"	Ch. 14 (L)

5.4 Application — backend (`order-inventory-platform/backend`)

File	Purpose and important contents	Used by / when	Depends on	Design rationale	If wrong or missing	Likely viva question	Chapter
`pom.xml`	Spring Boot 4.1.1 parent, Java 17, starters (webmvc, data-jpa, validation, flyway), `flyway-mysql` , `mysql-connector-j` (runtime), H2 + test starters (test), JaCoCo, security overrides `tomcat.version` 11.0.25 and `jackson-bom.version` 3.1.7, Sonar properties	Maven in Docker build stage and Jenkins "Backend Tests"	Maven Central	dependency versions managed by the Boot parent; overrides fix Trivy findings	no build; missing JaCoCo → no coverage in Sonar	"What does the parent POM give you?"	Ch. 7 (L)
`Dockerfile`	2 stages: `maven:3.9-eclipse-temurin-17-alpine` builds `app.jar` (tests skipped: CI runs them); `eclipse-temurin:17-`						

	`jre-alpine` runtime, `apk upgrade`, user 10001, `JAVA_OPTS` heap 75 % of limit, `HEALTHCHECK`, `exec java`	`docker compose build`, Jenkins "Docker Build"	`pom.xml`, `src/`	small runtime image without Maven/JDK; non-root numeric UID for `runAsNonRoot`	image fails to build or runs as root	"Why multi-stage?"	Ch. 9 (L)
`.dockerignore`	excludes `.git`, `target/`, IDE files, logs, `.env*`, keys	Docker build context	—	smaller context, no secrets in images	slower builds; risk of copying secrets	"What is the build context?"	Ch. 9 (L)
`src/main/java/.../OrderInventoryApplication.java`	`@SpringBootApplication` + `main`	JVM start	Spring Boot	standard entry point	app does not start	"What does @SpringBootApplication do?"	Ch. 7 (L)
`config/CorsConfig.java`	`WebMvcConfigurer` allowing configured origins on `/api/**`	every cross-origin request	`app.cors.allowed-origins` (env `CORS_ALLOWED_ORIGINS`)	same-origin in practice; configurable if UI moves	cross-origin UI would be blocked	"Why is CORS needed at all?"	Ch. 7 (L)
`config/RequestIdFilter.java`	reuses/creates `X-Request-Id`, validates it (`[A-Za-z0-9._:-]{1,128}`), puts it in MDC, echoes it in the response	first filter of every request (`HIGHEST_PRECEDENCE`)	SLF4J MDC	log correlation; prevents log injection	no request tracing in logs	"How do you trace one request?"	Ch. 7 (L)
`controller/HealthController.java`	`GET /api/health` (liveness, no DB), `GET /api/health/ready` (DB `isValid(2)`; 503 when down)	probes, ALB health checks, smoke tests, UI badge	`DataSource`	liveness must not restart pods during DB outages	restart loops or traffic to broken pods	"Readiness vs liveness?"	Ch. 7 (L)
`controller/OrderController.java`	`GET /api/orders`, `GET /{id}`, `POST` (201), `PUT /{id}/status`	HTTP	`OrderService`	thin controller; logic in service	order API missing	"Why PUT for status?"	Ch. 7 (L)
`controller/ProductController.java`	`GET`, `GET /{id}`, `POST` (201), `PUT /{id}`, `DELETE /{id}` (204)	HTTP	`ProductService`	REST resource conventions	product API missing	"Why 204 for delete?"	Ch. 7 (L)
`controller/UserController.java`	`GET`, `GET /{id}`, `POST` (201)	HTTP	`UserService`	—	user API missing	—	Ch. 7 (L)
`dto/*Request.java` (5 files)	request bodies with Bean Validation (`@NotNull`, `@NotBlank`, `@Size`, `@Min`, `@DecimalMin`, `@Email`, element-level `@Valid`)	`@Valid @RequestBody` in controllers	Jakarta Validation	validate at the edge, never expose entities	invalid data reaches services/DB	"Why DTOs instead of entities?"	Ch. 7 (L)

`dto/*Response.java` (5 files)	response shapes; `ProductResponse.calculateStatus` (≤0 OUT, ≤10 LOW, else IN); `HealthResponse` omits null fields	serialised to JSON by Jackson	—	stable API contract, computed fields	API contract changes accidentally	"Where is stock status computed?"	Ch. 7 (L)
`entity/User.java`, `Product.java`, `Order.java`, `OrderItem.java`, `OrderStatus.java`	JPA mappings to `users`, `products`, `orders`, `order_items`; `@PrePersist` timestamps; `Order` ↔ `OrderItem` one-to-many cascade; enum stored as string	Hibernate	Flyway schema (validated)	schema owned by Flyway; Hibernate only validates	app refuses to start (`ddl-auto=validate`)	"What is @ManyToOne fetch EAGER/LAZY?"	Ch. 7 (L)
`exception/BadRequestException.java`, `ResourceNotFoundException.java`	unchecked exceptions mapped to 400/404	services	—	business errors as types	—	—	Ch. 7 (L)
`exception/ErrorResponse.java`	error JSON: timestamp, status, error, message, details, path, **requestId** (from MDC)	`GlobalExceptionHandler`	MDC	consistent error shape with correlation id	inconsistent errors	—	Ch. 7 (L)
`exception/GlobalExceptionHandler.java`	`@RestControllerAdvice` mapping: 404, 400 (validation, malformed JSON, type mismatch, bad request), 409 (integrity, lock failure), 503 (DB unavailable), framework statuses kept, 500 generic	every exception thrown by controllers	Spring MVC	no stack traces/internal text to clients	500s with internals, wrong status codes	"What happens on DB outage mid-request?"	Ch. 7 (L)
`repository/*.java` (4)	Spring Data JPA interfaces; `findByIdForUpdate` with `@Lock(PESSIMISTIC_WRITE)` on `ProductRepositor-`						

	`y` and `OrderRepository`	services	Hibernate	derived queries + explicit row locks	overselling/double restock	"How do you prevent overselling?"	Ch. 7 (L)
`service/OrderService.java`	create order: validate user, lock products in ascending id order, check stock, deduct, compute total, save; status change: lock order, cancelled is final, restore stock once on cancel	`OrderController`	repositories	transaction + ordered locks = no oversell, no deadlock	data corruption	"Show me the two-buyers scenario."	Ch. 7 (L)
`service/ProductService.java`	CRUD; trims strings; delete + `flush()` to surface FK violation as 409	`ProductController`	`ProductRepository`	—	500 instead of 409 on delete	—	Ch. 7 (L)
`service/UserService.java`	list/get/create; email normalised `trim().toLowerCase(Locale.ROOT)` before uniqueness check	`UserController`	`UserRepository`	case-insensitive uniqueness	duplicates differing only by case	—	Ch. 7 (L)
`src/main/resources/application.properties`	port 8080, graceful shutdown 20 s, datasource defaults (local only, overridden by env), Hikari fail-fast timeouts, Flyway locations/baseline, `ddl-auto=validate`, `open-in-view=false`, CORS, ECS logging fields	Spring at startup	env vars `SPRING_DATASOURCE_*`, `CORS_ALLOWED_ORIGINS`, `APP_ENVIRONMENT`, `LOGGING_STRUCTURED_FORMAT_CONSOLE`	12-factor: same jar everywhere, config from environment	wrong DB, hanging requests	"How does the app get DB credentials on EKS?"	Ch. 7 (L)
`db/migration/V1__initial_schema.sql`	4 tables, PKs, unique email, FKs	Flyway on startup	MySQL / H2	versioned schema	app cannot start (validate fails)	"Why Flyway?"	Ch. 8 (L)
`db/migration/V2__indexes_and_integrity_checks.sql`	3 indexes; CHECK stock ≥ 0, price > 0, quantity > 0	Flyway	V1	DB-level guard for invariants	invariants only in code	"Why constraints if the API validates?"	Ch. 8 (L)
`db/seed/R__demo_data.sql`	repeatable migration inserting demo rows **only if `users` is empty**	Flyway, only when `classpath:db/seed` is added (Compose)	V1	never touches real data; never on AWS	demo DB empty (harmless)	"What is a repeatable migration?"	Ch. 8 (L)
`src/test/.../ApiIntegrationTest.java`	14 MockMvc tests over the real Spring context + H2: health, users, products, orders, cancellations, error mapping	`mvn verify`	`application-test.properties`	black-box tests of the real API	regressions undetected	"What do your tests cover?"	Ch. 7 (B)
`FailurePathWebTest.java`	4 `@WebMvcTest` tests with mocked `DataSource` / `ProductService`: readiness 503 while liveness 200, DB outage → 503 without leaking host, lock timeout → 409, unexpected → generic 500	`mvn verify`	Mockito	failure paths hard to trigger for real	—	—	Ch. 7 (L)
`OrderConcurrencyTest.java`	8 threads buy 1 unit each of a product with stock						

	3; asserts exactly 3 orders, stock 0	`mvn verify`	H2, real `OrderService`	proves the lock works (mutation-checked)	overselling undetected	"How do you test concurrency?"	Ch. 7 (L)
`RequestCorrelationTest.java`	3 tests: generates an id, propagates a safe incoming id to header and error body, replaces unsafe ids	`mvn verify`	—	—	—	—	Ch. 7 (B)
`dto/ProductResponseTest.java`	parameterised test of `calculateStatus` boundaries	`mvn verify`	—	boundary values 0/1/10/11	—	—	Ch. 7 (B)
`src/test/resources/application-test.properties`	H2 in-memory DB in MySQL mode for profile `test`; Flyway applies real migrations; `ddl-auto=validate`	tests with `@ActiveProfiles("test")`	H2	isolated, fast, never touches a real DB	tests hit a real database	"Why H2 for tests?"	Ch. 7 (L)

5.5 Application — frontend (`order-inventory-platform/frontend`)

File	Purpose and important contents	Used by / when	Depends on	Design rationale	If wrong or missing	Likely viva question	Chapter
`package.json`	scripts `dev`, `build`, `preview`, `test` (vitest run), `lint` (`--max-warnings=0`), `test:coverage`; deps React 18, react-router-dom 7; dev deps Vite 8, Vitest 5, ESLint 9, Testing Library, jsdom; `engines.node >= 20.19.0`	npm locally, Docker build, Jenkins	npm registry	—	no build/tests	"Dependencies vs devDependencies?"	Ch. 6 (L)
`package-lock.json`	exact resolved dependency tree (5,728 lines)	`npm ci`	—	reproducible installs	non-reproducible builds	"Why npm ci?"	Ch. 6 (C)
`vite.config.js`	React plugin; dev server on 5173 with `/api` proxy to `VITE_DEV_API_PROXY` or `localhost:8080`; Vitest jsdom + setup + V8 coverage	`npm run dev/build/test`	—	dev mirrors production same-origin routing	API calls fail in dev	"Why a proxy in dev?"	Ch. 6 (L)
`eslint.config.js`	flat config: JS + React + hooks + refresh rules, `prop-types` off	`npm run lint` (CI)	ESLint plugins	catch bugs (hooks rules) early	lint stage fails/misses issues	—	Ch. 6 (L)
`index.html`	root `<div id="root">`						

	`">`, module script `/src/main.jsx`, Inter font, inline SVG favicon	Vite entry; served by nginx	—	SPA shell	blank page	—	Ch. 6 (L)
`Dockerfile`	`node:24-alpine` builds `dist/` with `npm ci && npm run build`; `nginxinc/nginx-unprivileged:1.30-alpine` serves it as UID 101 on 8080; `BACKEND_URL` default; templates rendered by envsubst	Compose, Jenkins	`nginx/*`	non-root nginx; same image for Compose and EKS	—	"Why nginx in the frontend container?"	Ch. 9 (L)
`.dockerignore`	excludes `node_modules/`, `dist/`, `coverage/`, `.env*`, keys	Docker build	—	—	huge context, stale builds	—	Ch. 9 (L)
`nginx/default.conf.template`	access log with `rid=$request_id`; `listen 8080`; runtime DNS `resolver`; gzip; `/healthz` 200; `/api/` reverse proxy to `${BACKEND_URL}` adding `X-Request-Id`; `/assets/` cached 1 year immutable; SPA fallback `try_files ... /index.html` with `no-cache`	nginx at container start (envsubst)	`BACKEND_URL`, `NGINX_LOCAL_RESOLVERS`	same image works in Compose/K8s; deep links work; new deployments load immediately	404 on refresh, broken API proxy	"How does a deep link like /orders work?"	Ch. 6, Ch. 9 (L)
`nginx/security-headers.conf`	`X-Content-Type-Options`						

	`nosniff`, `X-Frame-Options DENY`, `Referrer-Policy`	included in each location	—	nginx does not inherit `add_header` into locations that set their own	missing headers on some paths	—	Ch. 9 (L)
`sonar-project.properties`	project key, sources = `src`, tests and exclusions, LCOV path	`sonar-scanner` in Jenkins	`coverage/lcov.info`	coverage in Sonar	Sonar analysis misconfigured	—	Ch. 14 (L)
`src/main.jsx`	`createRoot` + `StrictMode` + `BrowserRouter` + `App` + CSS	browser on load	—	—	blank page	—	Ch. 6 (L)
`src/App.jsx`	routes: `/login`, layout with `/dashboard`, `/products`, `/orders`, `/users`, `/` → dashboard, `*` → dashboard	React Router	pages	layout route with `<Outlet/>`	navigation broken	"How is routing done?"	Ch. 6 (L)
`src/services/api.js`	`API_BASE_URL` (`VITE_API_BASE_URL` or `/api`), `request()` with JSON headers, 15 s timeout (`AbortController`), 204 → null, error message from `details` / `message`, non-JSON 2xx treated as error, friendly network/timeout messages; `healthApi`, `usersApi`, `productsApi`, `ordersApi`	every page	`fetch`	one place for HTTP behaviour	inconsistent error handling	"What happens on 503?"	Ch. 6 (L)
`src/hooks/useToasts.js`	toast list state; stable `addToast` / `removeToast` via `useCallback`; ids from a `useRef` counter; auto-dismiss 4 s	pages	React	stable callbacks avoid effect loops	duplicate keys / re-render loops	—	Ch. 6 (L)
`src/components/*.jsx` (8)	`Header` (title + API health badge polling `/api/health` every 15 s), `Sidebar` (NavLinks), `Modal` (Escape + overlay close), `ConfirmDialog`, `StatCard`, `StatusBadg-`						

	`e` (order/stock badges), `Toast`, `Icons` (inline SVG)	pages	—	reusable presentational components	UI breaks	—	Ch. 6 (L; Icons C)
`src/pages/Dashboard.jsx`	loads users/products/orders with `Promise.allSettled`; KPIs; low stock (≤ 10); revenue excluding cancelled; recent 5 orders; error banner	route `/dashboard`	`api.js`	partial failure still shows data + error	—	—	Ch. 6 (B)
`src/pages/Products.jsx`	list, search, category filter, create/edit modal with validation, delete confirm; shows backend 409 message	`/products`	`productsApi`	—	—	—	Ch. 6 (B)
`src/pages/Orders.jsx`	list + filters; create-order modal with multiple lines, stock-aware validation and preview total; status modal (cancel warning, cancelled is final)	`/orders`	`ordersApi`, `usersApi`, `productsApi`	client validation for UX, server for truth	—	"Trace Create Order end to end."	Ch. 6 (L for logic, B for markup)
`src/pages/Users.jsx`	list, search, create with email regex validation, lower-cased email	`/users`	`usersApi`	—	—	—	Ch. 6 (B)
`src/pages/Login.jsx`	demo landing form; submit only navigates to `/dashboard` (**no authentication**)	`/login`	—	out of scope	—	"Is there authentication?" (answer: no)	Ch. 6 (L)
`src/index.css`	889 lines of design tokens (CSS variables), layout, tables, badges, modals, toasts, spinner	browser	—	—	ugly UI only	—	Ch. 6 (C)
`src/test/setup.js`, `src/test/helpers.js`	jest-dom matchers + cleanup; `jsonResponse()` and `mockApi()` fetch router that fails on unexpected requests	Vitest	—	deterministic UI tests without a backend	flaky tests	—	Ch. 6 (L)
`src/**/*.test.js(x)` (4 files, 21 tests)	API client (7), routing/pages (5), workflows (7), hook (2)	`npm test` / Jenkins	helpers	—	—	"How are frontend tests different from backend tests?"	Ch. 6 (B)

5.6 Application — local environment and logging

File	Purpose and important contents	Used by / when	Depends on	Design rationale	If wrong or missing	Likely viva question	Chapter
`order-inventory-platform/docker-compose.yml`	`mysql:8.4` (named volume, healthcheck `mysqladmin ping`), `backend` (env DB URL, Flyway + seed locations, waits for healthy MySQL, healthcheck `/api/health/ready`), `frontend` (port 3000→8080, waits for healthy backend); bridge network	`docker compose up -d --build`	`.env` (optional)	ordered start via health conditions; data persists in a volume	stack does not start / data lost	"How does the backend find MySQL?" (service name `mysql`)	Ch. 9 (L)
`docker-compose.logging.yml`	overlay: backend ECS logs; Elasticsearch, Kibana (127.0.0.1 only), Filebeat (root, Docker socket read-only)	`docker compose -f docker-compose.yml -f docker-compose.logging.yml up -d`	`logging/*`	optional; security disabled → localhost only	—	—	Ch. 15 (L)

`.env.example`	placeholders for root/app DB passwords, host ports, image tag	copied to `.env`	—	no real secrets in Git	—	—	Ch. 9 (L)
`README.md` (platform)	local usage notes	humans	—	—	—	—	—
`database/README.md`	schema summary, baseline behaviour, how to add `V3__…`	humans	—	—	—	—	Ch. 8
`logging/filebeat.yml`	Docker autodiscover for the 3 containers; decode JSON lines; add `app` field; ILM policy	Compose Filebeat	`ilm-policy.json`	—	—	—	Ch. 15 (L)
`logging/ilm-policy.json`	rollover 1 day / 1 GB, delete after 7 days	Filebeat setup	—	bounded storage	unbounded index growth	—	Ch. 15 (L)
`logging/kibana-saved-objects.ndjson`	data view `filebeat-*`, 4 saved searches, dashboard `oi-operations`	`kibana-setup.sh`	Kibana	dashboards as code	manual dashboard work	—	Ch. 15 (C)
`logging/kibana-setup.sh`	waits for Kibana `available`, imports the ndjson (`overwrite=true`)	manually / after resume	curl, node	idempotent import	—	—	Ch. 15 (L)

5.7 Kubernetes and Helm

File	Purpose and important contents	Used by / when	Depends on	Design rationale	If wrong or missing	Likely viva question	Chapter
`helm/order-inventory/Chart.yaml`	chart name/version 1.0.0, type application	Helm	—	—	no chart	—	Ch. 13 (L)
`values.yaml`	EKS defaults: image registry/tag (required), resources, replicas 2/2, rolling update 0/1, preStop 10 s, PDB min 1, ingress ALB annotations, config (CORS, ECS logs, JAVA_OPTS), `database.existingSecret: mysql-secret`	`helm upgrade`	—	one chart, environment overrides	—	"What does maxUnavailable 0 mean?"	Ch. 13 (L)
`values-minikube.yaml`	registry `local`, no Ingress, smaller resources	minikube acceptance test	—	—	—	—	Ch. 13 (L)
`templates/_helpers.tpl`	labels; `oi.image` fails without tag or with `latest`; pod security context (non-root, seccomp)	all templates	values	guardrails in templates	mutable deploys possible	"How do you forbid latest?"	Ch. 13 (L)
`templates/backend.yaml`	Deployment (probes, preStop, read-only root fs, Secret env, ConfigMap env, config checksum) + Service (ALB health-check						

	path `/api/health`)	Helm	Secret `mysql-secret`, ConfigMap	—	—	—	Ch. 13 (L)
`templates/frontend.yaml`	Deployment (FQDN `BACKEND_URL`, `/healthz` probes) + Service port 80 → 8080	Helm	—	—	—	—	Ch. 13 (L)
`templates/configmap.yaml`	`backend-config`: `APP_ENVIRONMENT`, `CORS_ALLOWED_ORIGINS`, `JAVA_OPTS`, optional `LOGGING_STRUCTURED_FORMAT_CONSOLE`	backend `envFrom`	values	non-secret config separated from secrets	—	—	Ch. 13 (L)
`templates/ingress.yaml`	(if enabled) Ingress class `alb`: `/api` → backend:8080, `/` → frontend:80	AWS Load Balancer Controller	annotations	path routing on one ALB	no ALB	"How does an Ingress become an ALB?"	Ch. 13 (L)
`templates/pdb.yaml`	PDB per app, `minAvailable: 1`	node drains/upgrades	—	voluntary disruptions keep 1 pod	—	—	Ch. 13 (L)
`templates/NOTES.txt`	message printed after install/upgrade	Helm	—	—	—	—	Ch. 13 (L)
`k8s/namespace.yaml`	Namespace `order-inventory`	bootstrap and local test	—	—	—	—	Ch. 13 (L)
`k8s/iam_policy.json`	upstream AWS Load Balancer Controller IAM policy (v3.6.0)	Terraform `aws_iam_policy.lbc`	—	copied from upstream, kept in Git	controller cannot create ALBs	—	Ch. 10 (C)
`k8s/logging/elk.yaml`	namespace `logging`; Elasticsearch (non-root, read-only rootfs, init copy of config, emptyDir); Kibana; Filebeat RBAC, ConfigMap, DaemonSet (root, read-only host log mounts, `DAC_READ_SEARCH`)	`kubectl apply` (cluster admin)	worker nodes	demo-sized logging without extra AWS cost	no central logs	—	Ch. 15 (L)
`k8s/testing/mysql-test-fixture.yaml`	namespace `oi-testdb`, PVC, MySQL 8.4 Deployment, Service — local tests only	`scripts/k8s-acceptance-test.sh`	Secret created by the script	realistic DB outage tests on minikube	—	—	Ch. 13 (B)

5.8 Terraform (`terraform/`)

File	Purpose and important contents	Used by / when	Depends on	Design rationale	If wrong or missing	Likely viva question	Chapter
`backend.tf`	S3 backend: bucket `order-inventory-tfstate-038215676861`, key, region, `encrypt`, `use_lockfile = true`	`terraform init`	bootstrap bucket	shared, encrypted, locked state	local state, no locking	"Where is your state and how is it locked?"	Ch. 10 (L)
`main.tf`	providers, VPC, subnets (public/private/database), IGW, NAT + EIP, route tables, ECR (immutable) + lifecycle, EKS SG/role/cluster (KMS secrets, endpoint access), node role + node group (`ignore_changes` desired size), OIDC, EBS CSI role, LBC role, add-ons, RDS SG/rule/subnets/route table, random password, Secrets Manager, subnet group, parameter group, RDS instance	`plan`/`apply`	variables, `kms.tf`, `monitoring.tf` (log groups)	—	—	many	Ch. 10 (L)
`jenkins.tf`	Jenkins SG + admin-only rules, EKS API rule from Jenkins SG, IAM role/policies (ECR push to 2 repos, describe cluster, put CI reports), instance profile, Ubuntu						

	24.04 AMI lookup, EC2 (IMDSv2, encrypted gp3 30 GB, `ignore_changes [ami, associate_public_ip_address]`), EKS access entry (edit, namespace-scoped), generated Jenkins/SonarQube admin passwords → Secrets Manager	apply	`main.tf`	least privilege, no stored AWS keys	—	"Why ignore_changes on the instance?"	Ch. 10 (L)
`kms.tf`	customer-managed KMS key (rotation on, 7-day deletion window) + alias + policy for CloudWatch/Budgets/SNS; EKS cluster role permission	apply	—	SNS encryption usable by CloudWatch; EKS secret envelope encryption	alarms silently undelivered / secrets unencrypted	—	Ch. 10 (L)
`storage.tf`	CI-reports bucket: private, owner-enforced, SSE-S3, versioned, 30-day expiry, TLS-only	apply; Jenkins uploads	—	bounded cost, secure defaults	—	—	Ch. 10 (L)
`monitoring.tf`	SNS topic (KMS) + policy, SQS audit queue + subscription, optional email, 9 alarms (RDS ×5, Jenkins ×2, nodes ×1, NAT ×1), RDS log groups (14 days), monthly budget	apply	`kms.tf`	verifiable alert delivery without email	no alerting	—	Ch. 15, Ch. 10 (L)
`variables.tf`	all inputs with defaults and validations (`0.0.0.0/0` rejected)	Terraform	—	guardrails as code	unsafe values accepted	—	Ch. 10 (L)
`outputs.tf`	22 outputs (IDs, endpoints, ECR URLs, LBC role, RDS address, Jenkins ID/IP/URL, buckets, SNS ARN, DB secret ARN, SQS URL)	scripts (`tfout`), humans	—	scripts read real IDs instead of hard-coding	scripts cannot find resources	—	Ch. 10 (L)
`terraform.tfvars`	region, project, env, VPC/public/private CIDRs, AZs (non-secret)	auto-loaded	—	—	—	—	Ch. 10 (L)
`environments/full.tfvars`	full profile: Jenkins on, 2 nodes (min 0, max 3), t3.large Jenkins, budget $350 (placeholder admin CIDR)	`-var-file`	—	profiles as files	—	—	Ch. 10 (L)
`environments/learning.tfvars`	cheaper profile: no Jenkins, 1 node, 1-day backups, budget $100	`-var-file`	—	—	—	—	Ch. 10 (L)
`.terraform.lock.hcl` , `bootstrap/.terraform.lock.hcl`	provider version + checksum locks	`terraform init`	—	reproducible providers	provider drift	"What is the lock file?"	Ch. 10 (C)
`bootstrap/main.tf`	creates the state bucket: versioning, AES256, public access block, TLS-only policy; local state	once, before the main stack	—	chicken-and-egg of remote state	—	"How was the state bucket created?"	Ch. 10 (L)

5.9 Ansible (`ansible/`)

File	Purpose and important contents	Used by / when	Depends on	Design rationale	If wrong or missing	Likely viva question	Chapter
`ansible.cfg`	inventory path, roles path, host key checking on, pipelining	ansible-playbook	—	—	—	—	Ch. 11 (L)
`group_vars/all.yml`	pinned tool versions (Maven 3.9.12, Node 24.22.0, Trivy 0.76.0, Helm 4.3.1, kubectl 1.36.5, Terraform 1.15.8, SonarScanner 8.1.0.6389), Jenkins port, `docker_start_service`	playbook (`vars_files`)	—	same versions as `ci/jenkins/Dockerfile`	toolchain drift	—	Ch. 11 (L)
`inventory.example.ini`	template host line	copied to `inventory.ini` (ignored)	—	—	—	—	Ch. 11 (L)
`requirements.yml`	collections `community.docker` ,						

	`ansible.posix`, `community.general`	`ansible-galaxy`	—	—	modules missing	—	Ch. 11 (L)
`playbooks/jenkins.yml`	host group `jenkins`, `become`, roles in order with conditions (`git_mirror` only for local checkout, `sonarqube` / `jenkins_casc` only when passwords given), post-task version banner	ansible-playbook	roles	—	—	—	Ch. 11 (L)
`run-playbook.sh`	runs Ansible in a `python:3.12-slim` container; host from `terraform output jenkins_public_ip`; secrets from Secrets Manager streamed as JSON on stdin; builds a temporary inventory	operator	Docker, AWS CLI, Terraform, SSH key	no local Ansible needed on Windows	—	"How do secrets reach the Jenkins host?"	Ch. 11 (L)
`roles/common/*`	base packages, unattended upgrades, SSH hardening (validated with `sshd -t`), reload handler	playbook	—	—	—	—	Ch. 11 (L)
`roles/docker/tasks/main.yml`	`docker.io`, service enabled	playbook	—	—	—	—	Ch. 11 (L)
`roles/build_tools/*`	JDK 21, Maven, Node, SonarScanner (unarchive with `creates`), symlinks, version checks for single-binary tools, Trivy/Helm/kubectl/Terraform/AWS CLI only when the version differs	playbook	`group_vars`	idempotent installs	—	"How is idempotency achieved?"	Ch. 11 (L)
`roles/jenkins/*`	Jenkins LTS apt repo (deb822, pinned key), install, docker group, systemd drop-in dir, start, wait for `/login`; restart handler	playbook	—	—	—	—	Ch. 11 (L)
`roles/git_mirror/tasks/main.yml`	bare repo `/srv/git/devops-project.git`, `safe.directory`	only when local checkout is enabled (historical)	—	used before the GitHub history was cleaned	—	—	Ch. 11 (L)
`roles/sonarqube/*`	sysctl for embedded Elasticsearch, SonarQube container on `127.0.0.1:9000` with volumes, wait UP, replace factory password, generate token into `/etc/jenkins-secrets/sonar_token` (`no_log`)	playbook	Docker	private SonarQube	—	—	Ch. 11 (L)
`roles/jenkins_casc/*` + `templates/casc.yaml.j2`	plugin manager + plugin list, render JCasC (admin user, matrix auth, global env, Sonar credential, pipeline job with `pollSCM` and repo URL), secrets env file `root:jenkins 0640`, systemd override (no setup wizard), wait for the job API	playbook	SonarQube token	Jenkins fully reproducible from code	manual click-ops	"What is JCasC?"	Ch. 11 (L)
`test/test-in-container.sh`	runs syntax check, ansible-lint, apply, second apply (must be `changed=0`) in a systemd Ubuntu 24.04 container	developer	Docker	test playbook without AWS	—	"How did you test Ansible?"	Ch. 11 (L)

5.10 CI (local) and scripts

File	Purpose and important contents	Used by / when	Depends on	Design rationale	If wrong or missing	Likely viva question	Chapter
`ci/docker-compose.ci.yml`	local SonarQube (127.0.0.1:19000) + Jenkins (127.0.0.1:18090), repo mounted read-only, Docker socket (local only)	`ci/start-local-ci.sh`	`ci/.env`	run the same Jenkinsfile without AWS (`DEPLOY=false`)	—	—	Ch. 12 (L)
`ci/jenkins/Dockerfile`	Jenkins LTS + pinned toolchain + plugins + JCasC	local CI	`plugins.txt`, `casc.yaml`	reference toolchain	—	—	Ch. 12 (L)
`ci/jenkins/casc.yaml`, `plugins.txt`	local JCasC (job with `DEPLOY=false` default), plugin list	local Jenkins	—	—	—	—	Ch. 12 (L)
`ci/start-local-ci.sh`	generates passwords into `ci/.env`, starts SonarQube,						

	replaces factory password, creates token, starts Jenkins	developer	Docker, node	secrets never printed	—	—	Ch. 12 (L)
`scripts/deploy-k8s.sh`	Helm upgrade with immutable tag, namespace-scoped preflight, Secret check, records previous good revision, `--wait`, rollout check, **rollback on failure**	Jenkins "Deploy to EKS", resume, tests	Helm, kubectl	single deploy path for CI, tests and resume	no automatic rollback	"How does rollback work?"	Ch. 12/Ch. 13 (L)
`scripts/smoke-test.sh`	20 functional checks (or 8 read-only with `SMOKE_READ_ONLY=1`) through any base URL	Jenkins, resume, humans	curl, node	proves the deployed system end to end	—	—	Ch. 12 (L)
`scripts/create-db-secret.sh`	reads Secrets Manager JSON and creates Secret `mysql-secret` (`url` with `sslMode=REQUIRED`, `username`, `password`) without printing	cluster bootstrap / rotation	AWS CLI, Terraform output	secret never on disk	backend cannot connect	—	Ch. 13 (L)
`scripts/install-lb-controller.sh`	Helm install of AWS Load Balancer Controller 3.6.0 with the IRSA role	cluster bootstrap	Terraform outputs	—	no ALB	—	Ch. 13 (L)
`scripts/create-alb-alarms.sh`	alarms for the controller-created ALB: ELB 5xx, target 5xx, and one unhealthy-hosts alarm per target group (4 in total with 2 target groups)	after deploy, resume	SNS ARN, Ingress	ALB is not Terraform-managed	—	—	Ch. 15 (L)
`scripts/aws-zero-downtime-test.sh`	load generator + 5 scenarios on real EKS						

	through the ALB	manual test	deploy-k8s, smoke-test	evidence of zero downtime	—	"How did you prove zero downtime?"	Ch. 13 (L)
`scripts/k8s-acceptance-test.sh`	same on minikube incl. DB outage and pod loss	manual test	MySQL fixture	—	—	—	Ch. 13 (B)
`scripts/compose-acceptance-test.sh`	Compose: build/start, smoke, persistence, DB outage, crash restart	manual test	Compose	—	—	—	Ch. 9 (B)
`scripts/lib/evaluation-common.sh`	shared helpers for suspend/resume (logging, `die`, `wait_until`, preflight, state queries, predicates)	sourced	AWS CLI, Terraform, kubectl	—	—	—	Ch. 16 (L)
`scripts/suspend-evaluation.sh` / `.ps1`	suspend (7 stages, `--plan`) / PowerShell wrapper	operator	lib	—	—	—	Ch. 16 (L)
`scripts/resume-evaluation.sh` / `.ps1`	resume (11 stages) / wrapper	operator	lib, manifest	—	—	—	Ch. 16 (L)
`aws/iam/order-inventory-deployer-iam.json`	inline policy `order-inventory-iam-scope` for the deployer: IAM write only on `order-inventory-*` roles/policies/profiles, OIDC providers for EKS, service-linked roles for 5 services (attached alongside AWS-managed `PowerUserAccess` + `IAMReadOnlyAccess`, verified 2026-10-25)	IAM	—	PowerUser cannot manage IAM; this scopes the IAM it needs	Terraform cannot create roles	"Why not AdministratorAccess?"	Ch. 14 (L)

5.11 Documentation

File	Content
`docs/API.md`	endpoint table, status codes, captured examples
`docs/ARCHITECTURE.md`	runtime, CI/CD, data model diagrams; key decisions
`docs/AWS_DEPLOYMENT.md`	deployment record, resource inventory, cost, verification table, Jenkins source of truth, teardown
`docs/OPERATIONS_CHECKLIST.md`	release/deploy/provision/teardown/periodic checklists
`docs/RUNBOOKS.md`	§1–§10 runbooks (local, provision, deploy, rollback, recovery, tracing, DB ops, teardown, SonarQube, suspend/resume)
`docs/SECURITY.md`	secret handling, history remediation, rotation, hardening, accepted risks, credential status
`docs/TESTING.md`	every test and its result with status labels
`docs/recovery/evaluation-manifest.json`	recovery manifest written by suspend/resume (no secrets)
`docs/COMPLETE_PROJECT_EXPLANATION_AND_VIVA_GUIDE.pdf`	this document

5.12 Generated, third-party and evidence files (explained by category)

Files	Category	Why no line-by-line teaching
`frontend/package-lock.json`	generated by npm	machine-written dependency graph with integrity hashes; regenerate with `npm install`, never hand-edit
`terraform/.terraform.lock.hcl`, `terraform/bootstrap/.terraform.lock.hcl`	generated by `terraform init`	provider versions + checksums; update with `terraform init -upgrade`
`k8s/iam_policy.json`	third-party	

	(AWS)	the official AWS Load Balancer Controller policy; changing it by hand risks breaking the controller. It grants EC2/ELB describe/create/modify actions needed to manage ALBs, target groups, security groups and tags
`frontend/src/index.css`	styling	presentation only; no behaviour except class names used by components
`frontend/src/components/Icons.jsx`	inline SVG paths	drawing coordinates, not logic
`logging/kibana-saved-objects.ndjson`	exported Kibana objects	generated by Kibana export; edited only through Kibana
`docs/evidence/*.txt` (9 files)	recorded command output	evidence, not code; each is cited where its result is used (`aws-zero-downtime-2026-10-19.txt`, `aws-rds-interruption-2026-10-19.txt`, `elk-eks-2026-10-19.txt`, `jenkins-github-build-7-…`, `jenkins-github-poll-build-8-…`, `jenkins-local-build-3.txt`, `k8s-acceptance-2026-10-18.txt`, `k8s-final-state.txt`, `ansible-container-test-2026-10-18.txt`)

5.13 Coverage checklist

Every tracked file appears in the tables of this chapter. Files marked **L** are explained line by line (or in small logical blocks) in the named chapter; **B** files (long JSX markup and test files with many similar cases) are explained block by block with every behaviour listed; **C** files are explained by category above. Known stale comments found while writing this document (documentation only; behaviour is correct):

File	Stale text	Reality
`terraform/main.tf` lines 18–19	"State is local by default … see backend.tf.example"	`terraform/backend.tf` configures the S3 backend; there is no `backend.tf.example`
`.gitignore` lines 53–55	secrets "(db_password, alert_email) are passed via TF_VAR_*"	the DB password is now generated by Terraform (`random_password.db_master`); only `alert_email` is optional input
`ansible/run-playbook.sh` line 9–10	default `JENKINS_REPO_URL` described as needed "while GitHub carries the pre-remediation history"	history was cleaned and pushed on 2026-10-19; the job uses GitHub (`JENKINS_REPO_URL=https://github.com/SamruddhiDoke/DevOps-Project.git` must be passed when re-running)
`docs/ARCHITECTURE.md` runtime diagram	`sns → Email`	no email is subscribed; delivery goes to the SQS audit queue
`frontend/src/components/Sidebar.jsx`	footer tag "Base Application (Phase 1)"	cosmetic label from the first project phase

CHAPTER 6

Frontend: React, JavaScript and Vite

`order-inventory-platform/frontend/`.

6.1 What the frontend is

A **single-page application (SPA)** written in React 18 (JSX), built by **Vite** into static files (`dist/index.html` + fingerprinted JS/CSS in `dist/assets/`) and served by **nginx** inside the frontend container. The browser downloads the app once; navigation between pages happens in the browser (React Router), and data comes from the backend REST API over `fetch()` calls to the **same origin** under `/api`.

Component tree (from `src/App.jsx`):

```
<StrictMode> <BrowserRouter> <App>
  /login                  → <Login/>
  (layout) <AppLayout>    → <Sidebar/> + <main><Outlet/></main>
      /dashboard          → <Dashboard/>  → <Header/>, <StatCard/>×n, <StatusBadge/>, <Modal/>
      /products           → <Products/>   → <Header/>, <Toast/>, <Modal/>, <ConfirmDialog/>,
<StatusBadge/>
      /orders             → <Orders/>     → <Header/>, <Toast/>, <Modal/>×3, <StatusBadge/>
      /users              → <Users/>      → <Header/>, <Toast/>, <Modal/>×2
      /                   → <Navigate to="/dashboard">
  *                       → <Navigate to="/dashboard">
```

State management: **local component state only** (`useState`, `useCallback`, `useEffect`) plus one custom hook (`useToasts`). There is no Redux/Context store: every page loads the data it needs from the API, and after each change it reloads from the API, so the server stays the single source of truth.

6.2 Build and tooling files

`package.json`

```
"type": "module",
```

Node treats `.js` files as ES modules (`import`/`export`), which `vite.config.js` and `eslint.config.js` use.

```
"scripts": { "dev": "vite", "build": "vite build", "preview": "vite preview",
             "test": "vitest run", "lint": "eslint . --max-warnings=0",
             "test:coverage": "vitest run --coverage" }
```

Script	Runs	Who uses it
`dev`	Vite dev server (port 5173, hot reload)	developer
`build`	production bundle into `dist/`	Dockerfile stage 1, Jenkins "Frontend Build"
`preview`	serves `dist/` locally	developer
`test`	Vitest once (not watch mode)	developer, CI
`lint`	ESLint; `--max-warnings=0` turns any warning into a failure	Jenkins

Dependencies: `react`, `react-dom` (rendering), `react-router-dom` (routing) are shipped to the browser. Dev dependencies are only for building/testing (Vite, Vitest, jsdom, Testing Library, ESLint plugins). `"engines": {"node": ">=20.19.0"}` documents the minimum Node version; the Jenkinsfile enforces it with a small `node -e` check in "Environment Check".

If removed/changed: without `--max-warnings=0` lint warnings would not fail CI; without `"type": "module"` the config files would fail to load.

`vite.config.js`

```
export default defineConfig({
  plugins: [react()],
```

`@vitejs/plugin-react` compiles JSX and enables React Fast Refresh in development.

```
  server: { port: 5173, host: true,
    proxy: { '/api': { target: process.env.VITE_DEV_API_PROXY || 'http://localhost:8080', changeOrigin:
true } } },
```

In development the browser talks to Vite on 5173. Requests to `/api/...` are forwarded to the backend (default `localhost:8080`, overridable with `VITE_DEV_API_PROXY`). This **mirrors production** where nginx or the ALB route `/api`, so `api.js` can always use the relative path `/api` and never needs CORS. `host: true` listens on all interfaces (useful inside containers/VMs). `changeOrigin` rewrites the `Host` header to the target.

```
  test: { environment: 'jsdom', setupFiles: './src/test/setup.js', restoreMocks: true,
    coverage: { provider: 'v8', reporter: ['text-summary', 'lcov'],
      include: ['src/**/*.{js,jsx}'], exclude: ['src/test/**', 'src/**/*.test.{js,jsx}', 'src/main.jsx'] }
},
```

Vitest runs tests in **jsdom** (a simulated browser DOM in Node). `restoreMocks` resets `vi.fn` mocks between tests. Coverage writes `coverage/lcov.info`, which SonarQube reads (`sonar-project.properties`). Test files and the bootstrap file are excluded so they do not distort coverage.

`eslint.config.js`

Flat config (ESLint 9). Line by line: ignore `dist` and `coverage`; for all `.js/.jsx` files use the latest ECMAScript, ES modules, browser + Node globals and JSX parsing; detect the React version; enable `js.configs.recommended` (bugs such as unused variables), `react.configs.recommended` and `jsx-runtime` (no need to import React for JSX), `react-hooks` recommended (rules of hooks, exhaustive dependencies); turn off `react/prop-types` (no PropTypes in this project) and warn on components exported together with non-components (Fast Refresh limitation). *Viva:* "Why lint in CI?" — it blocks a class of runtime bugs (e.g. a missing hook dependency causing stale data) before tests even run.

`index.html`

The SPA shell: `<div id="root"></div>` is where React renders; `<script type="module" src="/src/main.jsx">` is the entry Vite follows to build the bundle (Vite rewrites it to `/assets/index-<hash>.js` in `dist/`). The favicon is an inline SVG data URI; the Inter font is loaded from Google Fonts (the only external resource; if offline, the browser falls back to the system font).

`sonar-project.properties`

6.3 Entry point and routing

`src/main.jsx`

```
ReactDOM.createRoot(document.getElementById('root')).render(
  <StrictMode><BrowserRouter><App /></BrowserRouter></StrictMode>);
```

1. `createRoot` (React 18 concurrent root) attaches React to `#root`.
2. `StrictMode` runs extra development-only checks (double-invokes effects in dev to reveal side-effect bugs; no effect in production builds).
3. `BrowserRouter` uses the HTML5 History API, so URLs look like `/orders` (not `/#/orders`). This is why nginx needs the SPA fallback (`try_files $uri $uri/ /index.html`): when the user refreshes `/orders`, the server has no file `orders`, so it returns `index.html` and React Router renders the page.
4. `import './index.css'` — Vite bundles the stylesheet.

`src/App.jsx`

```
function AppLayout() {
  return (<div className="app-container"><Sidebar /><main className="main-content"><Outlet /></main>
</div>);
}
```

A **layout route**: the sidebar is rendered once and `<Outlet/>` is replaced by the matched child page.

```
<Route path="/login" element={<Login />} />
<Route element={<AppLayout />}>
  <Route path="/dashboard" element={<Dashboard />} /> … <Route path="/users" element={<Users />} />
  <Route path="/" element={<Navigate to="/dashboard" replace />} />
</Route>
<Route path="*" element={<Navigate to="/dashboard" replace />} />
```

`/login` is outside the layout (full-screen). `Navigate … replace` redirects without adding a history entry. The catch-all `*` sends unknown URLs to the dashboard (tested in `App.test.jsx`: "falls back to the dashboard for unknown routes").

6.4 API communication — `src/services/api.js` (line by line)

```
export const API_BASE_URL = (import.meta.env.VITE_API_BASE_URL || '/api').replace(/\/+$/, '');
```

- `import.meta.env.VITE_API_BASE_URL` is a **build-time** variable: Vite replaces it with a constant when it builds the bundle. It is not read at runtime. If unset (the normal case), the base is the **relative** `/api`, i.e. the same host that served the page.
- `.replace(/\/+$/, '')` removes trailing slashes so `"/api/" + "/users"` cannot become `//users`.

Every call sends and accepts JSON. `fetch` has no timeout of its own, so an `AbortController` aborts the request after 15 s; the caller's headers override the defaults (spread order).

```
  try {
    const response = await fetch(url, config);
    if (response.status === 204) { return null; }
```

204 No Content (product delete) has no body; returning `null` avoids a JSON parse error.

```
    const isJson = (response.headers.get('content-type') || '').includes('application/json');
    const data = isJson ? await response.json().catch(() => null) : null;
```

Parses JSON only when the server says it is JSON; a broken body becomes `null` instead of throwing.

```
    if (!response.ok) {
      const errorMessage = (data?.details && data.details.length ? data.details.join(', ') :
data?.message)
                           || `Request failed with status ${response.status}`;
      const error = new Error(errorMessage); error.status = response.status; error.data = data; throw
error;
    }
```

For 4xx/5xx the message shown to the user is, in order of preference: the backend's field-level validation `details` (e.g. `price: Price must be greater than 0`), the backend `message`, or a generic text with the status. `status` and the full `data` (including `requestId`) are attached for callers/tests.

Backend status	Backend body (GlobalExceptionHandler)	What the user sees
400 validation	`details: ["field: message", …]`	the joined field messages in a red toast
400 business rule	`message: "Insufficient stock for product 'X'. Available: 2, Requested: 5"`	that message
404	`message: "Order not found with id: 99"`	that message
409	"The request conflicts with existing data …" or "… being updated by another request. Please retry."	that message (e.g. deleting a product that has orders)
503	"The database is temporarily unavailable. Please retry shortly."	that message; header badge stays "API Online" because liveness does not check the DB
500	"An unexpected server error occurred"	that message (no internals)

```
    if (!isJson) {
      const error = new Error('Unexpected response from API. Check that /api is routed to the backend
service.');
      error.status = response.status; throw error;
    }
    return data;
```

A **2xx response that is not JSON** means `/api` was not routed to the backend (for example an SPA fallback returned `index.html` with 200). Treating it as an error makes routing mistakes visible instead of showing empty tables (tested: "treats a 200 HTML response (unrouted /api SPA fallback) as an error").

```
  } catch (err) {
    if (err.name === 'AbortError') throw new Error('The backend did not respond in time. Please try
again.');
    if (err instanceof TypeError) throw new Error('Unable to connect to backend server. Please verify the
Spring Boot service is running.');
    throw err;
  } finally { clearTimeout(timeoutId); }
```

`fetch` rejects with a `TypeError` for network failures (wording differs per browser), so the type is checked, not the text. `finally` always clears the timer.

The four API objects map 1:1 to backend endpoints: `healthApi.getHealth` → `GET /api/health`; `usersApi.getAll/getById/create`; `productsApi.getAll/getById/create/update/delete`; `ordersApi.getAll/getById/create/updateStatus` (`PUT /api/orders/{id}/status` with body `{"status": …}`). Bodies are serialised with `JSON.stringify`.

Why no secrets in frontend environment variables: every `VITE_*` value is compiled into the JavaScript that is downloaded by every visitor, who can read it in the browser's developer tools. The frontend therefore holds no credentials; the database password exists only in the backend pod (from a Kubernetes Secret).

6.5 Hook and components

`src/hooks/useToasts.js`

```
const [toasts, setToasts] = useState([]);
const nextId = useRef(0);
const removeToast = useCallback((id) => { setToasts((prev) => prev.filter((t) => t.id !== id)); }, []);
const addToast = useCallback((message, type = 'info') => {
  nextId.current += 1; const id = nextId.current;
  setToasts((prev) => [...prev, { id, message, type }]);
  setTimeout(() => removeToast(id), TOAST_DURATION_MS);
}, [removeToast]);
```

- `useRef` counter gives unique ids even for two toasts in the same millisecond (a `Date.now()` id would collide; tested).
- Functional `setToasts(prev => …)` avoids stale state when several toasts are added quickly.
- `useCallback` with stable dependencies keeps `addToast` the **same function across renders**. Pages use `addToast` inside `fetchData`'s `useCallback`, which is a dependency of `useEffect`; an unstable `addToast` would re-create `fetchData` every render and the effect would reload data in an endless loop (tested: "keeps callbacks stable across renders").

Components (`src/components/`)

Component	Logic worth knowing
`Header.jsx`	On mount calls `healthApi.getHealth()` and then every 15 s (`setInterval`); shows **API Online / Offline / Connecting…**. `isMounted` flag + `clearInterval` in the cleanup prevent state updates after unmount. It uses **liveness** (`/api/health`), so a DB outage does not turn it red.
`Sidebar.jsx`	`NavLink` with a class callback `({ isActive }) => …` highlights the current page.
`Modal.jsx`	renders nothing when closed; Escape key listener added/removed in an effect; clicking the overlay closes, clicking inside (`stopPropagation`) does not.
`ConfirmDialog.jsx`	`Modal` with a message and Cancel/Confirm buttons; danger styling by default.

`x`	presentational card; colours via CSS custom properties.
`StatusBadge.jsx`	maps `IN_STOCK` / `LOW_STOCK` / `OUT_OF_STOCK` and the 5 order statuses to labelled badges.
`Toast.jsx`	renders the toast list with an icon per type and a dismiss button.
`Icons.jsx`	inline SVG icon components (drawing paths only).

6.6 Pages

Shared data-loading pattern (Dashboard, Orders)

```
const fetchData = useCallback(async () => {
  try {
    const results = await Promise.allSettled([ordersApi.getAll(), usersApi.getAll(),
productsApi.getAll()]);
    const [ordersData, usersData, productsData] = results.map((r) => r.status === 'fulfilled' ? r.value ||
[] : []);
    setOrders(ordersData); setUsers(usersData); setProducts(productsData);
    const failure = results.find((r) => r.status === 'rejected');
    if (failure) { addToast(failure.reason?.message || 'Failed to fetch orders data', 'error'); }
  } finally { setLoading(false); }
}, [addToast]);
useEffect(() => { fetchData(); }, [fetchData]);
```

- `Promise.allSettled` (not `Promise.all`): the three requests run **in parallel**, and one failure does not hide the data of the others.
- The initial state is `loading = true`, so the effect only sets state **after** awaiting (comment in the code: avoids cascading renders flagged by the hooks lint rules). Manual refresh sets `loading` first.
- Loading, empty and error states: a spinner while loading; an "empty state" box with a hint when the (filtered) list is empty; a toast (Orders/Products/Users) or a banner (Dashboard) on errors.

`Dashboard.jsx`

Computed values: `lowStockProducts = products.filter(p => p.stockQuantity <= 10)`; `recentOrders = orders.slice(0, 5)` (the backend already returns newest first: `findAllByOrderByCreatedAtDesc`); `totalRevenue` = sum of `totalAmount` of orders whose status is **not** `CANCELLED`. A failed request shows `Failed to fetch dashboard data: <message>` instead of silently showing zeros (tested).

`Products.jsx`

- `validateForm`: name required; price > 0; stock ≥ 0 (mirrors the backend `@DecimalMin("0.01")`, `@Min(0)`).
- `handleSubmit` builds a **typed payload** (`parseFloat`, `parseInt`, trimmed strings, empty → `null`) and calls `create` or `update` depending on `isEditing`, then reloads the list.
- Delete asks for confirmation (`ConfirmDialog`); a **409** from the backend (product referenced by orders) is shown as the backend message (tested: "shows the backend message when deletion is refused (409)").

`Users.jsx`

`validateForm` requires a name and an email matching `/\S+@\S+\.\S+/`; the payload lower-cases the email; the backend also normalises it and rejects duplicates case-insensitively (tested in both layers).

`Login.jsx`

`Orders.jsx` — the main workflow

Key state: `orderUserId`, `orderItems` (array of `{productId, quantity}`), `orderErrors`, `submitting`, `isCreateModalOpen`, `isStatusModalOpen`, `orderToUpdate`, `newStatus`, filters `searchTerm`, `statusFilter`.

Function	What it does
`handleOpenCreateModal`	pre-selects the first user and product, one line with quantity 1
`handleAddItemRow` / `handleRemoveItemRow`	add a line / remove a line (never the last one)
`handleItemChange(index, field, value)`	immutable update of one line (`map` + spread)
`calculateOrderPreviewTotal`	price × quantity summed **for display only** — the backend computes the real total
`handleCreateOrder`	client validation (customer chosen; each line has a product; quantity ≥ 1; quantity ≤ current stock), then `ordersApi.create({ userId: Number(...), items: [{ productId: Number, quantity: parseInt }] })`, success toast, close modal, reload
`handleOpenStatusModal` / `handleUpdateStatus`	`ordersApi.updateStatus(id, newStatus)`; the Save button is **disabled when the order is already CANCELLED**; a warning explains that cancelling restores stock
`filteredOrders`	client-side filter by id/name/email text and status buttons

Products with `stockQuantity <= 0` are disabled in the product dropdown. Client-side stock checks improve the user experience but are **not trusted**: another user may buy the stock between page load and submit, so the backend re-checks under a row lock.

End-to-end trace: clicking Confirm Order

1. Browser: `handleCreateOrder` validates and calls `ordersApi.create(payload)`.
2. `request('/orders', {method:'POST', body: JSON.stringify(payload)})` → `fetch('/api/orders')` on the same origin, e.g. `http://k8s-orderinv-….elb.amazonaws.com/api/orders`.
3. On EKS the **ALB** matches the Ingress rule `path: /api, pathType: Prefix` and forwards to a **backend pod IP** in the backend target group (target type `ip`). In Docker Compose, **nginx** in the frontend container matches `location /api/` and proxies to `http://backend:8080` adding `X-Request-Id`.
4. Backend: `RequestIdFilter` sets the request id → `OrderController.createOrder` validates the DTO (`@Valid`) → `OrderService.createOrder` (one transaction): load user; lock each product row (`SELECT … FOR UPDATE`, ascending id); check stock; deduct; build `OrderItem`s with the current price; save order (cascade saves items) → commit.
5. Response `201 Created` with the `OrderResponse` JSON (id, user, total, status `PENDING`, items with subtotals) and header `X-Request-Id`.
6. Browser: `api.js` returns the parsed JSON → toast "Order created successfully" → modal closes → `fetchData()` reloads orders, users and products, so the new order and the reduced stock appear.

If step 4 fails with insufficient stock: 400 with the message → red toast with "Insufficient stock for product …"; nothing was written (the transaction rolled back).

6.7 Production build and how the container serves it

`no-cache` (so a new deployment is picked up on the next page load), answers `/healthz` itself for probes, and proxies `/api/` to the backend.

Concern	Local dev (`npm run dev`)	Docker Compose	EKS
Who serves the UI	Vite dev server :5173	nginx in `inventory-frontend` (host port 3000)	nginx in frontend pods behind the ALB
Who routes `/api`	Vite proxy → `localhost:8080`	nginx `location /api/` → `http://backend:8080`	the ALB Ingress rule → backend pods (nginx proxy exists but is not used for `/api` traffic from the ALB)
API base in JS	`/api`	`/api`	`/api`
Build-time vs runtime config	`VITE_*` at build time	nginx `BACKEND_URL` at container start (envsubst)	`BACKEND_URL = http://backend.order-inventory.svc.cluster.local:8080` from the Helm chart

6.8 Frontend tests (21 tests, Vitest + Testing Library)

`src/test/setup.js` adds jest-dom matchers (`toBeInTheDocument`, …) and unmounts components after each test. `src/test/helpers.js` provides `jsonResponse(body, status)` and `mockApi(routes)`, which replaces global `fetch` with a router keyed by `"METHOD /path"`; **any unexpected request throws**, so a test cannot silently pass while the page calls the wrong endpoint.

File	Tests
`services/api.test.js` (7)	JSON success; JSON request bodies; 204 → null; validation `details` surfaced; fallback to message then status; 200 HTML treated as error; network failure mapped to the friendly message
`App.test.jsx` (5)	`/` → dashboard with live KPIs; dashboard error shown when the API fails; products table with stock badges; users and orders pages; unknown route → dashboard
`pages/workflows.test.jsx` (7)	products: client validation before calling the API; typed create payload + success; 409 message on delete. users: invalid email rejected and duplicate-email error shown. orders: quantity above stock blocked then valid order submitted; cancel a pending order; cancelled order cannot be changed
`hooks/useToasts.test.js` (2)	unique ids in the same millisecond + expiry; stable callbacks

Frontend vs backend tests: the frontend tests run in jsdom with a **mocked** API and verify UI behaviour (rendering, validation, which request is sent, how responses are displayed). The backend tests run the real Spring context against a real (in-memory) database and verify business rules and HTTP semantics. Together with the smoke test (real browser-less HTTP against the deployed system) they cover the whole path.

Result (from `docs/TESTING.md`): 21/21 passed, lines 80 %, branches 67 %, lint 0 errors/0 warnings, build OK — VERIFIED, and re-run in every Jenkins build (#4, #7, #8).

6.9 Viva questions (frontend)

Question	Answer based on this project
Why React?	Component model for a data-heavy admin UI, large ecosystem (React Router, Testing Library), and the team knew it. The app needs no server-side rendering.
Why Vite instead of Create React App?	CRA is no longer maintained; Vite starts instantly (native ES modules in dev), builds with Rollup into hashed assets, and integrates Vitest with the same config file.

How does the browser find the backend?	It doesn't need to: it calls the relative path `/api` on the host it loaded the page from; the ALB (EKS), nginx (Compose) or Vite proxy (dev) routes `/api` to the backend.
Build-time vs runtime configuration?	`VITE_API_BASE_URL` is baked into the JS at build time (same image everywhere, so it is left unset). nginx's `BACKEND_URL` is substituted when the container starts, so one image works in Compose and Kubernetes.
Why must API secrets not be in `VITE_*` variables?	They become part of the public JavaScript bundle.
What happens on 400/404/500/503?	See the status-code table in the API communication section above: the backend's message (or field details) is shown in a toast; 503 means "database temporarily unavailable, retry"; 500 shows a generic message only.
What if the backend is down completely?	`fetch` throws a `TypeError` → "Unable to connect to backend server…"; the header badge shows "API Offline"; after 15 s without an answer: "The backend did not respond in time".
Why reload data after every change instead of updating state locally?	The server computes totals, stock and status; reloading guarantees the UI shows what was actually committed (e.g. stock after a concurrent order).
Is there authentication?	No. The login page only navigates. This is a documented limitation.

CHAPTER 7

Backend: Java and Spring Boot

`order-inventory-platform/backend/`; Java packages are under `src/main/java/com/example/orderinventory/`.

7.1 Architecture of the backend

A single Spring Boot 4.1.1 application (Java 17) in the classic **layered** design:

```
HTTP → RequestIdFilter → Controller (validation, HTTP mapping)
                              |
                              ▼
                         Service (@Transactional business rules)
                              |
                              ▼
                         Repository (Spring Data JPA) → Hibernate → HikariCP → MySQL
Exceptions anywhere → GlobalExceptionHandler → ErrorResponse JSON (+ requestId)
```

Package	Responsibility
`controller`	map URLs/HTTP methods to Java methods, validate request DTOs, choose status codes
`service`	business rules and transaction boundaries (stock reservation, cancellation, email uniqueness)
`repository`	database access (derived queries, row locks)
`entity`	JPA mappings of the 4 tables
`dto`	request/response shapes (the API contract); entities are never returned directly
`exception`	error types and the global HTTP error mapping
`config`	CORS and request-id correlation

Dependency injection: every class receives its collaborators through its **constructor** (e.g. `public OrderService(OrderRepository orderRepository, UserRepository userRepository, ProductRepository productRepository)`). Spring creates one instance (singleton bean) of each `@Service`, `@RestController`, `@Component`, `@Configuration` and repository interface, and passes them in. Constructor injection makes dependencies explicit and `final`, and lets tests construct or mock them.

7.2 Build: `pom.xml` (block by block)

Lines	Block	Meaning
7–12	`<parent>spring-boot-starter-parent 4.1.1`	inherits plugin configuration and a curated, mutually compatible set of dependency versions (so most dependencies have no `<version>`)
14–18	coordinates	artifact `order-inventory-backend` version `1.0.0` → `target/order-inventory-backend-1.0.0.jar`
21	`java.version` 17	compiles for Java 17 (LTS); the runtime image is Temurin 17 JRE
24–25	`tomcat.version` 11.0.25, `jackson-bom.version` 3.1.7	**security overrides** of versions managed by the parent, added after Trivy reported vulnerabilities (comment: remove when the parent manages these or newer)
28–30	`sonar.*`	project key/name for SonarQube; JaCoCo XML path for coverage

41–44	`spring-boot-starter-data-jpa`	Spring Data JPA + Hibernate + HikariCP pool
46–49	`spring-boot-starter-validation`	Hibernate Validator for `@Valid`, `@NotNull` …
52–59	`spring-boot-starter-flyway`, `flyway-mysql`	runs migrations at startup; MySQL dialect support
62–66	`mysql-connector-j` (`runtime`)	JDBC driver needed only at run time, not for compiling
69–73	`h2` (`test`)	in-memory database for tests, never packaged
76–87	`spring-boot-starter-webmvc-test`, `spring-boot-starter-test` (`test`)	MockMvc, JUnit 5, AssertJ, Mockito
92–95	`spring-boot-maven-plugin`	repackages the JAR into an executable "fat JAR" (`java -jar app.jar`)
98–116	`jacoco-maven-plugin`	`prepare-agent` instruments tests; `report` in the `test` phase writes `target/site/jacoco/jacoco.xml` used by Jenkins and SonarQube

`mvn -B clean verify` (Jenkins "Backend Tests") compiles, runs the 29 tests, writes JUnit XML (`target/surefire-reports`) and the JaCoCo report. The Docker build runs `mvn -B clean package -DskipTests` because the tests already ran in CI.

7.3 Configuration: `src/main/resources/application.properties` (line by line)

Line(s)	Property	Why
4–5	`spring.application.name`, `server.port=8080`	service name (also appears in ECS logs as `service.name`); container port
8–9	`server.shutdown=graceful`, `spring.lifecycle.timeout-per-shutdown-phase=20s`	on SIGTERM, Tomcat stops accepting new requests and lets in-flight requests finish (up to 20 s) — part of zero-downtime rollouts
17–20	`spring.datasource.url/username/password/driver`	**local defaults only** (Docker Compose MySQL on host port 3307, local-only password). On EKS they are overridden by the environment variables `SPRING_DATASOURCE_URL`, `SPRING_DATASOURCE_USERNAME`, `SPRING_DATASOURCE_PASSWORD` from the Kubernetes Secret (Spring Boot's relaxed binding: env vars override properties)
23	`hikari.maximum-pool-size=10`	at most 10 DB connections per pod (2 pods → 20; the RDS alarm threshold is 50)
24	`minimum-idle=2`	keep 2 warm connections
25	`connection-timeout=5000`	a request waits at most 5 s for a connection → fails fast with an exception mapped to **503** instead of hanging
26	`validation-timeout=3000`	max time to test a pooled connection
27	`max-lifetime=1800000`	recycle connections every 30 min (avoids server-side timeouts)
36	`spring.flyway.locations=classpath:db/migration`	where versioned migrations live; Compose adds `classpath:db/seed`

43	`spring.jpa.hibernate.ddl-auto=validate`	Hibernate **never changes the schema**; it only checks entities match the tables at startup (mismatch = startup failure, caught by tests)
44	`spring.jpa.open-in-view=false`	no DB session held open during view rendering; lazy loading outside services fails loudly instead of running hidden queries
45–46	`show-sql=false`, `format_sql=true`	no SQL in logs by default
53	`app.cors.allowed-origins=${CORS_ALLOWED_ORIGINS:*}`	read by `CorsConfig`; default `*` (UI is same-origin anyway)
59	`logging.level.com.example.orderinventory=INFO`	application log level
61	`logging.structured.ecs.service.environment=${APP_ENVIRONMENT:local}`	adds `service.environment` (`aws`, `minikube`, `local`) to ECS JSON log lines
62	`logging.pattern.correlation=[%X{requestId:-}]`	in plain-text logs, prints the MDC `requestId` in every line

ECS JSON logging itself is switched on by the environment variable `LOGGING_STRUCTURED_FORMAT_CONSOLE=ecs` (set by the Helm ConfigMap and the Compose logging overlay), not in this file.

7.4 Startup and cross-cutting configuration

`OrderInventoryApplication.java`

```
@SpringBootApplication
public class OrderInventoryApplication {
    public static void main(String[] args) { SpringApplication.run(OrderInventoryApplication.class, args);
}
}
```

`@SpringBootApplication` = `@Configuration` + `@EnableAutoConfiguration` (configure Tomcat, DataSource, JPA, Flyway, Jackson… from the classpath and properties) + `@ComponentScan` (find beans in this package and sub-packages). Startup order that matters: DataSource → **Flyway migrates** → Hibernate validates → web server starts accepting requests. If the database is unreachable at startup, the app fails to start (and Kubernetes restarts it until the DB is back); once running, DB outages are handled per request (503).

`config/CorsConfig.java`

```
public CorsConfig(@Value("${app.cors.allowed-origins:*}") String[] allowedOrigins) { … }
registry.addMapping("/api/**").allowedOrigins(allowedOrigins)
        .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS", "PATCH").allowedHeaders("*");
```

Comma-separated property → `String[]`. Only `/api/**` gets CORS headers. In this deployment the browser calls the same origin, so CORS is not exercised; it exists so the UI could be hosted elsewhere. *Security note:* `*` is acceptable here because there are no cookies/credentials; with authentication it must be narrowed.

`config/RequestIdFilter.java` (every line that matters)

```
@Component
@Order(Ordered.HIGHEST_PRECEDENCE)
public class RequestIdFilter extends OncePerRequestFilter {
```

A servlet filter registered automatically (`@Component`), run **first** (`HIGHEST_PRECEDENCE`) so even errors from other filters carry the id; `OncePerRequestFilter` guarantees one execution per request.

```
public static final String HEADER = "X-Request-Id";
public static final String MDC_KEY = "requestId";
private static final Pattern SAFE_ID = Pattern.compile("[A-Za-z0-9._:-]{1,128}");
```

Only short ids of safe characters are accepted from clients. A header containing a newline could otherwise forge fake log lines (**log injection**); tested by `replacesUnsafeIncomingRequestIds`.

```
String incoming = request.getHeader(HEADER);
String requestId = incoming != null && SAFE_ID.matcher(incoming).matches() ? incoming :
UUID.randomUUID().toString();
MDC.put(MDC_KEY, requestId);
response.setHeader(HEADER, requestId);
try { chain.doFilter(request, response); } finally { MDC.remove(MDC_KEY); }
```

Reuse nginx's id (nginx sets `X-Request-Id $request_id` when it proxies) or create a UUID; store it in the **MDC** (Mapped Diagnostic Context, a per-thread map that the logger adds to every log line); return it to the client; always remove it afterwards because Tomcat threads are reused for other requests.

7.5 REST endpoints

All bodies are JSON. Examples marked *captured* come from `docs/API.md` (Compose stack, 2026-10-18); examples marked *illustrative* show the shape only.

Method	Route	Controller → service → repository	DB effect	Success	Errors	Tests
GET	`/api/health`	`HealthController.getHealth`	none	200 `{"status":"UP"}`	—	`healthEndpointsReportUp`, `FailurePathWebTest`
GET	`/api/health/ready`	`HealthController.getReadiness` → `DataSource.getConnection().isValid(2)`	none	200 `{"status":"UP","database":"UP"}`	503 `{"status":"DOWN","database":"DOWN"}`	same
GET	`/api/users`	`UserController` → `UserService.getAllUsers` → `findAll`	read	200 list	503	`createsAndFetchesUsers`
GET	`/api/users/{id}`	→ `getUserById` → `findById`	read	200	400 non-numeric, 404	same
POST	`/api/users`	→				

		`createUser` → `existsByEmail`, `save`	insert `users`	201	400 validation / duplicate email	`rejectsDuplicateEmailCaseInsensitively`, `validatesUserInput`
GET	`/api/products`	`ProductController` → `getAllProducts` → `findAll`	read	200 list with computed `status`	503	`productCrudLifecycleWithStockStatus`
GET	`/api/products/{id}`	→ `getProductById`	read	200	400, 404	same
POST	`/api/products`	→ `createProduct` → `save`	insert `products`	201	400	`validatesProductInput`
PUT	`/api/products/{id}`	→ `updateProduct` → `findById`, `save`	update	200	400, 404	lifecycle test
DELETE	`/api/products/{id}`	→ `deleteProduct` → `existsById`, `deleteById`, `flush`	delete	204	404, **409** if referenced by orders	`deletingProductReferencedByOrdersReturnsConflict`, `lockTimeoutMapsTo409Retry`
GET	`/api/orders`	`OrderController` → `getAllOrders` → `findAllByOrderByCreatedAtDesc`	read	200 newest first, with items	503	order tests
GET	`/api/orders/{id}`	→ `getOrderById`	read	200	400, 404	`malformedAndInvalidRequests…`
POST	`/api/orders`	→ `createOrder` → `findById` (user), `findByIdForUpdate` (products),				

		`save`	insert `orders` + `order_items`, update `products.stock_quantity`	201	400 validation / empty items / insufficient stock, 404 unknown user/product, 409 lock timeout, 503	`creatingOrderDeductsStockAndComputesTotal`, `rejectsOrderExceedingStock…`, `duplicateLines…`, `validatesEachOrderLine`, `rejectsOrdersForUnknownUserOrProduct`, `OrderConcurrencyTest`
PUT	`/api/orders/{id}/status`	→ `updateOrderStatus` → `findByIdForUpdate` (order), product locks on cancel	update `orders.status`; on cancel update stock	200	400 invalid status / cancelled is final, 404	`cancellingRestoresStockOnceAndCancelledOrdersAreFinal`

Captured example (from `docs/API.md`):

```
POST /api/users
{"name":"Jane Smith","email":"jane.23621@example.com","phone":"+1-555-0199"}

HTTP/1.1 201
X-Request-Id: d3067ae2be233d876eebb10baa712b98
{"id":5,"name":"Jane Smith","email":"jane.23621@example.com","phone":"+1-555-0199","createdAt":"2026-10-18T16:24:27.756846249"}
```

Illustrative order request/response shape (fields from `OrderRequest`, `OrderResponse`, `OrderItemResponse`):

```
POST /api/orders
{"userId": 1, "items": [{"productId": 3, "quantity": 2}]}

201 Created
{"id": 12, "userId": 1, "userName": "…", "userEmail": "…", "totalAmount": 179.98, "status": "PENDING",
 "createdAt": "…", "items": [{"id": 20, "productId": 3, "productName": "…", "productCategory": "…",
 "quantity": 2, "price": 89.99, "subtotal": 179.98}]}
```

Controllers (all four follow the same pattern)

```
@RestController                       // every method's return value is written as JSON
@RequestMapping("/api/orders")        // URL prefix for the class
public class OrderController {
    private final OrderService orderService;
    public OrderController(OrderService orderService) { this.orderService = orderService; }   // DI

    @GetMapping public ResponseEntity<List<OrderResponse>> getAllOrders() { return
ResponseEntity.ok(orderService.getAllOrders()); }
    @GetMapping("/{id}") public ResponseEntity<OrderResponse> getOrderById(@PathVariable Long id) { … }
    @PostMapping public ResponseEntity<OrderResponse> createOrder(@Valid @RequestBody OrderRequest
request) {
        return new ResponseEntity<>(orderService.createOrder(request), HttpStatus.CREATED); }
    @PutMapping("/{id}/status") public ResponseEntity<OrderResponse> updateOrderStatus(@PathVariable Long
id,
            @Valid @RequestBody OrderStatusUpdateRequest request) { … }
}
```

- `@PathVariable Long id`: `/api/orders/abc` cannot be converted → `MethodArgumentTypeMismatchException` → 400.
- `@RequestBody`: Jackson converts JSON to the DTO; malformed JSON or an unknown enum value (`{"status":"LOST"}`) → `HttpMessageNotReadableException` → 400.
- `@Valid`: Bean Validation runs on the DTO; violations → `MethodArgumentNotValidException` → 400 with field `details`.
- `HttpStatus.CREATED` (201) for creation; `ResponseEntity.noContent()` (204) for delete.
- Controllers contain **no business logic**; they translate HTTP to service calls.

`HealthController.java` — the health model

```
@GetMapping public ResponseEntity<HealthResponse> getHealth() { return ResponseEntity.ok(new
HealthResponse("UP")); }
```

Liveness: answers whenever the JVM and Tomcat work. Used by Kubernetes `startupProbe` / `livenessProbe`, the ALB health check for the backend target group, the UI header badge and smoke tests. Deliberately **no DB check**: if the DB is down, restarting pods would not help and would cause restart storms.

```
@GetMapping("/ready")
public ResponseEntity<HealthResponse> getReadiness() {
    try (Connection connection = dataSource.getConnection()) {
        if (connection.isValid(DB_VALIDATION_TIMEOUT_SECONDS)) return ResponseEntity.ok(new
HealthResponse("UP", "UP"));
        log.warn("Readiness check failed: database connection is not valid");
    } catch (Exception ex) { log.warn("Readiness check failed: database unreachable ({})",
ex.getClass().getSimpleName()); }
    return ResponseEntity.status(HttpStatus.SERVICE_UNAVAILABLE).body(new HealthResponse("DOWN", "DOWN"));
}
```

Readiness: borrows a pooled connection and validates it (2-second timeout). Used by the Kubernetes `readinessProbe`, the pipeline's in-cluster check and smoke tests. When it fails, Kubernetes removes the pod from the Service endpoints (no traffic) without restarting it. Only the exception **class name** is logged, not the message (which can contain the JDBC host). `try-with-resources` returns the connection to the pool.

7.6 DTOs and validation

Request DTOs (Jakarta Bean Validation annotations):

DTO	Field rules
`UserRequest`	`name` `@NotBlank` `@Size(max=100)`; `email` `@NotBlank` `@Email` `@Size(max=100)`; `phone` `@Size(max=20)`
`ProductRequest`	`name` `@NotBlank` `@Size(max=150)`; `description` free; `price` `@NotNull` `@DecimalMin("0.01")`; `stockQuantity` `@NotNull` `@Min(0)`; `category` `@Size(max=50)`
`OrderRequest`	`userId` `@NotNull`; `items` `@NotEmpty` `List<@Valid OrderItemRequest>` (element-level `@Valid`: each line is validated; container-level `@Valid` is deprecated in Hibernate Validator 9)
`OrderItemRequest`	`productId` `@NotNull`; `quantity` `@NotNull` `@Min(1)`
`OrderStatusUpdateRequest`	`status` `@NotNull` `OrderStatus` (enum)

The sizes match the column lengths in `V1__initial_schema.sql`, so invalid data is rejected with a clear 400 before reaching the database. Response DTOs: `UserResponse`, `ProductResponse` (adds computed `status`), `OrderResponse` (adds `userName`, `userEmail`, `items`), `OrderItemResponse` (adds `productName`, `productCategory`, `subtotal`), `HealthResponse` (`@JsonInclude(NON_NULL)`, so liveness has no `database` field). `ProductResponse.calculateStatus`:

```
if (stock == null || stock <= 0) return "OUT_OF_STOCK";
else if (stock <= 10) return "LOW_STOCK";
else return "IN_STOCK";
```

Boundary-tested (`ProductResponseTest`: −1, 0 → OUT; 1, 10 → LOW; 11, 500 → IN; null → OUT).

Why DTOs instead of entities: the API shape is independent of the table structure, internal fields are not exposed, lazy-loaded associations are not serialised accidentally, and validation rules belong to the request, not the table.

7.7 Entities (JPA mapping)

Entity / table	Key mappings
`User` → `users`	`@Id` `@GeneratedValue(IDENTITY)` (MySQL `AUTO_INCREMENT`); `email` `unique = true, length = 100`; `createdAt` `updatable = false`, set in `@PrePersist`
`Product` → `products`	`price` `precision 10, scale 2` (`BigDecimal`, never `double` for money); `description` `TEXT`; `stockQuantity` `Integer`; `@PrePersist` sets `createdAt` and defaults stock to 0
`Order` → `orders`	`@ManyToOne(fetch = EAGER)` `@JoinColumn(name = "user_id")`; `@Enumerated(EnumType.STRING)` status (stored as text, so reordering the enum cannot corrupt data); `@OneToMany(mappedBy = "order", cascade = ALL, orphanRemoval = true, fetch = EAGER)` `List<OrderItem>` `items`; `@PrePersist` sets `createdAt` and default `PENDING`
`OrderItem` → `order_items`	`@ManyToOne(fetch = LAZY)` order (the owning side of the FK); `@ManyToOne(fetch = EAGER)` product; `quantity`; `price` = unit price **at the time of ordering**
`OrderStatus`	`PENDING, CONFIRMED, SHIPPED, DELIVERED, CANCELLED`

`cascade = ALL` means saving an `Order` also inserts its `OrderItem`s; `orphanRemoval` deletes items removed from the list. EAGER loading of items/user/product is acceptable at this data size (every order response needs them); with large data it would need pagination and fetch joins. Hibernate validates these mappings against the Flyway-created schema at startup (`ddl-auto=validate`).

7.8 Repositories

```
public interface ProductRepository extends JpaRepository<Product, Long> {
    List<Product> findByStockQuantityLessThanEqual(Integer threshold);
    List<Product> findByCategoryIgnoreCase(String category);
    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("select p from Product p where p.id = :id")
    Optional<Product> findByIdForUpdate(@Param("id") Long id);
}
```

`JpaRepository` provides `findAll`, `findById`, `save`, `deleteById`, `existsById`, `flush`, `count` … with no code. Method names like `findByCategoryIgnoreCase` are turned into queries by Spring Data. `findByIdForUpdate` is the key method: `PESSIMISTIC_WRITE` makes Hibernate issue `SELECT … FOR UPDATE` on MySQL, taking an **exclusive row lock** held until the transaction ends. `OrderRepository` has the same method for orders and `findAllByOrderByCreatedAtDesc` (newest first). `findByStockQuantityLessThanEqual`, `findByCategoryIgnoreCase`, `findByUserId`, `findByStatus`, `countByStatus`, `UserRepository.findByEmail` and `OrderItemRepository.findByOrderId` are declared but **not used** by the current services (available for future features; harmless).

7.9 Services, transactions and concurrency

Transaction boundaries

All three services are annotated `@Service @Transactional` at class level: **every public method runs in one database transaction** (propagation `REQUIRED`). Read methods use `@Transactional(readOnly = true)` (lets Hibernate skip dirty checking). Spring rolls back on any `RuntimeException`; `BadRequestException` and `ResourceNotFoundException` extend `RuntimeException`, so when a business rule fails **nothing is written** (e.g. an order whose second line has insufficient stock does not keep the deduction of the first line). *In general,* MySQL InnoDB's default isolation is REPEATABLE READ, but locking reads (`FOR UPDATE`) always read the latest committed row version, which is what the stock check needs.

`OrderService.createOrder` (line by line)

```
User user = userRepository.findById(request.getUserId())
        .orElseThrow(() -> new ResourceNotFoundException("User not found with id: " +
request.getUserId()));
```

Unknown user → 404.

```
if (request.getItems() == null || request.getItems().isEmpty()) throw new BadRequestException("Order must
contain at least one item");
```

Defensive check (the DTO's `@NotEmpty` normally catches this first) → 400.

```
Map<Long, Product> lockedProducts = new HashMap<>();
request.getItems().stream().map(OrderItemRequest::getProductId).distinct().sorted()
    .forEach(productId -> lockedProducts.put(productId, productRepository.findByIdForUpdate(productId)
        .orElseThrow(() -> new ResourceNotFoundException("Product not found with id: " + productId))));
```

1. Collect the distinct product ids of the order and **sort them ascending**.
2. Lock each product row with `SELECT … FOR UPDATE`, in that order, before any stock is checked.

- *Why lock first:* the stock read and the stock write must be atomic with respect to other orders.

- *Why sorted:* if order A locks product 1 then 2 while order B locks 2 then 1, each would wait for the other forever (**deadlock**). Locking in one global order (ascending id) makes that impossible.
- *Why* `distinct` *+ map:* the same product on two lines is locked once, and both lines deduct from the **same object**, so the second line sees the reduced stock (tested: `duplicateLinesForSameProductAreCheckedAgainstCombinedStock` — two lines of 2 against stock 3 → 400, stock unchanged).

```
Order order = new Order(); order.setUser(user); order.setStatus(OrderStatus.PENDING);
BigDecimal totalAmount = BigDecimal.ZERO; List<OrderItem> orderItems = new ArrayList<>();
for (OrderItemRequest itemReq : request.getItems()) {
    Product product = lockedProducts.get(itemReq.getProductId());
    if (product.getStockQuantity() < itemReq.getQuantity()) {
        log.warn("Order rejected for user {}: insufficient stock for product {} (available {}, requested
{})", …);
        throw new BadRequestException("Insufficient stock for product '" + product.getName() + "'.
Available: " + … );
    }
    product.setStockQuantity(product.getStockQuantity() - itemReq.getQuantity());
    productRepository.save(product);
    OrderItem orderItem = new OrderItem(); orderItem.setOrder(order); orderItem.setProduct(product);
    orderItem.setQuantity(itemReq.getQuantity()); orderItem.setPrice(product.getPrice());
    orderItems.add(orderItem);
    totalAmount = totalAmount.add(product.getPrice().multiply(BigDecimal.valueOf(itemReq.getQuantity())));
}
order.setTotalAmount(totalAmount); order.setItems(orderItems);
Order savedOrder = orderRepository.save(order);
log.info("Order {} created for user {}: {} line(s), total {}", …);
return mapToResponse(savedOrder);
```

- Stock check and deduction happen **while holding the lock**.
- The warning log line contains ids and numbers only (no email = no personal data in logs); it is the event traced in Kibana by `requestId` (evidence `elk-eks-2026-10-19.txt`).
- Price is copied into the order line, so later price changes do not alter past orders.
- Totals use `BigDecimal` (exact decimal arithmetic).
- `orderRepository.save(order)` inserts the order and, through `cascade = ALL`, its items.
- When the method returns, Spring **commits**: stock updates, order and items become visible together and the row locks are released.

Two requests for the last unit (the overselling question)

Stock of product 7 = 1. Requests A and B arrive at the same time on different pods (both pods use the same RDS).

Time	Transaction A	Transaction B
t1	`SELECT … FROM products WHERE id=7 FOR UPDATE` → gets the lock, reads stock 1	—
t2	—	`SELECT … FOR UPDATE` on id 7 → **blocks**, waiting for A's lock
t3	1 ≥ 1 → stock := 0, insert order + item	still waiting
t4	**COMMIT** → lock released	lock granted; reads the **committed** stock 0
t5	201 Created	0 < 1 → `BadRequestException` → **rollback** → 400 "Insufficient stock … Available: 0, Requested: 1"

`OrderConcurrencyTest` proves the behaviour: 8 threads each order 1 unit of a product with stock 3 → exactly 3 successes, stock 0, 3 orders. The test was mutation-checked (removing the lock makes it fail — `docs/TESTING.md`). Limitation: the test runs on H2 (MySQL mode), not on MySQL itself.

The mechanism is **pessimistic locking**. There is **no** optimistic locking (`@Version`) and **no** `SERIALIZABLE` isolation in this code.

`OrderService.updateOrderStatus`

```
Order order = orderRepository.findByIdForUpdate(id).orElseThrow(() -> new ResourceNotFoundException(…));
OrderStatus currentStatus = order.getStatus(); OrderStatus newStatus = request.getStatus();
if (currentStatus == OrderStatus.CANCELLED && newStatus != OrderStatus.CANCELLED) {
    log.warn(…); throw new BadRequestException("Order " + id + " is cancelled and cannot be moved to " +
newStatus);
}
if (newStatus == OrderStatus.CANCELLED && currentStatus != OrderStatus.CANCELLED) {
    order.getItems().stream().sorted(Comparator.comparing(item -> item.getProduct().getId()))
         .forEach(item -> { Product product =
productRepository.findByIdForUpdate(item.getProduct().getId())…;
                            product.setStockQuantity(product.getStockQuantity() + item.getQuantity());
                            productRepository.save(product); });
}
…
order.setStatus(newStatus); Order updatedOrder = orderRepository.save(order); return
mapToResponse(updatedOrder);
```

- The **order row is locked first**: if two "cancel" requests arrive together, the second waits, then sees status `CANCELLED` and does not restock again ("restore stock exactly once").
- **Cancelled is final**: re-opening would have to re-reserve stock that may have been sold meanwhile. Setting `CANCELLED` on an already-cancelled order is accepted (idempotent, no restock).
- Products are locked in ascending id order again (deadlock avoidance with concurrent orders).
- Other transitions are not restricted (e.g. `DELIVERED → PENDING` is allowed) — only the cancellation rule is enforced. Mention this honestly if asked about a full state machine.

`ProductService` and `UserService`

- `createProduct` / `updateProduct` trim strings, turn missing stock into 0, and save. Updating stock directly is an administrative action (not locked); order flows always lock.
- `deleteProduct`: `existsById` (404 if missing) → `deleteById` → **`flush()`** — forces the SQL `DELETE` now so a foreign-key violation (product used in `order_items`) surfaces inside the method as `DataIntegrityViolationException` → **409** with a clear message, instead of an error at commit time.
- `createUser`: `email.trim().toLowerCase(Locale.ROOT)` **before** `existsByEmail`, so `Jane@X.com` and `jane@x.com` are duplicates (400). `Locale.ROOT` avoids locale-specific lower-casing bugs (e.g. Turkish "I"). The database `UNIQUE` constraint is a second guard if two requests race (→ 409 via `DataIntegrityViolationException`).

7.10 Error handling: `GlobalExceptionHandler` and `ErrorResponse`

`ErrorResponse` fields: `timestamp`, `status`, `error`, `message`, `details` (field errors), `path`, `requestId` — read from the MDC in the constructor, so every error body carries the correlation id.

Exception (handler method)	Status	Message to client	Logged
`ResourceNotFoundException`	404	the exception message	—
`BadRequestException`	400	the exception message	(service logs WARN for business rejections)

`MethodArgumentNotValidException`	400 "Validation Failed"	"Input validation failed..." + `details: ["field: message", …]`	—
`IllegalArgumentException`	400	message	—
`HttpMessageNotReadableException`	400	"Malformed request body or invalid field value"	—
`MethodArgumentTypeMismatchException`	400	"Invalid value for parameter 'id'"	—
`DataIntegrityViolationException`	409	"The request conflicts with existing data ..."	WARN with root-cause class only
`PessimisticLockingFailureException`	409	"The resource is being updated by another request. Please retry."	WARN
`CannotCreateTransactionException`, `DataAccessResourceFailureException`	**503**	"The database is temporarily unavailable. Please retry shortly."	ERROR with method/URI
Spring MVC exceptions implementing `org.springframework.web.ErrorResponse` (no handler → 404, wrong method → 405, unsupported media type → 415)	their own status	Spring's detail text	—
any other `Exception`	500	"An unexpected server error occurred"	ERROR **with stack trace** (server side only)

What happens if the database becomes unavailable during an order? HikariCP cannot provide a connection within 5 s → Spring throws `CannotCreateTransactionException` when opening the transaction → 503 with the generic message (verified on AWS: RDS reboot → 3 × 503 on `/api/products` for ~1 s, liveness 200 throughout, 0 pod restarts — `docs/evidence/aws-rds-interruption-2026-10-19.txt`). If the connection breaks after the transaction started, the transaction is rolled back, so no half-written order exists; the client gets an error and can retry. Meanwhile readiness fails, so Kubernetes and the ALB stop sending new requests to the affected pods until the DB is back.

7.11 Tests (29, JUnit 5)

Test class	Type	What it proves
`ApiIntegrationTest` (14)	`@SpringBootTest` + MockMvc + H2 (real context, real migrations)	health, user create/get, duplicate email (case-insensitive), user validation, product CRUD + stock status, product validation, 409 delete, order totals and stock deduction, oversell rejected without changing stock, duplicate lines vs combined stock, per-line validation (`items[0].quantity`, `items[1].productId`), unknown user/product 404, empty items 400, cancel restores once + cancelled final, malformed JSON / unknown enum / non-numeric id / unknown route / wrong method / missing resource → proper 4xx (never 500)
`FailurePathWebTest` (4)	`@WebMvcTest` slice with `@MockitoBean DataSource` and `ProductService`	readiness 503 + liveness 200 during DB outage; DB outage → 503 without leaking `secret-host`; lock timeout → 409 "retry"; unexpected error → generic 500
`OrderConcurrencyTest` (1)	`@SpringBootTest`, 8 threads + `CountDownLatch` start gate	no overselling under real concurrency

`st` (3)	MockMvc	id generated (UUID format); incoming `nginx-abc123` propagated to header and error body; unsafe id with a newline replaced
`ProductResponseTest` (7)	parameterised unit test	stock status boundaries

`application-test.properties` switches the `test` profile to H2 in MySQL mode (`jdbc:h2:mem:…;MODE=MySQL;DATABASE_TO_LOWER=TRUE`), keeps Flyway (the real V1/V2 migrations run) and `ddl-auto=validate` (so entity/schema mismatches fail the build). `@BeforeEach cleanDatabase()` deletes all rows in FK order before each test for isolation. Result: 29/29, JaCoCo lines 70.97 %, branches 75 % (`docs/TESTING.md`), re-run in every Jenkins build (evidence for build #7 lists each class and count).

7.12 Build and run

Where	Command	Result
Developer	`mvn -B clean verify`	tests + `target/order-inventory-backend-1.0.0.jar` + coverage
Docker stage 1	`mvn -B dependency:go-offline` then `mvn -B clean package -DskipTests`	JAR in the builder image
Docker stage 2	`ENTRYPOINT ["sh","-c","exec java $JAVA_OPTS -jar app.jar"]`	JVM as PID 1, heap = 75 % of the container limit, exits on OOM

7.13 Viva questions (backend)

Question	Answer
How does the system prevent overselling?	Order creation runs in one transaction and locks each product row with `SELECT … FOR UPDATE` (ascending id) before checking and deducting stock; a second buyer waits and then sees the committed stock. A DB `CHECK` constraint forbids negative stock. Proven by `OrderConcurrencyTest`.
Why pessimistic and not optimistic locking?	Contention is on few hot rows (popular products) and a failed optimistic attempt would have to be retried by the client; waiting briefly for a row lock gives a simple, correct outcome. Optimistic locking (`@Version`) is better when conflicts are rare and reads dominate.
Why lock in ascending id order?	To prevent deadlocks between orders containing the same products in different orders.
What is `@Transactional` doing?	Opens a DB transaction before the method and commits after it; rolls back on runtime exceptions, so partial orders cannot exist.
What is the difference between 400, 404, 409, 503, 500 here?	Client mistake / missing record / conflict with current data or lock contention (retry) / database temporarily unavailable (retry) / unexpected bug (generic message, stack trace only in logs).
Why `ddl-auto=validate`?	Flyway owns schema changes (versioned, reviewable); Hibernate must never alter a production schema.
Why `BigDecimal` for prices?	Binary floating point cannot represent 0.10 exactly; money needs exact decimal arithmetic.
What is MDC?	A per-thread context map; values in it (here `requestId`) are added to every log line of that request.
Why does liveness not check the database?	A DB outage is not fixed by restarting the app; it would cause restart storms. Readiness removes the pod from traffic instead.
What if two users register the same email simultaneously?	Both may pass `existsByEmail`; the `UNIQUE` index rejects the second insert → `DataIntegrityViolationException` → 409.

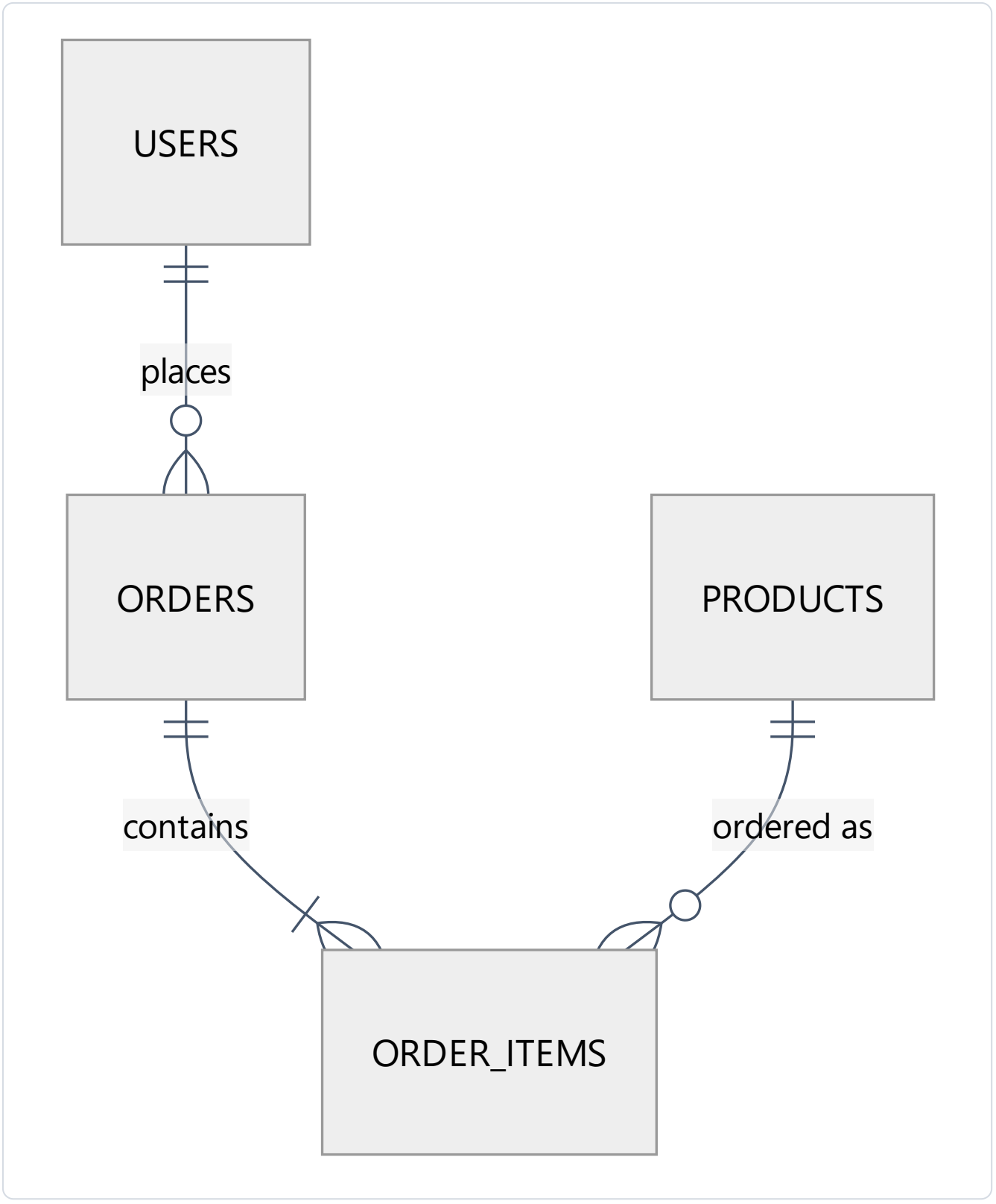
microservice?	It is one service (modular monolith) with users/products/orders sharing one DB transaction; see Microservices: an accurate description.

CHAPTER 8

MySQL, RDS and Flyway

8.1 Schema and relationships

Database `order_inventory_db` (MySQL 8.4). Defined only by Flyway migrations in `order-inventory-platform/backend/src/main/resources/db/migration/`.


USERS
places
ORDERS
PRODUCTS
contains
ordered as
ORDER_ITEMS

`V1__initial_schema.sql` (line by line)

```
CREATE TABLE users (
    id         BIGINT       NOT NULL AUTO_INCREMENT,   -- surrogate primary key generated by MySQL
    name       VARCHAR(100) NOT NULL,
    email      VARCHAR(100) NOT NULL,
    phone      VARCHAR(20),                            -- optional
    created_at DATETIME(6)  NOT NULL,                  -- microsecond precision (matches Java
LocalDateTime)
    PRIMARY KEY (id),
    CONSTRAINT uk_users_email UNIQUE (email)           -- one account per email (index + constraint)
);
```

```
CREATE TABLE products (
    id BIGINT NOT NULL AUTO_INCREMENT, name VARCHAR(150) NOT NULL, description TEXT,
    price DECIMAL(10,2) NOT NULL,                      -- exact money, up to 99,999,999.99
    stock_quantity INT NOT NULL, category VARCHAR(50), created_at DATETIME(6) NOT NULL,
    PRIMARY KEY (id));
```

```
CREATE TABLE orders (
    id BIGINT NOT NULL AUTO_INCREMENT, user_id BIGINT NOT NULL, total_amount DECIMAL(10,2) NOT NULL,
    status ENUM('CANCELLED','CONFIRMED','DELIVERED','PENDING','SHIPPED') NOT NULL,   -- only valid values
    created_at DATETIME(6) NOT NULL, PRIMARY KEY (id),
    CONSTRAINT fk_orders_user FOREIGN KEY (user_id) REFERENCES users (id));      -- order must belong to
an existing user
```

```
CREATE TABLE order_items (
    id BIGINT NOT NULL AUTO_INCREMENT, order_id BIGINT NOT NULL, product_id BIGINT NOT NULL,
    quantity INT NOT NULL, price DECIMAL(10,2) NOT NULL,                          -- unit price at order
time
    PRIMARY KEY (id),
    CONSTRAINT fk_order_items_order   FOREIGN KEY (order_id)   REFERENCES orders (id),
    CONSTRAINT fk_order_items_product FOREIGN KEY (product_id) REFERENCES products (id));
```

- Foreign keys have **no `ON DELETE CASCADE`**: deleting a product used by an order is refused by the database → the API returns 409. History is never silently deleted.
- *In general*, InnoDB creates an index for each foreign key automatically, so joins on `user_id`, `order_id`, `product_id` are indexed.
- The comment at the top explains the baseline behaviour (see Baselining below).

`V2__indexes_and_integrity_checks.sql`

```
CREATE INDEX idx_orders_status     ON orders (status);        -- filter by status
CREATE INDEX idx_orders_created_at ON orders (created_at);    -- "newest first" listing
CREATE INDEX idx_products_category ON products (category);    -- category filters
ALTER TABLE products    ADD CONSTRAINT chk_products_stock_non_negative CHECK (stock_quantity >= 0);
ALTER TABLE products    ADD CONSTRAINT chk_products_price_positive     CHECK (price > 0);
ALTER TABLE order_items ADD CONSTRAINT chk_order_items_quantity        CHECK (quantity > 0);
```

`CHECK` constraints (enforced by MySQL ≥ 8.0.16) are a **second line of defence** for invariants that the API already validates: even a bug or a manual SQL statement cannot create negative stock. Adding them in V2 (not editing V1) is the correct Flyway practice: V1 had already been applied to existing databases.

`db/seed/R__demo_data.sql`

A **repeatable** migration (prefix `R__`: re-applied when its checksum changes, after all versioned ones).

```
SET @seed_demo_data = (SELECT COUNT(*) = 0 FROM users);
INSERT INTO users (…) SELECT * FROM ( SELECT 1 AS id, 'Alex Morgan' AS name, … UNION ALL SELECT 2, … ) AS
seed WHERE @seed_demo_data;
```

Inserts 5 users, 8 products, 5 orders and 7 order items **only if `users` is empty**, so it can never overwrite real data. Aliases on the first `SELECT` avoid MySQL's "duplicate column name" error in derived tables. It is only active where `classpath:db/seed` is added to `SPRING_FLYWAY_LOCATIONS` (Docker Compose); **never on AWS**, so the RDS database contains only data created through the API (including smoke-test records with names like `smoke-<epoch>-<random>`).

8.2 How Flyway works here

1. Spring Boot starts, creates the DataSource, then runs Flyway **before** Hibernate and before the web server.
2. Flyway takes a lock (*in general*, MySQL uses a named lock) so that two pods starting at the same time do not migrate concurrently — observed on RDS: "two replicas, lock-coordinated" (`docs/AWS_DEPLOYMENT.md`).
3. It reads `flyway_schema_history` (creates it on first run), compares applied versions/checksums with the files `V1__…`, `V2__…` (ordered by version number), and applies only missing ones in order, each recorded with checksum, time and success flag.
4. A changed checksum of an applied migration makes validation fail at startup — protecting against edited history. Rule (`database/README.md`, `CONTRIBUTING.md`): never edit an applied migration; add `V3__…`.
5. Hibernate then validates the entities (`ddl-auto=validate`).

Observed on RDS after resume (2026-10-19 log): "Successfully validated 2 migrations", "Current version of schema `order_inventory_db`: 2", "Schema `order_inventory_db` is up to date. No migration necessary."

Why migrations are version-controlled: the schema changes together with the code that needs it, is reviewed in the same commit, and is applied identically in tests (H2), Compose, minikube and RDS. Rollback consideration: a new migration must stay backward compatible for one release, because a Helm rollback runs the previous application version against the newer schema.

8.3 Baselining

`spring.flyway.baseline-on-migrate=true`, `baseline-version=1`: if a database already contains tables but no `flyway_schema_history` (databases created before Flyway was adopted), Flyway records version 1 as "baseline" without running V1, then applies V2+. Verified on a real pre-existing local MySQL volume (rows unchanged, V2 applied — `docs/TESTING.md`). On RDS the database was empty, so V1 and V2 ran normally ("no baseline marker").

8.4 How the application gets its database configuration

Environment	Source
Tests	`application-test.properties` (H2 in memory)
Docker Compose	env in `docker-compose.yml`: `SPRING_DATASOURCE_URL=jdbc:mysql://mysql:3306/order_inventory_db?useSSL=false…`, user/password from `.env` or local defaults
EKS	Kubernetes Secret `mysql-secret` (keys `url`, `username`, `password`) → env vars `SPRING_DATASOURCE_*` in `helm/order-inventory/templates/backend.yaml`

The Secret is created by `scripts/create-db-secret.sh`, which reads the Secrets Manager secret `order-inventory/rds-master` (JSON with `username`, `password`, `host`, `port`, `dbname`, written by Terraform) and pipes it into `kubectl create secret … --dry-run=client -o yaml | kubectl apply -f -` without printing it. The URL is `jdbc:mysql://<rds-host>:3306/order_inventory_db?sslMode=REQUIRED&serverTimezone=UTC`. `sslMode=REQUIRED` = the driver refuses unencrypted connections. On EKS the Secret is envelope-encrypted with the project KMS key.

8.5 RDS networking and security (from `terraform/main.tf`)

Setting	Value	Effect
Subnets	`aws_db_subnet_group.mysql` = the two **database subnets** 10.0.21.0/24, 10.0.22.0/24	separate tier
Route table	`aws_route_table.database` has **no routes** besides the implicit local VPC route	no path to or from the internet (not even via NAT)
`publicly_accessible`	`false`	no public IP / public DNS resolution
Security group	`aws_security_group.rds` with one ingress rule: TCP 3306 **from the EKS cluster security group**	only pods/nodes of the cluster can connect; Jenkins and the internet cannot
Encryption at rest	`storage_encrypted = true`	data files, backups, snapshots encrypted (AWS-managed RDS key)
Encryption in transit	parameter `require_secure_transport = 1` + client `sslMode=REQUIRED`	plaintext connections rejected (verified: TLS cipher in use, `docs/TESTING.md`)
Engine	MySQL 8.4 (LTS; 8.0 would incur extended-support charges), db.t3.micro, gp3 20 GB autoscaling to 50 GB	
Memory tuning	`innodb_buffer_pool_size = {DBInstanceClassMemory*3/8}`, `performance_schema = 0`	fixed swapping on the 1 GiB instance (found by the FreeableMemory alarm; verified live)
Slow-query log	`slow_query_log = 1`, `long_query_time = 2`; exported with the error log to CloudWatch Logs (14-day retention)	diagnosis
Backups	`backup_retention_period = 7` (full profile), window 19:00–19:30 UTC; `copy_tags_to_snapshot`	daily snapshot + **point-in-time recovery** to any second within 7 days
Maintenance	Sunday 20:00–21:00 UTC; `auto_minor_version_upgrade = true`	
Lab settings	`skip_final_snapshot = true`, `deletion_protection = false`	allow a clean `terraform destroy` — which is exactly why destroy must not be used for routine suspension

Measured: RDS private address 10.0.22.34; port 3306 unreachable from the internet; RDS reboot caused ~1 s of 503s with no pod restarts (`docs/evidence/aws-rds-interruption-2026-10-19.txt`).

8.6 Connection handling during restarts

HikariCP keeps up to 10 connections per pod. When RDS restarts: existing connections break → the next query on them fails → Hikari discards them and opens new ones; requests that cannot get a connection within 5 s get **503**; readiness fails → pods are removed from the Service/ALB until a connection succeeds; liveness stays 200 → no restarts. When RDS is back, readiness passes and traffic resumes automatically.

8.7 Stopping RDS

`aws rds stop-db-instance` stops the **compute**; storage (gp3 volume), automated backups and the PITR window are kept and still billed (storage), compute is not. *General AWS rule:* a stopped instance is started automatically after **7 days**; the suspend script records this date (`rdsAutoRestartAfter`). While stopped, no new automated backups are taken. The resume script starts it and waits for `available` (≈ 3–4 minutes measured), then verifies the schema through Flyway's log lines and real reads/writes (smoke test).

How to verify the schema is intact (read-only, after resume):

```
kubectl -n order-inventory logs deploy/backend | grep -i flyway        # "up to date", version 2
curl -s $URL/api/products | head -c 300                                 # existing rows returned
```

Point-in-time restore (*general procedure*, not executed in this project): `aws rds restore-db-instance-to-point-in-time --source-db-instance-identifier order-inventory-mysql --target-db-instance-identifier <new-name> --restore-time <UTC time>` creates a **new** instance; the application would then be pointed at it via the Secret. Status: IMPLEMENTED (backups enabled, PITR window confirmed by the suspend script: "point-in-time recovery available up to ...") BUT the restore itself NOT VERIFIED.

8.8 Four different kinds of "state" (do not mix them up)

State	Where	Contains	Lost if ...	Protected by
Terraform state	S3 `order-inventory-tfstate-038215676861/order-inventory/terraform.tfstate`	Terraform's record of which AWS resources it manages, their IDs and attributes (including the generated RDS password)	bucket deleted	versioning, encryption, TLS-only policy, native lock file
Application data	RDS `order_inventory_db`	users, products, orders	instance deleted without snapshot	7-day backups/PITR, private network, encryption
Database backups	RDS automated snapshots + transaction logs (AWS-managed storage)	restorable copies	backup retention reduced/instance deleted (automated backups are deleted with the instance by default)	retention 7 days
Kubernetes state	EKS control plane (etcd, managed by AWS)	Deployments, Services, Ingress, Secrets, Helm release history (stored as Secrets)	cluster deleted	AWS-managed; Helm history allows rollback

Example: `terraform destroy` reads the Terraform state and deletes the RDS instance → application data and automated backups are gone (no final snapshot in this lab configuration). Scaling the node group to 0 changes neither the Terraform state (desired size is ignored), the database, nor the Kubernetes objects.

8.9 Viva questions

Question	Answer
Why Flyway?	Versioned, repeatable, reviewed schema changes applied automatically and identically everywhere; Hibernate only validates.
Why is RDS private?	It holds all business data; only the application needs it. Isolated subnets without internet routes + an SG that trusts only the EKS cluster SG + no public accessibility.
Is stopping RDS deleting data?	No — compute stops, storage and backups remain; AWS auto-starts after 7 days.

column?	New file `V3__add_x.sql`, backward compatible (nullable/default), update entity + DTO, run `mvn verify` (tests apply all migrations and validate).
Why `DATETIME(6)`?	Matches Java `LocalDateTime` precision; Hibernate validation expects it.
What protects against negative stock?	Row locks in the service and the `CHECK (stock_quantity >= 0)` constraint.
What is the difference between Terraform state and AWS state?	Terraform state is Terraform's last known record; AWS is the truth. `terraform plan` refreshes and compares; differences are drift (e.g. a stopped Jenkins without public IP).

CHAPTER 9

Docker and Docker Compose

9.1 What Docker does (and does not do) in this project

In general: a **container image** is a read-only, layered filesystem plus start-up metadata; a **container** is a process started from it, isolated with Linux namespaces and limited with cgroups, **sharing the host kernel**. A virtual machine instead runs a whole guest OS on virtualised hardware. Containers start in seconds and are small; VMs give stronger isolation.

In this project Docker:

- builds reproducible images for the backend and frontend (the same images run in Compose, minikube and EKS);
- runs the local full stack (MySQL + backend + frontend) and optional ELK with Docker Compose;
- runs the local CI controller and the Ansible controller (`ansible/run-playbook.sh`) as throwaway containers.

Docker does **not** schedule containers across machines, restart them on other hosts, load-balance or do rolling updates on AWS — that is Kubernetes (EKS). It also does not scan images itself (Trivy does) or store them centrally (ECR does).

9.2 Backend `Dockerfile` (line by line)

```
FROM maven:3.9-eclipse-temurin-17-alpine AS builder
```

Stage 1 ("builder"): an image with Maven and JDK 17. Named so stage 2 can copy from it.

```
WORKDIR /app
COPY pom.xml .
RUN mvn -B dependency:go-offline
```

Copy **only** `pom.xml` first and download all dependencies. Docker caches each instruction as a layer; as long as `pom.xml` is unchanged, this slow layer is reused and a code-only change rebuilds in seconds. `-B` = batch mode (no colours/prompts, CI-friendly).

```
COPY src ./src
RUN mvn -B clean package -DskipTests
```

Copy sources and build the fat JAR. Tests are skipped here because Jenkins already ran them in "Backend Tests" against the same commit (comment in the file); running them twice would only slow the build.

```
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
```

Stage 2: a **JRE-only** Alpine image (no Maven, no compiler, no source code) — smaller and with fewer packages that could contain vulnerabilities. Only what stage 2 copies ends up in the final image.

```
RUN apk upgrade --no-cache \
 && addgroup -S -g 10001 app && adduser -S -u 10001 -G app app
```

```
COPY --from=builder --chown=app:app /app/target/*.jar app.jar
USER 10001:10001
```

Copy the JAR from stage 1, owned by the app user. Run as **numeric** UID 10001: Kubernetes' `runAsNonRoot: true` can only verify a numeric user (a name could map to root).

```
ENV JAVA_OPTS="-XX:MaxRAMPercentage=75.0 -XX:+ExitOnOutOfMemoryError"
EXPOSE 8080
```

Heap = 75 % of the **container's** memory limit (1 GiB limit on EKS → ~768 MiB heap), leaving room for metaspace/threads. On OutOfMemoryError the JVM exits so the orchestrator restarts it, instead of limping on. `EXPOSE` documents the port (it does not publish it).

```
HEALTHCHECK --interval=15s --timeout=3s --start-period=60s --retries=3 \
  CMD wget -qO- http://127.0.0.1:8080/api/health > /dev/null || exit 1
```

Docker-level health check (used by plain Docker; Compose overrides it with `/api/health/ready`; Kubernetes ignores `HEALTHCHECK` and uses its own probes). 60 s start period for JVM start-up.

```
ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar app.jar"]
```

`sh -c` expands `$JAVA_OPTS`; `exec` **replaces** the shell with the JVM so Java is PID 1 and receives `SIGTERM` directly on `docker stop` / pod termination → Spring's graceful shutdown runs. Without `exec`, the shell would get the signal and the JVM would be killed after the grace period.

`.dockerignore` excludes `.git`, `target/`, IDE files, logs, `.env*`, `*.pem`, `*.key`, `README.md` from the **build context** (the files sent to the Docker daemon): faster builds and no secrets copied into a layer.

9.3 Frontend `Dockerfile` (line by line)

```
FROM node:24-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --no-audit --no-fund
COPY . .
RUN npm run build
```

Same caching idea: dependencies from `package.json` + `package-lock.json` first. `npm ci` installs **exactly** the lock file (fails if it does not match `package.json`) — reproducible; `--no-audit --no-fund` keeps CI output quiet (Trivy and `npm audit` are run separately). `npm run build` writes `dist/`.

```
FROM nginxinc/nginx-unprivileged:1.30-alpine
USER root
RUN apk upgrade --no-cache
USER 101
```

Runtime: the **unprivileged** nginx image runs as UID 101 and listens on 8080 (ports < 1024 need root). Temporarily root only to apply package updates.

```
ENV BACKEND_URL=http://backend:8080 \
    NGINX_ENTRYPOINT_LOCAL_RESOLVERS=1
```

Default upstream for `/api` = the Compose service name `backend`. `NGINX_ENTRYPOINT_LOCAL_RESOLVERS=1` asks the image's entrypoint to export the container's DNS server(s) from `/etc/resolv.conf` as `NGINX_LOCAL_RESOLVERS`, used by the `resolver` directive in the template.

```
COPY nginx/default.conf.template /etc/nginx/templates/default.conf.template
COPY nginx/security-headers.conf /etc/nginx/snippets/security-headers.conf
COPY --from=builder /app/dist /usr/share/nginx/html
EXPOSE 8080
HEALTHCHECK --interval=15s --timeout=3s --retries=3 CMD wget -qO- http://127.0.0.1:8080/healthz >
/dev/null || exit 1
```

The official nginx image renders every `/etc/nginx/templates/*.template` with `envsubst` at start-up into `/etc/nginx/conf.d/`, substituting **only variables that exist in the environment** — so nginx's own `$uri`, `$host` etc. stay untouched. The built SPA becomes the web root.

9.4 nginx configuration (`frontend/nginx/default.conf.template`)

Lines	Directive	Purpose
6–7, 10	`log_format with_request_id … rid=$request_id` / `access_log`	every access line contains nginx's per-request id, which is also sent to the backend → one id in both logs
11–14	`listen 8080`, `root`, `server_tokens off`	unprivileged port; hide the nginx version
18	`resolver ${NGINX_LOCAL_RESOLVERS} valid=10s ipv6=off`	resolve the backend name **at request time** (cached 10 s) rather than once at start-up, so nginx starts even if the backend is not up yet and follows IP changes
20–22	`gzip …`	compress text assets > 1 KB
24	`include …/security-headers.conf`	`nosniff`, `X-Frame-Options DENY` (no clickjacking), `Referrer-Policy`
27–31	`location = /healthz { return 200 "ok\n"; }`	probe endpoint independent of the backend (frontend liveness/readiness + ALB health check)
36–47	`location /api/` → `set $backend_upstream ${BACKEND_URL}; proxy_pass $backend_upstream;` + headers `Host`, `X-Request-Id $request_id`, `X-Real-IP`, `X-Forwarded-For`, `X-Forwarded-Proto`; connect timeout 5 s, read timeout 30 s	same-origin API for Compose/port-forward; using a **variable** in `proxy_pass` forces runtime DNS resolution
50–55	`location /assets/` → `expires 1y`, `Cache-Control "public, immutable"`, `try_files $uri =404`	hashed filenames can be cached forever; missing asset = real 404
58–62	`location /` → `Cache-Control "no-cache"`, `try_files $uri $uri/ /index.html`	SPA fallback for deep links; `index.html` always revalidated so new releases load immediately

Security headers are included again in each `location` that sets its own `add_header`, because nginx does not inherit server-level `add_header` into such blocks (comment in `security-headers.conf`).

9.5 Docker Compose (`order-inventory-platform/docker-compose.yml`)

Service	Key settings	Explanation
`mysql`	`image: mysql:8.4`, `restart: unless-stopped`, env `MYSQL_ROOT_PASSWORD`, `MYSQL_DATABASE=order_inventory_db`, `MYSQL_USER` / `MYSQL_PASSWORD` (from `.env` or local defaults), `ports: "${MYSQL_HOST_PORT:-3307}:3306"`, volume `inventory-mysql-data:/var/lib/mysql`, healthcheck `mysqladmin ping` (10 s, 12 retries, 30 s start period)	The app uses a **non-root application account** limited to its database. Host port 3307 avoids clashing with a local MySQL on 3306. The **named volume** keeps data across `docker compose down` / `up` (only `down -v` deletes it).
`backend`	`build: ./backend`, `image: inventory-backend:${IMAGE_TAG:-local}`, `ports: 8080:8080`, env `SPRING_DATASOURCE_URL=jdbc:mysql://mysql:3306/…`, user/password, `SPRING_FLYWAY_LOCATIONS=classpath:db/migration,classpath:db/seed`, `APP_ENVIRONMENT=local`, `depends_on: mysql: condition: service_healthy`, healthcheck `/api/health/ready`	**Service discovery**: on the user-defined bridge network, Docker's DNS resolves the service name `mysql` to the MySQL container. The backend starts only after MySQL is healthy; its own health requires the DB. Demo data is seeded locally only.
`frontend`	`build: ./frontend`, `ports: "${FRONTEND_HOST_PORT:-3000}:8080"`, `BACKEND_URL=http://backend:8080`, `depends_on: backend: condition: service_healthy`	Container port 8080 is published on host port 3000 → http://localhost:3000.
networks / volumes	`inventory-network` (bridge), `inventory-mysql-data`	isolated network for the three services; persistent data

Container port vs host port: `"3000:8080"` = host port 3000 forwards to container port 8080. Inside the network, containers talk to each other on container ports (`backend:8080`), never via host ports.

Restart policy: `unless-stopped` restarts a crashed container (verified: JVM killed from inside → restart count 0 → 1 in `compose-acceptance-test.sh`); it does not restart a container stopped by the user.

Start order and readiness: `depends_on` with `condition: service_healthy` waits for the health check, not just for the container to start. MySQL health = server answers; backend health = DB reachable.

`.env.example` documents the variables (placeholders `change-me-*`); copy it to `.env` (git-ignored).

Local commands

```
cd order-inventory-platform
Copy-Item .env.example .env                # optional: set your own local passwords/ports
docker compose up -d --build               # build images and start mysql, backend, frontend
docker compose ps                          # all three "healthy"
docker compose logs -f backend             # follow backend logs (Ctrl+C to stop following)
Invoke-RestMethod http://localhost:3000/api/health/ready
bash ../scripts/smoke-test.sh http://localhost:3000          # 20 functional checks (Git Bash)
docker compose -f docker-compose.yml -f docker-compose.logging.yml up -d   # optional ELK, Kibana on
http://localhost:5601
docker compose stop                        # stop, keep containers and data
docker compose down                        # remove containers and network, KEEP the data volume
# docker compose down -v                   # would DELETE the database volume - do not use routinely
```

Tests in containers: backend `mvn -B verify` and frontend `npm test` run on the host or in Jenkins, not in Compose. `scripts/compose-acceptance-test.sh` runs the whole Compose scenario (build, health, smoke test, persistence across recreation, DB outage → readiness/API 503 while liveness 200 → automatic recovery, crash auto-restart,

9.6 Image tagging and scanning

Context	Tag	Example
Compose	`inventory-backend:${IMAGE_TAG:-local}`	`inventory-backend:local`
Jenkins local name	`order-inventory-<svc>:<build>-<commit>`	`order-inventory-backend:8-18a2bc6`
ECR	`<account>.dkr.ecr.ap-south-1.amazonaws.com/order-inventory-<svc>:<build>-<commit>`	`…/order-inventory-backend:8-18a2bc6`

Jenkins also labels images with `org.opencontainers.image.revision=<full commit>` and `org.opencontainers.image.version=<tag>`. Trivy scans every image before it is pushed (fixable HIGH/CRITICAL fail the build) and ECR scans again on push. Latest results: 0 vulnerabilities in both images (Trivy 0.76.0, Alpine 3.24.2 — `docs/TESTING.md`); this means no **known, fixable** CVEs in the scanned packages at scan time, not that the images are free of all vulnerabilities.

9.7 Local vs production containers

Aspect	Compose	EKS
Database	MySQL container, plaintext (`useSSL=false`) on a private bridge network	RDS, TLS required
Health checks	Docker `healthcheck` (backend: readiness)	Kubernetes startup/liveness/readiness probes
Restart	`restart: unless-stopped`	Deployment/ReplicaSet controllers recreate pods anywhere
Filesystem	writable	backend `readOnlyRootFilesystem: true` + `/tmp` emptyDir; all capabilities dropped
Config/secrets	`.env`, environment	ConfigMap + KMS-encrypted Secret
Logs	plain text (ECS JSON with the logging overlay)	ECS JSON → Filebeat → Elasticsearch
Images	built locally	pulled from ECR by immutable tag

9.8 Viva questions

Question	Answer
Why multi-stage builds?	The final image contains only the runtime and the artefact, not build tools or source: smaller, faster to pull, fewer CVEs.
Why copy `pom.xml` / `package*.json` before the source?	Layer caching: dependencies are only downloaded again when the dependency files change.
Why non-root and a numeric UID?	Limits damage if the app is compromised; Kubernetes `runAsNonRoot` needs a numeric UID to verify.
Why `exec` in the entrypoint?	So the JVM is PID 1 and receives SIGTERM for graceful shutdown.
Container vs VM?	Container = isolated process sharing the host kernel; VM = full guest OS on a hypervisor.
How does the backend find MySQL in Compose?	Docker's embedded DNS resolves the service name `mysql` on the shared bridge network.
What does `depends_on: condition: service_healthy` add?	Waits for the dependency's health check to pass, not just for it to start.

	components at that time.

CHAPTER 10

Terraform: every file and resource

Terraform describes the desired AWS infrastructure in HCL files. `terraform plan` compares that description with the **state** (Terraform's record of what it created) refreshed against the real AWS APIs, and shows the differences; `terraform apply` makes them real. In this project the main stack manages **102 resources** (`docs/AWS_DEPLOYMENT.md`), plus 5 in the bootstrap stack.

Standard commands (from `terraform/`, profile exported, `admin.auto.tfvars` last so your real IP wins over the placeholder in the profile file):

```
export AWS_PROFILE=order-inventory AWS_REGION=ap-south-1
terraform init                                          # downloads providers, connects to the S3 backend
terraform fmt -check -recursive && terraform validate   # formatting + static validation
terraform plan -var-file=environments/full.tfvars -var-file=admin.auto.tfvars   # read-only diff
```

Never run `apply` or `destroy` to "demonstrate" something; a plan is enough to explain behaviour.

10.1 Files and variables

`backend.tf`

```
terraform { backend "s3" {
  bucket = "order-inventory-tfstate-038215676861"   # created by terraform/bootstrap
  key    = "order-inventory/terraform.tfstate"      # object path of the state
  region = "ap-south-1"
  encrypt      = true                               # server-side encryption of the state object
  use_lockfile = true                               # S3-native locking (Terraform >= 1.10): a .tflock
object
} }
```

Remote state lets the scripts (`tfout` in `scripts/lib/evaluation-common.sh`), Jenkins and any laptop read the same state. **Locking** prevents two `apply`s at the same time (the second waits/fails instead of corrupting state). The bucket name is not a secret.

`bootstrap/main.tf` — the chicken-and-egg

The state bucket must exist before the main stack can use it, so a tiny separate stack (with **local** state, containing no secrets) creates it once: `aws_s3_bucket.state` (name includes the account id → globally unique), `aws_s3_bucket_versioning` (every state version kept → recover from a bad apply), `aws_s3_bucket_server_side_encryption_configuration` (AES256), `aws_s3_bucket_public_access_block` (all four blocks on), `aws_s3_bucket_policy.state_tls_only` (deny any request with `aws:SecureTransport = false`). `default_tags` add `Project`, `ManagedBy`, `Purpose` to everything.

`main.tf` header

```
required_version = ">= 1.10.0"                          # S3 native locking needs 1.10+
required_providers { aws = "~> 5.0", tls = "~> 4.0", random = "~> 3.6" }   # "~>" = allow minor/patch
updates only
provider "aws" { region = var.aws_region
  default_tags { tags = { Project = var.project_name, Environment = var.environment, ManagedBy =
"Terraform" } } }
data "aws_caller_identity" "current" {}                # account id for ARNs/policies
locals {
  eks_public_access_cidrs = length(var.eks_public_access_cidrs) > 0 ? var.eks_public_access_cidrs :
var.admin_cidr_blocks
  cluster_name = "${var.project_name}-eks"   # order-inventory-eks
  app_namespace = "order-inventory"
}
```

Stale comment: lines 18–19 still say "State is local by default … backend.tf.example"; the S3 backend in `backend.tf` is what is actually used.

`default_tags` is why the scripts can find project resources by tag (e.g. the NAT gateway with `tag:Project=order-inventory`) and why Cost Explorer can group costs by project.

`.terraform.lock.hcl` pins the exact provider versions and checksums chosen at `init` so every machine uses the same provider build.

`variables.tf` (inputs and guardrails)

Variable	Default	Notes
`aws_region`	`ap-south-1`	
`project_name`, `environment`	`order-inventory`, `dev`	name prefix, tags
`vpc_cidr`	`10.0.0.0/16`	65,536 addresses
`availability_zones`	`["ap-south-1a","ap-south-1b"]`	two AZs (EKS requires subnets in ≥ 2 AZs)
`public_subnet_cidrs` / `private_subnet_cidrs` / `database_subnet_cidrs`	`10.0.1.0/24, 10.0.2.0/24` / `10.0.11.0/24, 10.0.12.0/24` / `10.0.21.0/24, 10.0.22.0/24`	three tiers
`eks_version`	`1.36`	description warns that extended-support versions cost 6× more per hour
`eks_public_access_cidrs`	`[]` (→ admin CIDRs)	**validation:** must not contain `0.0.0.0/0`
`node_instance_types`, `node_desired_size`, `node_min_size`, `node_max_size`	`t3.medium`, 2, 1, 3	the full profile overrides min to **0** for suspension
`db_engine_version`, `db_instance_class`, `db_username`, `db_backup_retention_days`	`8.4`, `db.t3.micro`, `admin`, 7	8.0 would be in paid extended support
`create_jenkins`	`true`	learning profile sets `false`
`admin_cidr_blocks`	`[]`	**validation:** no `0.0.0.0/0`, and required when Jenkins is created

`monthly_budget_usd`, `log_retention_days`, `create_alert_audit_queue`, `alert_email`	100, 14, true, `""`	full profile: budget 350

Variable files: `terraform.tfvars` (auto-loaded, non-secret network values), `environments/full.tfvars` (Jenkins, 2 nodes, min 0, max 3, t3.large, budget 350, **placeholder** admin CIDR `203.0.113.10/32` — a documentation address), `environments/learning.tfvars` (no Jenkins, 1 node, 1-day backups, budget 100), and the git-ignored `admin.auto.tfvars` (your real `/32`), passed **last** on the command line so it overrides the placeholder.

10.2 Network

Resource (block)	Why it exists	Security	Billing	During suspension	If deleted
`aws_vpc.main`	private network 10.0.0.0/16; DNS support + hostnames on (needed by EKS/RDS endpoints)	isolation boundary	free	kept	everything inside is lost
`aws_internet_gateway.igw`	internet access for public subnets	—	free	kept	ALB/Jenkins/NAT unreachable
`aws_subnet.public[0..1]`	ALB, NAT, Jenkins; tag `kubernetes.io/role/elb=1` lets the Load Balancer Controller auto-discover where to place internet-facing ALBs; `map_public_ip_on_launch = false`	nothing gets a public IP implicitly	free	kept	no ALB placement
`aws_subnet.private[0..1]`	EKS nodes/pods; tag `kubernetes.io/role/internal-elb=1`	no public IPs	free	kept	nodes cannot start
`aws_subnet.database[0..1]`	RDS only	own route table without internet routes	free	kept	RDS subnet group invalid
`aws_eip.nat`	static public IP of the NAT gateway (3.111.24.35)	—	public IPv4 ≈ $0.005/h	kept	NAT broken
`aws_nat_gateway.nat`	outbound internet for private subnets (pull images from ECR, reach AWS APIs, join the cluster); single NAT in public subnet 1, `depends_on` IGW	inbound connections impossible	≈ $0.056/h + data	**kept** (see the suspension table below)	nodes cannot join/pull images
`aws_route_table.public` + 2 associations	`0.0.0.0/0 → IGW`	—	free	kept	public subnets offline
`aws_route_table.private` + 2 associations	`0.0.0.0/0 → NAT`	—	free	kept	nodes have no egress
`aws_route_table.database` + 2 associations	**no routes** (only the implicit local route)	DB tier isolated from the internet	free	kept	—

10.3 Container registry

Resource	Details
`aws_ecr_repository.backend`, `.frontend`	`order-inventory-backend/-frontend`; `image_tag_mutability = "IMMUTABLE"` (a pushed tag can never be overwritten), `scan_on_push = true`, `force_delete = true` (allows destroy with images — another reason never to destroy casually). Billing: storage only (~150 MB). Kept while suspended.

The unrelated repository `frontend` in the same account is **not** managed by Terraform and must be left alone.

10.4 EKS

Resource	Why / key settings	Security	Billing	Suspension
`aws_security_group.eks_cluster` + egress rule	additional SG on the control-plane ENIs; Jenkins → API rule added in `jenkins.tf`	ingress only from Jenkins SG on 443	free	kept
`aws_iam_role.eks_cluster` (trust `eks.amazonaws.com`) + `AmazonEKSClusterPolicy`	lets the EKS service manage ENIs etc.	AWS-managed policy	free	kept
`aws_eks_cluster.main`	name `order-inventory-eks`, version 1.36; `access_config` `API_AND_CONFIG_MAP` (access entries + legacy aws-auth) with `bootstrap_cluster_creator_admin_permissions = true` (the deployer user that created it is cluster admin); `vpc_config` in the **private** subnets, endpoint **private + public**, `public_access_cidrs = local.eks_public_access_cidrs`; `encryption_config` = envelope-encrypt Kubernetes `secrets` with `aws_kms_key.app`; `precondition`: allow-list must not be empty; `depends_on` the IAM attachments	public API only for the admin /32; nodes and Jenkins use the private endpoint	**≈ $0.10/h, always** (even with 0 nodes)	kept
`aws_iam_role.eks_nodes` (trust `ec2.amazonaws.com`) + `AmazonEKSWorkerNodePolicy`, `AmazonEKS_CNI_Policy`, `AmazonEC2ContainerRegistryReadOnly`	what the nodes may do: join the cluster, manage pod IPs (VPC CNI), pull from ECR	managed policies	free	kept
`aws_eks_node_group.main`	`order-inventory-nodes`, private subnets, `t3.medium` ON_DEMAND, `scaling_config` desired/min/max from variables, `update_config.max_unavailable = 1`, `lifecycle { ignore_changes = [scaling_config[0].desired_size] }`	nodes in private subnets, EKS-managed cluster SG	2 × t3.medium ≈ $65/month total when running	**desired 0** (definition kept)
`data.tls_certificate.eks` + `aws_iam_openid_connect_provider.eks`	registers the cluster's OIDC issuer in IAM → **IRSA** (pods assume IAM roles via service-account tokens)	thumbprint from the issuer certificate	free	kept
`aws_iam_role.ebs_csi` (+ policy doc with conditions `sub = system:serviceaccount:kube-system:ebs-csi-controller-sa`, `aud = sts.amazonaws.com`) + `AmazonEBSCSIDriverPolicy`	role for the EBS CSI driver add-on	only that service account can assume it	free	kept

`.kube_proxy`	managed add-ons; `resolve_conflicts_on_* = OVERWRITE`; `depends_on` the node group (their pods need nodes)	—	free	kept (pods return with nodes)

The node-group `ignore_changes`, explained. The suspend script sets desired = 0 and the resume script sets it back to 2 with the AWS API. Without `ignore_changes`, the next `terraform plan` would show `desired_size: 0 → 2` (drift) and an `apply` would start nodes in the middle of a suspension. Ignoring only `desired_size` hands that one attribute to the scripts while Terraform still manages instance type, subnets, min and max. `node_min_size = 0` in `full.tfvars` was applied on 2026-10-19 (plan: exactly one in-place change `min_size 1 → 0`), because AWS rejects desired < min. Resume uses min 1 while running.

10.5 Database (`main.tf`, explained in detail in MySQL, RDS and Flyway)

Resource	Key settings
`aws_security_group.rds` + `aws_vpc_security_group_ingress_rule.rds_mysql`	3306 only from `aws_eks_cluster.main.vpc_config[0].cluster_security_group_id` (the EKS-created SG attached to every managed node)
`aws_subnet.database`, `aws_route_table.database`, associations	isolated DB tier
`random_password.db_master`	32 characters with a restricted special-character set (MySQL-safe); generated once, stored in state
`aws_secretsmanager_secret.db_master` + `_version`	`order-inventory/rds-master`, JSON `{username,password,host,port,dbname}`, 7-day recovery window
`aws_db_subnet_group.mysql`	the two database subnets
`aws_db_parameter_group.mysql`	family `mysql8.4`: `require_secure_transport=1`, slow log, buffer pool 3/8 of RAM, `performance_schema=0` (pending-reboot); `create_before_destroy` so a replacement group exists before the old one is removed
`aws_db_instance.mysql`	`order-inventory-mysql`, MySQL 8.4, db.t3.micro, gp3 20→50 GB, encrypted, `publicly_accessible = false`, 7-day backups, windows, CloudWatch log exports, `skip_final_snapshot = true`, `deletion_protection = false`, `depends_on` the log groups

Billing: instance hours (≈ $20.87/month estimate incl. storage when running), storage + backups beyond the free allowance when stopped. Suspension: **stopped** (storage and backups kept). Deleting it: all data gone (no final snapshot in this configuration). Secret: the password exists in the encrypted S3 state, Secrets Manager and the KMS-encrypted Kubernetes Secret — never in Git or logs.

10.6 Jenkins (`jenkins.tf`)

Resource	Key settings and reasons
`locals.jenkins_count`	1 or 0 from `create_jenkins` (all Jenkins resources use `count` / `for_each`)
`aws_security_group.jenkins`	named `order-inventory-jenkins-sg`
`aws_vpc_security_group_ingress_rule.jenkins_ssh` / `jenkins_web` (`for_each = toset(var.admin_cidr_blocks)`)	22 and 8080 **only** from each admin CIDR. `for_each` keyed by the CIDR string → when your IP changes, the old rule is destroyed and a new one created (this is what the resume script's targeted apply does)
`aws_vpc_security_group_egress_rule.jenkins_all_egress`	outbound anywhere (GitHub, ECR, package mirrors) — accepted risk AVD-AWS-0104

`aws_iam_role.jenkins` (trust `ec2.amazonaws.com`)	instance role → **no AWS keys stored in Jenkins**
`aws_iam_role_policy.jenkins_ecr_push`	`ecr:GetAuthorizationToken` (account-wide by design) + push/pull actions **only on the two app repositories**
`aws_iam_role_policy.jenkins_eks_access`	`eks:DescribeCluster` on this cluster only (needed by `aws eks update-kubeconfig`)
`aws_iam_role_policy.jenkins_ci_reports`	`s3:PutObject` into the CI-reports bucket only
`aws_iam_instance_profile.jenkins`	attaches the role to the instance
`data.aws_ami.ubuntu`	newest Canonical (`099720109477`) Ubuntu 24.04 amd64 gp3 AMI
`aws_instance.jenkins`	public subnet 1, key pair `UniversalKey`, the SG, the profile, `associate_public_ip_address = true`, **IMDSv2 required** (`http_tokens = "required"`, hop limit 2 so containers started by builds can still reach the metadata service), encrypted gp3 30 GB root volume, `lifecycle { ignore_changes = [ami, associate_public_ip_address] }`
`aws_eks_access_entry.jenkins` + `aws_eks_access_policy_association.jenkins`	the Jenkins role may use the cluster with `AmazonEKSEditPolicy` **scoped to the `order-inventory` namespace** (it cannot touch `kube-system`, other namespaces or cluster-scoped objects)
`random_password.ci_admin` (`jenkins-admin`, `sonarqube-admin`) + `aws_secretsmanager_secret.ci_admin` + `_version`	generated admin passwords (28 chars, mixed classes) in Secrets Manager `order-inventory/jenkins-admin` and `/sonarqube-admin`; read by `ansible/run-playbook.sh`

Why `ignore_changes = [ami, associate_public_ip_address]` was needed

- `ami`: `data.aws_ami.ubuntu` uses `most_recent = true`. When Canonical publishes a new image, the AMI id changes, and changing `ami` **forces replacement** of an instance. Without the ignore, a routine plan would propose destroying and recreating Jenkins (losing its disk).
- `associate_public_ip_address`: found during the suspension on 2026-10-19. A **stopped** instance has no public IP, and the AWS API then reports `associate_public_ip_address = false`. Terraform compared that with the configuration (`true`) and planned `false → true # forces replacement` — i.e. "destroy Jenkins and create a new one". Nothing was applied; the attribute was added to `ignore_changes`, after which the plan while suspended showed **no resource changes**, only the stale `jenkins_public_ip` / `jenkins_url` outputs.

Limits and risks of `ignore_changes` (do not present it as a universal fix): Terraform stops managing the ignored attribute, so a deliberate change (e.g. you really want a new AMI or no public IP) is silently not applied — you must use `terraform apply -replace=aws_instance.jenkins[0]` or remove the ignore temporarily. It hides drift for that attribute. It is appropriate here because the attribute only matters at creation time (AMI, public-IP association) or is intentionally controlled outside Terraform (node count).

10.7 Encryption, storage, monitoring

File / resource	Purpose
`kms.tf` `aws_kms_key.app` (rotation on, 7-day deletion window) + `aws_kms_alias.app` (`alias/order-inventory-app`)	customer-managed key (≈ $1/month). Key policy: account root administers; **CloudWatch and Budgets** may `Decrypt` / `GenerateDataKey*` (needed to publish to an encrypted SNS topic — the AWS-managed `aws/sns` key does not allow that, so alarms would be silently undelivered); **SNS** may use it (deliver to the encrypted SQS queue)
`aws_iam_role_policy.eks_cl-`	

`uster_kms`	the EKS cluster role may Encrypt/Decrypt/CreateGrant on this key for Secret envelope encryption
`storage.tf` `aws_s3_bucket.ci_reports` (`bucket_prefix`) + public access block + `BucketOwnerEnforced` + SSE-S3 + versioning + lifecycle (expire 30 days, non-current 7 days, abort multipart 1 day) + TLS-only policy	Jenkins uploads per-build reports to `s3://<bucket>/<job>/<build>/`; `force_destroy = true` (reproducible content)
`monitoring.tf`	SNS topic (KMS), topic policy, SQS audit queue + policy + subscription, optional email subscription, 9 alarms, 2 RDS log groups (14 days), monthly budget — see Monitoring and logging

10.8 Outputs (`outputs.tf`)

22 outputs, e.g. `eks_cluster_name`, `rds_endpoint`, `jenkins_instance_id`, `jenkins_public_ip`, `jenkins_url`, `aws_lbc_role_arn`, `db_secret_arn`, `alerts_sns_topic_arn`, `ci_reports_bucket`, `backend/frontend_ecr_repository_url`. Scripts read them with `terraform output -raw <name>` instead of hard-coding IDs (`tfout` in the evaluation library; `create-db-secret.sh`; `install-lb-controller.sh`; `create-alb-alarms.sh`; `run-playbook.sh`). Outputs are values **from state**: `jenkins_public_ip` still shows the last IP Terraform saw. After Jenkins restarts with a new IP, run `terraform apply -refresh-only -var-file=environments/full.tfvars -var-file=admin.auto.tfvars` (updates state/outputs only, changes no resources) before `ansible/run-playbook.sh`.

10.9 Every AWS resource: suspension, restoration and cost

Resource	Remains during suspension?	Restored by	Billed while suspended?
VPC, subnets, route tables, IGW, SGs, IAM roles/policies, OIDC provider, add-on definitions	yes	—	no
NAT gateway + EIP	yes	—	**yes** (≈ $0.056/h + ≈ $0.005/h)
EKS control plane	yes (ACTIVE)	—	**yes** (≈ $0.10/h)
Node group	definition yes, 0 instances	resume: `update-nodegroup-config`	no compute
RDS instance	stopped	resume: `start-db-instance`	storage/backups only
Jenkins EC2	stopped, EBS kept	resume: `start-instances`	EBS only
ECR repos/images	yes	—	storage
Secrets Manager ×3, KMS key	yes	—	yes ($0.40/secret, $1/key per month)
S3 buckets (state, CI reports)	yes	—	negligible
CloudWatch alarms (9 Terraform)	yes (INSUFFICIENT_DATA)	—	$0.10/alarm/month
ALB + target groups + 4 ALB alarms (not Terraform-managed)	**deleted**	resume: Helm Ingress + `create-alb-alarms.sh`	no

(`docs/AWS_DEPLOYMENT.md`, "Cost while suspended"). The EKS control plane cannot be paused.

10.10 Plan, refresh, apply, state and drift

- **refresh**: Terraform reads the real attributes of every resource in state (part of every plan).
- **plan**: refresh + diff with the configuration; nothing changes. Exit code with `-detailed-exitcode`: 0 = no changes, 2 = changes (including output-only changes), 1 = error.
- **apply**: executes a plan; `apply <saved-plan>` executes exactly that plan.
- **targeted plan** (`-target=…`): limits the plan to some resources and their dependencies; used by the resume script for the admin IP only (dry-run on 2026-10-25: *2 to add, 1 to change, 2 to destroy* — the two SG rules replaced and the cluster updated in place). Targeting is for exceptional, controlled cases.
- **drift**: real infrastructure differs from configuration/state. Checks performed: after apply (no changes), after the node-group change (no changes), while suspended (no resource changes after the Jenkins fix).
- **state is sensitive**: it contains every attribute, including `random_password` results. That is why the state is only in a private, encrypted, versioned, TLS-only bucket and `*.tfstate` is git-ignored. (Before the remediation, state files with a password had been committed to the public repository — see Security.)

10.11 Destruction and dependencies

Teardown order (`docs/RUNBOOKS.md` §8): delete the Ingress first (so the controller deletes the ALB, its target groups and security groups, which Terraform does not know about and which would otherwise block VPC deletion), uninstall the controller, delete the ALB alarms, then `terraform plan -destroy` → review → `terraform destroy`. Secrets enter a 7-day recovery window and the KMS key a 7-day pending-deletion window. For this project's evaluation period, **`terraform destroy` is forbidden**; suspension is used instead.

10.12 Viva questions

Question	Answer
Difference between Terraform state and actual AWS state?	State = Terraform's stored record; AWS = reality. Plan refreshes state from AWS and shows differences (drift) against the configuration.
Why remote state with locking?	Shared, durable, encrypted state; locking prevents concurrent applies corrupting it.
Why was `ignore_changes` required for the Jenkins instance?	A stopped instance reports no public IP association and new AMIs change the AMI id; both force replacement. Ignoring these creation-time attributes prevents destroying Jenkins.
Why does the node-group desired size live outside Terraform?	Suspend/resume change it at runtime; Terraform must not revert it.
How does Terraform get the DB password without anyone typing it?	`random_password` → Secrets Manager → `create-db-secret.sh` → Kubernetes Secret.
Why three subnet tiers?	Public (ALB, NAT, Jenkins), private (nodes with NAT egress), database (no internet route at all).
What does IRSA give you?	Pods get short-lived AWS credentials for one IAM role bound to one service account, instead of using the node's role.
What happens to the ALB on `terraform destroy`?	Nothing — Terraform did not create it; it must be removed first by deleting the Ingress, otherwise VPC deletion fails.
Why is `terraform.tfvars` committed but `admin.auto.tfvars` not?	The first contains only non-sensitive defaults; the second contains your personal IP.

CHAPTER 11

Ansible

11.1 Terraform vs Ansible in this project

	Terraform	Ansible
Works on	AWS APIs	the operating system of a host over SSH
Creates	the Jenkins EC2 instance, its security group, IAM role, generated admin passwords	packages, users, services, files and containers **inside** that instance
Model	declarative desired state + state file	ordered tasks, each idempotent (check, then change only if needed)
Here	`terraform/jenkins.tf`	`ansible/playbooks/jenkins.yml` + 7 roles

Terraform's comment in `jenkins.tf` states the split: "Terraform creates the instance, network access and IAM permissions only. Software ... is installed and kept up to date by Ansible". Ansible is **not** used for application deployments: the application is deployed by Jenkins with Helm (`scripts/deploy-k8s.sh`), because Kubernetes provides rolling updates, health checks and rollback that a host-level tool would have to reimplement.

11.2 Running it: `run-playbook.sh` (line by line)

```
win() { if command -v cygpath >/dev/null; then cygpath -w "$1"; else echo "$1"; fi; }
ANSIBLE_DIR="$(cd "$(dirname "$0")" && pwd)"
TF_DIR="$(win "$(cd "$ANSIBLE_DIR/../terraform" && pwd)")"
: "${SSH_KEY:?Set SSH_KEY to the private key of the EC2 key pair}"
export MSYS_NO_PATHCONV=1
```

Windows/Git Bash helpers: `cygpath -w` converts `/c/...` paths for the Windows `terraform.exe` and Docker; `MSYS_NO_PATHCONV=1` stops Git Bash from rewriting Linux paths like `/ansible` in `docker run` arguments. The script requires `SSH_KEY` (path to `UniversalKey.pem`).

```
HOST="$(terraform -chdir="$TF_DIR" output -raw jenkins_public_ip)"
BUCKET="$(terraform -chdir="$TF_DIR" output -raw ci_reports_bucket)"
REPO_URL="${JENKINS_REPO_URL:-file:///srv/git/devops-project.git}"
LOCAL_CHECKOUT=false; case "$REPO_URL" in file://*) LOCAL_CHECKOUT=true ;; esac
```

Target host and bucket from Terraform outputs. **Important trap:** the default `REPO_URL` is still the old on-host Git mirror (used while GitHub had the uncleaned history). Since 2026-10-19 the job must use GitHub, so **always** run it as:

```
export AWS_PROFILE=order-inventory AWS_REGION=ap-south-1
terraform -chdir=terraform apply -refresh-only -var-file=environments/full.tfvars -var-
file=admin.auto.tfvars   # update jenkins_public_ip after a stop/start (state only)
SSH_KEY=/c/Users/Siddesh/Downloads/UniversalKey.pem \
JENKINS_REPO_URL=https://github.com/SamruddhiDoke/DevOps-Project.git ./ansible/run-playbook.sh --check   #
dry run
SSH_KEY=/c/Users/Siddesh/Downloads/UniversalKey.pem \
JENKINS_REPO_URL=https://github.com/SamruddhiDoke/DevOps-Project.git ./ansible/run-playbook.sh           #
apply
```

Without `JENKINS_REPO_URL`, the job would be switched back to the (stale) mirror. Also: the jenkins_casc role restarts Jenkins when configuration changes — a running build is then interrupted (that is exactly what made build #6 fail on 2026-10-19), so do not run the playbook while a build is in progress.

```
secret() { aws secretsmanager get-secret-value --secret-id "$1" --query SecretString --output text | tr -d
'\r'; }
node -e '…process.stdout.write(JSON.stringify({ jenkins_casc_admin_password: jp, sonarqube_admin_password:
sp,
   jenkins_casc_repo_url: url, jenkins_casc_allow_local_checkout: local === "true",
   jenkins_casc_ci_reports_bucket: bucket, jenkins_casc_sonar_host_url: "http://127.0.0.1:9000" }));' \
   "$(secret order-inventory/jenkins-admin)" "$(secret order-inventory/sonarqube-admin)" "$REPO_URL"
"$LOCAL_CHECKOUT" "$BUCKET" \
| docker run --rm -i -v "$(win "$ANSIBLE_DIR")":/ansible:ro -v "$(win "$SSH_KEY")":/tmp/key.pem:ro \
    -e ANSIBLE_HOST_KEY_CHECKING=False -e ANSIBLE_FORCE_COLOR=0 python:3.12-slim sh -c '…'
```

- The two admin passwords are read from **Secrets Manager** and turned into a JSON document of Ansible extra variables, which is **piped on stdin** into a disposable `python:3.12-slim` container; inside, it is saved to `/tmp/extra-vars.json` (container-local, deleted with the container) and passed as `-e @/tmp/extra-vars.json`. So the secrets are not in the `docker run` arguments, the repository, files on the laptop or Ansible output.
- **Honest limitation:** on the laptop the secrets are passed to the local `node` process as command-line arguments (`"$(secret …)"`), so for the fraction of a second that `node` runs they are visible in the process list to other processes of the same Windows user. Reading them inside the `node` script (or piping the Secrets Manager JSON straight into the container) would remove that exposure. This was not changed while writing this document (working code is not modified for documentation).
- Inside the container: install `openssh-client`, `ansible-core==2.19.*`, the collections from `requirements.yml`; copy the key with mode 600; write a one-line inventory with the Jenkins IP; run `ansible-playbook -i /tmp/inventory.ini playbooks/jenkins.yml -e @/tmp/extra-vars.json "$@"` (so `--check` etc. are passed through). The repository is mounted **read-only**.
- `ANSIBLE_HOST_KEY_CHECKING=False` is set for this throwaway controller: the container starts with an empty `known_hosts` on every run, so it could never confirm the host key interactively. The trade-off is that the first connection is not protected against a man-in-the-middle; `ansible.cfg` keeps checking enabled for normal, persistent controllers.

11.3 Configuration files

File	Lines and meaning
`ansible.cfg`	`inventory = inventory.ini` (local, git-ignored), `roles_path = roles`, `host_key_checking = True`, `retry_files_enabled = False`, `interpreter_python = auto_silent`; `[ssh_connection] pipelining = True` (fewer SSH round-trips)
`inventory.example.ini`	`[jenkins]` group with one host `jenkins-server ansible_host=… ansible_user=ubuntu ansible_ssh_private_key_file=…` (template; `run-playbook.sh` generates its own)
`group_vars/all.yml`	pinned versions: Maven 3.9.12, Node 24.22.0, Trivy 0.76.0, Helm 4.3.1, kubectl 1.36.5, Terraform 1.15.8, SonarScanner 8.1.0.6389; `jenkins_port: 8080`; `docker_start_service: true` (false only in the container test). Kept in sync with `ci/jenkins/Dockerfile`
`requirements.yml`	collections `community.docker` (container module), `ansible.posix` (sysctl), `community.general` (git_config)

11.4 `playbooks/jenkins.yml` (line by line)

```
- name: Configure Jenkins CI server
  hosts: jenkins                # the inventory group
  become: true                  # privilege escalation: tasks run as root via sudo
  vars_files: [ ../group_vars/all.yml ]   # explicit, because run-playbook.sh's inventory lives in /tmp
  roles:
    - common
    - docker
    - build_tools
    - jenkins
    - role: git_mirror          ; when: jenkins_casc_allow_local_checkout | default(false) | bool
    - role: sonarqube           ; when: sonarqube_admin_password | default('') | length > 0
    - role: jenkins_casc        ; when: jenkins_casc_admin_password | default('') | length > 0
  post_tasks:  # print the first line of each tool's version (changed_when: false = never reports
"changed")
```

Roles run in order; the last three are conditional so the playbook can also run without secrets (base toolchain only). The post-task loop prints Java, Maven, Node, Trivy, Helm, kubectl, Terraform, SonarScanner, AWS CLI and Docker versions (`set -o pipefail; … | sed -n 1p`; `sed` instead of `head` avoids a SIGPIPE failure under `pipefail`).

11.5 Roles (task by task)

`common`

1. `apt` installs base packages (`ca-certificates curl unzip xz-utils git jq fontconfig unattended-upgrades openssh-server python3-debian`), `cache_valid_time: 3600` (no apt update if done within an hour).
2. `copy` `/etc/apt/apt.conf.d/20auto-upgrades` → daily automatic **security updates**.
3. `file` ensures `/run/sshd` exists (needed by `sshd -t`; Ubuntu 24.04 starts sshd via socket activation).
4. `copy` `/etc/ssh/sshd_config.d/10-hardening.conf`: `PasswordAuthentication no`, `KbdInteractiveAuthentication no`, `PermitRootLogin no`, with `validate: /usr/sbin/sshd -t -f %s` — an invalid file is **rejected before it replaces the old one** (prevents locking yourself out); `notify: Reload ssh` → handler reloads the service only when the file changed.

`docker`

`apt docker.io`; service started and enabled (`when: docker_start_service | bool`). Jenkins builds images with this daemon.

`build_tools`

- `openjdk-21-jdk-headless` (runs Jenkins; compiles the Java 17 backend).
- Maven, Node.js, SonarScanner: `unarchive` from the official URLs with `remote_src: true` and `creates:` pointing at the binary of that exact version → skipped when already installed (idempotent); a version change in `group_vars` installs the new version.
- Symlinks into `/usr/local/bin` (`mvn`, `node`, `npm`, `npx`, `sonar-scanner`).
- Single-binary tools: first a check task runs `trivy --version | grep -qF <version>` etc. with `failed_when: false`, `changed_when: false`; the install tasks for Trivy, Helm, kubectl (with `timeout: 120`, `retries: 3` because a download once timed out) and Terraform run **only when the check's `rc != 0`**.
- AWS CLI v2: check `/usr/local/bin/aws --version`; download + `/tmp/aws/install` only if missing (`creates: /usr/local/bin/aws`).

`jenkins`

Add the Jenkins LTS apt repository in deb822 format with its signing key (`signed_by` pinned to `jenkins.io-2026.key`), install `jenkins`, add the `jenkins` user to group `docker` (notify restart), create the systemd drop-in directory, start + enable, `flush_handlers` (apply pending restarts now), wait until `http://127.0.0.1:8080/login` returns 200 (30 × 10 s). Handler: `systemd_service` restart with `daemon_reload`.

`git_mirror` (historical)

Bare repository `/srv/git/devops-project.git` owned by the SSH user, plus `git config --system --add safe.directory` for that path (fixes Git's "dubious ownership" error when the `jenkins` user reads it). Only runs when `jenkins_casc_allow_local_checkout` is true (i.e. a `file://` URL). Not used since the job was switched to GitHub.

`sonarqube`

1. `assert` the admin password is ≥ 12 characters.
2. `sysctl` `vm.max_map_count=524288`, `fs.file-max=131072` (required by SonarQube's embedded Elasticsearch).
3. `python3-docker` (needed by the Docker module).
4. `community.docker.docker_container` `sonarqube:26.10.0.133078-community`, `restart_policy: unless-stopped`, published **only on** `127.0.0.1:9000` (not reachable from outside; admins use an SSH tunnel), named volumes for data/extensions/logs (persist on the EBS volume across stop/start), 512 MB heaps.
5. Wait until `/api/system/status` is `UP` (60 × 10 s).
6. Check whether the generated admin password already works (`/api/authentication/validate`, `no_log`); if not, change the factory default password to the generated one (`no_log`; the literal `admin` is SonarQube's documented default, marked `# gitleaks:allow`).
7. `/etc/jenkins-secrets` (root:jenkins 0750); if no token file exists, generate a `GLOBAL_ANALYSIS_TOKEN` and store it in `/etc/jenkins-secrets/sonar_token` (root:jenkins 0640, `no_log`), notify Jenkins restart.

`jenkins_casc`

1. `assert` admin password length.
2. Download the Jenkins plugin installation manager (pinned 2.15.0); write `/var/lib/jenkins/plugins.txt` (workflow-aggregator, pipeline-model-definition, git, junit, timestamper, coverage, configuration-as-code, credentials-binding, job-dsl, matrix-auth); install plugins **only if the list changed** (resolves versions compatible with the installed Jenkins WAR), notify restart.
3. Render `templates/casc.yaml.j2` to `/var/lib/jenkins/casc.yaml` (0640).
4. Read the Sonar token (`slurp`, `failed_when: false`, `no_log`) and write `/etc/jenkins-secrets/jenkins.env` with `JENKINS_ADMIN_PASSWORD` and `SONAR_TOKEN` (root:jenkins 0640, `no_log`).
5. systemd override: `JAVA_OPTS` disables the setup wizard (and enables local checkout only for `file://`), `CASC_JENKINS_CONFIG=/var/lib/jenkins/casc.yaml`, `EnvironmentFile=/etc/jenkins-secrets/jenkins.env`.
6. `flush_handlers`, then wait until `GET /job/order-inventory/api/json` with the admin password returns 200.

`templates/casc.yaml.j2` (Jenkins Configuration as Code):

Block	Meaning
`jenkins.securityRealm.local`	sign-up disabled; one user `admin` with password `${JENKINS_ADMIN_PASSWORD}` — a placeholder resolved by Jenkins from the environment file, so the file never contains the secret
`authorizationStrategy.globalMatrix`	`admin` has Overall/Administer; anonymous has nothing

`credentials`	string credential `sonarqube-token` = `${SONAR_TOKEN}` (used by `withCredentials` in the Jenkinsfile)
`unclassified.location.url`	`http://localhost:8080/`
`jobs` (Job DSL)	`pipelineJob('order-inventory')` with `pollSCM('H/5 * * * *')`, Git remote `jenkins_casc_repo_url`, branch `*/main`, `scriptPath('Jenkinsfile')`

Result: a Jenkins that can be destroyed and rebuilt identically from code — no manual clicks.

11.6 Idempotency, error handling, linting and testing

- **Idempotency:** modules describe state (`apt state: present`, `file`, `copy` with content) and only change what differs; command-like tasks use `creates:`, `when:` checks or `changed_when: false`. Proof: re-runs report `changed=0` (container test and real host, `docs/TESTING.md`).
- **Handlers** run once at the end of a play (or at `flush_handlers`) and only if notified — e.g. Jenkins is restarted once even if three tasks changed its configuration.
- **Error handling:** `assert` with clear messages, `validate:` for sshd, retries/until for slow services and downloads, `failed_when: false` only on explicit check tasks.
- **Secrets:** `no_log: true` on every task that handles a password or token.
- **Lint/test:** `ansible/test/test-in-container.sh` starts an Ubuntu 24.04 container with systemd, runs `--syntax-check`, `ansible-lint` (production profile), applies the playbook, applies it again and requires `changed=0`, then checks Jenkins health. Evidence: `docs/evidence/ansible-container-test-2026-10-18.txt` (29 ok, 0 failed; second run changed=0). Real host: first run 49 ok / 24 changed / 0 failed, repeat runs `changed=0`; the GitHub switch run on 2026-10-19: ok=38 changed=3 failed=0.

11.7 Safe commands

```
# Connectivity only (run inside the same container approach or with a local Ansible):
ansible -i ansible/inventory.ini jenkins -m ping
# Dry run (no changes) through the wrapper:
SSH_KEY=... JENKINS_REPO_URL=https://github.com/SamruddhiDoke/DevOps-Project.git ./ansible/run-playbook.sh
--check
# Apply, then apply again and expect "changed=0" in the PLAY RECAP:
SSH_KEY=... JENKINS_REPO_URL=https://github.com/SamruddhiDoke/DevOps-Project.git ./ansible/run-playbook.sh
```

Prerequisites: Jenkins EC2 running, its current IP in Terraform state (refresh-only apply), your IP allowed on port 22 (the resume script maintains this).

11.8 Viva questions

Question	Answer
Why both Terraform and Ansible?	Terraform provisions cloud resources through AWS APIs; Ansible configures software inside the server. Each is idempotent in its domain.
What is idempotency in Ansible?	Running the playbook again produces no changes when the host is already in the desired state (`changed=0`).
How do secrets reach Jenkins?	Terraform generates them into Secrets Manager → `run-playbook.sh` streams them on stdin into the Ansible container → Ansible writes a root:jenkins 0640 env file with `no_log` → JCasC reads `${…}` placeholders from the environment.
What are handlers?	Tasks run only when notified by a change, once, at the end of the play — e.g. restart Jenkins after its config changed.

`true` do?	Runs tasks with sudo as root on the target host.
Why is SonarQube bound to 127.0.0.1?	Only Jenkins on the same host needs it; admins use an SSH tunnel; nothing is exposed to the internet.
Why doesn't Ansible deploy the application?	Kubernetes/Helm provide rolling updates, readiness gating and rollback; the app runs on EKS, not on the Jenkins host.

CHAPTER 12

Jenkins and the CI/CD pipeline

12.1 Where Jenkins runs and how it is configured

Item	Value
Host	EC2 `i-03288b455e674765e`, t3.large, Ubuntu 24.04, public subnet, 30 GB encrypted gp3
Access	`http://<public-ip>:8080`, only from the admin /32 (security group); SSH 22 from the same /32
Configuration	Ansible roles + Jenkins Configuration as Code (`ansible/roles/jenkins_casc/templates/casc.yaml.j2`); see Ansible
Job	`order-inventory` (pipeline from SCM), Git remote `https://github.com/SamruddhiDoke/DevOps-Project.git`, branch `*/main`, script `Jenkinsfile`, trigger `pollSCM('H/5 * * * *')`
AWS credentials	**none stored**: the EC2 instance role (`order-inventory-jenkins-role`) via IMDSv2
Kubernetes permissions	EKS access entry: `AmazonEKSEditPolicy` in namespace `order-inventory` only
SonarQube	container on the same host, `127.0.0.1:9000`; token credential `sonarqube-token`
Persistence across stop/start	`/var/lib/jenkins` (jobs, build history, plugins) and the SonarQube Docker volumes are on the root EBS volume, which is kept when the instance stops; verified: after the resume, build history (#1–#7) was intact and build #8 ran

Why polling, not a webhook: a GitHub webhook needs GitHub's servers to reach Jenkins on port 8080, but the security group admits only the administrator's /32. Opening it to GitHub's hook IP ranges (or the internet) was deliberately not done. Polling means Jenkins asks GitHub every ~5 minutes ("H" spreads the minute to avoid load spikes) whether `main` has a new commit — outbound traffic only. Verified: build #8 shows "Started by an SCM change" after the push of `18a2bc6` (`docs/evidence/jenkins-github-poll-build-8-2026-10-19.txt`).

12.2 Pipeline-level settings (`Jenkinsfile` lines 10–43)

Setting	Meaning
`agent any`	run on the controller's executor (2 executors configured)
`parameters { booleanParam DEPLOY=true; booleanParam FULL_SMOKE_TEST=false }`	`DEPLOY=false` = CI-only run (local Jenkins); `FULL_SMOKE_TEST=true` runs the read/write smoke test against production (creates and cancels a test order). Parameters appear in the UI ("Build with Parameters") after the first run of this Jenkinsfile
`environment { AWS_REGION, APP_DIR, EKS_CLUSTER, K8S_NAMESPACE, HELM_RELEASE, REPORTS, TRIVY_CACHE_DIR }`	shared constants; Trivy's vulnerability DB is cached under `JENKINS_HOME` between builds
`options { disableConcurrentBuilds(); buildDiscarder(logRotator(numToKeepStr: '20')); timeout(60 min); timestamps() }`	never two deployments at once; keep 20 builds; abort hung builds; timestamped log lines

12.3 Every stage

Item	Explanation
Stage	**1. Checkout**
Purpose	get the exact commit and compute the immutable image tag
Commands	`checkout scm`; `GIT_SHORT=$(git rev-parse --short=7 HEAD)`; `IMAGE_TAG=${BUILD_NUMBER}-${GIT_SHORT}`; check that `backend/pom.xml`, both Dockerfiles, `package-lock.json`, `Chart.yaml`, `scripts/deploy-k8s.sh` exist; recreate `reports/`
Inputs	GitHub `main` (the revision polling detected)
Outputs	workspace, `IMAGE_TAG` e.g. `8-18a2bc6`
Failure conditions	GitHub unreachable, missing required file
Recovery	check network/GitHub; the log shows `Checking out Revision …`
Viva	"How is the image tag derived?" → build number + short commit: unique and traceable to source

Item	Explanation
Stage	**2. Environment Check**
Purpose	fail fast if the agent's toolchain is incomplete
Commands	`java -version`, `mvn -version`, `node --version` (+ assert ≥ 20.19), `npm`, `docker`, `trivy`, `helm`, `kubectl`, `aws --version`
Failure	a tool missing or Node too old
Recovery	re-run the Ansible playbook (installs pinned versions)

Item	Explanation
Stage	**3. Backend Tests**
Commands	`mvn -B clean verify` in `order-inventory-platform/backend`; `post always: junit(… surefire-reports/*.xml, allowEmptyResults: false)` (a run with **zero tests fails**), `recordCoverage` (JaCoCo), copy the coverage report into `reports/`
Outputs	test results and coverage in the Jenkins UI; JaCoCo XML for SonarQube
Failure	any of the 29 tests fails, compilation error
Recovery	open the test report; reproduce locally with `mvn verify`
Viva	"Why `allowEmptyResults: false`?" → a misconfiguration that runs no tests must not look green

Item	Explanation
Stage	**4. Frontend Build**
Commands	`npm ci --no-audit --no-fund`; `npm run lint` (0 warnings allowed); `npx vitest run --coverage --reporter=default --reporter=junit --outputFile.junit=reports/junit.xml`; `npm run build`; post: `junit` (no empty results), copy coverage
Failure	lint warning/error, failing test, build error

Condition	`when { expression { env.SONAR_HOST_URL } }` — skipped with a message if SonarQube is not configured (set globally by JCasC on AWS)
Commands	`withCredentials([string(credentialsId: 'sonarqube-token', variable: 'SONAR_TOKEN')])`; backend: `mvn … sonar-maven-plugin:5.8.0.7211:sonar -Dsonar.host.url=… -Dsonar.qualitygate.wait=true -Dsonar.qualitygate.timeout=300`; frontend: `sonar-scanner` with the same wait flags
Outputs	analysis in SonarQube; `QUALITY GATE STATUS: PASSED` in the log
Failure	quality gate FAILED (e.g. new bugs, coverage on new code below threshold), timeout
Recovery	open SonarQube via SSH tunnel (`ssh -L 9000:127.0.0.1:9000 ubuntu@<jenkins-ip>`), fix issues
Viva	"What is a quality gate?" → a pass/fail rule set on the analysis; `qualitygate.wait=true` makes the build wait for and obey it

Item	Explanation
Stage	**6. Dependency & IaC Scan** (Trivy)
Commands	(a) `trivy fs --scanners vuln` table report of all dependencies → `reports/trivy-dependencies.txt`; (b) gate: `--severity CRITICAL --exit-code 1`; (c) `trivy config --severity HIGH,CRITICAL --exit-code 1 --ignorefile .trivyignore.yaml --tf-vars terraform/environments/full.tfvars` over Terraform, Helm, Dockerfiles, Kubernetes YAML (skips `node_modules`, `.terraform`, `k8s/testing`) → `reports/trivy-iac.txt`; (d) `trivy fs --scanners secret --exit-code 1`
Failure	a CRITICAL dependency CVE, a new HIGH/CRITICAL misconfiguration, a secret in the working tree
Real example	build #3 on EC2 failed here: 5 HIGH findings in the new `k8s/logging/elk.yaml` → fixed by hardening the pods and one path-scoped exception (`docs/TESTING.md`)

Item	Explanation
Stage	**7. Helm Lint**
Commands	`helm lint helm/order-inventory --strict --set image.tag=${IMAGE_TAG} --set image.registry=example.invalid`; `helm template … > reports/rendered-manifests.yaml`
Purpose	the chart renders and is valid before any image is built; the rendered manifests are archived for review

Item	Explanation
Stage	**8. Docker Build**
Commands	for backend and frontend: `docker build --pull --label org.opencontainers.image.revision=${GIT_COMMIT} --label org.opencontainers.image.version=${IMAGE_TAG} -t order-inventory-${svc}:${IMAGE_TAG} ${APP_DIR}/${svc}`
Notes	`--pull` always fetches the newest base image (security fixes); labels make an image traceable to its commit

Item	Explanation
Stage	**9. Image Scan**
Commands	full table report per image; gate: `trivy image --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1`
Why `--ignore-unfixed`	a finding without an available fix cannot be resolved by rebuilding; gating on it would block all releases. Fixable HIGH/CRITICAL findings fail the build
Result	build #7/#8: 0 findings (Alpine 3.24.2)

Item	Explanation
Stage	**10. AWS Identity Check** (only if `DEPLOY`)
Commands	`aws sts get-caller-identity` → `AWS_ACCOUNT_ID`, `ECR_REGISTRY=<account>.dkr.ecr.ap-south-1.amazonaws.com`; `aws eks describe-cluster … --query cluster.status` must be `ACTIVE`
Failure	instance role missing, cluster not ACTIVE (e.g. while suspended)

Item	Explanation
Stage	**11. Push Images to ECR**
Commands	`aws ecr get-login-password
Note	runs **after** all gates — only tested and scanned images reach the registry. ECR is IMMUTABLE: pushing an existing tag fails

Item	Explanation
Stage	**12. Configure Kubernetes**
Commands	`aws eks update-kubeconfig`; `kubectl auth can-i patch deployments -n order-inventory`; `kubectl get deployments -n order-inventory`
Note	proves the namespace-scoped permission before deploying; Jenkins reaches the private API endpoint

Item	Explanation
Stage	**13. Deploy to EKS**
Commands	`env.DEPLOY_STARTED = 'true'`; `ROLLOUT_TIMEOUT=5m ./scripts/deploy-k8s.sh ${ECR_REGISTRY} ${IMAGE_TAG}` (explained below)
Outputs	new Helm revision; pods of the new version Ready; `.deploy/previous-revision`
Failure	pods not Ready within 5 min → the script **rolls back** to the previous good revision and exits 1
Real example	build #6: Jenkins was restarted by an Ansible run during this stage ("context canceled"); the pipeline's `post` rolled back to revision 8

Item	Explanation
Stage	**14. Kubernetes Verification**
Commands	`kubectl get pods,svc,ingress -o wide`; for both deployments read `.spec.template.spec.containers[0].image` and require it to end in `:${IMAGE_TAG}`
Purpose	proves the cluster actually runs the image this build produced

Item	Explanation
Stage	**15. Application Health Check**
Commands	in-cluster: `kubectl run health-check-${BUILD_NUMBER} --rm -i --restart=Never --image=curlimages/curl:8.10.1 -- curl -fsS http://backend:8080/api/health/ready`; read the ALB hostname from the Ingress (retry 30 × 10 s); wait until `http://$ALB_HOST/api/health` answers (new DNS names take minutes); then `SMOKE_READ_ONLY=1 ./scripts/smoke-test.sh` (8 checks) or the full 20-check test when `FULL_SMOKE_TEST`; save the hostname to `reports/alb-host.txt`
How Jenkins finds the endpoint	from the Ingress status at run time — never hard-coded, so it works after the ALB is recreated
Failure	readiness not UP, no ALB hostname, smoke check fails → `post failure` rollback

Item	Explanation
Stage	**16. Deployment Summary**
Output	build, commit, image tags, URL, rollback command

`post` section

- `failure`: if `DEPLOY_STARTED`, read `.deploy/previous-revision` and `helm rollback order-inventory <rev> --wait` — covers a release that became Ready but **failed the post-deploy checks** (deploy-k8s.sh already covers releases that never become Ready). Prints pods.
- `always`: `archiveArtifacts reports/**`; if deploying and `CI_REPORTS_BUCKET` is set, `aws s3 cp --recursive` to `s3://<bucket>/order-inventory/<build>/` (non-fatal); remove local images and `docker image prune -f` (never touches ECR or the cluster).

Artifacts per build: backend and frontend coverage, Trivy dependency/IaC/image reports, rendered manifests, JUnit results, ALB host.

12.4 Trace of one commit (real: `18a2bc6`, build #8, 2026-10-19 UTC)

1. `git push` to GitHub `main` (`ca5c22c..18a2bc6`).
2. Within ≤ 5 minutes the poll detects the change → **"Started by an SCM change"**.
3. Checkout `Revision 18a2bc6…` from `https://github.com/SamruddhiDoke/DevOps-Project.git`; tag `8-18a2bc6`.
4. 29 backend tests pass (21:05:12); SonarQube gates PASSED for backend (21:06:21) and frontend (21:07:04).
5. Trivy gates pass; images built, scanned, pushed to ECR as `order-inventory-{backend,frontend}:8-18a2bc6`.
6. `deploy-k8s.sh` → `helm upgrade` → Helm revision 14 (rolling update: one new pod at a time, old pods keep serving).
7. "Release healthy": backend 2, frontend 2, both images `…:8-18a2bc6` (21:11:20).
8. In-cluster readiness `{"database":"UP","status":"UP"}`; "All 8 read-only checks passed against http://k8s-orderinv-...-1122521152.ap-south-1.elb.amazonaws.com" (21:11:36). `Finished: SUCCESS`.

Evidence: `docs/evidence/jenkins-github-poll-build-8-2026-10-19.txt`. Helm revision history around it (from the evaluation logs): 11 = build #7, 12 = first suspend (Ingress disabled), 13 = resume test, **14 = build #8**, 15 = second suspend; the next resume creates 16.

12.5 `scripts/deploy-k8s.sh` (line by line)

```
REGISTRY="${1:?usage…}"; TAG="${2:?usage…}"; shift 2
NAMESPACE="${K8S_NAMESPACE:-order-inventory}"; RELEASE="${RELEASE:-order-inventory}";
TIMEOUT="${ROLLOUT_TIMEOUT:-5m}"
```

Required registry and tag; remaining arguments are passed to Helm (e.g. `-f values-minikube.yaml`).

```
[ "$TAG" != "latest" ] || { echo "Refusing to deploy the mutable tag 'latest'" >&2; exit 1; }
[ "$(kubectl auth can-i create deployments -n "$NAMESPACE" 2>/dev/null)" = yes ] || { …exit 1; }
kubectl -n "$NAMESPACE" get secret "${DB_SECRET:-mysql-secret}" >/dev/null || { …exit 1; }
```

Guards: never `latest` (also enforced in the chart helper); permission check that works with Jenkins' namespace-only access (reading the Namespace object itself would need cluster scope); DB Secret must exist.

Records the **last known-good revision** before changing anything (used by this script's rollback and the Jenkinsfile's `post failure`). Empty on the first install.

```
if helm upgrade --install "$RELEASE" "$CHART" --namespace "$NAMESPACE" \
     --set image.registry="$REGISTRY" --set image.tag="$TAG" --wait --timeout "$TIMEOUT" "$@"; then
  for d in backend frontend; do kubectl -n "$NAMESPACE" rollout status "deployment/$d" --timeout=60s; done
  …print deployments…; exit 0
fi
```

`upgrade --install` installs or upgrades; `--wait` makes Helm wait until all Deployments have their new pods **Ready** (readiness probes passing) — otherwise the command fails after the timeout. `rollout status` double-checks.

```
echo "Deployment of ${TAG} failed (pods not Ready within ${TIMEOUT})." >&2
kubectl get pods …; kubectl get events … | tail -15
if [ -n "$PREV_REVISION" ]; then helm -n "$NAMESPACE" rollback "$RELEASE" "$PREV_REVISION" --wait --
timeout "$TIMEOUT"; …
else echo "No previous revision (first install); leaving resources in place for diagnosis."; fi
exit 1
```

On failure: print diagnostics, **roll back** to the recorded revision and exit 1 so the pipeline fails. Because `maxUnavailable: 0`, the old pods were never removed while the new ones were failing, so users saw no errors (measured: scenarios C and D, 4,376 requests, 0 failures).

12.6 `scripts/smoke-test.sh` (what the checks are)

`request()` uses curl to call `BASE_URL + path`, storing status and body; `expect` compares the status; `json` extracts a field with Node. The run id `smoke-<epoch>-<random>` makes all test data unique, so the test can run repeatedly against a shared environment.

- Always: `GET /` contains `<div id="root">`; `GET /products` (SPA deep link) 200; `/api/health` UP; `/api/health/ready` database UP.
- Read-only mode (`SMOKE_READ_ONLY=1`, used by the pipeline): list products/users/orders 200; unknown API route 404 → **8 checks**.
- Full mode: create user (201), duplicate email with different case (400), get user; create product with stock 5 (`LOW_STOCK`), update stock to 20, invalid product (400); create order of 3 (201, total 37.5), stock 17, order of 999 (400), cancel (200), stock back to 20, reopen cancelled (400), delete referenced product (409); non-numeric id (400), missing order (404), unknown route (404) → **20 checks**.

12.7 Immutable tags and rollback

- Each build pushes `<build>-<commit>`; ECR refuses to overwrite a tag; the chart refuses `latest`. A tag therefore always identifies one exact, scanned image and the commit it came from.
- With `latest`, "what is running?" and "roll back to what?" have no reliable answer: the same name points to different images over time, nodes may cache an old `latest`, and a rollback might redeploy the broken image.
- Rollback options: automatic (`deploy-k8s.sh`, Jenkinsfile `post`), manual `helm -n order-inventory rollback order-inventory [revision]`, or redeploy any older tag: `./scripts/deploy-k8s.sh <registry> 7-ca5c22c`.

12.8 Local CI (`ci/`)

(git-ignored), starts SonarQube (`127.0.0.1:19000`), replaces its factory password, creates an analysis token, builds and starts a Jenkins image (`ci/jenkins/Dockerfile`: Jenkins 2.580.1 LTS + the same pinned toolchain + plugins + JCasC). The job reads the repository mounted read-only at `/repo` (`ALLOW_LOCAL_CHECKOUT` enabled only for this local, single-user controller) and defaults to `DEPLOY=false`. Used for local builds #1–#4 (`docs/TESTING.md`, `docs/evidence/jenkins-local-build-3.txt`).

12.9 Build history on AWS (from `docs/TESTING.md`)

Build	Result	Lesson
#1	failed at checkout	Git "dubious ownership" → `safe.directory`
#2	failed at deploy (exit 126)	scripts lacked the executable bit in Git (Windows) → `git update-index --chmod=+x`
#3	failed at IaC scan	the security gate worked (5 HIGH findings)
#4, #5	success	full pipeline incl. EKS deploy
#6	failed at deploy	Jenkins restarted mid-build by Ansible; automatic rollback
#7	success	first checkout from GitHub (manual trigger)
#8	success	first build triggered by a GitHub push (polling)

12.10 Troubleshooting failed builds

Symptom	Where to look / fix
Checkout fails	network, GitHub availability, repo URL in the job configuration (JCasC)
Tests fail	JUnit report in the build; reproduce locally
Quality gate failed	SonarQube UI through the SSH tunnel
Trivy gate failed	`reports/trivy-*.txt` artefacts; update base image/dependency or add a justified, narrow exception
ECR push denied	instance role policy; tag already exists (immutable)
`kubectl auth can-i` = no	EKS access entry / namespace
Deploy timeout + rollback	`kubectl -n order-inventory get events`, `kubectl logs` of the new pods (image pull error, readiness failing, DB secret)
ALB checks fail	`kubectl get ingress`, target health, Load Balancer Controller logs
Build stuck waiting	`options timeout` aborts at 60 min; check `disableConcurrentBuilds` queue

12.11 Viva questions

Question	Answer
How does a commit reach production?	See the commit trace above: push → poll → tests → quality gate → security gates → images → ECR → Helm rolling update → verification → (rollback on failure).
How does Jenkins roll back a bad release?	`deploy-k8s.sh` records the last good Helm revision, runs `helm upgrade --wait`, and on failure runs `helm rollback` to it; the Jenkinsfile `post failure` does the same for releases that fail the later checks.
How do you prove zero downtime?	Continuous requests through the public ALB during releases, failed releases and restarts: 9,532 requests, 0 non-200 (`docs/evidence/aws-zero-downtime-2026-10-19.txt`).

Jenkins?	The EC2 instance role provides temporary credentials via IMDSv2; nothing to leak or rotate.
Why not a webhook?	Jenkins is not reachable from GitHub by design (admin /32 only); polling is outbound-only. Trade-off: up to ~5 min delay.
What happens if two builds run at once?	They cannot: `disableConcurrentBuilds()`.
What does Jenkins do after a resume?	It polls GitHub and builds any commit pushed while it was stopped.
Difference between CI and CD here?	CI = stages 1–9 (build, test, analyse, scan) on every commit; CD = stages 10–15 (publish and deploy to EKS automatically, with verification and rollback).

CHAPTER 13

EKS, Kubernetes, Helm and the ALB

13.1 EKS: control plane vs worker nodes

Part	Who runs it	In this project	Billed
Control plane (API server, etcd, scheduler, controller manager)	AWS, in AWS-managed accounts, highly available	cluster `order-inventory-eks`, Kubernetes 1.36; API endpoint public (admin /32 only) + private (VPC)	per cluster-hour (≈ $0.10/h) **whether or not any nodes exist**
Worker nodes	EC2 instances in your account	managed node group `order-inventory-nodes`, 2 × t3.medium, private subnets, Amazon Linux 2023	per instance-hour while running
Add-ons	EKS-managed	`vpc-cni` (gives every pod a real VPC IP), `coredns` (cluster DNS), `kube-proxy` (Service routing), `aws-ebs-csi-driver`	run as pods on the nodes
AWS Load Balancer Controller	Helm chart 3.6.0 in `kube-system`, IRSA role `order-inventory-aws-lbc-role` (`scripts/install-lb-controller.sh`)	turns Ingress objects into ALBs	the ALB it creates is billed

That is why a suspended cluster (0 nodes) still costs ≈ $73/month: the control plane keeps running and keeps the cluster's state (Deployments, Services, Secrets, Helm history), which is what allows a 10-minute restore.

API access: `kubectl` from the laptop goes to the public endpoint, allowed only from `public_access_cidrs` (the admin /32); nodes and Jenkins use the private endpoint inside the VPC (Jenkins via the SG rule on 443). Authentication: the deployer IAM user is cluster admin (creator); the Jenkins role has an **access entry** with edit rights in namespace `order-inventory` only.

13.2 Kubernetes objects in the application namespace

```
Namespace order-inventory
├── Deployment backend   → ReplicaSet backend-<hash> (one per pod-template version) → Pod ×2
├── Deployment frontend  → ReplicaSet frontend-<hash>                                → Pod ×2
├── Service backend  (ClusterIP :8080)      Service frontend (ClusterIP :80 → 8080)
├── ConfigMap backend-config                Secret mysql-secret (url, username, password)
├── PodDisruptionBudget backend, frontend   Ingress order-inventory-ingress (class alb)
└── Helm release secrets sh.helm.release.v1.order-inventory.v<N>  (release history)
```

- **Deployment**: desired state (image, replicas, probes, update strategy). It manages **ReplicaSets**.
- **ReplicaSet**: keeps N identical pods of one pod-template version running. A rolling update creates a new ReplicaSet and scales it up while scaling the old one down; old ReplicaSets are kept (`revisionHistoryLimit: 10`) for `kubectl rollout undo` (this project uses Helm rollback instead).
- **Pod**: one running instance (here one container per pod) with its own VPC IP.
- **Service**: stable name and virtual IP in front of the pods matching its selector; only **Ready** pods are in its endpoints. `backend.order-inventory.svc.cluster.local` resolves through CoreDNS.
- **What happens when a pod crashes**: the container restarts in place (restartPolicy Always, with back-off); if the pod or node disappears, the ReplicaSet creates a replacement on any node with capacity. While not Ready it receives no traffic. Verified "pod loss → replacement Ready" on minikube.

13.3 The Helm chart (`helm/order-inventory/`)

Helm renders templates with values into Kubernetes manifests and installs them as a **release** with numbered **revisions** stored in the cluster. `helm history` lists them; `helm rollback <release> <rev>` re-applies an earlier revision as a new revision.

`Chart.yaml`

`apiVersion: v2`, `name: order-inventory`, `type: application`, chart `version: 1.0.0`, `appVersion: "1.0.0"` (the real version is the image tag, also shown in the `app.kubernetes.io/version` label).

`values.yaml` (every key)

Key	Value	Used by
`image.registry` / `image.tag`	`""` / `""` (set per release by `deploy-k8s.sh`)	`oi.image` helper
`image.pullPolicy`	`IfNotPresent` (immutable tags make this safe)	both Deployments
`environment`	`aws`	ConfigMap `APP_ENVIRONMENT`
`config.corsAllowedOrigins`, `config.logFormat: ecs`, `config.javaOpts`	ConfigMap	backend env
`database.existingSecret`	`mysql-secret`	backend `secretKeyRef`
`backend.replicas` / `frontend.replicas`	2 / 2	Deployments
`backend.resources`	requests 250m CPU / 512Mi, limits 500m / 1Gi	scheduling and limits
`frontend.resources`	requests 100m / 128Mi, limits 250m / 256Mi	
`rollingUpdate.maxUnavailable` / `maxSurge`	**0 / 1**	both Deployments
`preStopSleepSeconds`	10	preStop hook and grace periods
`podDisruptionBudget.enabled` / `minAvailable`	true / 1	`pdb.yaml`
`ingress.enabled`, `className: alb`, annotations	see the Ingress section below	`ingress.yaml`

`values-minikube.yaml` overrides: `image.registry: local`, `environment: minikube`, `ingress.enabled: false`, smaller resources.

`templates/_helpers.tpl`

```
{{- define "oi.labels" -}} app.kubernetes.io/part-of: order-inventory; app.kubernetes.io/managed-by: {{
.Release.Service }};
app.kubernetes.io/version: {{ .Values.image.tag | quote }}; helm.sh/chart: {{ printf "%s-%s" .Chart.Name
.Chart.Version }} {{- end }}
{{- define "oi.image" -}}
{{- $tag := required "image.tag is required (immutable build tag, never latest)" $root.Values.image.tag
-}}
{{- if eq $tag "latest" -}}{{- fail "image.tag must be an immutable build tag, not latest" -}}{{- end -}}
{{ printf "%s/%s:%s" registry repo tag }} (or "repo:tag" without a registry)
{{- define "oi.podSecurityContext" -}} runAsNonRoot: true; seccompProfile: { type: RuntimeDefault } {{-
end }}
```

`required` and `fail` make rendering **fail** without a tag or with `latest` (verified: "Chart refuses missing or `latest` tag"). The pod security context refuses to start containers running as root and applies the runtime's default seccomp syscall filter.

`templates/backend.yaml` (Deployment + Service)

Lines	Setting	Meaning
9–15	`replicas`, `revisionHistoryLimit: 10`, `strategy: RollingUpdate` with `maxUnavailable: 0`, `maxSurge: 1`	during an update never fewer than 2 Ready pods; at most 3 pods exist
16–22	selector/labels `app: backend`	Service and PDB select these pods
23–25	annotation `checksum/config: <sha256 of configmap.yaml>`	a change in the ConfigMap changes the pod template → automatic rolling restart (ConfigMap changes otherwise do not restart pods)
27	`terminationGracePeriodSeconds: preStop + 35` = 45 s	enough for 10 s preStop + 20 s graceful shutdown + margin
28–29	pod security context	non-root, seccomp
32–33	`image` via `oi.image`, `imagePullPolicy`	e.g. `…/order-inventory-backend:8-18a2bc6`
34–36	port `http` 8080	named port used by probes and the Service
37–39	`envFrom: configMapRef backend-config`	non-secret env
40–55	`SPRING_DATASOURCE_URL/USERNAME/PASSWORD` from `secretKeyRef` `mysql-secret`	credentials never in the chart
58–63	`startupProbe` `/api/health`, every 5 s, 36 failures allowed (= up to 180 s to start)	liveness checks begin only after start-up succeeded → slow JVM starts are not killed
64–70	`livenessProbe` `/api/health`, 10 s, timeout 3 s, 3 failures	restarts a hung JVM; no DB dependency
71–77	`readinessProbe` `/api/health/ready`, 5 s, timeout 3 s, 3 failures	traffic only when the DB is reachable
78–81	`preStop: sleep 10`	keeps serving while the endpoint and ALB deregistration propagate

87–94	`/tmp` as `emptyDir`	the JVM/Tomcat need a writable temp dir
90–91	resources from values	
96–113	Service `backend` ClusterIP 8080 → `http`; annotation `alb.ingress.kubernetes.io/healthcheck-path: /api/health`	the ALB target group health check uses liveness

`templates/frontend.yaml`

Same strategy; grace period preStop + 20 = 30 s; env `BACKEND_URL = http://backend.<namespace>.svc.cluster.local:8080` (fully-qualified because nginx's resolver ignores DNS search domains); readiness and liveness on `/healthz`; no privilege escalation, capabilities dropped (no read-only root filesystem: the nginx entrypoint writes the rendered config at start); Service `frontend` port 80 → 8080 with ALB health-check path `/healthz`.

`templates/configmap.yaml`, `pdb.yaml`, `ingress.yaml`, `NOTES.txt`

- ConfigMap `backend-config`: `APP_ENVIRONMENT`, `CORS_ALLOWED_ORIGINS`, `JAVA_OPTS`, `LOGGING_STRUCTURED_FORMAT_CONSOLE` (only if `logFormat` set).
- PDB per app with `minAvailable: 1`: a **voluntary** disruption (node drain, cluster upgrade) may not evict a pod if that would leave fewer than 1 available. (Scaling a Deployment to 0 is not an eviction, so the suspend script scales the apps to 0 first.)
- Ingress (only if `ingress.enabled`): class `alb`; rules `/api` (Prefix) → `backend:8080`, `/` (Prefix) → `frontend:80`. The longer prefix wins, so `/api/orders` goes to the backend and everything else to nginx.
- NOTES: printed after install (tag, revision, how to get the ALB hostname, rollback command).

13.4 Zero-downtime rolling update — how the pieces fit

1. `helm upgrade` changes the image tag → the Deployment creates a new ReplicaSet.
2. `maxSurge: 1`, `maxUnavailable: 0` → one **new** pod is added; no old pod is removed yet.
3. New pod: startup probe → readiness `/api/health/ready` passes → added to the Service endpoints and registered as an ALB target (target type `ip`); the ALB then health-checks it (`/api/health`, every 15 s, 2 successes = healthy). Note: no ALB **pod readiness gate** is configured (that would require the namespace label `elbv2.k8s.aws/pod-readiness-gate-inject`), so Kubernetes does not wait for the ALB's own health check before continuing; the ALB keeps routing to the remaining healthy old target meanwhile, and the 10 s preStop plus 20 s deregistration delay cover the hand-over. The measured result was 0 failed requests; adding the readiness gate would make this guarantee explicit.
4. Only now one **old** pod is terminated: it is removed from endpoints and ALB deregistration starts (`deregistration_delay` 20 s); meanwhile its `preStop` sleeps 10 s so requests still in flight or routed during propagation are served; then SIGTERM → Spring graceful shutdown finishes in-flight requests (≤ 20 s).
5. Repeat until all pods are new. `helm --wait` returns success when all new pods are Ready.
6. If new pods never become Ready (bad image, wrong DB secret), the old pods were never removed; after the timeout `deploy-k8s.sh` rolls back.

Evidence (AWS, through the public ALB, `docs/evidence/aws-zero-downtime-2026-10-19.txt`): rolling releases 1,712 and 1,702 requests, non-existent image 1,982, never-Ready release 2,394, restart 1,742 → **9,532 requests, 0 non-200**; both bad releases → deploy exit 1 and automatic rollback.

13.5 Ingress → ALB (AWS Load Balancer Controller)

Annotation	Effect
`alb.ingress.kubernetes.io/scheme: internet-facing`	ALB in the public subnets (found via the `kubernetes.io/role/elb` tag)
`target-type: ip`	targets are **pod IPs** directly (possible because of the VPC CNI), not node ports
`listen-ports: '[{"HTTP":80}]'`	HTTP listener on 80 (no HTTPS: no certificate)
`healthcheck-interval-seconds: "15"`, `healthy-threshold-count: "2"`	faster detection of new healthy targets
`target-group-attributes: deregistration_delay.timeout_seconds=20`	drain time for removed targets (default would be 300 s)
Service annotation `healthcheck-path`	`/api/health` (backend), `/healthz` (frontend)

How it works: the controller (pods in `kube-system`) watches Ingress objects with class `alb`, calls the ELBv2/EC2 APIs with its IRSA role and creates an ALB named `k8s-orderinv-orderinv-<hash>`, a security group, listener rules for the paths and one target group per Service (`k8s-orderinv-backend-…`, `k8s-orderinv-frontend-…`), then keeps target registrations in sync with the Ready pods. When the Ingress is deleted, the controller deletes all of it (an Ingress finalizer makes Kubernetes wait for that). The ALB is therefore **not** a Terraform resource, and its DNS name is assigned by AWS at creation time.

Why the ALB is removed during suspension and how the URL is found again: an ALB costs money per hour and needs nodes behind it anyway. Suspend sets `ingress.enabled=false` via Helm → the controller deletes the ALB (this must happen while the controller still has nodes to run on). Resume deploys the chart with defaults (`ingress.enabled: true`) → a **new** ALB with a **new** DNS name; the script reads it from `kubectl get ingress … -o jsonpath='{.status.loadBalancer.ingress[0].hostname}'`, waits for the ALB to be active, the targets healthy and DNS to resolve, and prints it.

13.6 Scaling: replicas vs nodes

	Scaling application replicas to 0	Scaling the node group to 0
Command	`kubectl -n order-inventory scale deploy backend frontend --replicas=0`	`aws eks update-nodegroup-config --scaling-config minSize=0,maxSize=3,desiredSize=0`
Effect	pods deleted; Deployments, Services, Secrets, Helm release remain	EC2 worker instances drained and terminated; **all** pods (including system pods) disappear
Cost effect	none by itself (nodes still run)	node compute cost → 0; control plane still billed
Restore	`kubectl scale … --replicas=2` or `helm upgrade`	`update-nodegroup-config desiredSize=2` → new instances join (~1 min measured)
Why both in suspend	apps first (fast, clean, avoids PDB-blocked evictions) then nodes	

There is no Horizontal Pod Autoscaler or Cluster Autoscaler in this project (fixed sizes).

13.7 Bootstrap scripts

- `scripts/install-lb-controller.sh`: reads cluster name, IRSA role ARN and VPC id from Terraform outputs; `helm upgrade --install aws-load-balancer-controller eks/aws-load-balancer-controller --version 3.6.0` in `kube-system` with `serviceAccount.annotations."eks\.amazonaws\.com/role-arn"=<role>` (IRSA); waits for the rollout.

`mysql-secret` with `kubectl create … --dry-run=client -o yaml | kubectl apply -f -` (create-or-update, idempotent); `unset` the variables; prints only the host. Alternative mode with explicit `DB_HOST/DB_USER/DB_PASSWORD` for minikube.

- `k8s/namespace.yaml`: the namespace object.

13.8 Database outage behaviour on Kubernetes

Two time scales:

- **Short outage (seconds):** readiness needs 3 failures × 5 s ≈ 15 s before a pod is marked unready, so pods stay in service; requests that need the database get a fast **503 from the application** ("database temporarily unavailable") instead of hanging. This is what the AWS RDS reboot showed: 3 × 503 on `/api/products` within ~1 s, liveness 88/88 OK, 0 restarts.
- **Longer outage:** readiness fails → Kubernetes removes the pods from the Service endpoints → the Load Balancer Controller (which registers only Ready endpoints as targets) deregisters them from the backend target group; with no healthy target the ALB itself answers 503 for `/api`. The frontend (static UI) keeps working. Liveness keeps passing → **no restarts**. When the DB returns, readiness passes and the pods are re-registered automatically. Minikube test: all backend pods unready, 0 restarts, automatic recovery.

13.9 Inspection commands

```
export AWS_PROFILE=order-inventory AWS_REGION=ap-south-1
aws eks update-kubeconfig --name order-inventory-eks --alias order-inventory-eks
kubectl get nodes -o wide
kubectl -n order-inventory get deploy,rs,pods,svc,ingress,pdb -o wide
kubectl -n order-inventory describe pod <pod>              # events: scheduling, pulls, probe failures
kubectl -n order-inventory get events --sort-by=.lastTimestamp | tail -20
kubectl -n order-inventory logs deploy/backend --tail=50
kubectl -n order-inventory rollout status deploy/backend
kubectl -n order-inventory rollout history deploy/backend
helm -n order-inventory list
helm -n order-inventory history order-inventory --max 10
helm -n order-inventory get values order-inventory          # user-supplied values of the current revision
kubectl -n kube-system get deploy aws-load-balancer-controller coredns
kubectl -n kube-system logs deploy/aws-load-balancer-controller --tail=50
```

13.10 Viva questions

Question	Answer
Why EKS instead of ECS?	Kubernetes skills and tooling are portable (Helm, kubectl, same chart on minikube and EKS — verified locally first); rich health/rollout primitives; ecosystem (Load Balancer Controller, Filebeat DaemonSet). ECS would be simpler and has no control-plane fee; it was a learning/portability trade-off.
Deployment vs ReplicaSet?	A ReplicaSet keeps N copies of one pod template; a Deployment manages ReplicaSets to roll between versions and roll back.
Readiness vs liveness?	Readiness = may this pod receive traffic now (DB reachable)? Liveness = is the process healthy or must it be restarted? Startup = has it finished starting (delays liveness).
What happens when all worker nodes are scaled to zero?	All pods are gone (they cannot run anywhere); the control plane keeps the objects; the Ingress-less app is offline; restoring nodes reschedules the pods automatically (Deployments still want 2 replicas unless scaled down).
Why does EKS still cost money with no nodes?	The managed control plane runs 24/7 and is billed per cluster-hour.

How does the ALB get created from an Ingress?	The AWS Load Balancer Controller watches Ingresses of class `alb` and creates/updates the ALB, listeners, rules and target groups through the AWS API using its IRSA role.
Why target-type ip?	The ALB sends traffic straight to pod IPs; health checks reflect individual pods; no extra kube-proxy hop.
Why `maxUnavailable: 0`?	Capacity never drops below the desired replica count during an update; new pods must be Ready before old ones go.
Why a preStop sleep?	Endpoint removal and ALB deregistration are asynchronous; sleeping keeps the pod serving until no new requests are routed to it.
What does a PDB protect against?	Voluntary evictions (drains, upgrades) taking down all replicas at once.
How is configuration separated from secrets?	ConfigMap for non-secret settings; Secret (KMS-encrypted in etcd) for credentials, injected as env vars.

CHAPTER 14

Security: SonarQube, Trivy, IAM and secrets

14.1 SonarQube (static analysis + quality gate)

In general: SonarQube analyses source code without running it and reports **bugs**, **vulnerabilities**, **security hotspots** (code needing review), **code smells** (maintainability) and **duplications**, plus test **coverage** imported from reports. A **quality gate** is a set of pass/fail conditions — by default ("Sonar way") focused on **new code**: no new issues above thresholds, coverage on new code ≥ 80 %, low duplication, reviewed hotspots.

In this project:

Item	Implementation
Server	SonarQube 26.10 Community, container on the Jenkins host, `127.0.0.1:9000` (Ansible role `sonarqube`)
Projects	`order-inventory-backend` (`pom.xml` properties, JaCoCo XML), `order-inventory-frontend` (`sonar-project.properties`, LCOV)
Pipeline	stage "Code Quality": Maven Sonar plugin and `sonar-scanner` with `-Dsonar.qualitygate.wait=true` → the step **fails if the gate fails**
Token	generated by Ansible into `/etc/jenkins-secrets/sonar_token`, exposed to Jenkins as credential `sonarqube-token` through JCasC
Result	quality gate PASSED for backend and frontend (builds #4, #7, #8 — e.g. `QUALITY GATE STATUS: PASSED` in `docs/evidence/jenkins-github-build-7-2026-10-19.txt`)

Access for humans: `ssh -L 9000:127.0.0.1:9000 ubuntu@<jenkins-ip> -i UniversalKey.pem` → http://localhost:9000 (password in Secrets Manager `order-inventory/sonarqube-admin`).

14.2 Trivy (vulnerability, misconfiguration and secret scanning)

Scan (Jenkinsfile)	What it checks	Gate
`trivy fs --scanners vuln` on `order-inventory-platform`	**dependency** vulnerabilities from `pom.xml` and `package-lock.json` (libraries we chose)	CRITICAL → fail
`trivy config` on the repository	**infrastructure misconfiguration** in Terraform (evaluated with `full.tfvars`), Helm/Kubernetes YAML, Dockerfiles (e.g. public endpoints, root containers, missing encryption)	HIGH/CRITICAL → fail, except the 5 documented IDs in `.trivyignore.yaml`
`trivy fs --scanners secret`	**secrets** (keys, tokens, passwords) in the working tree	any → fail
`trivy image` per image	**image** vulnerabilities in OS packages (Alpine) and application libraries inside the built image	fixable HIGH/CRITICAL → fail (`--ignore-unfixed`)

Difference: dependency scanning looks at what the code declares; image scanning looks at everything that actually ends up in the container (base OS packages, JRE, the built JAR); misconfiguration scanning looks at how infrastructure is configured, not at software versions.

History: before remediation the images had 11 (backend) and 68 (frontend) HIGH/CRITICAL findings, including a Spring Framework issue fixed only in Spring 7 → solved by upgrading to Spring Boot 4.1 (Spring 7), nginx 1.30, patched Tomcat/Jackson versions (`pom.xml` overrides) and `apk upgrade` in both Dockerfiles. Current: 0 findings in both images

and dependencies (`docs/TESTING.md`). **Scope of that statement:** "0 known vulnerabilities in the scanned packages with the vulnerability database of that day" — not "vulnerability-free": new CVEs appear daily, Trivy does not test application logic, and unfixed CVEs are not gated.

Accepted findings (`.trivyignore.yaml`) and how false positives are handled

ID	Finding	Why accepted	Mitigation
AVD-AWS-0040	EKS public endpoint enabled	admin `kubectl` without VPN/bastion	allow-list = admin /32
AVD-AWS-0041	public endpoint CIDR	same	`0.0.0.0/0` rejected by variable validation and precondition
AVD-AWS-0104	unrestricted egress (Jenkins, EKS SG)	package installs, ECR, GitHub, AWS APIs	ingress restricted
AVD-AWS-0132	CI bucket uses SSE-S3 not a CMK	reproducible build reports	private, TLS-only, versioned, 30-day expiry
KSV-0118 (**only** `k8s/logging/elk.yaml`)	container runs as root (Filebeat)	must read root-owned node log files	read-only root fs, read-only host mounts, all capabilities dropped except `DAC_READ_SEARCH`; all other pods non-root

Process: every exception is reviewed, justified in writing (statement field + `docs/SECURITY.md`), and scoped as narrowly as possible (KSV-0118 is path-scoped so a new root container elsewhere still fails the build). Build #3 shows the gate catching real issues: the first ELK manifest had 5 HIGH findings; 4 were fixed by hardening, 1 accepted with a path scope.

14.3 gitleaks and `.gitleaksignore`

gitleaks scans Git history for secrets. It was run manually before each commit during the remediation and on a fresh clone after the history rewrite (clean). `.gitleaksignore` lists **two fingerprints** (commit + file + rule + line) of reviewed false positives: SonarQube's documented factory default `admin:admin` in an old version of `ci/start-local-ci.sh`. Limitations: an ignore entry silences that exact finding forever, so it must be reviewed; fingerprints are tied to a commit, so the same text in a new commit is reported again; gitleaks is not part of the Jenkins pipeline (Trivy's secret scanner is).

14.4 Secret handling

Secret	Generated by	Stored in	Delivered to
RDS master password	Terraform `random_password`	Secrets Manager `order-inventory/rds-master`; encrypted S3 state	`create-db-secret.sh` → Kubernetes Secret `mysql-secret` (KMS envelope-encrypted) → backend env vars
Jenkins admin password	Terraform	Secrets Manager `order-inventory/jenkins-admin`	`run-playbook.sh` → Ansible (`no_log`) → `/etc/jenkins-secrets/jenkins.env` (root:jenkins 0640) → JCasC placeholder
SonarQube admin password	Terraform	Secrets Manager `order-inventory/sonarqube-admin`	Ansible changes the factory password
SonarQube analysis token	Ansible on the host	`/etc/jenkins-secrets/sonar_token`	Jenkins credential `sonarqube-token` (masked in logs by `withCredentials`)
AWS credentials for Jenkins	— (none)	EC2 instance role	temporary credentials from IMDSv2
AWS credentials			

for the operator	IAM user `order-inventory-deployer` access key	local AWS CLI profile `order-inventory`	—
Local Compose / local CI passwords	`.env` / `ci/start-local-ci.sh`	git-ignored files	—

Guards: `.gitignore`, Trivy secret scan in CI, gitleaks, `no_log`, `withCredentials`, KMS encryption of Kubernetes Secrets, private + TLS-only + versioned state bucket. Known limitation (honest): in `ansible/run-playbook.sh` the two admin passwords are passed as command-line arguments to a short-lived local `node` process before being streamed to the container on stdin (visible in the local process list during that moment; `docs/SECURITY.md` was corrected on 2026-10-25 to say so). Fix: read the secrets inside the Node script or pipe Secrets Manager output directly.

14.5 IAM least privilege

Identity	Permissions
`order-inventory-deployer` (operator, Terraform)	AWS-managed `PowerUserAccess` (everything except IAM/Organizations) + `IAMReadOnlyAccess` + inline `order-inventory-iam-scope` (`aws/iam/order-inventory-deployer-iam.json`): IAM write **only** on `order-inventory-*` roles/policies/instance profiles, EKS OIDC providers, and service-linked roles for 5 named services. Not `AdministratorAccess` ; never root.
Jenkins instance role	ECR login + push/pull on 2 repositories; `eks:DescribeCluster` on 1 cluster; `s3:PutObject` into 1 bucket; Kubernetes edit in 1 namespace
Node role	AWS-managed worker, CNI and ECR read-only policies
Load Balancer Controller	upstream policy, assumable only by `kube-system:aws-load-balancer-controller` (IRSA)
EBS CSI driver	AWS-managed policy, assumable only by `kube-system:ebs-csi-controller-sa`

The scripts refuse to run as the root user (`preflight` in `scripts/lib/evaluation-common.sh`).

14.6 Network exposure

Endpoint	Exposure
ALB	internet, **HTTP 80 only** (no HTTPS — no domain/certificate; accepted risk; do not use with real data)
EKS API	public endpoint restricted to the admin /32 + private endpoint
Jenkins UI 8080 / SSH 22	admin /32 only; SSH key-only, no root login, no passwords
SonarQube	`127.0.0.1` on the Jenkins host only
Kibana / Elasticsearch	ClusterIP only; access via `kubectl port-forward` (never published)
RDS	no public access, isolated subnets, SG = EKS cluster SG only, TLS required
Nodes	private subnets, no public IPs; egress via NAT

Verified on 2026-10-19: ALB only port 80 open; RDS port unreachable from the internet; Jenkins SG has no `0.0.0.0/0` rule (checked by the resume script on every run).

14.7 Container and workload security

Application: input validation, generic error messages (no stack traces/SQL to clients — tested), request ids validated against log injection, no emails in logs, configurable CORS. **No authentication** (documented limitation).

14.8 Historical secret removal and Git history

What happened (`docs/SECURITY.md`): early commits of the public repository contained the RDS master password (hard-coded in `terraform/main.tf`) and Terraform state files containing it.

- **Why deleting it in a later commit is not enough:** Git stores every version of every file. Anyone can check out the old commit (or browse it on GitHub) and read the password. History must be rewritten.
- **What was done:** password removed from configuration (now generated); state files untracked and ignored; full backups; history rewritten with `git filter-repo` (state files, `node_modules/`, build output removed from every commit; the literal replaced in all blobs), rehearsed on a disposable clone; verified locally (0 occurrences, gitleaks clean); pushed on 2026-10-19 with the owner's explicit approval using `git push --force-with-lease=main:9857ca7 origin main` (the lease makes the push fail if the remote had changed unexpectedly); verified on a fresh clone (32 commits, 0 occurrences, gitleaks clean).
- **Why the credential still had to be treated as compromised:** it was public for some time; clones, forks, caches and search indexes may keep it; GitHub still serves the old, unreferenced commits by SHA until its garbage collection or a Support purge. The protection is that the database it belonged to was **destroyed** and the password **never reused** — a new 32-character generated password protects the new instance. Rewriting history reduces exposure; **rotation** is what actually removes the risk.
- Open item (**BLOCKED** on GitHub Support): request a purge of cached views of the old commits (e.g. `8fbc012`).

14.9 Root account and MFA (owner actions)

From `docs/SECURITY.md` "Current credential status (2026-10-19)": the **root access key is still active** (deployment never used it; deactivation needs the root credentials and the owner's decision); the IAM user `Samruddhi` (admin) has **no MFA**; an unrelated IAM user `Ansible` has an active key; a legacy unused LBC role and policy remain. Best practice (*general*): no access keys for root at all, MFA on root and every human user, least-privilege users or IAM Identity Center for daily work.

14.10 Configured control vs verified outcome

Control	Configured	Verified outcome
Trivy gates	yes	yes — build #3 failed on real findings
SonarQube gate	yes	passes recorded; a failing gate was not demonstrated on AWS
Private RDS	yes	port unreachable from the internet (tested)
TLS to RDS	yes	TLS cipher observed in use (tested)
KMS encryption of Kubernetes Secrets	yes	configured on the cluster; encryption itself is not observable from outside (trust AWS configuration)
Alarm delivery	yes	verified via SQS
Budget alert	yes	**not verified** (threshold never reached)
Least-privilege Jenkins	yes	`kubectl auth can-i` checks in the pipeline; namespace limit by design
No secrets in images/logs	yes	Trivy secret scan; password found in 0 indexed log documents (ELK evidence)

14.11 Viva questions

Question	Answer
Why is deleting a password from the latest commit insufficient?	Git keeps all history; the old commit still contains it. Rewrite history **and** rotate the credential.
Why rotate even after rewriting history?	Copies may exist outside your control (clones, forks, caches); only rotation (or, here, destroying the database) makes the leaked value useless.
What does Trivy scan and how does a finding stop the pipeline?	Dependencies, IaC configuration, secrets, images; the gate commands use `--exit-code 1`, so a finding makes the shell step fail and Jenkins stops the build.
How do you handle false positives?	Review, justify, scope narrowly (`.trivyignore.yaml` statements, path scopes; `.gitleaksignore` fingerprints), document in `SECURITY.md`.
Does 0 findings mean secure?	No; it means no known (fixable, for the gate) issues in scanned components at scan time.
Why IMDSv2?	Requires a session token obtained with a PUT request, which blocks most SSRF attacks from stealing instance-role credentials.
What would you add for production security?	HTTPS, authentication/authorisation, WAF, least-privilege DB user, secrets via External Secrets Operator, GuardDuty/Security Hub, image signing, MFA everywhere, root key removal.

CHAPTER 15

Monitoring and logging

Two complementary systems:

	CloudWatch (+ SNS/SQS, Budgets)	ELK (Elasticsearch, Kibana, Filebeat)
Watches	**infrastructure metrics** emitted by AWS services (RDS CPU, memory, storage, connections; EC2 status/CPU; NAT errors; ALB 5xx/unhealthy hosts) and cost	**application and container logs** (every request, warning, error, with request id and Kubernetes metadata)
Question it answers	"Is something wrong and should someone be notified?"	"What exactly happened to this request / why did it fail?"
Where	AWS-managed	inside the EKS cluster (namespace `logging`) and optionally locally (Compose overlay)
Defined in	`terraform/monitoring.tf`, `scripts/create-alb-alarms.sh`	`k8s/logging/elk.yaml`, `order-inventory-platform/logging/*`, `docker-compose.logging.yml`

15.1 CloudWatch alarms

In general: an alarm evaluates one metric statistic over a **period**; after **evaluation_periods** consecutive breaching periods it goes `OK → ALARM`, and back when not breaching. `INSUFFICIENT_DATA` means no data points (e.g. the resource is stopped). Actions (here: publish to SNS) run on state transitions.

Terraform alarms (`terraform/monitoring.tf`; all have `alarm_actions` = SNS topic; all except the NAT alarm also notify on return to `OK`):

Alarm name	Namespace / metric / dimension	Statistic, period × evaluations	Threshold	Why
`order-inventory-rds-cpu-high`	AWS/RDS `CPUUtilization`, DB id	Average, 300 s × 2	> 80 %	sustained DB load
`order-inventory-rds-free-storage-low`	`FreeStorageSpace`	Minimum, 300 s × 1	< 2 GiB	disk exhaustion (storage autoscaling to 50 GB exists)
`order-inventory-rds-freeable-memory-low`	`FreeableMemory`	Average, 300 s × 2	< 25 MiB	**calibrated**: RDS's own OS leaves 40–60 MiB free on db.t3.micro even idle; 64 MiB fired permanently
`order-inventory-rds-swap-high`	`SwapUsage`	Average, 300 s × 3	> 200 MiB	sustained memory pressure (the meaningful signal on small instances)
`order-inventory-rds-connections-high`	`DatabaseConnections`	Maximum, 300 s × 1	> 50	pool is 10 per pod; 50 means something is leaking/scaling unexpectedly
`order-inventory-jenkins-status-check-failed`	AWS/EC2 `StatusCheckFailed`, instance id	Maximum, 60 s × 3	> 0	instance or host problem
`order-inventory-jenkins-cpu-high`	`CPUUtilization`	Average, 300 s × 3	> 85 %	overloaded CI host
`order-inventory-eks-nodes-cpu-high`	`CPUUtilization`, `AutoScalingGroupName` of the node group	Average, 300 s × 3	> 80 %	nodes need scaling

`errors`	AWS/NATGateway `ErrorPortAllocation`	Sum, 300 s × 1	> 0 (`treat_missing_data = notBreaching`)	private egress failing

ALB alarms (created by `scripts/create-alb-alarms.sh` because the ALB is not a Terraform resource; deleted by suspend, recreated by resume): `order-inventory-alb-5xx` (`HTTPCode_ELB_5XX_Count` Sum 300 s > 10), `order-inventory-alb-target-5xx` (`HTTPCode_Target_5XX_Count` Sum 300 s > 10), and one `order-inventory-alb-unhealthy-hosts-<tg>` per target group (`UnHealthyHostCount` Maximum 60 s × 3 > 0 — downtime detection). The script derives the CloudWatch dimension format `app/<name>/<id>` and `targetgroup/<name>/<id>` from the ARNs.

A real finding: `rds-freeable-memory-low` fired on real metrics: swap was growing under MySQL 8.4 default settings on the 1 GiB instance. The parameter group was tuned (buffer pool 3/8 of RAM, performance_schema off — verified live), swap dropped to ~4 MB, and the threshold was recalibrated plus the swap alarm added (`docs/TESTING.md`). This is monitoring working as intended.

Log groups: `/aws/rds/instance/order-inventory-mysql/error` and `/slowquery`, pre-created with 14-day retention (otherwise RDS creates them with "never expire").

During suspension: the 9 Terraform alarms remain (`INSUFFICIENT_DATA` — verified 2026-10-25; NAT alarm `OK` because missing data counts as not breaching); the 4 ALB alarms are deleted (their ALB no longer exists) and recreated for the new ALB on resume.

15.2 SNS → SQS (and email) notification path

```
CloudWatch alarm / AWS Budget ──publish──► SNS topic order-inventory-alerts (encrypted with KMS key alias/order-inventory-app)
                                                ├──► SQS queue order-inventory-alerts-audit (encrypted, 4-day retention)
                                                └──► email (only if TF_VAR_alert_email is set — currently NOT subscribed)
```

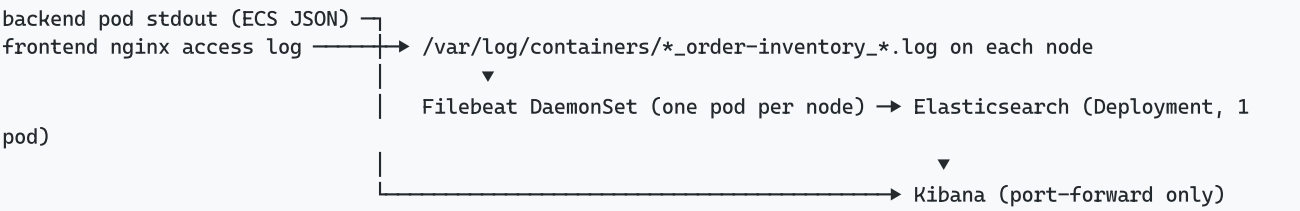
Resource	Detail
`aws_sns_topic.alerts`	`kms_master_key_id = aws_kms_key.app.id`
`aws_sns_topic_policy.alerts`	account owner may manage; **service principals `cloudwatch.amazonaws.com` and `budgets.amazonaws.com` may publish** only for this account (`aws:SourceAccount` condition)
`aws_sqs_queue.alerts_audit`	`message_retention_seconds = 345600` (4 days), KMS-encrypted
`aws_sqs_queue_policy.alerts_audit`	only SNS may send, and only from this topic (`aws:SourceArn`)
`aws_sns_topic_subscription.alerts_audit`	protocol `sqs`
`aws_sns_topic_subscription.alerts_email`	created only when `alert_email` is non-empty
KMS key policy	CloudWatch/Budgets/SNS may use the key — without this, publishing to an encrypted topic fails silently

Verification (VERIFIED, `docs/TESTING.md`): `aws cloudwatch set-alarm-state --alarm-name order-inventory-rds-cpu-high --state-value ALARM --state-reason test` → a message arrived in the SQS queue with the expected alarm name, new state and metric → alarm restored to `OK`. Reading the queue: `aws sqs receive-message --queue-url $(terraform -chdir=terraform output -raw alerts_audit_queue_url)`. Why an SQS "audit" queue: delivery can be proven and inspected from the CLI without a mailbox, and alerts are kept for 4 days.

15.3 AWS Budgets

`aws_budgets_budget.monthly` (`order-inventory-monthly`): COST, MONTHLY, limit `monthly_budget_usd` (350 in the full profile); notifications: **ACTUAL > 80 %** and **FORECASTED > 100 %** → SNS topic (and email if set). Limitations: budget data is updated a few times per day (not real time), it alerts but **does not stop spending**, actual spend is net of credits, and the notification path was never triggered because spend stayed below the threshold (**IMPLEMENTED BUT NOT VERIFIED**). Observed 2026-10-25: actual $0.00, forecast $7.13 for October as reported by AWS Budgets (indicative only).

15.4 ELK on EKS (`k8s/logging/elk.yaml`)



There is **no Logstash** in this project: Filebeat does the light parsing itself (`decode_json_fields`) and writes directly to Elasticsearch.

Block by block:

Object	Key settings	Why
Namespace `logging`		separation from the app
Deployment `elasticsearch` 8.19.5	pod security: `runAsNonRoot`, UID/GID 1000, `fsGroup`, seccomp; **initContainer** `copy-config` copies the default config into an `emptyDir` (`cp -R`) because the root filesystem is read-only; env `discovery.type=single-node`, `xpack.security.enabled=false`, heap 512 MB, disk-watermark checks disabled (emptyDir on the node root volume), `ES_TMPDIR=/tmp`; readiness `/_cluster/health?local=true`; requests 200m/1Gi, limit 1 CPU/1Gi; read-only root fs, no capabilities; volumes `data`, `config`, `logs`, `tmp` = emptyDir	demo-sized single node; **data is lost when the pod or node goes away** (accepted: logs are diagnostic, 7-day retention)
Service `elasticsearch` :9200	ClusterIP	only reachable inside the cluster
Deployment `kibana` 8.19.5	`ELASTICSEARCH_HOSTS` = in-cluster FQDN, Node heap 512 MB, readiness `/api/status` (up to 30 failures), non-root, read-only root fs, emptyDir data/tmp	
Service `kibana` :5601	ClusterIP	access only with `kubectl port-forward` — never published
ServiceAccount + ClusterRole + ClusterRoleBinding `filebeat`	get/list/watch namespaces, pods, nodes, replicasets	needed by `add_kubernetes_metadata`

	`app=order-inventory`; output to Elasticsearch; ILM policy `order-inventory-logs` (rollover 1 day / 1 GB, **delete after 7 days**)	
DaemonSet `filebeat`	env `NODE_NAME` from the downward API; runs as **root** (UID 0) but read-only root fs, no privilege escalation, all capabilities dropped except `DAC_READ_SEARCH`; mounts `/var/log/containers` and `/var/log/pods` read-only; registry in hostPath `/var/lib/filebeat-data`	must read root-owned log files on the node — the one accepted Trivy exception (KSV-0118, path-scoped)

Installed once by a cluster admin: `kubectl apply -f k8s/logging/elk.yaml`. Resources come from the existing nodes (no extra AWS cost). Suspend scales Elasticsearch and Kibana to 0; resume scales them to 1, waits for them and Filebeat, and checks Kibana `/api/status` through a temporary port-forward.

Kibana objects (`order-inventory-platform/logging/kibana-saved-objects.ndjson`, imported by `kibana-setup.sh`): data view `filebeat-*` (`oi-logs`), four saved searches — "Backend warnings and errors", "Order failures (rejected orders and status changes)", "Database availability problems", "Order lifecycle events" — and the dashboard **Order Inventory - Operations** (`oi-operations`). Because they are stored in Elasticsearch (emptyDir), they must be **re-imported after every resume** (see After the restore).

Local ELK (Compose)

`docker-compose.logging.yml` adds Elasticsearch/Kibana (ports bound to `127.0.0.1` only, security off — "do NOT expose this configuration on a network"), Filebeat with Docker autodiscover for the three named containers (`logging/filebeat.yml`, runs as root with the Docker socket mounted read-only), and switches the backend to ECS JSON. Same ILM policy (`logging/ilm-policy.json`).

15.5 Tracing one request with its request id

1. The browser (or curl) receives an error, e.g. 400 for an order exceeding stock; the JSON contains `"requestId": "1ef5064e-…"` and the response header `X-Request-Id` has the same value.
2. Where the id comes from: nginx generates `$request_id` and forwards it as `X-Request-Id` (Compose path), or the backend's `RequestIdFilter` generates a UUID (ALB path, since the ALB sends `/api` directly to the backend). The filter puts it in the MDC → every backend log line of that request contains it.
3. Kibana → Discover → data view `filebeat-*` → query `requestId : "1ef5064e-…"`.
4. Result recorded on EKS (`docs/evidence/elk-eks-2026-10-19.txt`): 1 document — `WARN OrderService 'Order rejected for user 1: insufficient stock for product 1 (available 20, requested 999999)'` with `kubernetes.namespace=order-inventory`, the pod name, container `backend`, `service.environment=aws`.
5. Privacy check from the same evidence: the RDS password appeared in **0** indexed documents.

15.6 Commands

```
aws cloudwatch describe-alarms --alarm-name-prefix order-inventory --query 'MetricAlarms[].
[AlarmName,StateValue]' --output table
aws sqs receive-message --queue-url "$(terraform -chdir=terraform output -raw alerts_audit_queue_url)" --
max-number-of-messages 5
kubectl -n logging get pods
kubectl -n logging port-forward svc/kibana 5601:5601            # then http://localhost:5601
bash order-inventory-platform/logging/kibana-setup.sh http://localhost:5601
kubectl -n order-inventory logs deploy/backend --tail=20       # raw ECS JSON lines
```

15.7 Viva questions

Question	Answer
Difference between CloudWatch and ELK here?	CloudWatch = AWS metrics, alarms, notifications; ELK = searchable application/container logs for diagnosis.
How do you know alerts are delivered?	Forced an alarm state; the message arrived in the SQS audit queue subscribed to the SNS topic.
Why a customer-managed KMS key for SNS?	CloudWatch can only publish to an encrypted topic if the key policy allows it; the AWS-managed SNS key does not.
Why do alarms show INSUFFICIENT_DATA now?	The environment is suspended; stopped resources emit no metrics.
Is the budget a hard limit?	No, it only notifies (ACTUAL 80 %, FORECASTED 100 %); it never fired.
Why no Logstash?	Not needed: Filebeat parses JSON and adds Kubernetes metadata; fewer components to run.
What happens to logs during suspension?	Elasticsearch data is on emptyDir and is lost when the nodes are removed; logging restarts empty on resume.
How would you make logging production-grade?	Managed OpenSearch or CloudWatch Logs/Container Insights, persistent storage, security enabled, longer retention, alerting on log patterns.

CHAPTER 16

Suspend and resume, line by line

Files: `scripts/lib/evaluation-common.sh` (shared library), `scripts/suspend-evaluation.sh`, `scripts/resume-evaluation.sh`, `scripts/suspend-evaluation.ps1`, `scripts/resume-evaluation.ps1`, `docs/recovery/evaluation-manifest.json`, plus Terraform lifecycle rules in `terraform/main.tf` (node group) and `terraform/jenkins.tf` (instance). Runbook: `docs/RUNBOOKS.md` §10.

16.1 Suspension is not destruction

	Suspend script	`terraform destroy`
Cluster, network, IAM, KMS, secrets, ECR, state	kept	deleted (secrets/KMS after 7-day windows)
Database	stopped, data + backups kept	**deleted without final snapshot** (`skip_final_snapshot = true`)
Jenkins	stopped, disk kept	instance and disk deleted
ALB	deleted by Kubernetes, recreated by Helm	must be deleted first or VPC deletion fails
Restore time	≈ 10 min, one command, verified	≈ 20+ min apply + Ansible + cluster bootstrap + first deployment
Cost while "off"	≈ $4/day estimate (control plane, NAT, storage)	≈ $0

16.2 Shared library `scripts/lib/evaluation-common.sh`

```
#!/usr/bin/env bash
# shellcheck disable=SC2034  # variables are used by the scripts that source this file
set -uo pipefail
```

Sourced (not executed) by both scripts. `-u`: using an unset variable is an error; `pipefail`: a pipeline fails if any part fails. `-e` is deliberately **not** set: every important command is checked explicitly and fails through `die` with a clear stage and reason.

```
REPO_ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/../.." && pwd)"
export AWS_PROFILE="${AWS_PROFILE:-order-inventory}"
export AWS_REGION="${AWS_REGION:-ap-south-1}"; export AWS_DEFAULT_REGION="$AWS_REGION"
EXPECTED_ACCOUNT="038215676861"; NAMESPACE="order-inventory"; RELEASE="order-inventory"
MANIFEST="$REPO_ROOT/docs/recovery/evaluation-manifest.json"; LOG_DIR="$REPO_ROOT/.deploy/evaluation-logs"
mkdir -p "$LOG_DIR" "$(dirname "$MANIFEST")"
```

Works from any current directory; profile and region default to the project's but can be overridden; the expected account is a constant (safety: wrong account → stop).

```
ts() { date +%H:%M:%S; }   log() { echo "[$(ts)] $*"; }
stage() { STAGE="$1"; echo; echo "[$(ts)] ===== $1 ====="; }
ok() { echo "[$(ts)]   OK  $*"; }   warn() { echo "[$(ts)]   WARN $*"; }
die() { …prints "FAILED in stage: $STAGE", "reason: $*", diagnostic commands (sts identity, nodegroup
        status, kubectl get nodes/pods/ingress/events, RDS status, Jenkins state), and
        "The scripts are idempotent: fix the cause and run the same command again."; exit 1; }
```

Uniform, timestamped output; `STAGE` always holds the current stage name for error messages.

```
wait_until() {                    # wait_until <timeout-seconds> <description> <command...>
  local timeout="$1" desc="$2"; shift 2; local start=$SECONDS
  until "$@" >/dev/null 2>&1; do
    [ $((SECONDS - start)) -ge "$timeout" ] && die "timed out after ${timeout}s waiting for: $desc"
    sleep 10
  done
  ok "$desc ($((SECONDS - start))s)"
}
```

Polls a predicate every 10 s with an explicit timeout; prints how long it took (these are the durations in the logs, e.g. "RDS available (165s)").

```
win_path() { if command -v cygpath >/dev/null 2>&1; then cygpath -w "$1"; else echo "$1"; fi; }
TF_DIR="$(win_path "$REPO_ROOT/terraform")"
tfout() { terraform -chdir="$TF_DIR" output -raw "$1" 2>/dev/null | tr -d '\r'; }
json_get() { node -e '…read file, walk dotted path, print value…' "$(win_path "$1")" "$2"; }
```

Windows Git Bash compatibility: `terraform.exe` and `node.exe` need Windows paths; `tr -d '\r'` removes Windows line endings from CLI output. `tfout` reads Terraform outputs from the remote state; `json_get` reads the manifest.

`preflight()`:

1. all tools present (`aws terraform kubectl helm node curl`) else `die`;
2. `aws sts get-caller-identity` → ARN and account; `die` if the profile does not work, the account is not `038215676861`, or the ARN ends in `:root`;
3. `terraform init -input=false` (connects to the S3 backend) else `die`;
4. `CLUSTER=$(tfout eks_cluster_name)`, `RDS_ID=$(tfout rds_endpoint | cut -d. -f1)` (the identifier is the first label of the endpoint), `JENKINS_ID=$(tfout jenkins_instance_id)`, `REGISTRY=<account>.dkr.ecr.<region>.amazonaws.com`, `NODEGROUP` from `aws eks list-nodegroups`, `STATE_BUCKET` parsed from `terraform/backend.tf`; `die` if any is empty;
5. `aws s3api head-bucket` on the state bucket; cluster status must be `ACTIVE`.

`ensure_kube_access()`: `aws eks update-kubeconfig --name $CLUSTER --alias $CLUSTER`, `kubectl config use-context`, `kubectl version --request-timeout=15s`; on failure: "cannot reach the Kubernetes API from this machine (is your public IP allowed? …)".

State queries (all strip `\r`, all read-only): `rds_status`, `ec2_state`, `ng_field <jmespath>`, `asg_instances` (instances in the node group's Auto Scaling group), `app_albs` (count of load balancers named `k8s-orderinv*`). `print_cost_summary` prints control plane, node instances, Jenkins state, RDS state, ALB count and NAT count. Predicates for `wait_until`: `rds_is <state>`, `ec2_is <state>`, `ng_active`, `no_asg_instances`, `no_app_albs`, `no_app_tgs`, `http_200 <url>`.

16.3 `suspend-evaluation.sh`

Lines	Code	What / why	Destructive?	Idempotency
10	`source …/lib/evaluation-common.sh`	load helpers	—	—
12	`PLAN_ONLY=false; [ "${1:-}" = "--plan" ] && PLAN_ONLY=true`	read-only mode	—	—
13	`exec > >(tee -a "$LOG_DIR/suspend-<timestamp>.log") 2>&1`	everything printed is also logged	—	—

20	read desired/min/max, instances, RDS, Jenkins, ALBs	current state	no	—
22–23	`ensure_kube_access` only if nodes exist	with 0 nodes there is nothing to do in Kubernetes	no	—
25–34	`--plan` : print what would happen + cost summary, `exit 0`	safe preview	no	—
36–76	**Stage 1 manifest:** if the backend Deployment exists, read the backend and frontend image tags (must be equal, else `die`), warn if not 2/2 ready, **verify both images exist in ECR** (else `die` : cannot guarantee restore), read the latest Helm revision and current ALB hostname, write the manifest JSON with Node (account, region, profile, backend bucket/key, cluster, node group sizes, RDS id + secret name, Jenkins id + secret name + repo, namespace/release/chart/revision/registry/repositories/ `lastKnownGoodTag` /replicas/ingress/ALB hostname, logging replicas, `suspendedAt` , `rdsAutoRestartAfter` = now + 7 days)	records exactly what resume must restore; **no secrets** (only secret *names*)	no (writes a local file)	
77–81	else: keep the existing manifest (app already scaled down) or `die` if none	re-running after a partial suspend must not overwrite good data with an empty state	no	✓
83–93	**Stage 2 ALB:** if an app ALB exists: requires nodes (else `die` "resume first"); `helm upgrade … --reuse-values --set ingress.enabled=false --wait --timeout 5m` ; wait (≤ 600 s) for 0 `k8s-orderinv` ALBs and (≤ 300 s) 0 target groups	deletes the Ingress through Helm (chart and values otherwise unchanged); the **controller deletes the ALB**, so this must run while the controller has nodes	**yes** (ALB deleted; recreatable)	skipped when no ALB
94–98	delete CloudWatch alarms named `order-inventory-alb*`	they refer to the deleted ALB	yes (recreated by resume)	no-op when none
100–107	**Stage 3:** `kubectl scale deploy backend frontend --replicas=0` ; ES/Kibana to 0; confirm `kubectl get pv` is empty	pods gone cleanly before node drain (scaling is not an eviction, so PDBs do not block it); no PersistentVolumes means nothing on node disks needs saving	pods removed (Deployments kept)	✓
109–116	**Stage 4:** if desired ≠ 0: wait node group ACTIVE, `update-nodegroup-config minSize=0,maxSize=<max>,desiredSize=0` ; wait (≤ 1500 s) for 0 ASG instances	EC2 workers terminated; Terraform ignores desired size, min 0 is allowed by `full.tfvars`	instances terminated (definition kept)	skipped if already 0; waiting is repeatable
118–127	**Stage 5:** record Jenkins root volume id, size, type, encryption; `stop-instances` if running/pending; nothing if stopped/stopping; `die` otherwise; wait for `stopped` (≤ 600 s)	proves the data volume exists before stopping	no (stop, not terminate)	✓
129–137	**Stage 6:** `stop-db-instance` only from `available` ; nothing if stopped/stopping; `die` for other states (e.g. `backing-up`); wait for `stopped` (≤ 1200 s); warn about the 7-day automatic start		no	✓
139–151	**Stage 7 verification:** cluster ACTIVE; ECR images for the manifest tag exist; the 3 secrets exist; state object exists in S3; RDS `LatestRestorableTime` (PITR); no app ALB remains (`die` otherwise)	proof that restore is possible	no	—
153–155	cost summary; "SUSPENDED. Restore with: bash scripts/resume-evaluation.sh"			

Measured 2026-10-19: 1396 s and 1383 s (two runs); node termination 1027–1028 s of that.

16.4 `resume-evaluation.sh`

Lines	Code	What / why
11–16	source library, log to `resume-<timestamp>.log`, `T0=$SECONDS`, `preflight`	
17–25	manifest must exist; read `lastKnownGoodTag`, `desiredSize`, `maxSize`; `NG_MIN_RUN=1`; `die` if incomplete; **both images for the tag must exist in ECR**	fail before changing anything
27–55	**Stage 1 admin IP:** `curl https://checkip.amazonaws.com` → this laptop's public IP; compare with the cluster's `publicAccessCidrs`. If missing: back up `terraform/admin.auto.tfvars` to `.bak`, write the new `/32`, run `terraform plan -target=aws_eks_cluster.main -target=…jenkins_ssh -target=… jenkins_web -out=…`, inspect the plan JSON with Node and **refuse** unless the only changes are an in-place `update` of `aws_eks_cluster.main` and create/delete of the two Jenkins SG rules; print the plan summary; `apply` exactly that saved plan	the only Terraform change resume may make; protects against accidental replacements. Dry-run of this branch on 2026-10-25: *2 to add, 1 to change, 2 to destroy*, guard accepts (not applied)
57–64	**Stage 2 RDS:** `stopped` → start; `available/starting/configuring-enhanced-monitoring/backing-up/modifying` → just log; `stopping` → wait until stopped, then start; other → `die`	handles the AWS auto-start (state `available`)
65–71	Jenkins: `stopped` → start; running/pending → log; `stopping` → wait then start	
72–79	node group: if desired < manifest desired: wait ACTIVE, `update-nodegroup-config minSize=1,maxSize=<max>,desiredSize=<2>`	the three start requests run in parallel in AWS; waiting happens later
81–91	**Stage 3:** `ensure_kube_access`; wait (≤ 1200 s) until ≥ 2 nodes `Ready`; print nodes; `kubectl rollout status` (≤ 600 s each) for `coredns`, `aws-load-balancer-controller`, `aws-node`, `kube-proxy`; Secret `mysql-secret` must exist	the controller must run before an Ingress can become an ALB
93–94	**Stage 4:** wait (≤ 1500 s) for RDS `available`	the backend needs the DB to become Ready
96–104	**Stage 5:** if the backend already has 2 replicas of `<registry>/order-inventory-backend:<tag>`, the Ingress exists and both deployments are 2/2 ready → skip; else `ROLLOUT_TIMEOUT=10m scripts/deploy-k8s.sh <registry> <tag>`	`helm upgrade` **without** `--reuse-values` → chart defaults: replicas 2/2 and `ingress.enabled: true`; automatic rollback on failure
106–125	**Stage 6:** wait for the Ingress hostname (≤ 600 s), ALB `active` (≤ 600 s), both `k8s-orderinv` target groups with ≥ 2 targets all `healthy` (≤ 900 s), and `http://<alb>/api/health` = 200 (≤ 600 s; covers DNS propagation)	
127–135	**Stage 7:** `/api/health`, `/api/health/ready`, `/` must be 200; print the last Flyway messages from the backend log; run the full read/write `smoke-test.sh` (20 checks)	proves DB connectivity, migrations state and real reads/writes
137–142	**Stage 8:** `create-alb-alarms.sh`; on failure only `warn` (the app is healthy)	
144–162	**Stage 9:** if namespace `logging` exists: scale ES/Kibana to 1, `rollout status` ES, Kibana, Filebeat; port-forward Kibana to 15601 in the background, poll `/api/status` for `available` (≤ 180 s), stop the port-forward	verifies ELK without exposing it
164–170	**Stage 10:** wait Jenkins `running`; read its **new** public IP from EC2 (not from Terraform state); wait for `/login` 200; `die` if its SG has any `0.0.0.0/0` rule	secure path check
172–177	**Stage 11:** manifest += `resumedAt`, `application.currentUrl`, `jenkins.publicIpAtResume`; remove `rdsAutoRestartAfter`	

194	summary block (URL, release, checks, Jenkins URL, Kibana command)	printed only after every check passed

16.5 PowerShell wrappers (`scripts/*-evaluation.ps1`)

```
$ErrorActionPreference = 'Stop'
$bash = @("$env:ProgramFiles\Git\bin\bash.exe", "${env:ProgramFiles(x86)}\Git\bin\bash.exe") | Where-
Object { Test-Path $_ } | Select-Object -First 1
if (-not $bash) { throw 'Git Bash not found: install Git for Windows' }
if (-not $env:AWS_PROFILE) { $env:AWS_PROFILE = 'order-inventory' }
Push-Location (Split-Path -Parent $PSScriptRoot)        # repository root
try { & $bash scripts/resume-evaluation.sh @args; exit $LASTEXITCODE } finally { Pop-Location }
```

Finds Git Bash, defaults the profile, runs the `.sh` with the same arguments (`@args` , e.g. `--plan`) from the repository root and returns its exit code. Run with `powershell -ExecutionPolicy Bypass -File scripts\…ps1` (bypass only for this process; the script is not signed). `.gitattributes` keeps `.ps1` files with CRLF.

16.6 Recovery manifest (`docs/recovery/evaluation-manifest.json`)

Fields at the last suspend: `awsAccount` , `awsRegion` , `awsProfile` ; `terraformBackend` {bucket, key, lock}; `eks` {cluster, nodegroup, desiredSize 2, minSize 1, maxSize 3}; `rds` {identifier, secret *name*}; `jenkins` {instanceId, adminSecret *name*, repo}; `application` {namespace, helmRelease, helmChart, helmRevisionAtSuspend 14, registry, repositories, **lastKnownGoodTag** `8-18a2bc6` , replicas, ingress, albHostnameAtSuspend}; `logging` {namespace, manifest, replicas}; `suspendedAt` ; `rdsAutoRestartAfter` . It contains no secret values and is committed so the restore information survives a lost laptop.

16.7 Order dependencies

Suspend: manifest → **ALB (needs controller → needs nodes)** → apps to 0 → nodes to 0 → Jenkins stop → RDS stop → verify. Doing nodes first would leave an orphaned ALB that nothing can delete (the script detects that and asks to resume first).

Resume: identity/backend → admin IP (else kubectl/Jenkins unreachable) → start RDS, Jenkins, nodes (parallel) → nodes Ready + add-ons (controller, DNS, CNI) → RDS available (backend readiness needs it) → Helm release (Ingress needs the controller) → ALB + healthy targets → functional tests → alarms (need the ALB) → ELK → Jenkins check → manifest.

16.8 Interrupted halfway — what happens

Every step checks state first, so the remedy is always "run the same script again":

Interrupted after	Re-running suspend	Re-running resume
manifest written	rewrites it from the still-running app	—
ALB deleted, apps still running	stage 2 skipped (no ALB); continues	Helm redeploys with Ingress
apps scaled to 0, nodes still up	manifest kept ("already scaled down"); continues	deploy-k8s restores 2/2
node scale-down requested	waits for 0 instances	scales back up
Jenkins/RDS stop requested	waits for `stopped`	waits for stopped, then starts (`stopping` branch)

16.9 What stays billable and why

See Exact shutdown instructions and the suspension table in the Terraform chapter: EKS control plane (cannot be paused), NAT gateway + EIP (nodes need it to start; kept deliberately), EBS of Jenkins, RDS storage/backups, Secrets Manager, KMS, alarms, ECR/S3 storage. Estimated ≈ $126/month on 2026-10-19 from list prices (not a verified bill).

16.10 Troubleshooting

Problem	Symptom (text printed by the scripts or AWS)	Fix
AWS credentials expired/invalid	`AWS CLI profile 'order-inventory' is not usable`	`aws configure --profile order-inventory` (new access key for the deployer user) and re-run
Wrong profile/account	`wrong AWS account (<id>), expected 038215676861` or `refusing to run as the root user`	`$env:AWS_PROFILE='order-inventory'` / `export AWS_PROFILE=order-inventory`
Terraform backend unavailable	`terraform init failed (S3 backend reachable?)` or `state bucket … not reachable`	check internet, credentials, bucket existence (`aws s3api head-bucket --bucket order-inventory-tfstate-038215676861`)
RDS fails to start	stuck in stage 4 / `timed out after 1500s waiting for: RDS available`	`aws rds describe-events --source-identifier order-inventory-mysql --source-type db-instance --duration 120`; re-run
Nodes Pending/NotReady	`timed out … node(s) Ready`	`kubectl get nodes`, `kubectl describe node`, node group health (`aws eks describe-nodegroup … --query nodegroup.health`), NAT gateway present
Jenkins has a new public IP	normal; printed in the summary	use the printed IP; `terraform apply -refresh-only …` before Ansible
ECR image not found	`ECR image order-inventory-backend:<tag> not found`	lifecycle policy keeps 20 images; pick an existing tag, edit `lastKnownGoodTag` in the manifest, re-run
Helm rollout times out	`Helm deployment of <tag> failed (deploy-k8s.sh rolled back if possible)`	`kubectl -n order-inventory get pods,events`; DB secret present? RDS available? re-run
ALB slow to become healthy	waiting on `ALB active` / target groups / `api/health returns 200`	new DNS can take minutes; `kubectl -n kube-system logs deploy/aws-load-balancer-controller`
DNS/network fails on the laptop	`could not determine this machine's public IP`, `cannot reach the Kubernetes API`	check connectivity/VPN; re-run (updates the allowed IP when it can determine it)
Script run twice	prints `already …`, `skipped`, 55 s total on a running environment (measured)	nothing to do — idempotent
RDS restarted itself after 7 days	RDS `available` while everything else is suspended	run suspend again: only stage 6 does work
ALB exists but no nodes	`an application ALB exists but the cluster has no nodes … resume first`	run resume, then suspend

16.11 Test record (2026-10-19)

`suspend --plan` (Git Bash and PowerShell wrapper)	correct preview, no changes
Suspend #1	exit 0, 1396 s; independent checks: 0 LBs, 0 k8s target groups, 0 running instances, RDS/Jenkins stopped, ECR (incl. unrelated `frontend`) intact, KMS enabled
`terraform plan` while suspended	found the Jenkins replacement risk → fixed with `ignore_changes` → no resource changes
Resume	exit 0, **9 min 14 s**, all stages, 20/20 smoke checks, new ALB
Resume again (idempotency)	exit 0, 55 s, nothing changed
Suspend #2 (after build #8)	exit 0, 1383 s, manifest tag `8-18a2bc6`
Admin-IP branch	targeted plan dry-run 2026-10-25 only (accepted by the guard); apply not yet executed

CHAPTER 17

Viva and cross-question bank

Format: **Q** — question; **Say** — a short answer in simple spoken English; **Deeper** — detail if the evaluator follows up. Level: (B) beginner, (I) intermediate, (A) advanced. Every answer matches the code in this repository; file references are given so you can show the evidence.

17.1 A. The hard cross-questions first

Q (A): Why did you choose EKS instead of ECS? Say: "I wanted the deployment to be portable and to learn Kubernetes properly: the same Helm chart runs on minikube and on EKS, and I tested rollouts and rollbacks locally before AWS." Deeper: Kubernetes gives readiness/liveness/startup probes, rolling-update control (`maxSurge` / `maxUnavailable`), PDBs, Helm revisions, DaemonSets (Filebeat) and the AWS Load Balancer Controller. Trade-off: EKS has a control-plane fee (≈ $0.10/h) and more complexity; ECS would be cheaper and simpler for two services.

Q (B): Why is RDS private? Say: "Only the backend needs the database. It sits in subnets with no route to the internet, has no public IP, and its security group accepts MySQL only from the EKS cluster." Deeper: `aws_route_table.database` has no routes, `publicly_accessible = false`, SG rule from `cluster_security_group_id`, TLS required (`require_secure_transport=1`), encrypted storage. Tested: port 3306 unreachable from the internet.

Q (B): Why use both Terraform and Ansible? Say: "Terraform creates cloud resources through the AWS API; Ansible installs and configures software inside the Jenkins server over SSH. Each is idempotent in its own area." Deeper: `terraform/jenkins.tf` creates instance, SG, IAM role, passwords; `ansible/roles/*` install JDK, Maven, Node, Docker, Trivy, Helm, kubectl, Terraform, AWS CLI, Jenkins, SonarQube and configure Jenkins as code.

Q (B): What is the difference between a Deployment and a ReplicaSet? Say: "A ReplicaSet keeps a fixed number of identical pods running. A Deployment manages ReplicaSets: for a new version it creates a new ReplicaSet and shifts pods over gradually, and it can go back." Deeper: `kubectl -n order-inventory get rs` shows one ReplicaSet per pod-template version (`revisionHistoryLimit: 10`).

Q (B): Difference between readiness and liveness probes? Say: "Liveness asks 'is the process healthy or should Kubernetes restart it?'; readiness asks 'can this pod take traffic right now?'. Our liveness `/api/health` does not touch the database; readiness `/api/health/ready` does. So a database outage takes pods out of traffic without restarting them." Deeper: plus a startup probe (up to 180 s) so slow JVM starts are not killed. Tested in `FailurePathWebTest`, the Compose acceptance test and the AWS RDS reboot (0 restarts).

Q (I): What happens when a pod crashes? Say: "Kubernetes restarts the container; if the pod or its node is gone, the ReplicaSet creates a new pod. While it is not ready it gets no traffic, and the other replica keeps serving." Deeper: two replicas per service, PDB `minAvailable: 1` for voluntary disruptions, `restartPolicy: Always` with back-off. Verified "pod loss → replacement Ready" on minikube.

Q (I): What happens when all worker nodes are scaled to zero? Say: "All pods stop because there is nowhere to run them. The control plane keeps every object — Deployments, Services, Secrets, Helm history — so when nodes come back, Kubernetes recreates the pods automatically." Deeper: in the suspend script the apps are scaled to 0 first and the ALB removed first, so nothing is orphaned; resume scales nodes to 2 and redeploys with Helm.

Q (B): Why does EKS still cost money when the nodes are stopped? Say: "AWS runs the Kubernetes control plane for us around the clock and bills it per cluster-hour, about ten cents an hour, independent of the number of nodes."

annotations in `values.yaml` (`internet-facing`, `target-type: ip`, health-check settings, deregistration delay 20 s), public subnets discovered by the `kubernetes.io/role/elb` tag.

Q (A): What happens if the database becomes unavailable during an order? Say: "Nothing half-done is saved, because the whole order is one transaction. The user gets a 503 'database temporarily unavailable, retry', and if the outage lasts, Kubernetes stops sending traffic to the backend pods until the database is back — without restarting them." Deeper: Hikari `connection-timeout=5000` → `CannotCreateTransactionException` → `GlobalExceptionHandler` 503; a broken connection mid-transaction → rollback. Measured on AWS: 3 × 503 during a ~1 s RDS interruption, 0 restarts (`docs/evidence/aws-rds-interruption-2026-10-19.txt`).

Q (A): How does the system prevent overselling? Say: "When an order is created, the backend locks each product row it needs — `SELECT … FOR UPDATE` — inside one transaction, checks the stock and deducts it before committing. A second buyer has to wait for the lock and then sees the new stock, so only one of them can take the last unit." Deeper: products locked in ascending id order to avoid deadlocks; duplicate lines share one locked object; `CHECK (stock_quantity >= 0)` in V2 as a database-level guard; `OrderConcurrencyTest`: 8 buyers, stock 3 → exactly 3 orders (mutation-checked). Lock wait timeout → 409 "retry". Limitation: the test runs on H2.

Q (B): Why use Flyway? Say: "The database schema is versioned like code. Each change is a numbered SQL file that runs once, in order, automatically at startup, identically in tests, locally and on RDS; Hibernate only validates it."

Q (B): Why immutable image tags? Say: "Each build gets a unique tag `<build>-<commit>` and the registry refuses to overwrite it, so a tag always means exactly one tested image. Rollback is just deploying an older tag." Deeper: ECR `image_tag_mutability = "IMMUTABLE"`; Helm helper fails on missing tag or `latest`; `deploy-k8s.sh` refuses `latest`; OCI revision labels link images to commits.

Q (A): How does Jenkins roll back a bad release? Say: "Before deploying, the deploy script records the last good Helm revision. Helm waits until the new pods are Ready; if they are not, the script rolls back to that revision and fails the build. If the release becomes Ready but fails the checks afterwards, the pipeline's failure handler rolls back too." Deeper: `scripts/deploy-k8s.sh` (`helm upgrade --wait`, `helm rollback $PREV_REVISION`), Jenkinsfile `post { failure }`. Verified on AWS: non-existent image and never-Ready release both rolled back automatically with 0 failed user requests.

Q (A): How do you prove zero downtime? Say: "I ran a load generator through the real public ALB during two version changes, a bad image, a release that could never become ready, and a restart. Every response was recorded: 9,532 requests, zero non-200." Deeper: `scripts/aws-zero-downtime-test.sh`; evidence `docs/evidence/aws-zero-downtime-2026-10-19.txt`. Mechanisms: `maxUnavailable: 0`, `maxSurge: 1`, readiness gating, preStop 10 s, graceful shutdown 20 s, ALB deregistration delay 20 s, PDBs.

Q (I): What is the difference between Terraform state and the actual AWS state? Say: "The state file is Terraform's memory of what it created. AWS is the truth. `terraform plan` refreshes the state from AWS and shows the differences against my code." Deeper: state in S3 (encrypted, versioned, locked); contains sensitive values (generated passwords) → never in Git. Drift example: stopped Jenkins reports no public IP.

Q (A): Why was `ignore_changes` required for the Jenkins instance? Say: "When Jenkins is stopped it has no public IP, and AWS reports `associate_public_ip_address = false`. Terraform saw that as a difference and planned to destroy and recreate Jenkins. That attribute only matters at creation, so I told Terraform to ignore it — same for the AMI, so a new Ubuntu image never replaces the server." Deeper: found in a plan while suspended, nothing was applied; afterwards the plan showed no resource changes. Limitation: ignored attributes are no longer managed (deliberate changes need `-replace`).

Q (A): Why is deleting a password from the latest commit insufficient? Say: "Git keeps every old version. Anyone can open the old commit and read it. You must rewrite history and, more importantly, rotate the credential." Deeper: `git filter-repo` rewrite, force-with-lease push (approved), fresh-clone verification; GitHub still serves old SHAs until purged; the leaked password's database was destroyed and never reused.

Q (I): What happens if the resume script stops halfway through? Say: "Run it again. Every step checks the current state first — for example it only starts RDS if it is stopped and only deploys if the release is not already running — so it continues where it stopped." Deeper: measured idempotent re-run: 55 s, no changes. Failures print `FAILED in stage`, the reason and diagnostic commands.

Q (B): Why does the ALB URL change after resume? Say: "Suspend deletes the Ingress so AWS deletes the load balancer to save cost. Resume creates a new Ingress, so AWS creates a new load balancer with a new DNS name. The script prints the new URL."

Q (A): What would you improve if this were a real production deployment? Say: "HTTPS with a certificate, real authentication, Multi-AZ RDS with deletion protection, autoscaling, GitHub webhooks or OIDC-based CI, managed logging, and canary releases." Deeper: see Future improvements.

Q (A): Which parts are production-style and which are simplified? Say: "Production-style: immutable tagged images, gated pipeline, zero-downtime rolling updates with automatic rollback, infrastructure as code with remote state, private database with TLS, least-privilege IAM, alarms, central logs. Simplified: no login, HTTP only, single NAT, single small database, demo-sized ELK, no autoscaling, fixed node count."

17.2 B. Project overview and design

#	Q	Say
1 (B)	What does the application do?	Manages users, products and orders; reserves stock when an order is placed and returns it when cancelled.
2 (B)	What is the DevOps part?	Automated build, test, quality and security gates, container packaging, deployment to Kubernetes on AWS without downtime, infrastructure as code, configuration management, monitoring, logging and cost control.
3 (I)	Is it microservices?	The platform is microservice-ready; the application is one Spring Boot service plus a separate frontend container. Splitting orders and inventory would need distributed transactions.
4 (I)	Why one region and two AZs?	Two AZs are required by EKS and give resilience for nodes and the ALB; one region keeps cost and complexity low.
5 (A)	What are the single points of failure?	One NAT gateway, single-AZ RDS, one Jenkins host, single-node Elasticsearch. Documented trade-offs for cost.

17.3 C. Frontend

#	Q	Say
1 (B)	Why React and Vite?	Component-based UI; Vite is fast, maintained, produces hashed static assets and shares config with Vitest.
2 (B)	How does the UI call the backend?	`fetch('/api/…')` on the same origin; the ALB/nginx/Vite proxy routes `/api` to the backend.
3 (I)	Build-time vs runtime config?	`VITE_*` is compiled into the bundle; nginx `BACKEND_URL` is substituted when the container starts.
4 (I)	Why no secrets in the frontend?	Everything in the bundle is downloadable by every user.
5 (I)	How are errors shown?	`api.js` turns non-2xx responses into errors with the backend's message or field details; pages show toasts.
6 (A)	Why reject a 200 HTML response from `/api`?	It means `/api` was routed to the SPA fallback; failing loudly exposes routing mistakes.

17.4 D. Java and Spring Boot

#	Q	Say
1 (B)	What does `@RestController` do?	Marks a class whose methods handle HTTP requests and return JSON.
2 (B)	What is dependency injection here?	Spring creates the services/repositories and passes them into constructors.
3 (I)	What does `@Transactional` do?	Wraps the method in a DB transaction; commit on success, rollback on runtime exceptions.
4 (I)	Why DTOs?	Stable API contract, validation at the edge, no accidental exposure of entity internals.
5 (I)	How is validation done?	Bean Validation annotations on request DTOs + `@Valid`; errors become 400 with field details.
6 (A)	Why constructor injection and `final` fields?	Explicit, immutable dependencies; easy to test.
7 (A)	What is `open-in-view=false`?	No DB session during response rendering; prevents hidden lazy-loading queries.
8 (I)	How is a request id generated and used?	`RequestIdFilter` reuses a safe `X-Request-Id` or creates a UUID, puts it in the MDC and the response; error bodies contain it; logs contain it.

17.5 E. REST and HTTP status codes

Code	Used for in this API
200	successful GET/PUT
201	created user/product/order
204	product deleted
400	validation error, malformed JSON, wrong type, insufficient stock, cancelled order change, duplicate email
404	missing user/product/order, unknown route
405	wrong HTTP method
409	conflict with existing data (FK/unique) or lock timeout (retry)
503	database temporarily unavailable; readiness down
500	unexpected error (generic message)

Q (I): Why PUT for status and not PATCH? — Say: "The client sends the complete new value of the status sub-resource `/orders/{id}/status`; PUT replaces it." (PATCH would also be defensible.)

17.6 F. SQL, MySQL, transactions, concurrency

#	Q	Say
1 (B)	What tables exist?	users, products, orders, order_items with foreign keys.
2 (I)	Why `DECIMAL(10,2)` for prices?	Exact money arithmetic.
3 (I)	Why store price in	

	order_items?	So past orders keep the price paid even if the product price changes.
4 (A)	Pessimistic vs optimistic locking?	Pessimistic: lock the row before changing it (used here). Optimistic: detect conflicts at commit with a version column and retry.
5 (A)	Isolation level?	MySQL default REPEATABLE READ; correctness comes from locking reads (`FOR UPDATE`) that see the latest committed data.
6 (A)	What is a deadlock and how is it avoided?	Two transactions waiting for each other's locks; avoided by locking products in ascending id order.
7 (I)	Why CHECK constraints if the API validates?	Defence in depth against bugs or manual SQL.

17.7 G. Docker and Compose

#	Q	Say
1 (B)	Container vs VM?	A container is an isolated process sharing the host kernel; a VM runs its own OS.
2 (B)	Why multi-stage builds?	Small runtime images without build tools.
3 (I)	How do containers find each other in Compose?	Docker DNS resolves service names (`mysql` , `backend`) on the shared network.
4 (I)	Why non-root?	Limits damage if compromised; required by `runAsNonRoot` .
5 (I)	What survives `docker compose down` ?	The named MySQL volume; only `down -v` deletes it.

17.8 H. Git and GitHub

#	Q	Say
1 (I)	What is `--force-with-lease` ?	A force push that only succeeds if the remote is still at the commit you expect, so you never overwrite someone else's new work.
2 (I)	Why `.gitattributes eol=lf` ?	Shell scripts with Windows line endings fail in Linux containers.
3 (I)	Why was build #2 exit 126?	Scripts were not executable in Git (Windows); fixed with `git update-index --chmod=+x` .

17.9 I. Terraform

#	Q	Say
1 (B)	What is IaC?	Infrastructure described in versioned code and created by a tool, reproducibly.
2 (B)	`plan` vs `apply` ?	Plan shows the changes; apply performs them.
3 (I)	Why remote state + locking?	Shared, durable, encrypted; no concurrent corruption.
4 (I)	What is a provider lock file?	Pins provider versions and checksums for reproducibility.
5 (A)	What does `-target` do and why is it risky?	Limits a plan to some resources; can leave the rest inconsistent, so it is only used for the controlled admin-IP update with a guard.

17.10 J. Ansible

#	Q	Say
1 (B)	What is a playbook / role?	A playbook maps hosts to roles; a role is a reusable set of tasks, handlers, defaults, templates.
2 (I)	What is idempotency?	Re-running changes nothing if the host is already correct (`changed=0` , verified).
3 (I)	What is JCasC?	Jenkins configured from a YAML file instead of clicking: users, credentials, job, global env.
4 (A)	How are secrets protected in Ansible?	Streamed on stdin into the controller container, `no_log` on tasks, files `root:jenkins 0640` , placeholders in JCasC.

17.11 K. Jenkins and CI/CD

#	Q	Say
1 (B)	What is CI vs CD?	CI: build and test every change; CD: automatically release tested changes to the environment.
2 (I)	How is the pipeline triggered?	SCM polling of GitHub every 5 minutes (verified by build #8).
3 (I)	Why no webhook?	Jenkins accepts traffic only from the admin IP.
4 (I)	Where do AWS credentials come from?	The EC2 instance role; nothing stored in Jenkins.
5 (A)	What stops a vulnerable image reaching production?	Trivy image gate before the ECR push; ECR scan on push as a second look.
6 (A)	What happens after a resume?	Jenkins polls and builds any commit pushed while it was off.

17.12 L. SonarQube and Trivy

#	Q	Say
1 (B)	What does SonarQube do?	Static analysis for bugs, vulnerabilities, smells, duplication, coverage; the quality gate decides pass/fail.
2 (I)	What does Trivy scan?	Dependencies, IaC misconfigurations, secrets and images.
3 (I)	Why `--ignore-unfixed` ?	Unfixable CVEs cannot be resolved by rebuilding; gating on them would block every release.
4 (A)	Example of the gate working?	Build #3 failed on 5 HIGH IaC findings in the ELK manifest.

17.13 M. AWS services

#	Q	Say
1 (B)	What is a VPC / subnet?	A private network / a range of it in one AZ.
2 (B)	Public vs private subnet?	Public has a route to the internet gateway; private goes out through NAT and cannot be reached from outside.

4 (I)	What is IRSA?	IAM roles for Kubernetes service accounts via the cluster's OIDC provider.
5 (I)	What is KMS used for?	Encrypting Kubernetes Secrets and the SNS topic/SQS queue.
6 (I)	What is in Secrets Manager?	RDS master credentials, Jenkins and SonarQube admin passwords (generated by Terraform).
7 (I)	What is IMDSv2?	Token-based instance metadata access; blocks SSRF credential theft.
8 (A)	Why is the deployer user not admin?	PowerUser + IAM read + IAM write scoped to `order-inventory-*`. Least privilege; never root.
9 (I)	What is S3 used for?	Terraform state (bootstrap bucket) and CI reports (30-day expiry).

17.14 N. Monitoring, logging, cost, recovery

#	Q	Say
1 (B)	What alarms exist?	5 RDS, 2 Jenkins, 1 node CPU, 1 NAT (Terraform) + 4 ALB (script).
2 (I)	How did you verify alerts?	Forced an alarm; message arrived in the SQS queue subscribed to SNS.
3 (I)	Did monitoring find a real problem?	Yes: RDS memory/swap on db.t3.micro → parameter tuning + recalibrated alarm.
4 (I)	How do you trace a failed request?	Use the `requestId` from the error response in Kibana.
5 (I)	How do you control cost?	Suspend/resume scripts, budget alerts, small instance types, lifecycle policies, log retention.
6 (A)	What is your recovery strategy for data?	RDS 7-day automated backups with point-in-time restore (restore itself not exercised), versioned state bucket, immutable images.
7 (A)	RDS stop limit?	AWS auto-starts a stopped instance after 7 days; re-run suspend.

CHAPTER 18

Presentation scripts

Simple spoken English. Every claim below is implemented and verified (evidence in `docs/TESTING.md` and `docs/evidence/`). Do not memorise word for word; keep the order of ideas.

18.1 30-second introduction

"My project is an order and inventory management system — customers, products and orders — but the focus is the DevOps platform around it. Every change I push to GitHub is tested, checked for quality and security, packaged into containers and deployed to Kubernetes on AWS with zero downtime, and it rolls back by itself if the new version is unhealthy. All the AWS infrastructure is written in Terraform, the build server is configured with Ansible, and I measured zero failed requests across releases and failures."

18.2 One-minute introduction

"The application has a React frontend, a Spring Boot REST backend and a MySQL database. The interesting business rule is stock: when two customers try to buy the last item at the same time, only one may succeed — I solve that with database row locks inside a transaction and I have a concurrency test that proves it.

Around it I built the delivery platform. Jenkins watches the GitHub repository, runs 29 backend and 21 frontend tests, a SonarQube quality gate and Trivy security scans, builds two Docker images with an immutable tag made of build number and commit, pushes them to Amazon ECR and deploys them to Amazon EKS with Helm. The rollout adds a new pod before removing an old one, waits for health checks, and rolls back automatically on failure. Terraform created the VPC, EKS, RDS, ECR, IAM, encryption keys, monitoring and the Jenkins server; Ansible configured Jenkins and SonarQube. CloudWatch alarms notify through SNS, and logs go to an ELK stack where I can trace any request by its id. Finally, to control cost, one script suspends the whole environment and another restores and verifies it in about ten minutes."

18.3 Three-minute technical explanation

1. **Application (30 s).** "Spring Boot service with layered design — controllers, services, repositories. Flyway owns the schema; Hibernate only validates it. Orders lock product rows in ascending id order, check and deduct stock in one transaction; cancelling restores stock exactly once. Errors are mapped centrally: 400, 404, 409 for conflicts, 503 when the database is down — never stack traces."
2. **Health model (20 s).** "Liveness does not check the database; readiness does. So during a database outage the pods leave the load balancer but are not restarted."
3. **Pipeline (50 s).** "Checkout and tag, environment check, backend tests with coverage, frontend lint/tests/ build, SonarQube quality gate, Trivy for dependencies, infrastructure code and secrets, Helm lint, Docker build, Trivy image gate, then — only if everything passed — ECR push, Helm deploy with automatic rollback, verification that the right image runs, an in-cluster readiness check and a smoke test through the public load balancer."
4. **Infrastructure (40 s).** "Three subnet tiers in two availability zones: public for the ALB, NAT and Jenkins; private for the EKS nodes; isolated database subnets with no internet route for RDS. State is in an encrypted, versioned S3 bucket with locking. Jenkins has no stored AWS keys — it uses an instance role — and it may only edit one Kubernetes namespace."
5. **Operations (40 s).** "Thirteen CloudWatch alarms to SNS with verified delivery to an SQS audit queue, a budget alert, ELK with request-id tracing, and the suspend/resume scripts with a recovery manifest."

18.4 Five-minute architecture explanation (follow diagram 1 and 2 in Architecture and flows)

- "Start at the user: the browser connects on port 80 to an Application Load Balancer in the two public subnets. The load balancer is not in Terraform — Kubernetes creates it from my Ingress through the AWS Load Balancer Controller. It sends `/api` to the backend pods and everything else to the frontend pods, directly to the pod IPs."
- "The pods run on two t3.medium worker nodes in private subnets, one per availability zone. They have no public IPs; to pull images from ECR and reach AWS APIs they go out through the NAT gateway."
- "The backend connects to RDS MySQL 8.4 in database subnets that have no route to the internet. The RDS security group only allows the EKS cluster's security group, and the connection must use TLS. Credentials are generated by Terraform into Secrets Manager and copied into a Kubernetes Secret that is encrypted with my KMS key."
- "Jenkins runs on its own EC2 instance in a public subnet but only my IP can reach it. It talks to the cluster through the private API endpoint and has edit rights only in the application namespace."
- "Monitoring: CloudWatch watches RDS, Jenkins, the nodes, the NAT gateway and the load balancer; alarms go to an encrypted SNS topic; logs go to Elasticsearch in the cluster through Filebeat."
- "Everything except the load balancer is created by Terraform from code; the state lives in S3. Jenkins software is installed by Ansible."
- "For cost, the environment is suspended between demos: nodes to zero, load balancer removed, Jenkins and RDS stopped, everything else kept."

18.5 Live demonstration script (5–10 minutes)

Use the Live demonstration plan. Spoken lines:

1. "Here is the running system through the load balancer address the resume script printed."
2. "I create an order for two units — the stock goes down. I cancel it — the stock comes back. A cancelled order cannot be reopened."
3. "If I order more than is in stock, the API returns 400 with a clear message and a request id."
4. "This is Jenkins. This build was triggered by a push to GitHub. Each stage is a gate; images only reach ECR after tests, quality gate and scans pass."
5. "These are the immutable image tags in ECR — build number and commit."
6. "In Kubernetes: two deployments, their ReplicaSets and pods, services, and the ingress which is the load balancer. Helm keeps every revision, so I can roll back with one command."
7. "Here is the zero-downtime evidence: 9,532 requests during releases and failures, zero errors."
8. "In Kibana I search the request id from the failed order and find the exact backend log line with its pod."
9. "And this is how I keep cost down: the suspend script and what remains billed."

18.6 CI/CD pipeline — detailed explanation

"The pipeline is the `Jenkinsfile` in the repository, so it is versioned with the code. Jenkins polls GitHub every five minutes; a new commit starts a build. The tag is build number plus short commit. First I run the cheap, fast checks: tool versions, backend tests with coverage — a build with zero tests fails on purpose — then frontend lint, tests and build. Then SonarQube analyses both and the build waits for the quality gate. Then Trivy: critical dependency CVEs, high or critical misconfigurations in Terraform, Helm and Dockerfiles — with five reviewed exceptions — and secrets. Helm lint renders the chart. Only then do I build images, and Trivy scans them; fixable high or critical

do not become ready. After that the pipeline verifies that both deployments run exactly this tag, checks readiness inside the cluster and runs a smoke test through the public load balancer. If anything fails after the deploy, the failure handler rolls back to the recorded revision. Reports are archived in Jenkins and in S3."

18.7 Zero-downtime deployment — detailed explanation

"Zero downtime needs several pieces working together. One: the rolling update has `maxUnavailable` zero and `maxSurge` one, so Kubernetes first starts one new pod and only removes an old pod after the new one is ready. Two: readiness checks the database, so a pod only receives traffic when it can really serve requests. Three: when an old pod is stopped, it first sleeps ten seconds in a preStop hook, because removing it from the load balancer takes a moment; then Spring Boot shuts down gracefully and finishes requests in flight. Four: the load balancer has a twenty-second deregistration delay. Five: PodDisruptionBudgets keep at least one pod during node maintenance. And if a new version never becomes ready, the old pods were never removed, so users see nothing and the script rolls back. I proved it on AWS with continuous traffic: 9,532 requests during two releases, a broken image, a release that could not reach the database and a restart — zero errors."

18.8 AWS architecture — detailed explanation

Use the five-minute architecture explanation above plus: "IAM is least privilege: my deployer user is a PowerUser with IAM rights only for resources named `order-inventory`; Jenkins can push to two repositories, describe one cluster and write to one bucket; the load balancer controller and the EBS driver use IRSA roles bound to single service accounts. Encryption: KMS for Kubernetes secrets and alerts, encrypted RDS and EBS, encrypted S3 with TLS-only policies. Network: three tiers, security groups referencing each other instead of IP ranges, administrator access limited to one IP address."

18.9 Security and cost optimisation explanation

"Security: no secrets in Git — passwords are generated by Terraform into Secrets Manager; an old password had leaked in the public history, so I rewrote the history, verified it on a fresh clone, and the database that used it had already been destroyed, so the value is useless. Containers run as non-root with read-only file systems and no Linux capabilities. Trivy and SonarQube gates run on every build. The database is private and TLS-only; Jenkins and the Kubernetes API accept only my IP.

Cost: small instance types, ECR lifecycle and S3 expiry rules, log retention limits, a budget alarm, and the suspend script: from roughly 311 dollars a month running to roughly 126 a month suspended, by my estimate from list prices. The remaining cost is mostly the EKS control plane and the NAT gateway, which I keep so that restore is reliable."

18.10 Honest limitations and future improvements

"Things I would change for real production: HTTPS with a certificate — it is HTTP only now; real authentication — the login page is only a demo; Multi-AZ RDS with deletion protection and final snapshots; a dedicated least-privilege database user instead of the master user; autoscaling for pods and nodes; a webhook or GitHub Actions with OIDC instead of polling; managed logging instead of a single-node Elasticsearch; canary or blue-green releases; and removing the last owner-side items: deactivating the root access key and enabling MFA for all human users."

CHAPTER 19

Glossary

Columns: definition · analogy · how it applies here · related file/command · a likely viva question.

19.1 Networking and AWS

Term	Definition	Analogy	In this project	File / command	Viva question
VPC	a private, isolated network in AWS	a gated campus	`10.0.0.0/16`	`aws_vpc.main`	"What is your VPC CIDR?"
Subnet	a slice of the VPC in one Availability Zone	a building on the campus	2 public, 2 private, 2 database	`aws_subnet.*`	"Why three tiers?"
Availability Zone	an isolated data-centre group in a region	two separate buildings with own power	ap-south-1a, 1b	`terraform.tfvars`	"Why two AZs?"
Route table	rules saying where traffic for an address range goes	road signs	public→IGW, private→NAT, database→none	`aws_route_table.*`	"Why can't RDS reach the internet?"
Internet gateway	VPC's door to the internet	main gate	for public subnets	`aws_internet_gateway.igw`	—
NAT gateway	lets private resources make outbound connections only	a one-way exit turnstile	nodes pull images through it	`aws_nat_gateway.nat`	"Why keep NAT while suspended?"
Elastic IP	a static public IPv4 address	a permanent phone number	NAT gateway IP	`aws_eip.nat`	"Why does Jenkins' IP change?" (it has no EIP)
Security group	a stateful firewall attached to resources	a guest list at a door	RDS allows only the cluster SG; Jenkins only your /32	`aws_security_group.*`	"How is RDS protected?"
CIDR /32	a range containing exactly one IP address	one specific house	admin access	`admin.auto.tfvars`	"What does /32 mean?"
IAM role / policy	identity with permissions that services can assume / a permission document	a staff badge / the badge's access list	Jenkins role, node role, LBC role	`jenkins.tf`, `main.tf`	"Why roles instead of keys?"
Least privilege	grant only what is needed	a key that opens one room	Jenkins: 2 ECR repos, 1 cluster, 1 bucket, 1 namespace	`jenkins_ecr_push` etc.	"Show least privilege."
IRSA / OIDC provider	Kubernetes service accounts assume IAM roles via an OIDC trust	a visitor pass issued by reception	LBC and EBS CSI roles	`aws_iam_openid_connect_provider.eks`	"How does the controller get AWS permissions?"
IMDSv2	session-token-protected instance metadata service	a door needing a fresh code	required on Jenkins	`metadata_options`	"Why IMDSv2?"
KMS / CMK	managed encryption keys / a key you control	a bank vault key	Kubernetes Secrets, SNS, SQS	`kms.tf`	"Why a customer-managed key?"
Secrets Manager	managed storage for secrets	a safe-deposit box	RDS, Jenkins, SonarQube passwords	`order-inventory/*`	"Where is the DB password?"
ECR	AWS container image registry	a warehouse for packaged goods	2 IMMUTABLE repos, scan on push	`aws_ecr_repository.*`	"Why immutable?"
RDS	managed relational database	a hosted database with a caretaker	MySQL 8.4 db.t3.micro	`aws_db_instance.mysql`	"Why managed DB?"

				`n_period`	"How would you restore yesterday's data?"
ALB	layer-7 HTTP load balancer	a receptionist routing visitors by request	created from the Ingress	`ingress.yaml`	"How is it created?"
Target group	the set of backends an ALB rule sends to, with health checks	a department's staff list	backend and frontend TGs (pod IPs)	—	"What is target-type ip?"
CloudWatch alarm	metric threshold that changes state and triggers actions	a smoke detector	13 alarms	`monitoring.tf`, `create-alb-alarms.sh`	"What does INSUFFICIENT_DATA mean?"
SNS / SQS	pub/sub notification topic / message queue	a loudspeaker / a mailbox	alerts → audit queue	`aws_sns_topic.alerts`, `aws_sqs_queue.alerts_audit`	"How did you verify alerts?"
AWS Budgets	cost thresholds with notifications	a spending alert on a card	$350, 80 % actual / 100 % forecast	`aws_budgets_budget.monthly`	"Does a budget stop spending?" (no)

19.2 Containers and orchestration

Term	Definition	Analogy	In this project	File / command	Viva question
Image	read-only template of a container's filesystem + start command	a recipe-packed meal kit	backend, frontend	`Dockerfile`	"Image vs container?"
Container	isolated process started from an image	a cooked meal from the kit	in Compose and pods	`docker compose up`	"Container vs VM?"
Multi-stage build	build in one stage, copy only results to the final stage	cook in the kitchen, serve only the plate	Maven/Node builders → JRE/nginx	Dockerfiles	"Why?"
Registry	stores and serves images	warehouse	ECR	`docker push`	—
Immutable tag	a tag that can never point to another image	a serial number	`8-18a2bc6`	Jenkinsfile `IMAGE_TAG`	"Why not latest?"
Kubernetes	container orchestrator	an air-traffic controller	EKS 1.36	`kubectl`	—
Control plane / node	the cluster's brain / the machines running pods	head office / branch offices	AWS-managed / 2 × t3.medium	—	"Why does EKS cost with 0 nodes?"
Pod	smallest deployable unit (one or more containers)	one worker at a desk	backend/frontend pods	`kubectl get pods`	—
Deployment / ReplicaSet	manages versions / keeps N identical pods	a manager / a team of identical workers	2 + 2 replicas	`backend.yaml`	"Difference?"
Service	stable name/IP for a set of ready pods	a department phone number	`backend:8080`, `frontend:80`	`kubectl get svc`	"How do pods find each other?"
Ingress	HTTP routing rules into the cluster	a building directory	`/api` → backend, `/` → frontend	`ingress.yaml`	"Ingress vs Service?"
ConfigMap / Secret	non-secret / secret configuration for pods	a notice board / a sealed envelope	`backend-config` / `mysql-secret`	templates	"How are credentials injected?"

Resource requests/limits	guaranteed / maximum CPU and memory	reserved seat / seat-belt limit	backend 250m/512Mi → 500m/1Gi	`values.yaml`	"Why limits?"
Rolling update	replace pods gradually	changing train carriages one at a time while it runs	maxSurge 1, maxUnavailable 0	`values.yaml`	"How is downtime avoided?"
PodDisruptionBudget	minimum pods kept during voluntary disruptions	minimum staff on duty	minAvailable 1	`pdb.yaml`	—
DaemonSet	one pod per node	a security guard on every floor	Filebeat	`elk.yaml`	—
Helm chart / release / revision	package of templates / installed instance / numbered version of it	app installer / installed app / app version history	`helm/order-inventory`, release `order-inventory`	`helm history`	"How do you roll back?"
Namespace	a scope for names and permissions	a department	`order-inventory`, `logging`	`k8s/namespace.yaml`	"Why is Jenkins limited to one namespace?"
Security context	process privileges of a pod/container	the rules of a visitor badge	non-root, read-only rootfs, no capabilities	templates	—

19.3 Delivery, quality and operations

Term	Definition	Analogy	In this project	File / command	Viva question
Infrastructure as Code	infrastructure defined in versioned files	architectural blueprints	Terraform	`terraform/`	—
Terraform state	Terraform's record of managed resources	an inventory ledger	S3, encrypted, locked	`backend.tf`	"Why sensitive?"
State drift	real infrastructure differs from code/state	the ledger no longer matches the shelves	stopped Jenkins public IP	`terraform plan`	"How do you detect drift?"
`ignore_changes`	tell Terraform not to manage an attribute	"don't correct this note in the ledger"	node desired size, Jenkins AMI/public-IP flag	`main.tf`, `jenkins.tf`	"Why needed? Risks?"
Configuration management	automated, repeatable server configuration	a checklist robot	Ansible roles	`ansible/`	—
Idempotency	repeating an operation has no extra effect	pressing a lift button twice	Ansible `changed=0`; scripts re-run safely	`run-playbook.sh`, `*-evaluation.sh`	"Prove idempotency."
CI / CD	continuous integration / continuous delivery-deployment	quality-control line / automatic shipping	Jenkinsfile stages	`Jenkinsfile`	"Difference?"
Quality gate	pass/fail conditions on code analysis	an exam pass mark	SonarQube "PASSED"	Code Quality stage	—
Static analysis	analysing code without running it	proof-reading	SonarQube	—	—
Vulnerability scanning	matching components to known CVEs	checking food against recall lists	Trivy fs/image, ECR scan	Jenkinsfile	"Does 0 findings mean secure?"
Misconfiguration scanning	checking IaC against security rules	building-code inspection	`trivy config`	`.trivyignore.yaml`	—
Secrets management	generating, storing and delivering secrets safely	a key-management office	Terraform → Secrets Manager → K8s Secret	`create-db-secret.sh`	—
Database migration	versioned schema change script	renovation plan with numbered steps	Flyway V1, V2	`db/migration/`	"Why Flyway?"

Rollback	return to the previous good version	undo	`helm rollback`	`deploy-k8s.sh`	"How automatic is it?"
Zero downtime	no failed user requests during changes	changing tyres while the car drives	9,532 requests, 0 errors	evidence file	"How proven?"
Observability	understanding a system from its outputs (metrics, logs)	a dashboard and black box	CloudWatch + ELK	—	—
Correlation / request id	one id attached to all logs of a request	a parcel tracking number	`X-Request-Id`, MDC	`RequestIdFilter.java`	"How do you trace a request?"
Structured logging (ECS)	logs as JSON fields following the Elastic Common Schema	a form instead of free text	backend logs	`LOGGING_STRUCTURED_FORMAT_CONSOLE=ecs`	—
ILM	index lifecycle management (rollover, delete)	automatic archive shredding	delete after 7 days	`ilm-policy.json`	—
Recovery manifest	record of what to restore	a packing list	`evaluation-manifest.json`	suspend/resume	"What is in it?"
Suspend / resume	stop billable compute while keeping data, then restore	putting a shop in standby rather than demolishing it	evaluation scripts	`scripts/*-evaluation.sh`	"Why not destroy?"
Pessimistic lock	lock a row before changing it	reserving the last ticket while paying	`SELECT … FOR UPDATE`	`ProductRepository.findByIdForUpdate`	"How is overselling prevented?"
Transaction	all-or-nothing group of DB operations	a bank transfer	`@Transactional` services	`OrderService`	—
Graceful shutdown	finish in-flight work before exiting	serving the customers in the queue before closing	20 s	`application.properties`	—

CHAPTER 20

Quick reference

20.1 Identity and names

Fact	Value
Repository root	`C:\Users\Siddesh\Desktop\DevOps Project`
AWS account / region / CLI profile	`038215676861` / `ap-south-1` / `order-inventory`
Expected identity	`arn:aws:iam::038215676861:user/order-inventory-deployer` (never root)
EKS cluster / node group	`order-inventory-eks` / `order-inventory-nodes`
RDS / Jenkins EC2	`order-inventory-mysql` / `i-03288b455e674765e`
Namespace / Helm release	`order-inventory` / `order-inventory`
ECR repositories	`order-inventory-backend`, `order-inventory-frontend`
Secrets (names only)	`order-inventory/rds-master`, `order-inventory/jenkins-admin`, `order-inventory/sonarqube-admin`
Terraform state	`s3://order-inventory-tfstate-038215676861/order-inventory/terraform.tfstate`
Restored release	`lastKnownGoodTag` in `docs/recovery/evaluation-manifest.json` (`8-18a2bc6` at the last suspend)
GitHub	`https://github.com/SamruddhiDoke/DevOps-Project`, branch `main`

20.2 Commands

Purpose	Command
Check identity	`aws sts get-caller-identity --profile order-inventory`
Restore (PowerShell)	`powershell -ExecutionPolicy Bypass -File scripts\resume-evaluation.ps1`
Restore (Git Bash)	`bash scripts/resume-evaluation.sh`
Preview suspend	`powershell -ExecutionPolicy Bypass -File scripts\suspend-evaluation.ps1 --plan`
Suspend (PowerShell)	`powershell -ExecutionPolicy Bypass -File scripts\suspend-evaluation.ps1`
Suspend (Git Bash)	`bash scripts/suspend-evaluation.sh`
Application URL	`kubectl -n order-inventory get ingress order-inventory-ingress -o jsonpath='{.status.loadBalancer.ingress[0].hostname}'`
Jenkins IP	`aws ec2 describe-instances --instance-ids i-03288b455e674765e --query 'Reservations[0].Instances[0].PublicIpAddress' --output text`
Jenkins password to clipboard	`aws secretsmanager get-secret-value --secret-id order-inventory/jenkins-admin --query SecretString --output text --profile order-inventory --region ap-south-1 | Set-Clipboard`
Readiness	`curl -s $URL/api/health/ready` → `{"status":"UP","database":"UP"}`
Read-only smoke test	`SMOKE_READ_ONLY=1 bash scripts/smoke-test.sh $URL`

	`history order-inventory`
Manual rollback	`helm -n order-inventory rollback order-inventory <revision>`
Kibana	`kubectl -n logging port-forward svc/kibana 5601:5601` then `bash order-inventory-platform/logging/kibana-setup.sh http://localhost:5601`
Alarms	`aws cloudwatch describe-alarms --alarm-name-prefix order-inventory --query 'MetricAlarms[].[AlarmName,StateValue]' --output table`
Local fallback	`cd order-inventory-platform; docker compose up -d --build` → http://localhost:3000

20.3 Five numbers to remember

Number	Meaning	Evidence
9 min 14 s	measured full restore and verification	resume log 2026-10-19, `docs/AWS_DEPLOYMENT.md`
9,532 / 0	requests / errors during releases, failed releases and a restart through the ALB	`docs/evidence/aws-zero-downtime-2026-10-19.txt`
29 + 21	backend + frontend automated tests, run in every build	`docs/TESTING.md`
8 → 3	concurrent buyers vs stock 3 → exactly 3 orders, stock 0	`OrderConcurrencyTest`
≈ $4/day	estimated suspended cost (control plane, NAT, storage)	estimate 2026-10-19

20.4 Never do

- `terraform destroy` to suspend (deletes the database without a final snapshot).
- Run `ansible/run-playbook.sh` without `JENKINS_REPO_URL=https://github.com/SamruddhiDoke/DevOps-Project.git`, or while a build is running.
- Show the Jenkins password on the projector.
- Rely on `terraform output jenkins_public_ip` after a restart (stale until `terraform apply -refresh-only`).